

FOORT

1.0

Generated by Doxygen 1.14.0

1 Namespace Index	1
1.1 Namespace List	1
2 Hierarchical Index	3
2.1 Class Hierarchy	3
3 Class Index	5
3.1 Class List	5
4 File Index	9
4.1 File List	9
5 Namespace Documentation	11
5.1 Config Namespace Reference	11
5.1.1 Detailed Description	12
5.1.2 Typedef Documentation	12
5.1.2.1 ConfigCollection	12
5.1.2.2 SettingError	12
5.1.3 Function Documentation	12
5.1.3.1 GetGeodesicIntegrator()	12
5.1.3.2 GetMesh()	12
5.1.3.3 GetMetric()	13
5.1.3.4 GetOutputHandler()	13
5.1.3.5 GetSource()	13
5.1.3.6 GetViewScreen()	14
5.1.3.7 InitializeDiagnostics()	14
5.1.3.8 InitializeScreenOutput()	14
5.1.3.9 InitializeTerminations()	15
5.1.4 Variable Documentation	15
5.1.4.1 Output_Important_Default	15
5.1.4.2 Output_Other_Default	15
5.2 ConfigReader Namespace Reference	15
5.2.1 Detailed Description	16
5.2.2 Variable Documentation	16
5.2.2.1 MaxStreamSize	16
5.3 Integrators Namespace Reference	16
5.3.1 Function Documentation	16
5.3.1.1 GetAdaptiveStep()	16
5.3.1.2 GetFullIntegratorDescription()	17
5.3.1.3 IntegrateGeodesicStep_RK4()	17
5.3.1.4 IntegrateGeodesicStep_Verlet()	17
5.3.2 Variable Documentation	18
5.3.2.1 delta_nodiv0	18
5.3.2.2 Derivative_hval	18

5.3.2.3 epsilon	18
5.3.2.4 IntegratorDescription	18
5.3.2.5 SmallestPossibleStepsize	18
5.3.2.6 VerletVelocityTolerance	19
5.4 tk Namespace Reference	19
5.5 tk::internal Namespace Reference	19
5.6 Utilities Namespace Reference	19
5.6.1 Detailed Description	19
5.6.2 Function Documentation	19
5.6.2.1 GetDiagNameStrings()	19
5.6.2.2 GetFirstLineInfoString()	20
5.6.2.3 GetTimeStampString()	20
6 Class Documentation	21
6.1 tk::internal::band_matrix Class Reference	21
6.1.1 Constructor & Destructor Documentation	21
6.1.1.1 band_matrix() [1/2]	21
6.1.1.2 band_matrix() [2/2]	22
6.1.1.3 ~band_matrix()	22
6.1.2 Member Function Documentation	22
6.1.2.1 dim()	22
6.1.2.2 l_solve()	22
6.1.2.3 lu_decompose()	22
6.1.2.4 lu_solve()	22
6.1.2.5 num_lower()	22
6.1.2.6 num_upper()	22
6.1.2.7 operator() [1/2]	22
6.1.2.8 operator() [2/2]	23
6.1.2.9 r_solve()	23
6.1.2.10 resize()	23
6.1.2.11 saved_diag() [1/2]	23
6.1.2.12 saved_diag() [2/2]	23
6.1.3 Member Data Documentation	23
6.1.3.1 m_lower	23
6.1.3.2 m_upper	23
6.2 BicubicSplineInterpolator Class Reference	24
6.2.1 Constructor & Destructor Documentation	24
6.2.1.1 BicubicSplineInterpolator()	24
6.2.2 Member Function Documentation	24
6.2.2.1 compute_cell_coefficients()	24
6.2.2.2 compute_spline_coefficients()	25
6.2.2.3 estimate_dx()	25

6.2.2.4 estimate_dxdy()	25
6.2.2.5 estimate_dy()	25
6.2.2.6 interpolate()	25
6.2.2.7 set_grid()	25
6.2.3 Member Data Documentation	25
6.2.3.1 m_coeffs	25
6.2.3.2 m_theta	26
6.2.3.3 m_x	26
6.3 BosonStarMetric Class Reference	26
6.3.1 Detailed Description	27
6.3.2 Constructor & Destructor Documentation	27
6.3.2.1 BosonStarMetric()	27
6.3.3 Member Function Documentation	28
6.3.3.1 getFullDescriptionStr()	28
6.3.3.2 getMetric_dd()	28
6.3.3.3 getMetric_uu()	28
6.3.3.4 read_data()	29
6.3.4 Member Data Documentation	29
6.3.4.1 m_m_filename	29
6.3.4.2 m_mSpline	29
6.3.4.3 m_num_lines	29
6.3.4.4 m_Phi_filename	29
6.3.4.5 m_Phi_infinity	30
6.3.4.6 m_PhiSpline	30
6.4 BoundarySphereTermination Class Reference	30
6.4.1 Detailed Description	31
6.4.2 Constructor & Destructor Documentation	31
6.4.2.1 BoundarySphereTermination()	31
6.4.3 Member Function Documentation	31
6.4.3.1 CheckTermination()	31
6.4.3.2 getFullDescriptionStr()	32
6.4.4 Member Data Documentation	32
6.4.4.1 TermOptions	32
6.5 BoundarySphereTermOptions Struct Reference	32
6.5.1 Detailed Description	33
6.5.2 Constructor & Destructor Documentation	33
6.5.2.1 BoundarySphereTermOptions()	33
6.5.3 Member Data Documentation	33
6.5.3.1 rLogScale	33
6.5.3.2 SphereRadius	33
6.6 ClosestRadiusDiagnostic Class Reference	34
6.6.1 Detailed Description	35

6.6.2 Constructor & Destructor Documentation	35
6.6.2.1 ClosestRadiusDiagnostic()	35
6.6.3 Member Function Documentation	35
6.6.3.1 FinalDataValDistance()	35
6.6.3.2 getFinalDataVal()	36
6.6.3.3 getFullDataStr()	36
6.6.3.4 getFullDescriptionStr()	36
6.6.3.5 getNameStr()	36
6.6.3.6 Reset()	37
6.6.3.7 UpdateData()	37
6.6.4 Member Data Documentation	37
6.6.4.1 DiagOptions	37
6.6.4.2 m_ClosestRadius	37
6.7 ClosestRadiusOptions Struct Reference	37
6.7.1 Detailed Description	38
6.7.2 Constructor & Destructor Documentation	38
6.7.2.1 ClosestRadiusOptions()	38
6.7.3 Member Data Documentation	38
6.7.3.1 RLogScale	38
6.8 ConfigReader::ConfigCollection Class Reference	38
6.8.1 Member Typedef Documentation	40
6.8.1.1 ConfigSettingValue	40
6.8.2 Member Function Documentation	40
6.8.2.1 DisplayCollection()	40
6.8.2.2 DisplaySetting()	41
6.8.2.3 DisplayTabs()	41
6.8.2.4 Exists()	41
6.8.2.5 GetSettingIndex()	41
6.8.2.6 IsCollection() [1/2]	42
6.8.2.7 IsCollection() [2/2]	42
6.8.2.8 LookupValue() [1/2]	42
6.8.2.9 LookupValue() [2/2]	43
6.8.2.10 LookupValueInteger() [1/7]	43
6.8.2.11 LookupValueInteger() [2/7]	44
6.8.2.12 LookupValueInteger() [3/7]	44
6.8.2.13 LookupValueInteger() [4/7]	45
6.8.2.14 LookupValueInteger() [5/7]	45
6.8.2.15 LookupValueInteger() [6/7]	45
6.8.2.16 LookupValueInteger() [7/7]	46
6.8.2.17 NrSettings()	46
6.8.2.18 operator[]() [1/2]	46
6.8.2.19 operator[]() [2/2]	46

6.8.2.20 ReadCollection()	47
6.8.2.21 ReadFile()	47
6.8.2.22 ReadSettingName()	47
6.8.2.23 ReadSettingSpecificChar()	48
6.8.2.24 ReadSettingValue()	48
6.8.3 Member Data Documentation	48
6.8.3.1 m_Settings	48
6.9 ConfigReader::ConfigReaderException Class Reference	49
6.9.1 Constructor & Destructor Documentation	49
6.9.1.1 ConfigReaderException()	49
6.9.2 Member Function Documentation	49
6.9.2.1 trace()	49
6.9.3 Member Data Documentation	49
6.9.3.1 m_settingtrace	49
6.10 ConfigReader::ConfigCollection::ConfigSetting Struct Reference	50
6.10.1 Member Data Documentation	50
6.10.1.1 SettingName	50
6.10.1.2 SettingValue	50
6.11 Diagnostic Class Reference	50
6.11.1 Detailed Description	51
6.11.2 Constructor & Destructor Documentation	51
6.11.2.1 Diagnostic() [1/2]	51
6.11.2.2 Diagnostic() [2/2]	51
6.11.2.3 ~Diagnostic()	51
6.11.3 Member Function Documentation	51
6.11.3.1 DecideUpdate()	51
6.11.3.2 FinalDataValDistance()	52
6.11.3.3 getFinalDataVal()	52
6.11.3.4 getFullDataStr()	52
6.11.3.5 getFullDescriptionStr()	52
6.11.3.6 getNameStr()	52
6.11.3.7 Reset()	53
6.11.3.8 UpdateData()	53
6.11.4 Member Data Documentation	53
6.11.4.1 m_OwnerGeodesic	53
6.11.4.2 m_StepsSinceUpdated	53
6.12 DiagnosticOptions Struct Reference	53
6.12.1 Detailed Description	54
6.12.2 Constructor & Destructor Documentation	54
6.12.2.1 DiagnosticOptions()	54
6.12.2.2 ~DiagnosticOptions()	54
6.12.3 Member Data Documentation	54

6.12.3.1 theUpdateFrequency	54
6.13 EmissionModel Struct Reference	54
6.13.1 Detailed Description	55
6.13.2 Constructor & Destructor Documentation	55
6.13.2.1 ~EmissionModel()	55
6.13.3 Member Function Documentation	55
6.13.3.1 GetEmission()	55
6.13.3.2 getFullDescriptionStr()	55
6.14 EquatorialEmissionDiagnostic Class Reference	56
6.14.1 Detailed Description	57
6.14.2 Constructor & Destructor Documentation	58
6.14.2.1 EquatorialEmissionDiagnostic()	58
6.14.3 Member Function Documentation	58
6.14.3.1 FinalDataValDistance()	58
6.14.3.2 getFinalDataVal()	58
6.14.3.3 getFullDataStr()	58
6.14.3.4 getFullDescriptionStr()	59
6.14.3.5 getNameStr()	59
6.14.3.6 Reset()	59
6.14.3.7 UpdateData()	59
6.14.4 Member Data Documentation	59
6.14.4.1 DiagOptions	59
6.14.4.2 m_Intensity	60
6.15 EquatorialEmissionOptions Struct Reference	60
6.15.1 Detailed Description	61
6.15.2 Constructor & Destructor Documentation	61
6.15.2.1 EquatorialEmissionOptions()	61
6.15.3 Member Data Documentation	61
6.15.3.1 EquatPassUpperBound	61
6.15.3.2 GeometricFudgeFactor	61
6.15.3.3 RedShiftPower	61
6.15.3.4 RLogScale	61
6.15.3.5 TheEmissionModel	62
6.15.3.6 TheFluidVelocityModel	62
6.16 EquatorialPassesDiagnostic Class Reference	62
6.16.1 Detailed Description	63
6.16.2 Constructor & Destructor Documentation	63
6.16.2.1 EquatorialPassesDiagnostic()	63
6.16.3 Member Function Documentation	63
6.16.3.1 FinalDataValDistance()	63
6.16.3.2 getFinalDataVal()	64
6.16.3.3 getFullDataStr()	64

6.16.3.4 getFullDescriptionStr()	64
6.16.3.5 getNameStr()	65
6.16.3.6 Reset()	65
6.16.3.7 UpdateData()	65
6.16.4 Member Data Documentation	65
6.16.4.1 DiagOptions	65
6.16.4.2 m_EquatPasses	65
6.16.4.3 m_PrevTheta	66
6.17 EquatorialPassesOptions Struct Reference	66
6.17.1 Detailed Description	67
6.17.2 Constructor & Destructor Documentation	67
6.17.2.1 EquatorialPassesOptions()	67
6.17.3 Member Data Documentation	67
6.17.3.1 Threshold	67
6.18 FlatSpaceMetric Class Reference	67
6.18.1 Detailed Description	68
6.18.2 Constructor & Destructor Documentation	68
6.18.2.1 FlatSpaceMetric()	68
6.18.3 Member Function Documentation	68
6.18.3.1 getFullDescriptionStr()	68
6.18.3.2 getMetric_dd()	69
6.18.3.3 getMetric_uu()	69
6.19 FluidVelocityModel Struct Reference	69
6.19.1 Detailed Description	70
6.19.2 Constructor & Destructor Documentation	70
6.19.2.1 FluidVelocityModel()	70
6.19.2.2 ~FluidVelocityModel()	70
6.19.3 Member Function Documentation	70
6.19.3.1 GetFourVelocityd()	70
6.19.3.2 getFullDescriptionStr()	70
6.19.4 Member Data Documentation	71
6.19.4.1 m_theMetric	71
6.20 FourColorScreenDiagnostic Class Reference	71
6.20.1 Detailed Description	72
6.20.2 Constructor & Destructor Documentation	72
6.20.2.1 FourColorScreenDiagnostic()	72
6.20.3 Member Function Documentation	72
6.20.3.1 FinalDataValDistance()	72
6.20.3.2 getFinalDataVal()	73
6.20.3.3 getFullDataStr()	73
6.20.3.4 getFullDescriptionStr()	73
6.20.3.5 getNameStr()	73

6.20.3.6 Reset()	74
6.20.3.7 UpdateData()	74
6.20.4 Member Data Documentation	74
6.20.4.1 m_quadrant	74
6.21 GeneralCircularRadialFluid Struct Reference	74
6.21.1 Detailed Description	76
6.21.2 Constructor & Destructor Documentation	76
6.21.2.1 GeneralCircularRadialFluid()	76
6.21.3 Member Function Documentation	76
6.21.3.1 FindISCO()	76
6.21.3.2 GetChrstrRaisedDer()	76
6.21.3.3 GetCircularVelocityd()	77
6.21.3.4 GetFourVelocityd()	77
6.21.3.5 getFullDescriptionStr()	78
6.21.3.6 GetInsideISCOCircularVelocityd()	78
6.21.3.7 GetRadialVelocityd()	78
6.21.4 Member Data Documentation	78
6.21.4.1 m_betaPhi	78
6.21.4.2 m_betaR	79
6.21.4.3 m_ISCOexists	79
6.21.4.4 m_ISCOLowerbound	79
6.21.4.5 m_ISCOpPhi	79
6.21.4.6 m_ISCOpt	79
6.21.4.7 m_ISCOr	79
6.21.4.8 m_ISCOUpperbound	79
6.21.4.9 m_subKeplerParam	80
6.22 GeneralSingularityTermination Class Reference	80
6.22.1 Detailed Description	81
6.22.2 Constructor & Destructor Documentation	81
6.22.2.1 GeneralSingularityTermination()	81
6.22.3 Member Function Documentation	81
6.22.3.1 CheckTermination()	81
6.22.3.2 getFullDescriptionStr()	82
6.22.3.3 SingularityToString()	82
6.22.4 Member Data Documentation	82
6.22.4.1 TermOptions	82
6.23 GeneralSingularityTermOptions Struct Reference	82
6.23.1 Detailed Description	83
6.23.2 Constructor & Destructor Documentation	83
6.23.2.1 GeneralSingularityTermOptions()	83
6.23.3 Member Data Documentation	83
6.23.3.1 Epsilon	83

6.23.3.2 OutputToConsole	84
6.23.3.3 rLogScale	84
6.23.3.4 Singularities	84
6.24 Geodesic Class Reference	84
6.24.1 Detailed Description	85
6.24.2 Constructor & Destructor Documentation	85
6.24.2.1 Geodesic() [1/3]	85
6.24.2.2 Geodesic() [2/3]	85
6.24.2.3 Geodesic() [3/3]	85
6.24.3 Member Function Documentation	86
6.24.3.1 getAllOutputStr()	86
6.24.3.2 getCurrentLambda()	86
6.24.3.3 getCurrentPos()	86
6.24.3.4 getCurrentVel()	87
6.24.3.5 getDiagnosticFinalValue()	87
6.24.3.6 getScreenIndex()	87
6.24.3.7 getTermCondition()	87
6.24.3.8 operator=()	87
6.24.3.9 Reset()	87
6.24.3.10 Update()	88
6.24.4 Member Data Documentation	88
6.24.4.1 m_AllDiagnostics	88
6.24.4.2 m_AllTerminations	88
6.24.4.3 m_curLambda	88
6.24.4.4 m_CurrentPos	88
6.24.4.5 m_CurrentVel	89
6.24.4.6 m_ScreenIndex	89
6.24.4.7 m_TermCond	89
6.24.4.8 m_theIntegrator	89
6.24.4.9 m_theMetric	89
6.24.4.10 m_theSource	89
6.25 GeodesicOutputHandler Class Reference	89
6.25.1 Detailed Description	91
6.25.2 Constructor & Destructor Documentation	91
6.25.2.1 GeodesicOutputHandler() [1/2]	91
6.25.2.2 GeodesicOutputHandler() [2/2]	91
6.25.3 Member Function Documentation	91
6.25.3.1 GetFileName()	91
6.25.3.2 getFullDescriptionStr()	92
6.25.3.3 NewGeodesicOutput()	92
6.25.3.4 OpenForFirstTime()	92
6.25.3.5 OutputFinished()	92

6.25.3.6 PrepareForOutput()	92
6.25.3.7 WriteCachedOutputToFile()	93
6.25.4 Member Data Documentation	93
6.25.4.1 m_AllCachedData	93
6.25.4.2 m_CurrentFullFiles	93
6.25.4.3 m_CurrentGeodesicsInFile	93
6.25.4.4 m_DiagNames	93
6.25.4.5 m_FileExtension	93
6.25.4.6 m_FilePrefix	94
6.25.4.7 m_FirstLineInfoString	94
6.25.4.8 m_nrGeodesicsPerFile	94
6.25.4.9 m_nrOutputsToCache	94
6.25.4.10 m_PrevCached	94
6.25.4.11 m_PrintFirstLineInfo	94
6.25.4.12 m_TimeStamp	94
6.25.4.13 m_WriteToConsole	95
6.26 GeodesicPositionDiagnostic Class Reference	95
6.26.1 Detailed Description	96
6.26.2 Constructor & Destructor Documentation	96
6.26.2.1 GeodesicPositionDiagnostic()	96
6.26.3 Member Function Documentation	96
6.26.3.1 FinalDataValDistance()	96
6.26.3.2 getFinalDataVal()	97
6.26.3.3 getFullDataStr()	97
6.26.3.4 getFullDescriptionStr()	97
6.26.3.5 getNameStr()	98
6.26.3.6 Reset()	98
6.26.3.7 UpdateData()	98
6.26.4 Member Data Documentation	98
6.26.4.1 DiagOptions	98
6.26.4.2 m_AllSavedPoints	98
6.27 GeodesicPositionOptions Struct Reference	99
6.27.1 Detailed Description	99
6.27.2 Constructor & Destructor Documentation	99
6.27.2.1 GeodesicPositionOptions()	99
6.27.3 Member Data Documentation	100
6.27.3.1 OutputNrSteps	100
6.28 GLMJohnsonSUEmission Struct Reference	100
6.28.1 Detailed Description	101
6.28.2 Constructor & Destructor Documentation	101
6.28.2.1 GLMJohnsonSUEmission()	101
6.28.3 Member Function Documentation	101

6.28.3.1 GetEmission()	101
6.28.3.2 getFullDescriptionStr()	101
6.28.4 Member Data Documentation	102
6.28.4.1 m_gamma	102
6.28.4.2 m_mu	102
6.28.4.3 m_sigma	102
6.29 Grid Class Reference	102
6.29.1 Detailed Description	103
6.29.2 Constructor & Destructor Documentation	103
6.29.2.1 Grid()	103
6.29.2.2 ~Grid()	103
6.29.3 Member Function Documentation	103
6.29.3.1 initialize_from_file()	103
6.29.3.2 operator>() [1/2]	103
6.29.3.3 operator>() [2/2]	104
6.29.4 Member Data Documentation	104
6.29.4.1 data	104
6.29.4.2 N_col	104
6.29.4.3 N_row	104
6.29.4.4 size	104
6.30 HorizonTermination Class Reference	104
6.30.1 Detailed Description	105
6.30.2 Constructor & Destructor Documentation	105
6.30.2.1 HorizonTermination()	105
6.30.3 Member Function Documentation	106
6.30.3.1 CheckTermination()	106
6.30.3.2 getFullDescriptionStr()	106
6.30.4 Member Data Documentation	106
6.30.4.1 TermOptions	106
6.31 HorizonTermOptions Struct Reference	106
6.31.1 Detailed Description	107
6.31.2 Constructor & Destructor Documentation	107
6.31.2.1 HorizonTermOptions()	107
6.31.3 Member Data Documentation	107
6.31.3.1 AtHorizonEps	107
6.31.3.2 HorizonRadius	108
6.31.3.3 rLogScale	108
6.32 InputCertainPixelsMesh Class Reference	108
6.32.1 Detailed Description	109
6.32.2 Constructor & Destructor Documentation	109
6.32.2.1 InputCertainPixelsMesh() [1/3]	109
6.32.2.2 InputCertainPixelsMesh() [2/3]	109

6.32.2.3 InputCertainPixelsMesh() [3/3]	109
6.32.3 Member Function Documentation	110
6.32.3.1 EndCurrentLoop()	110
6.32.3.2 GeodesicFinished()	110
6.32.3.3 getCurNrGeodesics()	110
6.32.3.4 getFullDescriptionStr()	111
6.32.3.5 getNewInitConds()	111
6.32.3.6 IsFinished()	111
6.32.4 Member Data Documentation	111
6.32.4.1 m_Finished	111
6.32.4.2 m_PixelsToIntegrate	112
6.32.4.3 m_RowColumnSize	112
6.32.4.4 m_TotalPixels	112
6.33 JohannsenMetric Class Reference	112
6.33.1 Detailed Description	114
6.33.2 Constructor & Destructor Documentation	114
6.33.2.1 JohannsenMetric() [1/2]	114
6.33.2.2 JohannsenMetric() [2/2]	114
6.33.3 Member Function Documentation	114
6.33.3.1 getFullDescriptionStr()	114
6.33.3.2 getMetric_dd()	115
6.33.3.3 getMetric_uu()	116
6.33.4 Member Data Documentation	116
6.33.4.1 m_alpha13Param	116
6.33.4.2 m_alpha22Param	116
6.33.4.3 m_alpha52Param	116
6.33.4.4 m_aParam	117
6.33.4.5 m_eps3Param	117
6.34 KerrMetric Class Reference	117
6.34.1 Detailed Description	118
6.34.2 Constructor & Destructor Documentation	119
6.34.2.1 KerrMetric() [1/2]	119
6.34.2.2 KerrMetric() [2/2]	119
6.34.3 Member Function Documentation	119
6.34.3.1 getFullDescriptionStr()	119
6.34.3.2 getMetric_dd()	119
6.34.3.3 getMetric_uu()	119
6.34.4 Member Data Documentation	120
6.34.4.1 m_aParam	120
6.34.4.2 m_mParam	120
6.35 KerrSchildMetric Class Reference	120
6.35.1 Detailed Description	122

6.35.2 Constructor & Destructor Documentation	122
6.35.2.1 KerrSchildMetric() [1/2]	122
6.35.2.2 KerrSchildMetric() [2/2]	122
6.35.3 Member Function Documentation	122
6.35.3.1 getFullDescriptionStr()	122
6.35.3.2 getMetric_dd()	122
6.35.3.3 getMetric_uu()	123
6.35.4 Member Data Documentation	123
6.35.4.1 m_aParam	123
6.36 MankoNovikovMetric Class Reference	123
6.36.1 Detailed Description	125
6.36.2 Constructor & Destructor Documentation	125
6.36.2.1 MankoNovikovMetric() [1/2]	125
6.36.2.2 MankoNovikovMetric() [2/2]	125
6.36.3 Member Function Documentation	125
6.36.3.1 getFullDescriptionStr()	125
6.36.3.2 getMetric_dd()	125
6.36.3.3 getMetric_uu()	126
6.36.4 Member Data Documentation	126
6.36.4.1 m_alpha3Param	126
6.36.4.2 m_alphaParam	126
6.36.4.3 m_aParam	126
6.36.4.4 m_kParam	127
6.37 Mesh Class Reference	127
6.37.1 Detailed Description	127
6.37.2 Constructor & Destructor Documentation	128
6.37.2.1 Mesh()	128
6.37.2.2 ~Mesh()	128
6.37.3 Member Function Documentation	128
6.37.3.1 EndCurrentLoop()	128
6.37.3.2 GeodesicFinished()	128
6.37.3.3 getCurNrGeodesics()	128
6.37.3.4 getFullDescriptionStr()	128
6.37.3.5 getNewInitConds()	129
6.37.3.6 IsFinished()	129
6.37.4 Member Data Documentation	129
6.37.4.1 m_DistanceDiagnostic	129
6.38 Metric Class Reference	129
6.38.1 Detailed Description	130
6.38.2 Constructor & Destructor Documentation	130
6.38.2.1 ~Metric()	130
6.38.2.2 Metric()	130

6.38.3 Member Function Documentation	130
6.38.3.1 getChristoffel_udd()	130
6.38.3.2 getFullDescriptionStr()	131
6.38.3.3 getKretschmann()	131
6.38.3.4 getMetric_dd()	131
6.38.3.5 getMetric_uu()	132
6.38.3.6 getRiemann_uddd()	132
6.38.3.7 getrLogScale()	132
6.38.4 Member Data Documentation	132
6.38.4.1 m_rLogScale	132
6.38.4.2 m_Symmetries	133
6.39 NaTermination Class Reference	133
6.39.1 Detailed Description	134
6.39.2 Constructor & Destructor Documentation	134
6.39.2.1 NaTermination()	134
6.39.3 Member Function Documentation	134
6.39.3.1 CheckTermination()	134
6.39.3.2 getFullDescriptionStr()	134
6.39.4 Member Data Documentation	135
6.39.4.1 TermOptions	135
6.40 NaTermOptions Struct Reference	135
6.40.1 Detailed Description	136
6.40.2 Constructor & Destructor Documentation	136
6.40.2.1 NaTermOptions()	136
6.40.3 Member Data Documentation	136
6.40.3.1 OutputToConsole	136
6.41 NoSource Class Reference	136
6.41.1 Detailed Description	137
6.41.2 Constructor & Destructor Documentation	137
6.41.2.1 NoSource()	137
6.41.3 Member Function Documentation	137
6.41.3.1 getFullDescriptionStr()	137
6.41.3.2 getSource()	137
6.42 SquareSubdivisionMesh::PixelInfo Struct Reference	138
6.42.1 Detailed Description	138
6.42.2 Constructor & Destructor Documentation	139
6.42.2.1 PixelInfo()	139
6.42.3 Member Data Documentation	139
6.42.3.1 DiagValue	139
6.42.3.2 Index	139
6.42.3.3 LowerNbrIndex	139
6.42.3.4 RightNbrIndex	139

6.42.3.5 SubdivideLevel	139
6.42.3.6 Weight	140
6.43 SquareSubdivisionMeshV2::PixelInfo Struct Reference	140
6.43.1 Detailed Description	140
6.43.2 Constructor & Destructor Documentation	141
6.43.2.1 PixelInfo()	141
6.43.3 Member Data Documentation	141
6.43.3.1 DiagValue	141
6.43.3.2 DownNbr	141
6.43.3.3 Index	141
6.43.3.4 LeftNbr	141
6.43.3.5 RightNbr	141
6.43.3.6 SEdiagNbr	141
6.43.3.7 SubdivideLevel	142
6.43.3.8 UpNbr	142
6.43.3.9 Weight	142
6.44 RasheedLarsenMetric Class Reference	142
6.44.1 Detailed Description	144
6.44.2 Constructor & Destructor Documentation	144
6.44.2.1 RasheedLarsenMetric() [1/2]	144
6.44.2.2 RasheedLarsenMetric() [2/2]	144
6.44.3 Member Function Documentation	144
6.44.3.1 getFullDescriptionStr()	144
6.44.3.2 getMetric_dd()	144
6.44.3.3 getMetric_uu()	145
6.44.4 Member Data Documentation	145
6.44.4.1 m_aParam	145
6.44.4.2 m_mParam	145
6.44.4.3 m_pParam	145
6.44.4.4 m_qParam	146
6.45 RotatingBosonStarMetric Class Reference	146
6.45.1 Constructor & Destructor Documentation	147
6.45.1.1 RotatingBosonStarMetric()	147
6.45.1.2 ~RotatingBosonStarMetric()	147
6.45.2 Member Function Documentation	148
6.45.2.1 getFullDescriptionStr()	148
6.45.2.2 getMetric_dd()	148
6.45.2.3 getMetric_uu()	148
6.45.3 Member Data Documentation	149
6.45.3.1 m_fInterpolator	149
6.45.3.2 m_gInterpolator	149
6.45.3.3 m_grid_f	149

6.45.3.4 m_grid_g	149
6.45.3.5 m_grid_l	149
6.45.3.6 m_grid_Omega	149
6.45.3.7 m_lInterpolator	149
6.45.3.8 m_OmegaInterpolator	149
6.45.3.9 m_OmegaSign	149
6.46 SimpleSquareMesh Class Reference	150
6.46.1 Detailed Description	151
6.46.2 Constructor & Destructor Documentation	151
6.46.2.1 SimpleSquareMesh() [1/2]	151
6.46.2.2 SimpleSquareMesh() [2/2]	151
6.46.3 Member Function Documentation	151
6.46.3.1 EndCurrentLoop()	151
6.46.3.2 GeodesicFinished()	151
6.46.3.3 getCurNrGeodesics()	152
6.46.3.4 getFullDescriptionStr()	152
6.46.3.5 getNewInitConds()	152
6.46.3.6 IsFinished()	153
6.46.4 Member Data Documentation	153
6.46.4.1 m_Finished	153
6.46.4.2 m_RowColumnSize	153
6.46.4.3 m_TotalPixels	153
6.47 SingularityMetric Class Reference	153
6.47.1 Detailed Description	154
6.47.2 Constructor & Destructor Documentation	154
6.47.2.1 SingularityMetric()	154
6.47.3 Member Function Documentation	155
6.47.3.1 getSingularities()	155
6.47.4 Member Data Documentation	155
6.47.4.1 m_AllSingularities	155
6.48 Source Class Reference	155
6.48.1 Detailed Description	156
6.48.2 Constructor & Destructor Documentation	156
6.48.2.1 Source()	156
6.48.2.2 ~Source()	156
6.48.3 Member Function Documentation	156
6.48.3.1 getFullDescriptionStr()	156
6.48.3.2 getSource()	157
6.48.4 Member Data Documentation	157
6.48.4.1 m_theMetric	157
6.49 SphericalHorizonMetric Class Reference	157
6.49.1 Detailed Description	158

6.49.2 Constructor & Destructor Documentation	158
6.49.2.1 SphericalHorizonMetric() [1/2]	158
6.49.2.2 SphericalHorizonMetric() [2/2]	158
6.49.3 Member Function Documentation	159
6.49.3.1 getHorizonRadius()	159
6.49.4 Member Data Documentation	159
6.49.4.1 m_HorizonRadius	159
6.50 tk::spline Class Reference	159
6.50.1 Member Enumeration Documentation	160
6.50.1.1 bd_type	160
6.50.1.2 spline_type	160
6.50.2 Constructor & Destructor Documentation	161
6.50.2.1 spline() [1/2]	161
6.50.2.2 spline() [2/2]	161
6.50.3 Member Function Documentation	161
6.50.3.1 deriv()	161
6.50.3.2 find_closest()	161
6.50.3.3 get_x()	161
6.50.3.4 get_x_max()	161
6.50.3.5 get_x_min()	162
6.50.3.6 get_y()	162
6.50.3.7 make_monotonic()	162
6.50.3.8 operator()()	162
6.50.3.9 set_boundary()	162
6.50.3.10 set_coeffs_from_b()	162
6.50.3.11 set_points()	162
6.50.4 Member Data Documentation	162
6.50.4.1 m_b	162
6.50.4.2 m_c	163
6.50.4.3 m_c0	163
6.50.4.4 m_d	163
6.50.4.5 m_left	163
6.50.4.6 m_left_value	163
6.50.4.7 m_made_monotonic	163
6.50.4.8 m_right	163
6.50.4.9 m_right_value	163
6.50.4.10 m_type	163
6.50.4.11 m_x	163
6.50.4.12 m_y	164
6.51 SquareSubdivisionMesh Class Reference	164
6.51.1 Detailed Description	166
6.51.2 Constructor & Destructor Documentation	166

6.51.2.1 SquareSubdivisionMesh() [1/2]	166
6.51.2.2 SquareSubdivisionMesh() [2/2]	166
6.51.3 Member Function Documentation	166
6.51.3.1 EndCurrentLoop()	166
6.51.3.2 Explnt()	166
6.51.3.3 GeodesicFinished()	167
6.51.3.4 getCurNrGeodesics()	167
6.51.3.5 getFullDescriptionStr()	167
6.51.3.6 getNewInitConds()	167
6.51.3.7 InitializeFirstGrid()	168
6.51.3.8 IsFinished()	168
6.51.3.9 SubdivideAndQueue()	168
6.51.3.10 UpdateAllNeighbors()	168
6.51.3.11 UpdateAllWeights()	169
6.51.4 Member Data Documentation	169
6.51.4.1 m_AllPixels	169
6.51.4.2 m_CurrentPixelQueue	169
6.51.4.3 m_CurrentPixelQueueDone	169
6.51.4.4 m_InfinitePixels	169
6.51.4.5 m_InitialPixels	169
6.51.4.6 m_InitialSubDivideToFinal	170
6.51.4.7 m_IterationPixels	170
6.51.4.8 m_MaxPixels	170
6.51.4.9 m_MaxSubdivide	170
6.51.4.10 m_PixelsLeft	170
6.51.4.11 m_RowColumnSize	170
6.52 SquareSubdivisionMeshV2 Class Reference	171
6.52.1 Detailed Description	173
6.52.2 Constructor & Destructor Documentation	173
6.52.2.1 SquareSubdivisionMeshV2() [1/2]	173
6.52.2.2 SquareSubdivisionMeshV2() [2/2]	173
6.52.3 Member Function Documentation	173
6.52.3.1 EndCurrentLoop()	173
6.52.3.2 Explnt()	173
6.52.3.3 GeodesicFinished()	174
6.52.3.4 getCurNrGeodesics()	174
6.52.3.5 GetDown()	174
6.52.3.6 getFullDescriptionStr()	175
6.52.3.7 GetLeft()	175
6.52.3.8 getNewInitConds()	175
6.52.3.9 GetRight()	175
6.52.3.10 GetUp()	176

6.52.3.11 InitializeFirstGrid()	176
6.52.3.12 IsFinished()	176
6.52.3.13 SubdivideAndQueue()	176
6.52.3.14 UpdateAllWeights()	177
6.52.4 Member Data Documentation	177
6.52.4.1 m_ActivePixels	177
6.52.4.2 m_AllPixels	177
6.52.4.3 m_CurrentPixelQueue	177
6.52.4.4 m_CurrentPixelQueueDone	177
6.52.4.5 m_CurrentPixelUpdating	178
6.52.4.6 m_InfinitePixels	178
6.52.4.7 m_InitialPixels	178
6.52.4.8 m_InitialSubDivideToFinal	178
6.52.4.9 m_IterationPixels	178
6.52.4.10 m_MaxPixels	178
6.52.4.11 m_MaxSubdivide	178
6.52.4.12 m_PixelsIntegrated	178
6.52.4.13 m_PixelsLeft	179
6.52.4.14 m_RowColumnSize	179
6.53 ST3CrMetric Class Reference	179
6.53.1 Detailed Description	180
6.53.2 Constructor & Destructor Documentation	181
6.53.2.1 ST3CrMetric()	181
6.53.3 Member Function Documentation	181
6.53.3.1 f_om_phi()	181
6.53.3.2 f_phi()	181
6.53.3.3 get_omega()	182
6.53.3.4 getFullDescriptionStr()	182
6.53.3.5 getMetric_dd()	182
6.53.3.6 getMetric_uu()	183
6.53.4 Member Data Documentation	183
6.53.4.1 m_lambda	183
6.53.4.2 m_P	183
6.53.4.3 m_q0	183
6.54 Termination Class Reference	184
6.54.1 Detailed Description	184
6.54.2 Constructor & Destructor Documentation	184
6.54.2.1 Termination() [1/2]	184
6.54.2.2 Termination() [2/2]	185
6.54.2.3 ~Termination()	185
6.54.3 Member Function Documentation	185
6.54.3.1 CheckTermination()	185

6.54.3.2 DecideUpdate()	185
6.54.3.3 getFullDescriptionStr()	185
6.54.3.4 Reset()	186
6.54.4 Member Data Documentation	186
6.54.4.1 m_OwnerGeodesic	186
6.54.4.2 m_StepsSinceUpdated	186
6.55 TerminationOptions Struct Reference	186
6.55.1 Detailed Description	187
6.55.2 Constructor & Destructor Documentation	187
6.55.2.1 TerminationOptions()	187
6.55.2.2 ~TerminationOptions()	187
6.55.3 Member Data Documentation	187
6.55.3.1 UpdateEveryNSteps	187
6.56 ThetaSingularityTermination Class Reference	187
6.56.1 Detailed Description	188
6.56.2 Constructor & Destructor Documentation	188
6.56.2.1 ThetaSingularityTermination()	188
6.56.3 Member Function Documentation	189
6.56.3.1 CheckTermination()	189
6.56.3.2 getFullDescriptionStr()	189
6.56.4 Member Data Documentation	189
6.56.4.1 TermOptions	189
6.57 ThetaSingularityTermOptions Struct Reference	189
6.57.1 Detailed Description	190
6.57.2 Constructor & Destructor Documentation	190
6.57.2.1 ThetaSingularityTermOptions()	190
6.57.3 Member Data Documentation	190
6.57.3.1 ThetaSingEpsilon	190
6.58 TimeOutTermination Class Reference	191
6.58.1 Detailed Description	192
6.58.2 Constructor & Destructor Documentation	192
6.58.2.1 TimeOutTermination()	192
6.58.3 Member Function Documentation	192
6.58.3.1 CheckTermination()	192
6.58.3.2 getFullDescriptionStr()	192
6.58.3.3 Reset()	193
6.58.4 Member Data Documentation	193
6.58.4.1 m_CurNrSteps	193
6.58.4.2 TermOptions	193
6.59 TimeOutTermOptions Struct Reference	193
6.59.1 Detailed Description	194
6.59.2 Constructor & Destructor Documentation	194

6.59.2.1 TimeOutTermOptions()	194
6.59.3 Member Data Documentation	194
6.59.3.1 MaxSteps	194
6.60 Utilities::Timer Class Reference	194
6.60.1 Detailed Description	195
6.60.2 Member Typedef Documentation	195
6.60.2.1 Clock	195
6.60.2.2 Second	195
6.60.3 Member Function Documentation	195
6.60.3.1 elapsed()	195
6.60.3.2 reset()	196
6.60.4 Member Data Documentation	196
6.60.4.1 m_beg	196
6.61 UpdateFrequency Struct Reference	196
6.61.1 Detailed Description	196
6.61.2 Member Data Documentation	196
6.61.2.1 UpdateFinish	196
6.61.2.2 UpdateNSteps	197
6.61.2.3 UpdateStart	197
6.62 ViewScreen Class Reference	197
6.62.1 Detailed Description	198
6.62.2 Constructor & Destructor Documentation	198
6.62.2.1 ViewScreen() [1/2]	198
6.62.2.2 ViewScreen() [2/2]	198
6.62.3 Member Function Documentation	199
6.62.3.1 ConstructVielbein()	199
6.62.3.2 EndCurrentLoop()	199
6.62.3.3 GeodesicFinished()	199
6.62.3.4 getCurNrGeodesics()	199
6.62.3.5 getFullDescriptionStr()	199
6.62.3.6 IsFinished()	200
6.62.3.7 SetNewInitialConditions()	200
6.62.4 Member Data Documentation	200
6.62.4.1 m_Direction	200
6.62.4.2 m_GeodType	200
6.62.4.3 m_Metric_dd	200
6.62.4.4 m_Pos	201
6.62.4.5 m_rLogScale	201
6.62.4.6 m_ScreenCenter	201
6.62.4.7 m_ScreenSize	201
6.62.4.8 m_theMesh	201
6.62.4.9 m_theMetric	201

6.62.4.10 m_Vielbein	201
7 File Documentation	203
7.1 FOORT/src/Config.cpp File Reference	203
7.1.1 Detailed Description	203
7.2 FOORT/src/Config.h File Reference	203
7.2.1 Detailed Description	205
7.3 Config.h	205
7.4 FOORT/src/ConfigReader.cpp File Reference	206
7.4.1 Detailed Description	206
7.5 FOORT/src/ConfigReader.h File Reference	207
7.5.1 Detailed Description	207
7.6 ConfigReader.h	208
7.7 FOORT/src/Diagnostics.cpp File Reference	210
7.7.1 Detailed Description	210
7.7.2 Function Documentation	211
7.7.2.1 CreateDiagnosticVector()	211
7.8 FOORT/src/Diagnostics.h File Reference	212
7.8.1 Detailed Description	213
7.8.2 Typedef Documentation	213
7.8.2.1 DiagBitflag	213
7.8.2.2 DiagnosticUniqueVector	214
7.8.3 Function Documentation	214
7.8.3.1 CreateDiagnosticVector()	214
7.8.4 Variable Documentation	214
7.8.4.1 Diag_ClosestRadius	214
7.8.4.2 Diag_EquatorialEmission	214
7.8.4.3 Diag_EquatorialPasses	214
7.8.4.4 Diag_FourColorScreen	215
7.8.4.5 Diag_GeodesicPosition	215
7.8.4.6 Diag_None	215
7.9 Diagnostics.h	215
7.10 FOORT/src/DiagnosticsEmission.cpp File Reference	220
7.10.1 Detailed Description	220
7.11 FOORT/src/DiagnosticsEmission.h File Reference	221
7.11.1 Detailed Description	221
7.12 DiagnosticsEmission.h	222
7.13 FOORT/src/Geodesic.cpp File Reference	223
7.13.1 Detailed Description	224
7.14 FOORT/src/Geodesic.h File Reference	224
7.14.1 Detailed Description	224
7.15 Geodesic.h	225

7.16 FOORT/src/Geometry.h File Reference	226
7.16.1 Detailed Description	228
7.16.2 Macro Definition Documentation	228
7.16.2.1 LARGE_COUNTER_MAX	228
7.16.2.2 PIXEL_MAX	228
7.16.3 Typedef Documentation	228
7.16.3.1 FourIndex	228
7.16.3.2 largecounter	229
7.16.3.3 OneIndex	229
7.16.3.4 pixelcoord	229
7.16.3.5 Point	229
7.16.3.6 real	229
7.16.3.7 ScreenIndex	229
7.16.3.8 ScreenPoint	229
7.16.3.9 Singularity	229
7.16.3.10 SingularityCoord	230
7.16.3.11 ThreeIndex	230
7.16.3.12 TwoIndex	230
7.16.4 Function Documentation	230
7.16.4.1 operator*() [1/2]	230
7.16.4.2 operator*() [2/2]	230
7.16.4.3 operator+()	231
7.16.4.4 operator-()	231
7.16.4.5 operator/()	232
7.16.4.6 toString() [1/3]	232
7.16.4.7 toString() [2/3]	233
7.16.4.8 toString() [3/3]	233
7.16.5 Variable Documentation	234
7.16.5.1 dimension	234
7.16.5.2 pi	234
7.17 Geometry.h	234
7.18 FOORT/src/Grid.cpp File Reference	236
7.19 FOORT/src/Grid.h File Reference	236
7.20 Grid.h	236
7.21 FOORT/src/InputOutput.cpp File Reference	237
7.21.1 Detailed Description	237
7.21.2 Function Documentation	238
7.21.2.1 GetLoopMessageFrequency()	238
7.21.2.2 ScreenOutput()	238
7.21.2.3 SetLoopMessageFrequency()	238
7.21.2.4 SetOutputLevel()	238
7.22 FOORT/src/InputOutput.h File Reference	239

7.22.1 Detailed Description	240
7.22.2 Enumeration Type Documentation	240
7.22.2.1 OutputLevel	240
7.22.3 Function Documentation	240
7.22.3.1 GetLoopMessageFrequency()	240
7.22.3.2 ScreenOutput()	241
7.22.3.3 SetLoopMessageFrequency()	241
7.22.3.4 SetOutputLevel()	241
7.23 InputOutput.h	242
7.24 FOORT/src/Integrators.cpp File Reference	243
7.24.1 Detailed Description	243
7.25 FOORT/src/Integrators.h File Reference	244
7.25.1 Detailed Description	245
7.25.2 Typedef Documentation	245
7.25.2.1 GeodesicIntegratorFunc	245
7.26 Integrators.h	245
7.27 FOORT/src/Interpolator.cpp File Reference	246
7.28 FOORT/src/Interpolator.h File Reference	246
7.29 Interpolator.h	247
7.30 FOORT/src/Main.cpp File Reference	248
7.30.1 Detailed Description	248
7.30.2 Function Documentation	248
7.30.2.1 main()	248
7.31 FOORT/src/Mesh.cpp File Reference	249
7.31.1 Detailed Description	249
7.32 FOORT/src/Mesh.h File Reference	249
7.32.1 Detailed Description	250
7.33 Mesh.h	250
7.34 FOORT/src/Metric.cpp File Reference	255
7.35 FOORT/src/Metric.h File Reference	255
7.35.1 Detailed Description	256
7.36 Metric.h	256
7.37 FOORT/src/Spline.cpp File Reference	260
7.37.1 Detailed Description	260
7.38 FOORT/src/Spline.h File Reference	260
7.38.1 Detailed Description	261
7.39 Spline.h	261
7.40 FOORT/src/Terminations.cpp File Reference	263
7.40.1 Detailed Description	263
7.40.2 Function Documentation	264
7.40.2.1 CreateTerminationVector()	264
7.41 FOORT/src/Terminations.h File Reference	264

7.41.1 Detailed Description	266
7.41.2 Typedef Documentation	266
7.41.2.1 TermBitflag	266
7.41.2.2 TerminationUniqueVector	267
7.41.3 Enumeration Type Documentation	267
7.41.3.1 Term	267
7.41.4 Function Documentation	267
7.41.4.1 CreateTerminationVector()	267
7.41.5 Variable Documentation	268
7.41.5.1 Term_BoundarySphere	268
7.41.5.2 Term_GeneralSingularity	268
7.41.5.3 Term_Horizon	268
7.41.5.4 Term_NaN	268
7.41.5.5 Term_None	268
7.41.5.6 Term_ThetaSingularity	268
7.41.5.7 Term_TimeOut	268
7.42 Terminations.h	269
7.43 FOORT/src/Utilities.cpp File Reference	273
7.43.1 Detailed Description	273
7.44 FOORT/src/Utilities.h File Reference	273
7.44.1 Detailed Description	274
7.45 Utilities.h	275
7.46 FOORT/src/ViewScreen.cpp File Reference	275
7.46.1 Detailed Description	275
7.47 FOORT/src/ViewScreen.h File Reference	276
7.47.1 Detailed Description	276
7.47.2 Enumeration Type Documentation	277
7.47.2.1 GeodesicType	277
7.48 ViewScreen.h	277

Index	279
--------------	------------

Chapter 1

Namespace Index

1.1 Namespace List

Here is a list of all namespaces with brief descriptions:

Config	Namespace for all configuration functions that initialize objects based on configuration file . . .	11
ConfigReader	Namespace to put all ConfigReader objects	15
Integrators		16
tk		19
tk::internal		19
Utilities	Namespace that contains our utility functions	19

Chapter 2

Hierarchical Index

2.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

tk::internal::band_matrix	21
BicubicSplineInterpolator	24
ConfigReader::ConfigCollection	38
ConfigReader::ConfigCollection::ConfigSetting	50
Diagnostic	50
ClosestRadiusDiagnostic	34
EquatorialPassesDiagnostic	62
EquatorialEmissionDiagnostic	56
FourColorScreenDiagnostic	71
GeodesicPositionDiagnostic	95
DiagnosticOptions	53
ClosestRadiusOptions	37
EquatorialPassesOptions	66
EquatorialEmissionOptions	60
GeodesicPositionOptions	99
EmissionModel	54
GLMJohnsonSUEmission	100
FluidVelocityModel	69
GeneralCircularRadialFluid	74
Geodesic	84
GeodesicOutputHandler	89
Grid	102
Mesh	127
InputCertainPixelsMesh	108
SimpleSquareMesh	150
SquareSubdivisionMesh	164
SquareSubdivisionMeshV2	171
Metric	129
BosonStarMetric	26
FlatSpaceMetric	67
RotatingBosonStarMetric	146
SingularityMetric	153
ST3CrMetric	179

SphericalHorizonMetric	157
JohannsenMetric	112
KerrMetric	117
KerrSchildMetric	120
MankoNovikovMetric	123
RasheedLarsenMetric	142
SquareSubdivisionMesh::PixelInfo	138
SquareSubdivisionMeshV2::PixelInfo	140
std::runtime_error	
ConfigReader::ConfigReaderException	49
Source	155
NoSource	136
tk::spline	159
Termination	184
BoundarySphereTermination	30
GeneralSingularityTermination	80
HorizonTermination	104
NaNTermination	133
ThetaSingularityTermination	187
TimeOutTermination	191
TerminationOptions	186
BoundarySphereTermOptions	32
GeneralSingularityTermOptions	82
HorizonTermOptions	106
NaNTermOptions	135
ThetaSingularityTermOptions	189
TimeOutTermOptions	193
Utilities::Timer	194
UpdateFrequency	196
ViewScreen	197

Chapter 3

Class Index

3.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

tk::internal::band_matrix	21
BicubicSplineInterpolator	24
BosonStarMetric	
Boson star metric with solitonic potential	26
BoundarySphereTermination	
BoundarySphereTermination : Terminate geodesics if they reach outside of a boundary sphere	30
BoundarySphereTermOptions	
Options for BoundarySphereTermination	32
ClosestRadiusDiagnostic	
Diagnostic that keeps track of the closest (true, not log) radius that the geodesic passes through	34
ClosestRadiusOptions	
Options for ClosestRadiusDiagnostic	37
ConfigReader::ConfigCollection	38
ConfigReader::ConfigReaderException	49
ConfigReader::ConfigCollection::ConfigSetting	50
Diagnostic	
Abstract base class for all diagnostics	50
DiagnosticOptions	
Base class for DiagnosticOptions . Other Diagnostics can inherit from here if they require more options	53
EmissionModel	
Emmision model abstract base class	54
EquatorialEmissionDiagnostic	
Diagnostic that calculates brightness intensity for the geodesic, based on a specified equatorial disc emission model	56
EquatorialEmissionOptions	
Options for EquatorialEmissionDiagnostic	60
EquatorialPassesDiagnostic	
Diagnostic that counts the number of passes through the equatorial plane	62
EquatorialPassesOptions	
Options for EquatorialPassesDiagnostic	66
FlatSpaceMetric	
Flat space metric in 4D	67
FluidVelocityModel	
Fluid velocity model abstract base class	69

FourColorScreenDiagnostic	
The four color screen diagnostic	71
GeneralCircularRadialFluid	
General circular radial fluid velocity model	74
GeneralSingularityTermination	
GeneralSingularityTermination : Terminate geodesics if they get too close to one of a number of singularities (of arbitrary codimension)	80
GeneralSingularityTermOptions	
Options for GeneralSingularityTermination	82
Geodesic	
Geodesic class: an instance of this class is created for each Geodesic that is integrated	84
GeodesicOutputHandler	
// GeodesicOutputHandler handles all of the output to file. It gets passed all of the output strings for every Geodesic , it then stores this data until it eventually writes all data to the appropriate files	89
GeodesicPositionDiagnostic	
Geodesic position tracker	95
GeodesicPositionOptions	
Options for GeodesicPositionDiagnostic	99
GLMJohnsonSUEmission	
The Johnson SU emission model used in GLM 2020, PRD 102(12):124004	100
Grid	
A class for a 2D grid that contains the values of the necessary functions	102
HorizonTermination	
HorizonTermination : Terminate geodesics if they get too close to the horizon	104
HorizonTermOptions	
Options for HorizonTermination	106
InputCertainPixelsMesh	
Mesh which integrates only certain user-inputted pixels	108
JohannsenMetric	
Johannsen black hole metric	112
KerrMetric	
The Kerr metric	117
KerrSchildMetric	
Kerr metric in Kerr-Schild coordinates	120
MankoNovikovMetric	
Mano-Novikov metric (with angular momentum and M3 parameter turned on)	123
Mesh	
Abstract base Mesh class	127
Metric	
The abstract base class for all Metrics	129
NaNTermination	
NaNTermination : Terminate geodesics if they contain a NaN in position or velocity	133
NaNTermOptions	
Options for NaNTermination	135
NoSource	
NoSource class	136
SquareSubdivisionMesh::PixelInfo	
A struct the Mesh uses to keep all information about a given pixel	138
SquareSubdivisionMeshV2::PixelInfo	
A struct the Mesh uses to keep all information about a given pixel	140
RasheedLarsenMetric	
Rasheed-Larsen black hole	142
RotatingBosonStarMetric	
.	146
SimpleSquareMesh	
A simple square mesh that will integrate a square of evenly spaced pixels	150
SingularityMetric	
Abstract base class for a metric with an arbitrary number of singularities (of arbitrary codimension)	153

Source	
Abstract Source base class	155
SphericalHorizonMetric	
Abstract base class for a metric with a spherical horizon (i.e. horizon at constant radius r)	157
tk::spline	159
SquareSubdivisionMesh	
Adaptive subdivision Mesh	164
SquareSubdivisionMeshV2	
Adaptive subdivision Mesh	171
ST3CrMetric	
Ring fuzzball metric	179
Termination	
Abstract base class for all Terminations	184
TerminationOptions	
Base class for all TerminationOptions structs. Other TerminationOptions can inherit from here if they require more options	186
ThetaSingularityTermination	
ThetaSingularityTermination : Terminate geodesics if they get too close to a polar coordinate singularity	187
ThetaSingularityTermOptions	
Options for ThetaSingularityTermination	189
TimeOutTermination	
TimeOutTermination : Terminate geodesics if they take too many steps	191
TimeOutTermOptions	
Options for TimeOutTermination	193
Utilities::Timer	
Timer class to keep track of elapsed time	194
UpdateFrequency	
Struct to hold the information for a Diagnostic to update itself	196
ViewScreen	
This class is in charge of converting a pixel on the screen into physical initial conditions for a geodesic	197

Chapter 4

File Index

4.1 File List

Here is a list of all files with brief descriptions:

FOORT/src/ Config.cpp	203
FOORT/src/ Config.h	
Functions that read the configuration file and initialize all objects	203
FOORT/src/ ConfigReader.cpp	206
FOORT/src/ ConfigReader.h	
Classes used to read in a configuration file and store the settings	207
FOORT/src/ Diagnostics.cpp	210
FOORT/src/ Diagnostics.h	
Declarations of abstract base Diagnostic class and all its descendants. All Definitions in Diagnostics.cpp	212
FOORT/src/ DiagnosticsEmission.cpp	220
FOORT/src/ DiagnosticsEmission.h	
Declarations of emission and fluid velocity models used for equatorial disc emission	221
FOORT/src/ Geodesic.cpp	223
FOORT/src/ Geodesic.h	
Declarations of abstract base Source class and all its descendants. Declaration of Geodesic class	224
FOORT/src/ Geometry.h	
Definitions and some operations with geometric objects.alignas	226
FOORT/src/ Grid.cpp	236
FOORT/src/ Grid.h	236
FOORT/src/ InputOutput.cpp	237
FOORT/src/ InputOutput.h	
Declarations of functions & classes that deal with the output to files and to the screen	239
FOORT/src/ Integrators.cpp	243
FOORT/src/ Integrators.h	
Declarations of integrator functions that integrate geodesic equation	244
FOORT/src/ Interpolator.cpp	246
FOORT/src/ Interpolator.h	246
FOORT/src/ Main.cpp	
Main function for FOORT	248
FOORT/src/ Mesh.cpp	249
FOORT/src/ Mesh.h	
Declarations of abstract base Mesh class and all its descendants	249
FOORT/src/ Metric.cpp	255

FOORT/src/ Metric.h	
Declarations of abstract base Metric class and all its descendants	255
FOORT/src/ Spline.cpp	
Simple cubic spline interpolation library without external dependencies	260
FOORT/src/ Spline.h	
Simple cubic spline interpolation library without external dependencies	260
FOORT/src/ Terminations.cpp	263
FOORT/src/ Terminations.h	
Declarations of abstract base Termination class and all its descendants	264
FOORT/src/ Utilities.cpp	273
FOORT/src/ Utilities.h	
Declarations of various utility functions, in a Utilities namespace	273
FOORT/src/ ViewScreen.cpp	275
FOORT/src/ ViewScreen.h	
Declarations of ViewScreen class	276

Chapter 5

Namespace Documentation

5.1 Config Namespace Reference

Namespace for all configuration functions that initialize objects based on configuration file.

Typedefs

- using [ConfigCollection](#) = [ConfigReader::ConfigCollection](#)
- using [SettingError](#) = `std::invalid_argument`

Functions

- void [InitializeScreenOutput](#) (const [ConfigCollection](#) &theCfg)
Initialize screen output options.
- std::unique_ptr< [Metric](#) > [GetMetric](#) (const [ConfigCollection](#) &theCfg)
Use configuration to create the correct [Metric](#) with specified parameters.
- std::unique_ptr< [Source](#) > [GetSource](#) (const [ConfigCollection](#) &theCfg, const [Metric](#) *const theMetric)
Use configuration to create the correct [Source](#) with specified parameters.
- void [InitializeDiagnostics](#) (const [ConfigCollection](#) &theCfg, [DiagBitflag](#) &alldiags, [DiagBitflag](#) &valdiag, const [Metric](#) *const theMetric)
Use configuration to set the [Diagnostics](#) bitflag appropriately; initialize all [DiagnosticOptions](#) for all [Diagnostics](#) that are turned on; and set bitflag for diagnostic to be used for coarseness evaluating in [Mesh](#).
- void [InitializeTerminations](#) (const [ConfigCollection](#) &theCfg, [TermBitflag](#) &allterms, const [Metric](#) *const theMetric)
Use configuration to set the [Termination](#) bitflag appropriately; and initialize all [TerminationOptions](#) for all [Terminations](#) that are turned on.
- std::unique_ptr< [ViewScreen](#) > [GetViewScreen](#) (const [ConfigCollection](#) &theCfg, [DiagBitflag](#) valdiag, const [Metric](#) *const theMetric)
Use configuration to create the [ViewScreen](#) object with specified parameters.
- std::unique_ptr< [Mesh](#) > [GetMesh](#) (const [ConfigCollection](#) &theCfg, [DiagBitflag](#) valdiag)
Use configuration to create the [Mesh](#) object with specified parameters.
- [GeodesicIntegratorFunc](#) [GetGeodesicIntegrator](#) (const [ConfigCollection](#) &theCfg)
Use configuration to set the [GeodesicIntegratorFunc](#) pointer to the correct integrator function.
- std::unique_ptr< [GeodesicOutputHandler](#) > [GetOutputHandler](#) (const [ConfigCollection](#) &theCfg, [DiagBitflag](#) alldiags, [DiagBitflag](#) valdiag, std::string FirstLineInfo)
Use configuration to create the [GeodesicOutputHandler](#) object with specified parameters.

Variables

- constexpr auto [Output_Important_Default](#) = [OutputLevel::Level_0_WARNING](#)
- constexpr auto [Output_Other_Default](#) = [OutputLevel::Level_1_PROC](#)

5.1.1 Detailed Description

Namespace for all configuration functions that initialize objects based on configuration file.

5.1.2 Typedef Documentation

5.1.2.1 ConfigCollection

```
using Config::ConfigCollection = ConfigReader::ConfigCollection
```

5.1.2.2 SettingError

```
using Config::SettingError = std::invalid_argument
```

5.1.3 Function Documentation

5.1.3.1 GetGeodesicIntegrator()

```
GeodesicIntegratorFunc Config::GetGeodesicIntegrator (  
    const ConfigCollection & theCfg)
```

Use configuration to set the [GeodesicIntegratorFunc](#) pointer to the correct integrator function.

Parameters

<i>theCfg</i>	Pointer to the configuration object
---------------	-------------------------------------

Returns

[GeodesicIntegratorFunc](#) function

5.1.3.2 GetMesh()

```
std::unique_ptr< Mesh > Config::GetMesh (  
    const ConfigCollection & theCfg,  
    DiagBitflag valdiag)
```

Use configuration to create the [Mesh](#) object with specified parameters.

Parameters

<i>theCfg</i>	Pointer to the configuration object
<i>valdiag</i>	Diagnostic bitflag used for coarseness evaluation in the Mesh

Returns

Pointer to the [Mesh](#) object

5.1.3.3 GetMetric()

```
std::unique_ptr< Metric > Config::GetMetric (
    const ConfigCollection & theCfg)
```

Use configuration to create the correct [Metric](#) with specified parameters.

Parameters

<i>theCfg</i>	Pointer to the configuration object
---------------	-------------------------------------

Returns

[Metric](#) pointer

5.1.3.4 GetOutputHandler()

```
std::unique_ptr< GeodesicOutputHandler > Config::GetOutputHandler (
    const ConfigCollection & theCfg,
    DiagBitflag alldiags,
    DiagBitflag valdiag,
    std::string FirstLineInfo)
```

Use configuration to create the [GeodesicOutputHandler](#) object with specified parameters.

Parameters

<i>theCfg</i>	Pointer to the configuration object
<i>alldiags</i>	Diagnostics bitflags
<i>valdiag</i>	Diagnostic bitflag used for coarseness evaluation in the Mesh
<i>FirstLineInfo</i>	String to be written as the first line in the output file

Returns

Pointer to the [GeodesicOutputHandler](#) object

5.1.3.5 GetSource()

```
std::unique_ptr< Source > Config::GetSource (
    const ConfigCollection & theCfg,
    const Metric *const theMetric)
```

Use configuration to create the correct [Source](#) with specified parameters.

Parameters

<i>theCfg</i>	Pointer to the configuration object
<i>theMetric</i>	Pointer to the Metric object

Returns

Pointer to the [Source](#) object

5.1.3.6 GetViewScreen()

```
std::unique_ptr< ViewScreen > Config::GetViewScreen (
    const ConfigCollection & theCfg,
    DiagBitflag valdiag,
    const Metric *const theMetric)
```

Use configuration to create the [ViewScreen](#) object with specified parameters.

Parameters

<i>theCfg</i>	Pointer to the configuration object
<i>valdiag</i>	Diagnostic bitflag used for coarseness evaluation in the Mesh
<i>theMetric</i>	Pointer to the Metric object

Returns

Pointer to the [ViewScreen](#) object

5.1.3.7 InitializeDiagnostics()

```
void Config::InitializeDiagnostics (
    const ConfigCollection & theCfg,
    DiagBitflag & alldiags,
    DiagBitflag & valdiag,
    const Metric *const theMetric)
```

Use configuration to set the Diagnostics bitflag appropriately; initialize all [DiagnosticOptions](#) for all Diagnostics that are turned on; and set bitflag for diagnostic to be used for coarseness evaluating in [Mesh](#).

Parameters

<i>theCfg</i>	Pointer to the configuration object
<i>alldiags</i>	Pointer to the diagnostics bitflags
<i>valdiag</i>	Pointer to the diagnostics used for coarseness evaluation in the Mesh
<i>theMetric</i>	Pointer to the Metric object

5.1.3.8 InitializeScreenOutput()

```
void Config::InitializeScreenOutput (
    const ConfigCollection & theCfg)
```

Initialize screen output options.

Parameters

<i>theCfg</i>	Pointer to the configuration object
---------------	-------------------------------------

5.1.3.9 InitializeTerminations()

```
void Config::InitializeTerminations (
    const ConfigCollection & theCfg,
    TermBitflag & allterms,
    const Metric *const theMetric)
```

Use configuration to set the [Termination](#) bitflag appropriately; and initialize all [TerminationOptions](#) for all Terminations that are turned on.

Parameters

<i>theCfg</i>	Pointer to the configuration object
<i>allterms</i>	Pointer to the termination bitflags
<i>theMetric</i>	Pointer to the Metric object

5.1.4 Variable Documentation

5.1.4.1 Output_Important_Default

```
auto Config::Output_Important_Default = OutputLevel::Level_0_WARNING [constexpr]
```

5.1.4.2 Output_Other_Default

```
auto Config::Output_Other_Default = OutputLevel::Level_1_PROC [constexpr]
```

5.2 ConfigReader Namespace Reference

Namespace to put all [ConfigReader](#) objects.

Classes

- class [ConfigCollection](#)
- class [ConfigReaderException](#)

Variables

- const long long [MaxStreamSize](#) {std::numeric_limits<std::streamsize>::max() }

5.2.1 Detailed Description

Namespace to put all [ConfigReader](#) objects.

5.2.2 Variable Documentation

5.2.2.1 MaxStreamSize

```
const long long ConfigReader::MaxStreamSize {std::numeric_limits<std::streamsize>::max() }
```

5.3 Integrators Namespace Reference

Functions

- `std::string GetFullIntegratorDescription ()`
Description string getter for the Integrator.
- `real GetAdaptiveStep (Point curpos, OneIndex curvel)`
Determine the (affine parameter) step size to take.
- `void IntegrateGeodesicStep_RK4 (Point curpos, OneIndex curvel, Point &nextpos, OneIndex &nextvel, real &stepsize, const Metric *theMetric, const Source *theSource)`
Integrate the geodesic equation by one step using Runge-Kutta-4.
- `void IntegrateGeodesicStep_Verlet (Point curpos, OneIndex curvel, Point &nextpos, OneIndex &nextvel, real &stepsize, const Metric *theMetric, const Source *theSource)`
Integrate the geodesic equation by one step using the Verlet algorithm.

Variables

- `constexpr real delta_nodiv0 = 1e-20`
This is used to avoid dividing by zero.
- `real Derivative_hval {1e-7}`
The spacing used in stencils for the calculation of numerical derivatives (e.g. in Christoffel symbols)
- `std::string IntegratorDescription {"RK4"}`
The name of the integrator selected.
- `real epsilon {0.03}`
This is the base step size to be taken (the integrator will adapt this if necessary)
- `real SmallestPossibleStepsize {1e-12}`
The affine parameter must always go forward by at least this amount.
- `real VerletVelocityTolerance {0.001}`
Tolerance on the change in velocity in the iteration loop of the Verlet integration algorithm.

5.3.1 Function Documentation

5.3.1.1 GetAdaptiveStep()

```
real Integrators::GetAdaptiveStep (
    Point curpos,
    OneIndex curvel)
```

Determine the (affine parameter) step size to take.

algorithm as in Raptor (eqs (21)-(24)), which is taken from Noble et al. (2007) & Dolence et al. (2009)

Parameters

<i>curpos</i>	current position
<i>curvel</i>	current velocity

Returns

real

5.3.1.2 GetFullIntegratorDescription()

```
std::string Integrators::GetFullIntegratorDescription ()
```

Description string getter for the Integrator.

Returns

std::string

5.3.1.3 IntegrateGeodesicStep_RK4()

```
void Integrators::IntegrateGeodesicStep_RK4 (
    Point curpos,
    OneIndex curvel,
    Point & nextpos,
    OneIndex & nextvel,
    real & stepsize,
    const Metric * theMetric,
    const Source * theSource)
```

Integrate the geodesic equation by one step using Runge-Kutta-4.

Parameters

<i>curpos</i>	current position
<i>curvel</i>	current velocity
<i>nextpos</i>	reference to next position
<i>nextvel</i>	reference to next velocity
<i>stepsize</i>	reference to the stepsize
<i>theMetric</i>	pointer to the Metric object
<i>theSource</i>	pointer to the Source object

5.3.1.4 IntegrateGeodesicStep_Verlet()

```
void Integrators::IntegrateGeodesicStep_Verlet (
    Point curpos,
    OneIndex curvel,
    Point & nextpos,
    OneIndex & nextvel,
    real & stepsize,
    const Metric * theMetric,
    const Source * theSource)
```

Integrate the geodesic equation by one step using the Verlet algorithm.

Parameters

<i>curpos</i>	current position
<i>curvel</i>	current velocity
<i>nextpos</i>	reference to next position
<i>nextvel</i>	reference to next velocity
<i>stepsize</i>	reference to the stepsize
<i>theMetric</i>	pointer to the Metric object
<i>theSource</i>	pointer to the Source object

5.3.2 Variable Documentation**5.3.2.1 `delta_nodiv0`**

```
real Integrators::delta_nodiv0 = 1e-20 [constexpr]
```

This is used to avoid dividing by zero.

5.3.2.2 `Derivative_hval`

```
real Integrators::Derivative_hval {1e-7} [inline]
```

The spacing used in stencils for the calculation of numerical derivatives (e.g. in Christoffel symbols)

5.3.2.3 `epsilon`

```
real Integrators::epsilon {0.03} [inline]
```

This is the base step size to be taken (the integrator will adapt this if necessary)

5.3.2.4 `IntegratorDescription`

```
std::string Integrators::IntegratorDescription {"RK4"} [inline]
```

The name of the integrator selected.

5.3.2.5 `SmallestPossibleStepsize`

```
real Integrators::SmallestPossibleStepsize {1e-12} [inline]
```

The affine parameter must always go forward by at least this amount.

5.3.2.6 VerletVelocityTolerance

```
real Integrators::VerletVelocityTolerance {0.001} [inline]
```

Tolerance on the change in velocity in the iteration loop of the Verlet integration algorithm.

5.4 tk Namespace Reference

Namespaces

- namespace [internal](#)

Classes

- class [spline](#)

5.5 tk::internal Namespace Reference

Classes

- class [band_matrix](#)

5.6 Utilities Namespace Reference

Namespace that contains our utility functions.

Classes

- class [Timer](#)
[Timer](#) class to keep track of elapsed time.

Functions

- `std::string GetTimeStampString ()`
Get a string of the current time (in a format that can be used to append to file names)
- `std::vector< std::string > GetDiagNameStrings (DiagBitflag alldiags, DiagBitflag valdiag)`
Helper function to get all [Diagnostic](#) Names (for outputting to files)
- `std::string GetFirstLineInfoString (const Metric *theMetric, const Source *theSource, DiagBitflag alldiags, DiagBitflag valdiag, TermBitflag allterms, const ViewScreen *theView)`
This returns the full string to be written to every output file as its first line.

5.6.1 Detailed Description

Namespace that contains our utility functions.

5.6.2 Function Documentation

5.6.2.1 GetDiagNameStrings()

```
std::vector< std::string > Utilities::GetDiagNameStrings (
    DiagBitflag alldiags,
    DiagBitflag valdiag)
```

Helper function to get all [Diagnostic](#) Names (for outputting to files)

Parameters

<i>alldiags</i>	bitflag for all Diagnostics
<i>valdiag</i>	bitflag for the value diagnostic

Returns

`std::vector<std::string>`

5.6.2.2 GetFirstLineInfoString()

```
std::string Utilities::GetFirstLineInfoString (
    const Metric * theMetric,
    const Source * theSource,
    DiagBitflag alldiags,
    DiagBitflag valdiag,
    TermBitflag allterms,
    const ViewScreen * theView)
```

This returns the full string to be written to every output file as its first line.

Parameters

<i>theMetric</i>	pointer to the Metric object
<i>theSource</i>	pointer to the Source object
<i>alldiags</i>	bitflag for all Diagnostics
<i>valdiag</i>	bitflag for the value diagnostic
<i>allterms</i>	bitflag for all Terminations
<i>theView</i>	pointer to the ViewScreen object

Returns

`std::string`

5.6.2.3 GetTimeStampString()

```
std::string Utilities::GetTimeStampString ()
```

Get a string of the current time (in a format that can be used to append to file names)

Returns

`std::string`

Chapter 6

Class Documentation

6.1 tk::internal::band_matrix Class Reference

```
#include <Spline.h>
```

Public Member Functions

- [band_matrix](#) ()
- [band_matrix](#) (int [dim](#), int [n_u](#), int [n_l](#))
- [~band_matrix](#) ()
- void [resize](#) (int [dim](#), int [n_u](#), int [n_l](#))
- int [dim](#) () const
- int [num_upper](#) () const
- int [num_lower](#) () const
- double & [operator\(\)](#) (int [i](#), int [j](#))
- double [operator\(\)](#) (int [i](#), int [j](#)) const
- double & [saved_diag](#) (int [i](#))
- double [saved_diag](#) (int [i](#)) const
- void [lu_decompose](#) ()
- std::vector< double > [r_solve](#) (const std::vector< double > &[b](#)) const
- std::vector< double > [l_solve](#) (const std::vector< double > &[b](#)) const
- std::vector< double > [lu_solve](#) (const std::vector< double > &[b](#), bool [is_lu_decomposed](#)=false)

Private Attributes

- std::vector< std::vector< double > > [m_upper](#)
- std::vector< std::vector< double > > [m_lower](#)

6.1.1 Constructor & Destructor Documentation

6.1.1.1 band_matrix() [1/2]

```
tk::internal::band_matrix::band_matrix () [inline]
```

6.1.1.2 band_matrix() [2/2]

```
tk::internal::band_matrix::band_matrix (
    int dim,
    int n_u,
    int n_l)
```

6.1.1.3 ~band_matrix()

```
tk::internal::band_matrix::~~band_matrix () [inline]
```

6.1.2 Member Function Documentation

6.1.2.1 dim()

```
int tk::internal::band_matrix::dim () const
```

6.1.2.2 l_solve()

```
std::vector< double > tk::internal::band_matrix::l_solve (
    const std::vector< double > & b) const
```

6.1.2.3 lu_decompose()

```
void tk::internal::band_matrix::lu_decompose ()
```

6.1.2.4 lu_solve()

```
std::vector< double > tk::internal::band_matrix::lu_solve (
    const std::vector< double > & b,
    bool is_lu_decomposed = false)
```

6.1.2.5 num_lower()

```
int tk::internal::band_matrix::num_lower () const
```

6.1.2.6 num_upper()

```
int tk::internal::band_matrix::num_upper () const
```

6.1.2.7 operator()() [1/2]

```
double & tk::internal::band_matrix::operator() (
    int i,
    int j)
```

6.1.2.8 operator>() [2/2]

```
double tk::internal::band_matrix::operator() (
    int i,
    int j) const
```

6.1.2.9 r_solve()

```
std::vector< double > tk::internal::band_matrix::r_solve (
    const std::vector< double > & b) const
```

6.1.2.10 resize()

```
void tk::internal::band_matrix::resize (
    int dim,
    int n_u,
    int n_l)
```

6.1.2.11 saved_diag() [1/2]

```
double & tk::internal::band_matrix::saved_diag (
    int i)
```

6.1.2.12 saved_diag() [2/2]

```
double tk::internal::band_matrix::saved_diag (
    int i) const
```

6.1.3 Member Data Documentation

6.1.3.1 m_lower

```
std::vector<std::vector<double> > tk::internal::band_matrix::m_lower [private]
```

6.1.3.2 m_upper

```
std::vector<std::vector<double> > tk::internal::band_matrix::m_upper [private]
```

The documentation for this class was generated from the following files:

- FOORT/src/[Spline.h](#)
- FOORT/src/[Spline.cpp](#)

6.2 BicubicSplineInterpolator Class Reference

```
#include <Interpolator.h>
```

Public Member Functions

- [BicubicSplineInterpolator](#) (const std::string x_file="RotatingBosonStar/x.txt", const std::string th_file="RotatingBosonStar/theta.txt")
- void [set_grid](#) (const [Grid](#) *grid)
- double [interpolate](#) (double p_x, double p_theta, bool allow_extrapolation=false) const

Private Member Functions

- void [compute_spline_coefficients](#) (const [Grid](#) *grid)
- double [estimate_dx](#) (const [Grid](#) *grid, size_t j, size_t i) const
- double [estimate_dy](#) (const [Grid](#) *grid, size_t j, size_t i) const
- double [estimate_dxdy](#) (const [Grid](#) *grid, size_t j, size_t i) const
- void [compute_cell_coefficients](#) (std::array< double, 16 > &coeffs, double f00, double f10, double f01, double f11, double fx00, double fx10, double fx01, double fx11, double fy00, double fy10, double fy01, double fy11, double fxy00, double fxy10, double fxy01, double fxy11) const

Private Attributes

- std::vector< double > [m_x](#)
- std::vector< double > [m_theta](#)
- std::vector< std::array< double, 16 > > [m_coeffs](#)

6.2.1 Constructor & Destructor Documentation

6.2.1.1 BicubicSplineInterpolator()

```
BicubicSplineInterpolator::BicubicSplineInterpolator (
    const std::string x_file = "RotatingBosonStar/x.txt",
    const std::string th_file = "RotatingBosonStar/theta.txt") [inline]
```

6.2.2 Member Function Documentation

6.2.2.1 compute_cell_coefficients()

```
void BicubicSplineInterpolator::compute_cell_coefficients (
    std::array< double, 16 > & coeffs,
    double f00,
    double f10,
    double f01,
    double f11,
    double fx00,
    double fx10,
    double fx01,
    double fx11,
    double fy00,
    double fy10,
    double fy01,
    double fy11,
    double fxy00,
    double fxy10,
    double fxy01,
    double fxy11) const [private]
```

6.2.2.2 compute_spline_coefficients()

```
void BicubicSplineInterpolator::compute_spline_coefficients (
    const Grid * grid) [private]
```

6.2.2.3 estimate_dx()

```
double BicubicSplineInterpolator::estimate_dx (
    const Grid * grid,
    size_t j,
    size_t i) const [private]
```

6.2.2.4 estimate_dxdy()

```
double BicubicSplineInterpolator::estimate_dxdy (
    const Grid * grid,
    size_t j,
    size_t i) const [private]
```

6.2.2.5 estimate_dy()

```
double BicubicSplineInterpolator::estimate_dy (
    const Grid * grid,
    size_t j,
    size_t i) const [private]
```

6.2.2.6 interpolate()

```
double BicubicSplineInterpolator::interpolate (
    double p_x,
    double p_theta,
    bool allow_extrapolation = false) const
```

6.2.2.7 set_grid()

```
void BicubicSplineInterpolator::set_grid (
    const Grid * grid) [inline]
```

6.2.3 Member Data Documentation

6.2.3.1 m_coeffs

```
std::vector<std::array<double, 16> > BicubicSplineInterpolator::m_coeffs [private]
```

6.2.3.2 m_theta

```
std::vector<double> BicubicSplineInterpolator::m_theta [private]
```

6.2.3.3 m_x

```
std::vector<double> BicubicSplineInterpolator::m_x [private]
```

The documentation for this class was generated from the following files:

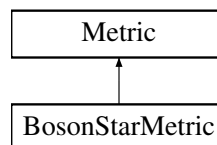
- FOORT/src/[Interpolator.h](#)
- FOORT/src/[Interpolator.cpp](#)

6.3 BosonStarMetric Class Reference

Boson star metric with solitonic potential.

```
#include <Metric.h>
```

Inheritance diagram for BosonStarMetric:



Public Member Functions

- [BosonStarMetric](#) (double Phi_infinity, int num_lines, bool rLogScale=false, std::string Phi_filename="Phi.dat", std::string m_filename="m.dat")
Construct a new Boson Star [Metric](#) object.
- [TwoIndex getMetric_dd](#) (const [Point](#) &p) const final
BosonStar metric getter, indices down.
- [TwoIndex getMetric_uu](#) (const [Point](#) &p) const final
BosonStar metric getter, indices up.
- std::string [getFullDescriptionStr](#) () const final
BosonStar metric description string getter.

Public Member Functions inherited from [Metric](#)

- virtual [~Metric](#) ()=default
- [Metric](#) (bool rlogscale=false)
Construct a new [Metric](#) object.
- virtual [ThreeIndex getChristoffel_udd](#) (const [Point](#) &p) const
Return the Christoffel symbols of the metric (indices up, down, down)
- virtual [FourIndex getRiemann_uddd](#) (const [Point](#) &p) const
Riemann tensor of the metric (indices up, down, down, down)
- virtual [real getKretschmann](#) (const [Point](#) &p) const
Get the Kretschmann scalar at a point.
- bool [getrLogScale](#) () const
Get whether we are using a logarithmic radial scale.

Protected Member Functions

- void `read_data` ()
function to read the data

Protected Attributes

- const double `m_Phi_infinity`
The value of Phi at infinity, before rescaling.
- const int `m_num_lines`
- std::string `m_Phi_filename`
filename with Phi data
- std::string `m_m_filename`
filename with m data
- tk::spline `m_PhiSpline`
The spline interpolator for Phi.
- tk::spline `m_mSpline`
The spline interpolator for m.

Protected Attributes inherited from `Metric`

- std::vector< int > `m_Symmetries` {}
The symmetries (coordinate Killing vectors) of the metric. Should be set by descendant constructor.
- const bool `m_rLogScale`
Are we using a logarithmic r coordinate?

6.3.1 Detailed Description

Boson star metric with solitonic potential.

Note

Implementation by Seppe Staelens

The default files correspond to $\sigma = 0.06$ and $\phi_c = 0.044$

6.3.2 Constructor & Destructor Documentation

6.3.2.1 BosonStarMetric()

```
BosonStarMetric::BosonStarMetric (
    double Phi_infinity,
    int num_lines,
    bool rLogScale = false,
    std::string Phi_filename = "Phi.dat",
    std::string m_filename = "m.dat")
```

Construct a new Boson Star `Metric` object.

Parameters

<i>Phi_infinity</i>	value of Phi = log(lapse) at infinity
<i>num_lines</i>	number of lines in the data files
<i>rLogScale</i>	whether we are using a logarithmic radial scale
<i>Phi_filename</i>	filename for the Phi data
<i>m_filename</i>	filename for the mass aspect data

6.3.3 Member Function Documentation

6.3.3.1 getFullDescriptionStr()

```
std::string BosonStarMetric::getFullDescriptionStr () const [final], [virtual]
```

BosonStar metric description string getter.

Returns

std::string

Reimplemented from [Metric](#).

6.3.3.2 getMetric_dd()

```
TwoIndex BosonStarMetric::getMetric_dd (
    const Point & p) const [final], [virtual]
```

BosonStar metric getter, indices down.

Parameters

<i>p</i>	Point at which to evaluate the metric
----------	---

Returns

[TwoIndex](#)

Implements [Metric](#).

6.3.3.3 getMetric_uu()

```
TwoIndex BosonStarMetric::getMetric_uu (
    const Point & p) const [final], [virtual]
```

BosonStar metric getter, indices up.

Parameters

p	Point at which to evaluate the metric
-----	---------------------------------------

Returns

[TwoIndex](#)Implements [Metric](#).**6.3.3.4 read_data()**

```
void BosonStarMetric::read_data () [protected]
```

function to read the data

Read the numerically obtained boson star metric data from the files.

6.3.4 Member Data Documentation**6.3.4.1 m_m_filename**

```
std::string BosonStarMetric::m_m_filename [protected]
```

filename with m data

6.3.4.2 m_mSpline

```
tk::spline BosonStarMetric::m_mSpline [protected]
```

The spline interpolator for m.

6.3.4.3 m_num_lines

```
const int BosonStarMetric::m_num_lines [protected]
```

Number of lines to be read from the data files. Mainly to avoid reading the data at extreme distances, which could upset the spline interpolation.

6.3.4.4 m_Phi_filename

```
std::string BosonStarMetric::m_Phi_filename [protected]
```

filename with Phi data

6.3.4.5 m_Phi_infinity

```
const double BosonStarMetric::m_Phi_infinity [protected]
```

The value of Phi at infinity, before rescaling.

6.3.4.6 m_PhiSpline

```
tk::spline BosonStarMetric::m_PhiSpline [protected]
```

The spline interpolator for Phi.

The documentation for this class was generated from the following files:

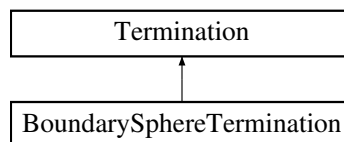
- FOORT/src/[Metric.h](#)
- FOORT/src/[Metric.cpp](#)

6.4 BoundarySphereTermination Class Reference

[BoundarySphereTermination](#): Terminate geodesics if they reach outside of a boundary sphere.

```
#include <Terminations.h>
```

Inheritance diagram for BoundarySphereTermination:



Public Member Functions

- [BoundarySphereTermination](#) ([Geodesic](#) *const theGeodesic)
- [Term CheckTermination](#) () final
Check if the boundary sphere has been reached.
- std::string [getFullDescriptionStr](#) () const final
Description string getter for the Boundary Sphere [Termination](#).

Public Member Functions inherited from [Termination](#)

- [Termination](#) ()=delete
- [Termination](#) ([Geodesic](#) *const theGeodesic)
- virtual void [Reset](#) ()
Reset to 0 to start integrating a new geodesic.
- virtual [~Termination](#) ()=default

Static Public Attributes

- static std::unique_ptr< [BoundarySphereTermOptions](#) > [TermOptions](#)
The options that the [BoundarySphereTermination](#) keeps (contains the radius of the boundary sphere)

Additional Inherited Members

Protected Member Functions inherited from [Termination](#)

- bool [DecideUpdate](#) ([largecounter](#) UpdateNSteps)
Returns true if the [Termination](#) should update its internal status.

Protected Attributes inherited from [Termination](#)

- [Geodesic](#) *const [m_OwnerGeodesic](#)
The geodesic that owns the [Termination](#) (a const pointer to the [Geodesic](#))
- [largecounter](#) [m_StepsSinceUpdated](#) {}
The termination is itself in charge of keeping track of how many steps it has been since it has been updated. The [Termination](#)'s [TerminationOptions](#) struct tells it how many steps it needs to wait between updates.

6.4.1 Detailed Description

[BoundarySphereTermination](#): Terminate geodesics if they reach outside of a boundary sphere.

6.4.2 Constructor & Destructor Documentation

6.4.2.1 BoundarySphereTermination()

```
BoundarySphereTermination::BoundarySphereTermination (
    Geodesic *const theGeodesic) [inline]
```

6.4.3 Member Function Documentation

6.4.3.1 CheckTermination()

```
Term BoundarySphereTermination::CheckTermination () [final], [virtual]
```

Check if the boundary sphere has been reached.

Returns

[Term](#)

Implements [Termination](#).

6.4.3.2 getFullDescriptionStr()

```
std::string BoundarySphereTermination::getFullDescriptionStr () const [final], [virtual]
```

Description string getter for the Boundary Sphere [Termination](#).

Returns

std::string

Implements [Termination](#).

6.4.4 Member Data Documentation

6.4.4.1 TermOptions

```
std::unique_ptr< BoundarySphereTermOptions > BoundarySphereTermination::TermOptions [static]
```

The options that the [BoundarySphereTermination](#) keeps (contains the radius of the boundary sphere)

The documentation for this class was generated from the following files:

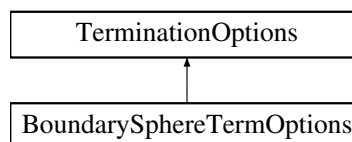
- FOORT/src/[Terminations.h](#)
- FOORT/src/[Config.cpp](#)
- FOORT/src/[Terminations.cpp](#)

6.5 BoundarySphereTermOptions Struct Reference

Options for [BoundarySphereTermination](#).

```
#include <Terminations.h>
```

Inheritance diagram for BoundarySphereTermOptions:



Public Member Functions

- [BoundarySphereTermOptions](#) ([real](#) theRadius, bool therLogScale, [largecounter](#) Nsteps)

Public Member Functions inherited from [TerminationOptions](#)

- [TerminationOptions](#) ([largecounter](#) Nsteps)
- virtual [~TerminationOptions](#) ()=default

Public Attributes

- const [real](#) [SphereRadius](#)
Radius of the boundary sphere.
- const [bool](#) [rLogScale](#)
Whether we are using a logarithmic r coordinate.

Public Attributes inherited from [TerminationOptions](#)

- const [largecounter](#) [UpdateEveryNSteps](#)
Number of steps between updates.

6.5.1 Detailed Description

Options for [BoundarySphereTermination](#).

Keeps track of the boundary sphere radius, whether we are using a logarithmic r coordinate.

6.5.2 Constructor & Destructor Documentation

6.5.2.1 [BoundarySphereTermOptions\(\)](#)

```
BoundarySphereTermOptions::BoundarySphereTermOptions (  
    real theRadius,  
    bool theLogScale,  
    largecounter Nsteps) [inline]
```

6.5.3 Member Data Documentation

6.5.3.1 [rLogScale](#)

```
const bool BoundarySphereTermOptions::rLogScale
```

Whether we are using a logarithmic r coordinate.

6.5.3.2 [SphereRadius](#)

```
const real BoundarySphereTermOptions::SphereRadius
```

Radius of the boundary sphere.

The documentation for this struct was generated from the following file:

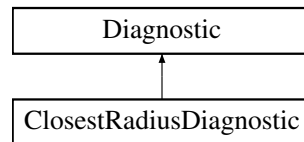
- FOORT/src/[Terminations.h](#)

6.6 ClosestRadiusDiagnostic Class Reference

[Diagnostic](#) that keeps track of the closest (true, not log) radius that the geodesic passes through.

```
#include <Diagnostics.h>
```

Inheritance diagram for ClosestRadiusDiagnostic:



Public Member Functions

- [ClosestRadiusDiagnostic](#) ([Geodesic](#) *const theGeodesic)
- void [Reset](#) () final
ClosestRadiusDiagnostic functions.
- void [UpdateData](#) () final
Check to see if we have travelled closer to the center and update the closest radius.
- std::string [getFullDataStr](#) () const final
Return the closest radius as a string.
- std::vector< [real](#) > [getFinalDataVal](#) () const final
Return the closest radius as a vector of size one.
- [real](#) [FinalDataValDistance](#) (const std::vector< [real](#) > &val1, const std::vector< [real](#) > &val2) const final
Return the absolute value of the difference between the closest radii.
- std::string [getNameStr](#) () const final
Simple name without spaces.
- std::string [getFullDescriptionStr](#) () const final
More descriptive string (with spaces)

Public Member Functions inherited from [Diagnostic](#)

- [Diagnostic](#) ()=delete
- [Diagnostic](#) ([Geodesic](#) *const theGeodesic)
- virtual [~Diagnostic](#) ()=default

Static Public Attributes

- static std::unique_ptr< [ClosestRadiusOptions](#) > [DiagOptions](#)

Private Attributes

- [real](#) [m_ClosestRadius](#) {-1}

Additional Inherited Members

Protected Member Functions inherited from [Diagnostic](#)

- bool [DecideUpdate](#) (const [UpdateFrequency](#) &myUpdateFrequency)
Returns true if the [Diagnostic](#) should update its initial status.

Protected Attributes inherited from [Diagnostic](#)

- [Geodesic](#) *const [m_OwnerGeodesic](#)
- [largecounter](#) [m_StepsSinceUpdated](#) {}

6.6.1 Detailed Description

[Diagnostic](#) that keeps track of the closest (true, not log) radius that the geodesic passes through.

6.6.2 Constructor & Destructor Documentation

6.6.2.1 ClosestRadiusDiagnostic()

```
ClosestRadiusDiagnostic::ClosestRadiusDiagnostic (
    Geodesic *const theGeodesic) [inline]
```

6.6.3 Member Function Documentation

6.6.3.1 FinalDataValDistance()

```
real ClosestRadiusDiagnostic::FinalDataValDistance (
    const std::vector< real > & val1,
    const std::vector< real > & val2) const [final], [virtual]
```

Return the absolute value of the difference between the closest radii.

Parameters

<i>val1</i>	first closest radius
<i>val2</i>	second closest radius

Returns

[real](#)

Implements [Diagnostic](#).

6.6.3.2 getFinalDataVal()

```
std::vector< real > ClosestRadiusDiagnostic::getFinalDataVal () const [final], [virtual]
```

Return the closest radius as a vector of size one.

Returns

std::vector<real>

Implements [Diagnostic](#).

6.6.3.3 getFullDataStr()

```
std::string ClosestRadiusDiagnostic::getFullDataStr () const [final], [virtual]
```

Return the closest radius as a string.

Returns

std::string

Implements [Diagnostic](#).

6.6.3.4 getFullDescriptionStr()

```
std::string ClosestRadiusDiagnostic::getFullDescriptionStr () const [final], [virtual]
```

More descriptive string (with spaces)

Returns

std::string

Reimplemented from [Diagnostic](#).

6.6.3.5 getNameStr()

```
std::string ClosestRadiusDiagnostic::getNameStr () const [final], [virtual]
```

Simple name without spaces.

Returns

std::string

Implements [Diagnostic](#).

6.6.3.6 Reset()

```
void ClosestRadiusDiagnostic::Reset () [final], [virtual]
```

[ClosestRadiusDiagnostic](#) functions.

Reset the closest radius to -1 and call base class Reset function

Reimplemented from [Diagnostic](#).

6.6.3.7 UpdateData()

```
void ClosestRadiusDiagnostic::UpdateData () [final], [virtual]
```

Check to see if we have travelled closer to the center and update the closest radius.

Implements [Diagnostic](#).

6.6.4 Member Data Documentation

6.6.4.1 DiagOptions

```
std::unique_ptr< ClosestRadiusOptions > ClosestRadiusDiagnostic::DiagOptions [static]
```

6.6.4.2 m_ClosestRadius

```
real ClosestRadiusDiagnostic::m_ClosestRadius {-1} [private]
```

The documentation for this class was generated from the following files:

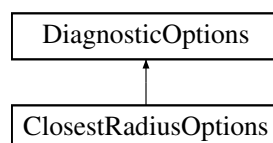
- FOORT/src/[Diagnostics.h](#)
- FOORT/src/[Config.cpp](#)
- FOORT/src/[Diagnostics.cpp](#)

6.7 ClosestRadiusOptions Struct Reference

Options for [ClosestRadiusDiagnostic](#).

```
#include <Diagnostics.h>
```

Inheritance diagram for ClosestRadiusOptions:



Public Member Functions

- [ClosestRadiusOptions](#) (bool rlog, [UpdateFrequency](#) thefrequency)

Public Member Functions inherited from [DiagnosticOptions](#)

- [DiagnosticOptions](#) ([UpdateFrequency](#) thefrequency)
- virtual [~DiagnosticOptions](#) ()=default

Public Attributes

- const bool [RLogScale](#)

Public Attributes inherited from [DiagnosticOptions](#)

- const [UpdateFrequency](#) theUpdateFrequency

6.7.1 Detailed Description

Options for [ClosestRadiusDiagnostic](#).

Inherits from [DiagnosticOptions](#), and adds the flag for using log(r) scale.

6.7.2 Constructor & Destructor Documentation

6.7.2.1 ClosestRadiusOptions()

```
ClosestRadiusOptions::ClosestRadiusOptions (
    bool rlog,
    UpdateFrequency thefrequency) [inline]
```

6.7.3 Member Data Documentation

6.7.3.1 RLogScale

```
const bool ClosestRadiusOptions::RLogScale
```

The documentation for this struct was generated from the following file:

- FOORT/src/[Diagnostics.h](#)

6.8 ConfigReader::ConfigCollection Class Reference

```
#include <ConfigReader.h>
```

Classes

- struct [ConfigSetting](#)

Public Member Functions

- bool [Exists](#) (std::string_view SettingName) const
Check if a setting with the given name exists in the collection.
- int [NrSettings](#) () const
Return the total number of settings in the collection.
- bool [IsCollection](#) (std::string_view SettingName) const
Check if a setting with the given name is a collection.
- bool [IsCollection](#) (int SettingIndex) const
Check if a setting with the given index is a collection.
- const [ConfigCollection](#) & [operator\[\]](#) (std::string_view CollectionName) const
Access a subcollection by name.
- const [ConfigCollection](#) & [operator\[\]](#) (int CollectionIndex) const
Access a subcollection by index.
- template<class OutputType>
bool [LookupValue](#) (std::string_view SettingName, OutputType &theOutput) const
Looks up the value of a setting with the given name and puts it in the output variable.
- template<class OutputType>
bool [LookupValue](#) (int SettingIndex, OutputType &theOutput) const
Looks up the value of a setting with the given index and puts it in the output variable.
- bool [LookupValueInteger](#) (std::string_view SettingName, int &theOutput) const
If the output is int, we only look up to see if the value is an int.
- bool [LookupValueInteger](#) (std::string_view SettingName, long &theOutput) const
If the output is long, the setting value can either be a long or an int, so we check for both.
- bool [LookupValueInteger](#) (std::string_view SettingName, long long &theOutput) const
If the output is long long, the setting value can either be a long long, a long, or an int, so we check for all.
- bool [LookupValueInteger](#) (std::string_view SettingName, unsigned int &theOutput) const
If the output is unsigned int, the setting value can either be an unsigned int, an int, a long, or a long long, so we check for all.
- bool [LookupValueInteger](#) (std::string_view SettingName, unsigned long &theOutput) const
If the output is unsigned long, the setting value can either be an unsigned long, an unsigned int, an int, a long, or a long long, so we check for all.
- bool [LookupValueInteger](#) (std::string_view SettingName, unsigned long long &theOutput) const
If the output is unsigned long long, the setting value can either be an unsigned long long, an unsigned long, an unsigned int, an int, a long, or a long long, so we check for all.
- template<class OutputType>
bool [LookupValueInteger](#) (int SettingIndex, OutputType &theOutput) const
This simply defers to the setting-name-based implementation of the function with given output type.
- void [DisplayCollection](#) (std::ostream &OutputStream, int Indent=0) const
Output the entire collection to a given output stream.
- bool [ReadFile](#) (const std::string &FileName)
Read in a configuration file.

Private Types

- using [ConfigSettingValue](#)

Private Member Functions

- int [GetSettingIndex](#) (std::string_view SettingName) const
Return the index of a specific setting in the collection.
- void [DisplaySetting](#) (std::ostream &OutputStream, int SettingIndex, int Indent) const
Output a single setting to the output stream.
- void [DisplayTabs](#) (std::ostream &OutputStream, int NrTabs) const
Output a given number of tabs to the output stream.
- void [ReadCollection](#) (std::ifstream &InputFile)
Read in an entire collection (including subcollections). Pre: stream is positioned right after '{' (or at beginning of file). Post: stream is positioned right after '}' (or at EOF).
- void [ReadSettingName](#) (std::ifstream &InputFile, std::string &theName)
Read in a setting name. Pre: next non-whitespace character in stream is beginning of name. Post: stream has just read in all characters of name (but no more); returns "" if there is no more setting to read in the current collection, in this case '}' has been read in (for subcollections).
- void [ReadSettingSpecificChar](#) (std::ifstream &InputFile, char theChar) const
Read in one specific character only (not '/'). Pre: next non-whitespace character in stream is this character. Post: stream is just after character.
- void [ReadSettingValue](#) (std::ifstream &InputFile, [ConfigSettingValue](#) &theValue)
Read in setting value (could be subcollection). Pre: next non-whitespace character in stream is beginning of value (can be '{' for subcollection, digit or '-' or '.' or 'e' for number, "" for string, or 't'/'f' for boolean). Post: Value has entirely been read in (i.e. next character should be ';').

Private Attributes

- std::vector< [ConfigSetting](#) > [m_Settings](#) {}

6.8.1 Member Typedef Documentation

6.8.1.1 ConfigSettingValue

```
using ConfigReader::ConfigCollection::ConfigSettingValue [private]
```

Initial value:

```
std::variant<bool,
              int, long, long long,
              double,
              std::string,
              std::unique_ptr<ConfigCollection>>
```

6.8.2 Member Function Documentation

6.8.2.1 DisplayCollection()

```
void ConfigCollection::DisplayCollection (
    std::ostream & OutputStream,
    int Indent = 0) const
```

Output the entire collection to a given output stream.

Parameters

<i>OutputStream</i>	pointer to an outputstream
<i>Indent</i>	indentation level

6.8.2.2 DisplaySetting()

```
void ConfigCollection::DisplaySetting (
    std::ostream & OutputStream,
    int SettingIndex,
    int Indent) const [private]
```

Output a single setting to the output stream.

Parameters

<i>OutputStream</i>	pointer to an outputstream
<i>SettingIndex</i>	index of the setting to output
<i>Indent</i>	indentation level

6.8.2.3 DisplayTabs()

```
void ConfigCollection::DisplayTabs (
    std::ostream & OutputStream,
    int NrTabs) const [private]
```

Output a given number of tabs to the output stream.

Parameters

<i>OutputStream</i>	pointer to an outputstream
<i>NrTabs</i>	number of tabs to output

6.8.2.4 Exists()

```
bool ConfigCollection::Exists (
    std::string_view SettingName) const
```

Check if a setting with the given name exists in the collection.

Parameters

<i>SettingName</i>	The name of the setting to check
--------------------	----------------------------------

Returns

true
false

6.8.2.5 GetSettingIndex()

```
int ConfigCollection::GetSettingIndex (
    std::string_view SettingName) const [private]
```

Return the index of a specific setting in the collection.

Parameters

<i>SettingName</i>	The name of the setting to find
--------------------	---------------------------------

Returns

int

6.8.2.6 IsCollection() [1/2]

```
bool ConfigCollection::IsCollection (
    int SettingIndex) const
```

Check if a setting with the given index is a collection.

Parameters

<i>SettingIndex</i>	The index of the setting to check
---------------------	-----------------------------------

Returns

true

false

6.8.2.7 IsCollection() [2/2]

```
bool ConfigCollection::IsCollection (
    std::string_view SettingName) const
```

Check if a setting with the given name is a collection.

Parameters

<i>SettingName</i>	The name of the setting to check
--------------------	----------------------------------

Returns

true

false

6.8.2.8 LookupValue() [1/2]

```
template<class OutputType>
bool ConfigReader::ConfigCollection::LookupValue (
    int SettingIndex,
    OutputType & theOutput) const
```

Looks up the value of a setting with the given index and puts it in the output variable.

Template Parameters

<i>OutputType</i>	
-------------------	--

Parameters

<i>SettingIndex</i>	Setting index
<i>theOutput</i>	Pointer to the output variable

Returns

true
false

6.8.2.9 LookupValue() [2/2]

```
template<class OutputType>
bool ConfigReader::ConfigCollection::LookupValue (
    std::string_view SettingName,
    OutputType & theOutput) const
```

Looks up the value of a setting with the given name and puts it in the output variable.

Template Parameters

<i>OutputType</i>	
-------------------	--

Parameters

<i>SettingName</i>	Setting name
<i>theOutput</i>	Pointer to the output variable

Returns

true
false

6.8.2.10 LookupValueInteger() [1/7]

```
template<class OutputType>
bool ConfigReader::ConfigCollection::LookupValueInteger (
    int SettingIndex,
    OutputType & theOutput) const
```

This simply defers to the setting-name-based implementation of the function with given output type.

Template Parameters

<i>OutputType</i>	
-------------------	--

Parameters

<i>SettingIndex</i>	Setting index
<i>theOutput</i>	Pointer to the output variable

Returns

true
false

6.8.2.11 LookupValueInteger() [2/7]

```
bool ConfigCollection::LookupValueInteger (
    std::string_view SettingName,
    int & theOutput) const
```

If the output is int, we only look up to see if the value is an int.

Parameters

<i>SettingName</i>	Name of the setting to look up
<i>theOutput</i>	integer ouput

Returns

true
false

6.8.2.12 LookupValueInteger() [3/7]

```
bool ConfigCollection::LookupValueInteger (
    std::string_view SettingName,
    long & theOutput) const
```

If the output is long, the setting value can either be a long or an int, so we check for both.

Parameters

<i>SettingName</i>	Name of the setting to look up
<i>theOutput</i>	long output

Returns

true
false

6.8.2.13 LookupValueInteger() [4/7]

```
bool ConfigCollection::LookupValueInteger (
    std::string_view SettingName,
    long long & theOutput) const
```

If the output is long long, the setting value can either be a long long, a long, or an int, so we check for all.

Parameters

<i>SettingName</i>	Name of the setting to look up
<i>theOutput</i>	long long output

Returns

true
false

6.8.2.14 LookupValueInteger() [5/7]

```
bool ConfigCollection::LookupValueInteger (
    std::string_view SettingName,
    unsigned int & theOutput) const
```

If the output is unsigned int, the setting value can either be an unsigned int, an int, a long, or a long long, so we check for all.

Parameters

<i>SettingName</i>	Name of the setting to look up
<i>theOutput</i>	unsigned int output

Returns

true
false

6.8.2.15 LookupValueInteger() [6/7]

```
bool ConfigCollection::LookupValueInteger (
    std::string_view SettingName,
    unsigned long & theOutput) const
```

If the output is unsigned long, the setting value can either be an unsigned long, an unsigned int, an int, a long, or a long long, so we check for all.

Parameters

<i>SettingName</i>	Name of the setting to look up
<i>theOutput</i>	unsigned long output

Returns

true
false

6.8.2.16 LookupValueInteger() [7/7]

```
bool ConfigCollection::LookupValueInteger (
    std::string_view SettingName,
    unsigned long long & theOutput) const
```

If the output is unsigned long long, the setting value can either be an unsigned long long, an unsigned long, an unsigned int, an int, a long, or a long long, so we check for all.

Parameters

<i>SettingName</i>	Name of the setting to look up
<i>theOutput</i>	unsigned long long output

Returns

true
false

6.8.2.17 NrSettings()

```
int ConfigCollection::NrSettings () const
```

Return the total number of settings in the collection.

Returns

int

6.8.2.18 operator[]() [1/2]

```
const ConfigCollection & ConfigCollection::operator[] (
    int CollectionIndex) const
```

Access a subcollection by index.

Parameters

<i>CollectionIndex</i>	The index of the subcollection to access
------------------------	--

Returns

const [ConfigCollection](#)&

6.8.2.19 operator[]() [2/2]

```
const ConfigCollection & ConfigCollection::operator[] (
    std::string_view CollectionName) const
```

Access a subcollection by name.

Parameters

<i>CollectionName</i>	The name of the subcollection to access
-----------------------	---

Returns

const [ConfigCollection](#)&

6.8.2.20 ReadCollection()

```
void ConfigCollection::ReadCollection (
    std::ifstream & InputFile) [private]
```

Read in an entire collection (including subcollections). Pre: stream is positioned right after '{' (or at beginning of file). Post: stream is positioned right after '}' (or at EOF).

Parameters

<i>InputFile</i>	pointer to an input file stream
------------------	---------------------------------

6.8.2.21 ReadFile()

```
bool ConfigCollection::ReadFile (
    const std::string & FileName)
```

Read in a configuration file.

Parameters

<i>FileName</i>	File name of the configuration file
-----------------	-------------------------------------

Returns

true if succesful

false if the file does not exist, or if there is a parse/syntax error

6.8.2.22 ReadSettingName()

```
void ConfigCollection::ReadSettingName (
    std::ifstream & InputFile,
    std::string & theName) [private]
```

Read in a setting name. Pre: next non-whitespace character in stream is beginning of name. Post: stream has just read in all characters of name (but no more); returns "" if there is no more setting to read in the current collection, in this case '}' has been read in (for subcollections).

Parameters

<i>InputFile</i>	Pointer to the input file stream
<i>theName</i>	Pointer to the name of the setting

6.8.2.23 ReadSettingSpecificChar()

```
void ConfigCollection::ReadSettingSpecificChar (
    std::ifstream & InputFile,
    char theChar) const [private]
```

Read in one specific character only (not '/'). Pre: next non-whitespace character in stream is this character. Post: stream is just after character.

Parameters

<i>InputFile</i>	Pointer to the input file stream
<i>theChar</i>	The character to read in

6.8.2.24 ReadSettingValue()

```
void ConfigCollection::ReadSettingValue (
    std::ifstream & InputFile,
    ConfigSettingValue & theValue) [private]
```

Read in setting value (could be subcollection). Pre: next non-whitespace character in stream is beginning of value (can be '{' for subcollection, digit or '-' or '.' or 'e' for number, '"' for string, or 't'/'f' for boolean). Post: Value has entirely been read in (i.e. next character should be ';').

Parameters

<i>InputFile</i>	Pointer to the input file stream
<i>theValue</i>	Pointer to the setting value

6.8.3 Member Data Documentation**6.8.3.1 m_Settings**

```
std::vector<ConfigSetting> ConfigReader::ConfigCollection::m_Settings {} [private]
```

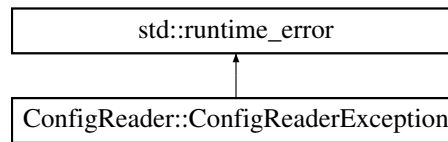
The documentation for this class was generated from the following files:

- FOORT/src/[ConfigReader.h](#)
- FOORT/src/[ConfigReader.cpp](#)

6.9 ConfigReader::ConfigReaderException Class Reference

```
#include <ConfigReader.h>
```

Inheritance diagram for ConfigReader::ConfigReaderException:



Public Member Functions

- [ConfigReaderException](#) (const std::string &error, std::vector< int > settingtrace={})
- std::vector< int > [trace](#) () const

Private Attributes

- std::vector< int > [m_settingtrace](#)

6.9.1 Constructor & Destructor Documentation

6.9.1.1 ConfigReaderException()

```
ConfigReader::ConfigReaderException::ConfigReaderException (
    const std::string & error,
    std::vector< int > settingtrace = {}) [inline]
```

6.9.2 Member Function Documentation

6.9.2.1 trace()

```
std::vector< int > ConfigReader::ConfigReaderException::trace () const [inline]
```

6.9.3 Member Data Documentation

6.9.3.1 m_settingtrace

```
std::vector<int> ConfigReader::ConfigReaderException::m_settingtrace [private]
```

The documentation for this class was generated from the following file:

- FOORT/src/[ConfigReader.h](#)

6.10 ConfigReader::ConfigCollection::ConfigSetting Struct Reference

Public Attributes

- `std::string` [SettingName](#)
- [ConfigSettingValue](#) `SettingValue`

6.10.1 Member Data Documentation

6.10.1.1 SettingName

`std::string` `ConfigReader::ConfigCollection::ConfigSetting::SettingName`

6.10.1.2 SettingValue

[ConfigSettingValue](#) `ConfigReader::ConfigCollection::ConfigSetting::SettingValue`

The documentation for this struct was generated from the following file:

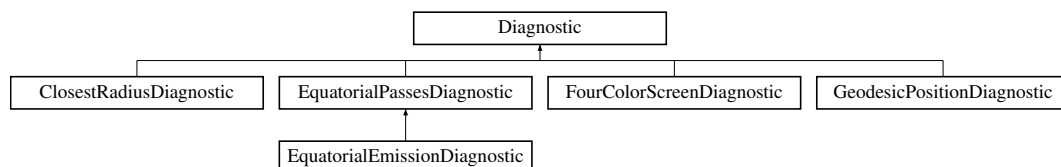
- FOORT/src/[ConfigReader.h](#)

6.11 Diagnostic Class Reference

Abstract base class for all diagnostics.

```
#include <Diagnostics.h>
```

Inheritance diagram for Diagnostic:



Public Member Functions

- [Diagnostic](#) ()=delete
 - [Diagnostic](#) ([Geodesic](#) *const theGeodesic)
 - virtual void [Reset](#) ()
 - *[Diagnostic](#) (abstract base class) functions.*
 - virtual [~Diagnostic](#) ()=default
 - virtual void [UpdateData](#) ()=0
 - virtual `std::string` [getFullDataStr](#) () const =0
 - virtual `std::vector< real >` [getFinalDataVal](#) () const =0
 - virtual `real` [FinalDataValDistance](#) (const `std::vector< real >` &val1, const `std::vector< real >` &val2) const =0
 - virtual `std::string` [getNameStr](#) () const =0
 - virtual `std::string` [getFullDescriptionStr](#) () const
- Base class just returns the short name of the [Diagnostic](#).*

Protected Member Functions

- bool [DecideUpdate](#) (const [UpdateFrequency](#) &myUpdateFrequency)
Returns true if the [Diagnostic](#) should update its initial status.

Protected Attributes

- [Geodesic](#) *const [m_OwnerGeodesic](#)
- [largecounter](#) [m_StepsSinceUpdated](#) {}

6.11.1 Detailed Description

Abstract base class for all diagnostics.

6.11.2 Constructor & Destructor Documentation

6.11.2.1 Diagnostic() [1/2]

```
Diagnostic::Diagnostic () [delete]
```

6.11.2.2 Diagnostic() [2/2]

```
Diagnostic::Diagnostic (
    Geodesic *const theGeodesic) [inline]
```

6.11.2.3 ~Diagnostic()

```
virtual Diagnostic::~~Diagnostic () [virtual], [default]
```

6.11.3 Member Function Documentation

6.11.3.1 DecideUpdate()

```
bool Diagnostic::DecideUpdate (
    const UpdateFrequency & myUpdateFrequency) [protected]
```

Returns true if the [Diagnostic](#) should update its initial status.

Should be called from within [UpdateData\(\)](#) with the appropriate [DiagnosticOptions::theUpdateFrequency](#).

Parameters

<i>myUpdateFrequency</i>	UpdateFrequency struct
--------------------------	--

Returns

true
false

6.11.3.2 FinalDataValDistance()

```
virtual real Diagnostic::FinalDataValDistance (
    const std::vector< real > & val1,
    const std::vector< real > & val2) const [pure virtual]
```

Implemented in [ClosestRadiusDiagnostic](#), [EquatorialEmissionDiagnostic](#), [EquatorialPassesDiagnostic](#), [FourColorScreenDiagnostic](#), and [GeodesicPositionDiagnostic](#).

6.11.3.3 getFinalDataVal()

```
virtual std::vector< real > Diagnostic::getFinalDataVal () const [pure virtual]
```

Implemented in [ClosestRadiusDiagnostic](#), [EquatorialEmissionDiagnostic](#), [EquatorialPassesDiagnostic](#), [FourColorScreenDiagnostic](#), and [GeodesicPositionDiagnostic](#).

6.11.3.4 getFullDataStr()

```
virtual std::string Diagnostic::getFullDataStr () const [pure virtual]
```

Implemented in [ClosestRadiusDiagnostic](#), [EquatorialEmissionDiagnostic](#), [EquatorialPassesDiagnostic](#), [FourColorScreenDiagnostic](#), and [GeodesicPositionDiagnostic](#).

6.11.3.5 getFullDescriptionStr()

```
std::string Diagnostic::getFullDescriptionStr () const [virtual]
```

Base class just returns the short name of the [Diagnostic](#).

This function is pure virtual in the base class.

Returns

`std::string`

Reimplemented in [ClosestRadiusDiagnostic](#), [EquatorialEmissionDiagnostic](#), [EquatorialPassesDiagnostic](#), [FourColorScreenDiagnostic](#), and [GeodesicPositionDiagnostic](#).

6.11.3.6 getNameStr()

```
virtual std::string Diagnostic::getNameStr () const [pure virtual]
```

Implemented in [ClosestRadiusDiagnostic](#), [EquatorialEmissionDiagnostic](#), [EquatorialPassesDiagnostic](#), [FourColorScreenDiagnostic](#), and [GeodesicPositionDiagnostic](#).

6.11.3.7 Reset()

```
void Diagnostic::Reset () [virtual]
```

[Diagnostic](#) (abstract base class) functions.

Reset [Diagnostic](#) object steps to zero.

Reimplemented in [ClosestRadiusDiagnostic](#), [EquatorialEmissionDiagnostic](#), [EquatorialPassesDiagnostic](#), [FourColorScreenDiagnostic](#), and [GeodesicPositionDiagnostic](#).

6.11.3.8 UpdateData()

```
virtual void Diagnostic::UpdateData () [pure virtual]
```

Implemented in [ClosestRadiusDiagnostic](#), [EquatorialEmissionDiagnostic](#), [EquatorialPassesDiagnostic](#), [FourColorScreenDiagnostic](#), and [GeodesicPositionDiagnostic](#).

6.11.4 Member Data Documentation

6.11.4.1 m_OwnerGeodesic

```
Geodesic* const Diagnostic::m_OwnerGeodesic [protected]
```

6.11.4.2 m_StepsSinceUpdated

```
largecounter Diagnostic::m_StepsSinceUpdated {} [protected]
```

The documentation for this class was generated from the following files:

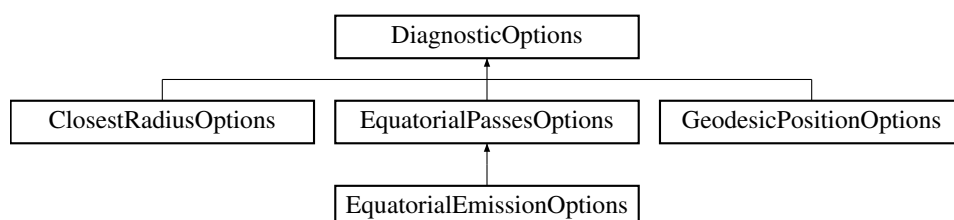
- [FOORT/src/Diagnostics.h](#)
- [FOORT/src/Diagnostics.cpp](#)

6.12 DiagnosticOptions Struct Reference

Base class for [DiagnosticOptions](#). Other Diagnostics can inherit from here if they require more options.

```
#include <Diagnostics.h>
```

Inheritance diagram for [DiagnosticOptions](#):



Public Member Functions

- [DiagnosticOptions](#) ([UpdateFrequency](#) thefrequency)
- virtual [~DiagnosticOptions](#) ()=default

Public Attributes

- const [UpdateFrequency](#) theUpdateFrequency

6.12.1 Detailed Description

Base class for [DiagnosticOptions](#). Other Diagnostics can inherit from here if they require more options.

6.12.2 Constructor & Destructor Documentation

6.12.2.1 DiagnosticOptions()

```
DiagnosticOptions::DiagnosticOptions (
    UpdateFrequency thefrequency) [inline]
```

6.12.2.2 ~DiagnosticOptions()

```
virtual DiagnosticOptions::~~DiagnosticOptions () [virtual], [default]
```

6.12.3 Member Data Documentation

6.12.3.1 theUpdateFrequency

```
const UpdateFrequency DiagnosticOptions::theUpdateFrequency
```

The documentation for this struct was generated from the following file:

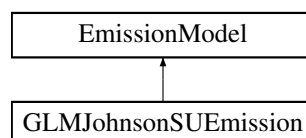
- FOORT/src/[Diagnostics.h](#)

6.13 EmissionModel Struct Reference

Emmision model abstract base class.

```
#include <DiagnosticsEmission.h>
```

Inheritance diagram for EmissionModel:



Public Member Functions

- virtual [~EmissionModel](#) ()=default
- virtual [real GetEmission](#) (const [Point](#) &p) const =0
- virtual std::string [getFullDescriptionStr](#) () const

Emission model functions.

6.13.1 Detailed Description

Emmision model abstract base class.

6.13.2 Constructor & Destructor Documentation

6.13.2.1 ~EmissionModel()

```
virtual EmissionModel::~~EmissionModel () [virtual], [default]
```

6.13.3 Member Function Documentation

6.13.3.1 GetEmission()

```
virtual real EmissionModel::GetEmission (
    const Point & p) const [pure virtual]
```

Implemented in [GLMJohnsonSUEmission](#).

6.13.3.2 getFullDescriptionStr()

```
std::string EmissionModel::getFullDescriptionStr () const [virtual]
```

Emission model functions.

Base class default string getter

Returns

std::string

Reimplemented in [GLMJohnsonSUEmission](#).

The documentation for this struct was generated from the following files:

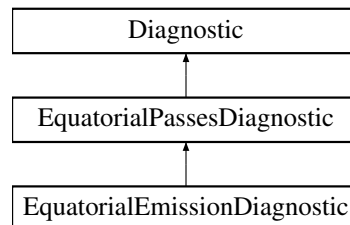
- FOORT/src/[DiagnosticsEmission.h](#)
- FOORT/src/[DiagnosticsEmission.cpp](#)

6.14 EquatorialEmissionDiagnostic Class Reference

[Diagnostic](#) that calculates brightness intensity for the geodesic, based on a specified equatorial disc emission model.

```
#include <Diagnostics.h>
```

Inheritance diagram for EquatorialEmissionDiagnostic:



Public Member Functions

- [EquatorialEmissionDiagnostic](#) ([Geodesic](#) *const theGeodesic)
- void [Reset](#) () final
EquatorialEmissionDiagnostic functions.
- void [UpdateData](#) () final
Update the intensity based on the emission model and the number of equatorial passes.
- std::string [getFullDataStr](#) () const final
Return the intensity and number of equatorial passes as a string.
- std::vector< [real](#) > [getFinalDataVal](#) () const final
Return the intensity and number of equatorial passes as a vector of size two.
- [real](#) [FinalDataValDistance](#) (const std::vector< [real](#) > &val1, const std::vector< [real](#) > &val2) const final
Return the distance between two geodesics based on their intensity and number of equatorial passes.
- std::string [getNameStr](#) () const final
Simple name without spaces.
- std::string [getFullDescriptionStr](#) () const final
More descriptive string (with spaces)

Public Member Functions inherited from [EquatorialPassesDiagnostic](#)

- [EquatorialPassesDiagnostic](#) ([Geodesic](#) *const theGeodesic)
- void [Reset](#) () override
EquatorialPassesDiagnostic functions.
- void [UpdateData](#) () override
Check to see if we have crossed the equatorial plane and update the number of passes.
- std::string [getFullDataStr](#) () const override
Return the number of equatorial passes as a string.
- std::vector< [real](#) > [getFinalDataVal](#) () const override
Return the number of equatorial passes as a vector of size one.
- [real](#) [FinalDataValDistance](#) (const std::vector< [real](#) > &val1, const std::vector< [real](#) > &val2) const override
Return the absolute value of the difference between the number of equatorial passes.
- std::string [getNameStr](#) () const override
Simple name without spaces.
- std::string [getFullDescriptionStr](#) () const override
More descriptive string (with spaces)

Public Member Functions inherited from [Diagnostic](#)

- [Diagnostic](#) ()=delete
- [Diagnostic](#) ([Geodesic](#) *const theGeodesic)
- virtual [~Diagnostic](#) ()=default

Static Public Attributes

- static std::unique_ptr< [EquatorialEmissionOptions](#) > [DiagOptions](#)

Static Public Attributes inherited from [EquatorialPassesDiagnostic](#)

- static std::unique_ptr< [EquatorialPassesOptions](#) > [DiagOptions](#)
Needs the extra option of a threshold.

Private Attributes

- [real](#) [m_Intensity](#) {0.0}

Additional Inherited Members

Protected Member Functions inherited from [Diagnostic](#)

- [bool](#) [DecideUpdate](#) (const [UpdateFrequency](#) &myUpdateFrequency)
Returns true if the [Diagnostic](#) should update its initial status.

Protected Attributes inherited from [EquatorialPassesDiagnostic](#)

- [int](#) [m_EquatPasses](#) {0}
Keeps track of how many passes have been made.

Protected Attributes inherited from [Diagnostic](#)

- [Geodesic](#) *const [m_OwnerGeodesic](#)
- [largecounter](#) [m_StepsSinceUpdated](#) {}

6.14.1 Detailed Description

[Diagnostic](#) that calculates brightness intensity for the geodesic, based on a specified equatorial disc emission model.

Inherits from [EquatorialPassesDiagnostic](#), since it needs to keep track of when it passes through the equatorial plane. As a result it also keeps track of the number of equatorial passes. Both the intensity profile and fluid velocity profile are specified in the options struct.

6.14.2 Constructor & Destructor Documentation

6.14.2.1 EquatorialEmissionDiagnostic()

```
EquatorialEmissionDiagnostic::EquatorialEmissionDiagnostic (
    Geodesic *const theGeodesic) [inline]
```

6.14.3 Member Function Documentation

6.14.3.1 FinalDataValDistance()

```
real EquatorialEmissionDiagnostic::FinalDataValDistance (
    const std::vector< real > & val1,
    const std::vector< real > & val2) const [final], [virtual]
```

Return the distance between two geodesics based on their intensity and number of equatorial passes.

Parameters

<i>val1</i>	first geodesic intensity and number of equatorial passes
<i>val2</i>	second geodesic intensity and number of equatorial passes

Returns

[real](#)

Implements [Diagnostic](#).

6.14.3.2 getFinalDataVal()

```
std::vector< real > EquatorialEmissionDiagnostic::getFinalDataVal () const [final], [virtual]
```

Return the intensity and number of equatorial passes as a vector of size two.

Returns

std::vector<real>

Implements [Diagnostic](#).

6.14.3.3 getFullDataStr()

```
std::string EquatorialEmissionDiagnostic::getFullDataStr () const [final], [virtual]
```

Return the intensity and number of equatorial passes as a string.

Returns

std::string

Implements [Diagnostic](#).

6.14.3.4 getFullDescriptionStr()

```
std::string EquatorialEmissionDiagnostic::getFullDescriptionStr () const [final], [virtual]
```

More descriptive string (with spaces)

Returns

std::string

Reimplemented from [Diagnostic](#).

6.14.3.5 getNameStr()

```
std::string EquatorialEmissionDiagnostic::getNameStr () const [final], [virtual]
```

Simple name without spaces.

Returns

std::string

Implements [Diagnostic](#).

6.14.3.6 Reset()

```
void EquatorialEmissionDiagnostic::Reset () [final], [virtual]
```

[EquatorialEmissionDiagnostic](#) functions.

Reset the intensity and number of equatorial passes to zero and call base class Reset function

Reimplemented from [Diagnostic](#).

6.14.3.7 UpdateData()

```
void EquatorialEmissionDiagnostic::UpdateData () [final], [virtual]
```

Update the intensity based on the emission model and the number of equatorial passes.

Implements [Diagnostic](#).

6.14.4 Member Data Documentation

6.14.4.1 DiagOptions

```
std::unique_ptr< EquatorialEmissionOptions > EquatorialEmissionDiagnostic::DiagOptions [static]
```

6.14.4.2 m_Intensity

```
real EquatorialEmissionDiagnostic::m_Intensity {0.0} [private]
```

The documentation for this class was generated from the following files:

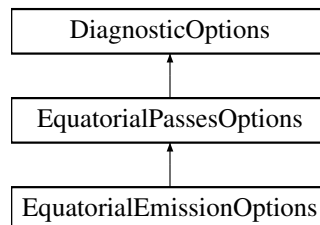
- FOORT/src/Diagnostics.h
- FOORT/src/Config.cpp
- FOORT/src/Diagnostics.cpp

6.15 EquatorialEmissionOptions Struct Reference

Options for [EquatorialEmissionDiagnostic](#).

```
#include <Diagnostics.h>
```

Inheritance diagram for EquatorialEmissionOptions:



Public Member Functions

- [EquatorialEmissionOptions](#) ([real](#) thefudgefactor, int equatupper, std::unique_ptr< [EmissionModel](#) > theemission, std::unique_ptr< [FluidVelocityModel](#) > thefluidmodel, bool rlog, int theredshiftpower, [real](#) thethreshold, [UpdateFrequency](#) thefrequency)

Public Member Functions inherited from [EquatorialPassesOptions](#)

- [EquatorialPassesOptions](#) ([real](#) thethreshold, [UpdateFrequency](#) thefrequency)

Public Member Functions inherited from [DiagnosticOptions](#)

- [DiagnosticOptions](#) ([UpdateFrequency](#) thefrequency)
- virtual [~DiagnosticOptions](#) ()=default

Public Attributes

- const [real](#) GeometricFudgeFactor
- const int EquatPassUpperBound
- const bool RLogScale
- const int RedShiftPower
- const std::unique_ptr< [EmissionModel](#) > TheEmissionModel
- const std::unique_ptr< [FluidVelocityModel](#) > TheFluidVelocityModel

Public Attributes inherited from [EquatorialPassesOptions](#)

- const [real](#) [Threshold](#)

Public Attributes inherited from [DiagnosticOptions](#)

- const [UpdateFrequency](#) [theUpdateFrequency](#)

6.15.1 Detailed Description

Options for [EquatorialEmissionDiagnostic](#).

Inherits from [EquatorialPassesOptions](#), and adds the fudge factor, upper bound, emission model, fluid velocity model, redshift power, and threshold.

6.15.2 Constructor & Destructor Documentation

6.15.2.1 EquatorialEmissionOptions()

```
EquatorialEmissionOptions::EquatorialEmissionOptions (
    real thefudgefactor,
    int equatupper,
    std::unique_ptr< EmissionModel > theemission,
    std::unique_ptr< FluidVelocityModel > thefluidmodel,
    bool rlog,
    int theredshiftpower,
    real thethreshold,
    UpdateFrequency thefrequency) [inline]
```

6.15.3 Member Data Documentation

6.15.3.1 EquatPassUpperBound

```
const int EquatorialEmissionOptions::EquatPassUpperBound
```

6.15.3.2 GeometricFudgeFactor

```
const real EquatorialEmissionOptions::GeometricFudgeFactor
```

6.15.3.3 RedShiftPower

```
const int EquatorialEmissionOptions::RedShiftPower
```

6.15.3.4 RLogScale

```
const bool EquatorialEmissionOptions::RLogScale
```

6.15.3.5 TheEmissionModel

```
const std::unique_ptr<EmissionModel> EquatorialEmissionOptions::TheEmissionModel
```

6.15.3.6 TheFluidVelocityModel

```
const std::unique_ptr<FluidVelocityModel> EquatorialEmissionOptions::TheFluidVelocityModel
```

The documentation for this struct was generated from the following file:

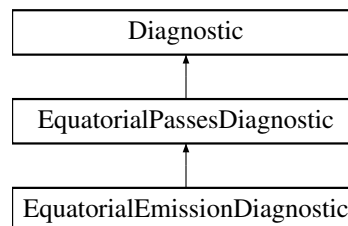
- FOORT/src/[Diagnostics.h](#)

6.16 EquatorialPassesDiagnostic Class Reference

[Diagnostic](#) that counts the number of passes through the equatorial plane.

```
#include <Diagnostics.h>
```

Inheritance diagram for EquatorialPassesDiagnostic:



Public Member Functions

- [EquatorialPassesDiagnostic](#) ([Geodesic](#) *const theGeodesic)
- void [Reset](#) () override
EquatorialPassesDiagnostic functions.
- void [UpdateData](#) () override
Check to see if we have crossed the equatorial plane and update the number of passes.
- std::string [getFullDataStr](#) () const override
Return the number of equatorial passes as a string.
- std::vector< [real](#) > [getFinalDataVal](#) () const override
Return the number of equatorial passes as a vector of size one.
- [real](#) [FinalDataValDistance](#) (const std::vector< [real](#) > &val1, const std::vector< [real](#) > &val2) const override
Return the absolute value of the difference between the number of equatorial passes.
- std::string [getNameStr](#) () const override
Simple name without spaces.
- std::string [getFullDescriptionStr](#) () const override
More descriptive string (with spaces)

Public Member Functions inherited from [Diagnostic](#)

- [Diagnostic](#) ()=delete
- [Diagnostic](#) ([Geodesic](#) *const theGeodesic)
- virtual [~Diagnostic](#) ()=default

Static Public Attributes

- static std::unique_ptr< [EquatorialPassesOptions](#) > [DiagOptions](#)
Needs the extra option of a threshold.

Protected Attributes

- int [m_EquatPasses](#) {0}
Keeps track of how many passes have been made.

Protected Attributes inherited from [Diagnostic](#)

- [Geodesic](#) *const [m_OwnerGeodesic](#)
- [largecounter](#) [m_StepsSinceUpdated](#) {}

Private Attributes

- [real](#) [m_PrevTheta](#) {-1}
Keeps track of the previous theta angle, so that we can compare with current theta angle.

Additional Inherited Members

Protected Member Functions inherited from [Diagnostic](#)

- bool [DecideUpdate](#) (const [UpdateFrequency](#) &myUpdateFrequency)
Returns true if the [Diagnostic](#) should update its initial status.

6.16.1 Detailed Description

[Diagnostic](#) that counts the number of passes through the equatorial plane.

6.16.2 Constructor & Destructor Documentation

6.16.2.1 EquatorialPassesDiagnostic()

```
EquatorialPassesDiagnostic::EquatorialPassesDiagnostic (
    Geodesic *const theGeodesic) [inline]
```

6.16.3 Member Function Documentation

6.16.3.1 FinalDataValDistance()

```
real EquatorialPassesDiagnostic::FinalDataValDistance (
    const std::vector< real > & val1,
    const std::vector< real > & val2) const [override], [virtual]
```

Return the absolute value of the difference between the number of equatorial passes.

Parameters

<i>val1</i>	first number of equatorial passes
<i>val2</i>	second number of equatorial passes

Returns

real

Implements [Diagnostic](#).

6.16.3.2 getFinalDataVal()

```
std::vector< real > EquatorialPassesDiagnostic::getFinalDataVal () const [override], [virtual]
```

Return the number of equatorial passes as a vector of size one.

Returns

std::vector<real>

Implements [Diagnostic](#).

6.16.3.3 getFullDataStr()

```
std::string EquatorialPassesDiagnostic::getFullDataStr () const [override], [virtual]
```

Return the number of equatorial passes as a string.

Returns

std::string

Implements [Diagnostic](#).

6.16.3.4 getFullDescriptionStr()

```
std::string EquatorialPassesDiagnostic::getFullDescriptionStr () const [override], [virtual]
```

More descriptive string (with spaces)

Returns

std::string

Reimplemented from [Diagnostic](#).

6.16.3.5 getNameStr()

```
std::string EquatorialPassesDiagnostic::getNameStr () const [override], [virtual]
```

Simple name without spaces.

Returns

std::string

Implements [Diagnostic](#).

6.16.3.6 Reset()

```
void EquatorialPassesDiagnostic::Reset () [override], [virtual]
```

[EquatorialPassesDiagnostic](#) functions.

Reset the number of equatorial passes and the previous theta angle and call base class Reset function

Reimplemented from [Diagnostic](#).

6.16.3.7 UpdateData()

```
void EquatorialPassesDiagnostic::UpdateData () [override], [virtual]
```

Check to see if we have crossed the equatorial plane and update the number of passes.

Implements [Diagnostic](#).

6.16.4 Member Data Documentation

6.16.4.1 DiagOptions

```
std::unique_ptr< EquatorialPassesOptions > EquatorialPassesDiagnostic::DiagOptions [static]
```

Needs the extra option of a threshold.

6.16.4.2 m_EquatPasses

```
int EquatorialPassesDiagnostic::m_EquatPasses {0} [protected]
```

Keeps track of how many passes have been made.

6.16.4.3 m_PrevTheta

```
real EquatorialPassesDiagnostic::m_PrevTheta {-1} [private]
```

Keeps track of the previous theta angle, so that we can compare with current theta angle.

The documentation for this class was generated from the following files:

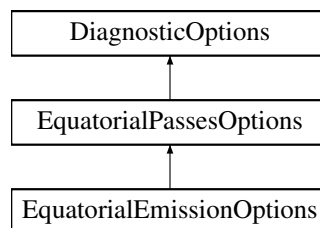
- FOORT/src/[Diagnostics.h](#)
- FOORT/src/[Config.cpp](#)
- FOORT/src/[Diagnostics.cpp](#)

6.17 EquatorialPassesOptions Struct Reference

Options for [EquatorialPassesDiagnostic](#).

```
#include <Diagnostics.h>
```

Inheritance diagram for EquatorialPassesOptions:



Public Member Functions

- [EquatorialPassesOptions](#) ([real](#) thethreshold, [UpdateFrequency](#) thefrequency)

Public Member Functions inherited from [DiagnosticOptions](#)

- [DiagnosticOptions](#) ([UpdateFrequency](#) thefrequency)
- virtual [~DiagnosticOptions](#) ()=default

Public Attributes

- const [real](#) Threshold

Public Attributes inherited from [DiagnosticOptions](#)

- const [UpdateFrequency](#) theUpdateFrequency

6.17.1 Detailed Description

Options for [EquatorialPassesDiagnostic](#).

Inherits from [DiagnosticOptions](#), and adds the threshold for counting passes.

6.17.2 Constructor & Destructor Documentation

6.17.2.1 EquatorialPassesOptions()

```
EquatorialPassesOptions::EquatorialPassesOptions (
    real thethreshold,
    UpdateFrequency thefrequency) [inline]
```

6.17.3 Member Data Documentation

6.17.3.1 Threshold

```
const real EquatorialPassesOptions::Threshold
```

The documentation for this struct was generated from the following file:

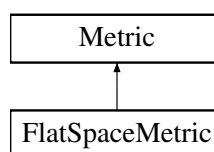
- FOORT/src/[Diagnostics.h](#)

6.18 FlatSpaceMetric Class Reference

Flat space metric in 4D.

```
#include <Metric.h>
```

Inheritance diagram for FlatSpaceMetric:



Public Member Functions

- [FlatSpaceMetric](#) (bool rlogscale=false)
Construct a new Flat Space [Metric](#) object.
- [TwoIndex getMetric_dd](#) (const [Point](#) &p) const final
Flat metric getter, indices down.
- [TwoIndex getMetric_uu](#) (const [Point](#) &p) const final
Flat metric getter, indices up.
- std::string [getFullDescriptionStr](#) () const final
Flat metric description string getter.

Public Member Functions inherited from [Metric](#)

- virtual [~Metric](#) ()=default
- [Metric](#) (bool rlogscale=false)
Construct a new [Metric](#) object.
- virtual [ThreeIndex](#) [getChristoffel_udd](#) (const [Point](#) &p) const
Return the Christoffel symbols of the metric (indices up, down, down)
- virtual [FourIndex](#) [getRiemann_uddd](#) (const [Point](#) &p) const
Riemann tensor of the metric (indices up, down, down, down)
- virtual [real](#) [getKretschmann](#) (const [Point](#) &p) const
Get the Kretschmann scalar at a point.
- bool [getrLogScale](#) () const
Get whether we are using a logarithmic radial scale.

Additional Inherited Members

Protected Attributes inherited from [Metric](#)

- std::vector< int > [m_Symmetries](#) {}
The symmetries (coordinate Killing vectors) of the metric. Should be set by descendant constructor.
- const bool [m_rLogScale](#)
Are we using a logarithmic r coordinate?

6.18.1 Detailed Description

Flat space metric in 4D.

6.18.2 Constructor & Destructor Documentation

6.18.2.1 FlatSpaceMetric()

```
FlatSpaceMetric::FlatSpaceMetric (
    bool rlogscale = false)
```

Construct a new Flat Space [Metric](#) object.

Parameters

<i>rlogscale</i>	
------------------	--

6.18.3 Member Function Documentation

6.18.3.1 getFullDescriptionStr()

```
std::string FlatSpaceMetric::getFullDescriptionStr () const [final], [virtual]
```

Flat metric description string getter.

Returns

std::string

Reimplemented from [Metric](#).

6.18.3.2 getMetric_dd()

```
TwoIndex FlatSpaceMetric::getMetric_dd (
    const Point & p) const [final], [virtual]
```

Flat metric getter, indices down.

Parameters

p	Point at which to evaluate the metric
-----	---------------------------------------

Returns

TwoIndex

Implements [Metric](#).

6.18.3.3 getMetric_uu()

```
TwoIndex FlatSpaceMetric::getMetric_uu (
    const Point & p) const [final], [virtual]
```

Flat metric getter, indices up.

Parameters

p	Point at which to evaluate the metric
-----	---------------------------------------

Returns

TwoIndex

Implements [Metric](#).

The documentation for this class was generated from the following files:

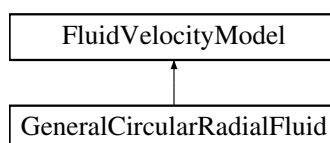
- FOORT/src/[Metric.h](#)
- FOORT/src/[Metric.cpp](#)

6.19 FluidVelocityModel Struct Reference

Fluid velocity model abstract base class.

```
#include <DiagnosticsEmission.h>
```

Inheritance diagram for FluidVelocityModel:



Public Member Functions

- [FluidVelocityModel](#) (const [Metric](#) *const theMetric)
- virtual [~FluidVelocityModel](#) ()=default
- virtual [OneIndex](#) [GetFourVelocityd](#) (const [Point](#) &p) const =0
- virtual std::string [getFullDescriptionStr](#) () const

Base class default string getter.

Protected Attributes

- const [Metric](#) *const [m_theMetric](#)

6.19.1 Detailed Description

Fluid velocity model abstract base class.

6.19.2 Constructor & Destructor Documentation

6.19.2.1 FluidVelocityModel()

```
FluidVelocityModel::FluidVelocityModel (
    const Metric *const theMetric) [inline]
```

6.19.2.2 ~FluidVelocityModel()

```
virtual FluidVelocityModel::~~FluidVelocityModel () [virtual], [default]
```

6.19.3 Member Function Documentation

6.19.3.1 GetFourVelocityd()

```
virtual OneIndex FluidVelocityModel::GetFourVelocityd (
    const Point & p) const [pure virtual]
```

Implemented in [GeneralCircularRadialFluid](#).

6.19.3.2 getFullDescriptionStr()

```
std::string FluidVelocityModel::getFullDescriptionStr () const [virtual]
```

Base class default string getter.

Returns

std::string

Reimplemented in [GeneralCircularRadialFluid](#).

6.19.4 Member Data Documentation

6.19.4.1 m_theMetric

```
const Metric* const FluidVelocityModel::m_theMetric [protected]
```

The documentation for this struct was generated from the following files:

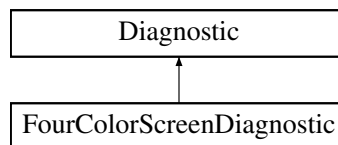
- FOORT/src/DiagnosticsEmission.h
- FOORT/src/DiagnosticsEmission.cpp

6.20 FourColorScreenDiagnostic Class Reference

The four color screen diagnostic.

```
#include <Diagnostics.h>
```

Inheritance diagram for FourColorScreenDiagnostic:



Public Member Functions

- [FourColorScreenDiagnostic](#) ([Geodesic](#) *const theGeodesic)
- void [Reset](#) () final
FourColorScreen functions.
- void [UpdateData](#) () override
Update the quadrant based on the geodesic's final position.
- std::string [getFullDataStr](#) () const final
Return the value of the quadrant as a string.
- std::vector< [real](#) > [getFinalDataVal](#) () const final
Return the value of the quadrant as a vector of size one.
- [real](#) [FinalDataValDistance](#) (const std::vector< [real](#) > &val1, const std::vector< [real](#) > &val2) const final
Discrete metric for distance: return 0 if the quadrants are the same, 1 if they are not.
- std::string [getNameStr](#) () const final
Simple name without spaces.
- std::string [getFullDescriptionStr](#) () const final
Return full name, possibly with spaces.

Public Member Functions inherited from [Diagnostic](#)

- [Diagnostic](#) ()=delete
- [Diagnostic](#) ([Geodesic](#) *const theGeodesic)
- virtual [~Diagnostic](#) ()=default

Private Attributes

- int `m_quadrant` {0}

Note initialization to 0; this means the default value returned will be 0 (e.g. if [Term::BoundarySphere](#) is not reached)

Additional Inherited Members

Protected Member Functions inherited from [Diagnostic](#)

- bool `DecideUpdate` (const [UpdateFrequency](#) &myUpdateFrequency)

Returns true if the [Diagnostic](#) should update its initial status.

Protected Attributes inherited from [Diagnostic](#)

- [Geodesic](#) *const `m_OwnerGeodesic`
- largecounter `m_StepsSinceUpdated` {}

6.20.1 Detailed Description

The four color screen diagnostic.

This diagnostic associates one of four colors based on the quadrant that the geodesic finishes in. Will ONLY return a color if the geodesic indeed finishes because it passes through the boundary sphere!

6.20.2 Constructor & Destructor Documentation

6.20.2.1 FourColorScreenDiagnostic()

```
FourColorScreenDiagnostic::FourColorScreenDiagnostic (
    Geodesic *const theGeodesic) [inline]
```

6.20.3 Member Function Documentation

6.20.3.1 FinalDataValDistance()

```
real FourColorScreenDiagnostic::FinalDataValDistance (
    const std::vector< real > & val1,
    const std::vector< real > & val2) const [final], [virtual]
```

Discrete metric for distance: return 0 if the quadrants are the same, 1 if they are not.

Parameters

<i>val1</i>	first quadrant value
<i>val2</i>	second quadrant value

Returns

real

Implements [Diagnostic](#).

6.20.3.2 getFinalDataVal()

```
std::vector< real > FourColorScreenDiagnostic::getFinalDataVal () const [final], [virtual]
```

Return the value of the quadrant as a vector of size one.

Returns

std::vector<real>

Implements [Diagnostic](#).

6.20.3.3 getFullDataStr()

```
std::string FourColorScreenDiagnostic::getFullDataStr () const [final], [virtual]
```

Return the value of the quadrant as a string.

Returns

std::string

Implements [Diagnostic](#).

6.20.3.4 getFullDescriptionStr()

```
std::string FourColorScreenDiagnostic::getFullDescriptionStr () const [final], [virtual]
```

Return full name, possibly with spaces.

Returns

std::string

Reimplemented from [Diagnostic](#).

6.20.3.5 getNameStr()

```
std::string FourColorScreenDiagnostic::getNameStr () const [final], [virtual]
```

Simple name without spaces.

Returns

std::string

Implements [Diagnostic](#).

6.20.3.6 Reset()

```
void FourColorScreenDiagnostic::Reset () [final], [virtual]
```

FourColorScreen functions.

Reset the quadrant to its default option; also call base class Reset function

Reimplemented from [Diagnostic](#).

6.20.3.7 UpdateData()

```
void FourColorScreenDiagnostic::UpdateData () [override], [virtual]
```

Update the quadrant based on the geodesic's final position.

Implements [Diagnostic](#).

6.20.4 Member Data Documentation

6.20.4.1 m_quadrant

```
int FourColorScreenDiagnostic::m_quadrant {0} [private]
```

Note initialization to 0; this means the default value returned will be 0 (e.g. if [Term::BoundarySphere](#) is not reached)

The documentation for this class was generated from the following files:

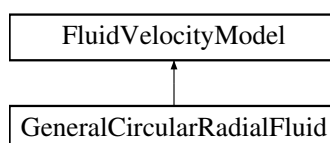
- FOORT/src/[Diagnostics.h](#)
- FOORT/src/[Diagnostics.cpp](#)

6.21 GeneralCircularRadialFluid Struct Reference

General circular radial fluid velocity model.

```
#include <DiagnosticsEmission.h>
```

Inheritance diagram for GeneralCircularRadialFluid:



Public Member Functions

- **GeneralCircularRadialFluid** (**real** subKeplerParam, **real** betar, **real** betaphi, const **Metric** *const theMetric, **real** ISCO_lowerbound, **real** ISCO_upperbound)
- **OneIndex GetFourVelocityd** (const **Point** &p) const final
Get the local four-velocity of the fluid according to the GeneralCircularRadial model.
- std::string **getFullDescriptionStr** () const final
Description string getter for the GeneralCircularRadial fluid velocity model.

Public Member Functions inherited from FluidVelocityModel

- **FluidVelocityModel** (const **Metric** *const theMetric)
- virtual **~FluidVelocityModel** ()=default

Private Member Functions

- **OneIndex GetCircularVelocityd** (const **Point** &p, bool subKeplerianOn=true) const
Get the circular velocity for a point, with optional subKeplerian rescaling.
- **OneIndex GetInsideISCOCircularVelocityd** (const **Point** &p) const
Returns velocity for non-geodetic motion where $E, L = (E, L)_{ISCO}$ and radially falling inward.
- **OneIndex GetRadialVelocityd** (const **Point** &p) const
Returns radial velocity for pure infalling matter with $E = 1$ and $L = 0$ at infinity.
- void **FindISCO** ()
Helper function to find the ISCO of the spacetime, if it exists.
- **TwoIndex GetChrstrRaisedDer** (**real** r) const
Helper function to get $\partial_r(g^{ab})\Gamma^{bc}_r g^{cd}$

Private Attributes

- const **real** m_subKeplerParam
Sub-Keplerian parameter (0 = Keplerian, 1 = sub-Keplerian)
- const **real** m_betaR
Radial velocity parameter.
- const **real** m_betaPhi
Azimuthal velocity parameter.
- bool m_ISCOexists {false}
Flag to indicate if ISCO exists for this Metric.
- **real** m_ISCO { -1.0 }
ISCO radius.
- **real** m_ISCOlowerbound
Lower bound for ISCO radius search.
- **real** m_ISCOupperbound
Upper bound for ISCO radius search.
- **real** m_ISCOpt {}
ISCO t momentum.
- **real** m_ISCOpphi {}
ISCO phi momentum.

Additional Inherited Members

Protected Attributes inherited from [FluidVelocityModel](#)

- const [Metric](#) *const [m_theMetric](#)

6.21.1 Detailed Description

General circular radial fluid velocity model.

This fluid velocity model has three tuneable parameters and represents fluid travelling at a mix of (sub)Keplerian circular orbits and radially infalling orbits in the equatorial plane. Details can be found in e.g. <https://doi.org/10.1103/PhysRevD.107.043030>.

6.21.2 Constructor & Destructor Documentation

6.21.2.1 GeneralCircularRadialFluid()

```
GeneralCircularRadialFluid::GeneralCircularRadialFluid (
    real subKeplerParam,
    real betar,
    real betaphi,
    const Metric *const theMetric,
    real ISCO_lowerbound,
    real ISCO_upperbound) [inline]
```

6.21.3 Member Function Documentation

6.21.3.1 FindISCO()

```
void GeneralCircularRadialFluid::FindISCO () [private]
```

Helper function to find the ISCO of the spacetime, if it exists.

This helper function finds the ISCO. We use a binary search to converge on the ISCO value; the initial outer bounds for a metric with horizon are the horizon radius and $10 \times (\text{horizon radius})$ (in true radii, not log(r) coordinates)

6.21.3.2 GetChrstrRaisedDer()

```
TwoIndex GeneralCircularRadialFluid::GetChrstrRaisedDer (
    real r) const [private]
```

Helper function to get $\partial_r(g^{ab})\Gamma^r_{bc}g^{cd}$

This function calculates $\partial_r(g^{ab})\Gamma^r_{bc}g^{cd}$. We do so by calculating the central difference with $O(h^4)$ accuracy, but taking the sqrt of the usual h used for derivatives

Parameters

<i>r</i>	Radius at which to calculate the derivative
----------	---

Returns

[TwoIndex](#)

6.21.3.3 GetCircularVelocityd()

```
OneIndex GeneralCircularRadialFluid::GetCircularVelocityd (
    const Point & p,
    bool subKeplerianOn = true) const [private]
```

Get the circular velocity for a point, with optional subKeplerian rescaling.

Returns the velocity for circular (sub)Keplerian motion (outside the ISCO). First the circular geodesic is calculated, then if subKeplerianOn == true, the orbit is reparametrized to non-geodetic subKeplerian circular motion according to the subKeplerian parameter

Parameters

<i>p</i>	Point at which to calculate the velocity
<i>subKeplerianOn</i>	Boolean to indicate whether to rescale the angular momentum for sub-Keplerian motion

Returns

[OneIndex](#) circular velocity with index down. If no solution, returns all zeros

6.21.3.4 GetFourVelocityd()

```
OneIndex GeneralCircularRadialFluid::GetFourVelocityd (
    const Point & p) const [final], [virtual]
```

Get the local four-velocity of the fluid according to the GeneralCircularRadial model.

Parameters

<i>p</i>	Point at which to calculate the four-velocity
----------	---

Returns

[OneIndex](#) Local four-velocity with index down

Implements [FluidVelocityModel](#).

6.21.3.5 getFullDescriptionStr()

```
std::string GeneralCircularRadialFluid::getFullDescriptionStr () const [final], [virtual]
```

Description string getter for the GeneralCircularRadial fluid velocity model.

Returns

std::string

Reimplemented from [FluidVelocityModel](#).

6.21.3.6 GetInsideISCOCircularVelocityd()

```
OneIndex GeneralCircularRadialFluid::GetInsideISCOCircularVelocityd (
    const Point & p) const [private]
```

Returns velocity for non-geodetic motion where $E, L = (E, L)_{\text{ISCO}}$ and radially falling inward.

Parameters

p	Point at which to calculate the velocity
-----	--

Returns

[OneIndex](#)

6.21.3.7 GetRadialVelocityd()

```
OneIndex GeneralCircularRadialFluid::GetRadialVelocityd (
    const Point & p) const [private]
```

Returns radial velocity for pure infalling matter with $E = 1$ and $L = 0$ at infinity.

Parameters

p	Point at which to calculate the velocity
-----	--

Returns

[OneIndex](#)

6.21.4 Member Data Documentation

6.21.4.1 m_betaPhi

```
const real GeneralCircularRadialFluid::m_betaPhi [private]
```

Azimuthal velocity parameter.

6.21.4.2 m_betaR

```
const real GeneralCircularRadialFluid::m_betaR [private]
```

Radial velocity parameter.

6.21.4.3 m_ISCOexists

```
bool GeneralCircularRadialFluid::m_ISCOexists {false} [private]
```

Flag to indicate if ISCO exists for this [Metric](#).

6.21.4.4 m_ISCOlowerbound

```
real GeneralCircularRadialFluid::m_ISCOlowerbound [private]
```

Lower bound for ISCO radius search.

6.21.4.5 m_ISCOpphi

```
real GeneralCircularRadialFluid::m_ISCOpphi {} [private]
```

ISCO phi momentum.

6.21.4.6 m_ISCOpt

```
real GeneralCircularRadialFluid::m_ISCOpt {} [private]
```

ISCO t momentum.

6.21.4.7 m_ISCOr

```
real GeneralCircularRadialFluid::m_ISCOr {-1.0} [private]
```

ISCO radius.

6.21.4.8 m_ISCOupperbound

```
real GeneralCircularRadialFluid::m_ISCOupperbound [private]
```

Upper bound for ISCO radius search.

6.21.4.9 m_subKeplerParam

```
const real GeneralCircularRadialFluid::m_subKeplerParam [private]
```

Sub-Keplerian parameter (0 = Keplerian, 1 = sub-Keplerian)

The documentation for this struct was generated from the following files:

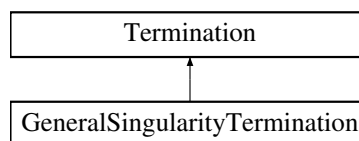
- FOORT/src/[DiagnosticsEmission.h](#)
- FOORT/src/[DiagnosticsEmission.cpp](#)

6.22 GeneralSingularityTermination Class Reference

[GeneralSingularityTermination](#): Terminate geodesics if they get too close to one of a number of singularities (of arbitrary codimension)

```
#include <Terminations.h>
```

Inheritance diagram for GeneralSingularityTermination:



Public Member Functions

- [GeneralSingularityTermination](#) ([Geodesic](#) *const theGeodesic)
- [Term CheckTermination](#) () final
Check if the geodesic is too close to a singularity.
- std::string [getFullDescriptionStr](#) () const final
Description string getter for the General [Singularity Termination](#).

Public Member Functions inherited from [Termination](#)

- [Termination](#) ()=delete
- [Termination](#) ([Geodesic](#) *const theGeodesic)
- virtual void [Reset](#) ()
Reset to 0 to start integrating a new geodesic.
- virtual [~Termination](#) ()=default

Static Public Attributes

- static std::unique_ptr< [GeneralSingularityTermOptions](#) > [TermOptions](#)
Options (contains horizon radius, if we are using logarithmic r coordinate, and distance allowed from the horizon)

Private Member Functions

- `std::string SingularityToString` (int singnr) const
List the singularities in a human-readable format.

Additional Inherited Members

Protected Member Functions inherited from [Termination](#)

- `bool DecideUpdate` (largecounter UpdateNSteps)
Returns true if the [Termination](#) should update its internal status.

Protected Attributes inherited from [Termination](#)

- `Geodesic *const m_OwnerGeodesic`
The geodesic that owns the [Termination](#) (a const pointer to the [Geodesic](#))
- `largecounter m_StepsSinceUpdated` {}
The termination is itself in charge of keeping track of how many steps it has been since it has been updated. The [Termination](#)'s [TerminationOptions](#) struct tells it how many steps it needs to wait between updates.

6.22.1 Detailed Description

[GeneralSingularityTermination](#): Terminate geodesics if they get too close to one of a number of singularities (of arbitrary codimension)

6.22.2 Constructor & Destructor Documentation

6.22.2.1 GeneralSingularityTermination()

```
GeneralSingularityTermination::GeneralSingularityTermination (
    Geodesic *const theGeodesic) [inline]
```

6.22.3 Member Function Documentation

6.22.3.1 CheckTermination()

```
Term GeneralSingularityTermination::CheckTermination () [final], [virtual]
```

Check if the geodesic is too close to a singularity.

Returns

[Term](#)

Implements [Termination](#).

6.22.3.2 getFullDescriptionStr()

```
std::string GeneralSingularityTermination::getFullDescriptionStr () const [final], [virtual]
```

Description string getter for the General [Singularity Termination](#).

Returns

std::string

Implements [Termination](#).

6.22.3.3 SingularityToString()

```
std::string GeneralSingularityTermination::SingularityToString (
    int singnr) const [private]
```

List the singularities in a human-readable format.

Returns

std::string

6.22.4 Member Data Documentation

6.22.4.1 TermOptions

```
std::unique_ptr< GeneralSingularityTermOptions > GeneralSingularityTermination::TermOptions
[static]
```

Options (contains horizon radius, if we are using logarithmic r coordinate, and distance allowed from the horizon)

The documentation for this class was generated from the following files:

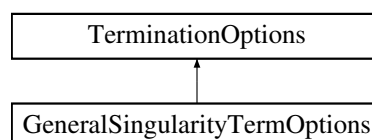
- FOORT/src/[Terminations.h](#)
- FOORT/src/[Config.cpp](#)
- FOORT/src/[Terminations.cpp](#)

6.23 GeneralSingularityTermOptions Struct Reference

Options for [GeneralSingularityTermination](#).

```
#include <Terminations.h>
```

Inheritance diagram for GeneralSingularityTermOptions:



Public Member Functions

- [GeneralSingularityTermOptions](#) (std::vector< [Singularity](#) > sings, [real](#) eps, bool consoleoutputon, bool therlogscale, [largecounter](#) Nsteps)

Public Member Functions inherited from [TerminationOptions](#)

- [TerminationOptions](#) ([largecounter](#) Nsteps)
- virtual [~TerminationOptions](#) ()=default

Public Attributes

- const std::vector< [Singularity](#) > [Singularities](#)
Vector of singularities.
- const [real](#) [Epsilon](#)
Minimum flat distance the geodesic is allowed to be from any singularity.
- const bool [OutputToConsole](#)
Whether to output a message to the console when a singularity is encountered.
- const bool [rLogScale](#)
Whether we are using a logarithmic r coordinate or not.

Public Attributes inherited from [TerminationOptions](#)

- const [largecounter](#) [UpdateEveryNSteps](#)
Number of steps between updates.

6.23.1 Detailed Description

Options for [GeneralSingularityTermination](#).

Keeps track of the singularities, allowed distance and whether we are using a logarithmic r coordinate.

6.23.2 Constructor & Destructor Documentation

6.23.2.1 GeneralSingularityTermOptions()

```
GeneralSingularityTermOptions::GeneralSingularityTermOptions (
    std::vector< Singularity > sings,
    real eps,
    bool consoleoutputon,
    bool therlogscale,
    largecounter Nsteps) [inline]
```

6.23.3 Member Data Documentation

6.23.3.1 Epsilon

```
const real GeneralSingularityTermOptions::Epsilon
```

Minimum flat distance the geodesic is allowed to be from any singularity.

6.23.3.2 OutputToConsole

```
const bool GeneralSingularityTermOptions::OutputToConsole
```

Whether to output a message to the console when a singularity is encountered.

6.23.3.3 rLogScale

```
const bool GeneralSingularityTermOptions::rLogScale
```

Whether we are using a logarithmic r coordinate or not.

6.23.3.4 Singularities

```
const std::vector<Singularity> GeneralSingularityTermOptions::Singularities
```

Vector of singularities.

The documentation for this struct was generated from the following file:

- FOORT/src/[Terminations.h](#)

6.24 Geodesic Class Reference

[Geodesic](#) class: an instance of this class is created for each [Geodesic](#) that is integrated.

```
#include <Geodesic.h>
```

Public Member Functions

- [Geodesic](#) ()=delete
- [Geodesic](#) (const [Geodesic](#) &)=delete
- [Geodesic](#) & operator= (const [Geodesic](#) &)=delete
- [Geodesic](#) (const [Metric](#) *const theMetric, const [Source](#) *const theSource, [DiagBitflag](#) diagbit, [DiagBitflag](#) valdiagbit, [TermBitflag](#) termbit, [GeodesicIntegratorFunc](#) theIntegrator)

Construct a new [Geodesic](#) object. Arguments initialize private member variables that remain the same over all geodesics.
- void [Reset](#) ([ScreenIndex](#) scrindex, [Point](#) initpos, [OneIndex](#) initvel)

[Geodesic](#) (and descendant classes) functions.
- [Term](#) [Update](#) ()

This makes the [Geodesic](#) integrate itself one step; then the [Geodesic](#) loops through all Terminations and Diagnostics to update.
- [Term](#) [getTermCondition](#) () const

Get the current termination condition.
- [Point](#) [getCurrentPos](#) () const

Get the current position.
- [OneIndex](#) [getCurrentVel](#) () const

Get the current velocity.
- [real](#) [getCurrentLambda](#) () const

Get the current value of the affine parameter.
- [ScreenIndex](#) [getScreenIndex](#) () const

Get the screen index.
- std::vector< std::string > [getAllOutputStr](#) () const

Get the complete output that should be written to the output files.
- std::vector< [real](#) > [getDiagnosticFinalValue](#) () const

Get the "value" (from the [Diagnostic](#) that was set to the value [Diagnostic](#)) that is associated to the [Geodesic](#).

Private Attributes

- [Term m_TermCond](#) {[Term::Uninitialized](#)}
As long as this is [Term::Continue](#), not done integrating yet.
- [Point m_CurrentPos](#) {}
Current position.
- [OneIndex m_CurrentVel](#) {}
Current proper velocity.
- [real m_curLambda](#) {0.0}
Current value of affine parameter (starts at 0.0)
- [ScreenIndex m_ScreenIndex](#) {}
- `const Metric *const m_theMetric`
const pointer to the [Metric](#)
- `const Source *const m_theSource`
const pointer to the [Source](#) for the rhs of the geodesic equation
- `const DiagnosticUniqueVector m_AllDiagnostics`
Vector of unique pointers to Diagnostics. An instance of each [Diagnostic](#) is created for the [Geodesic](#), so the [Geodesic](#) is the owner of these objects.
- `const TerminationUniqueVector m_AllTerminations`
Vector of unique pointers to Terminations. An instance of each [Termination](#) is created for the [Geodesic](#), so the [Geodesic](#) is the owner of these objects.
- `const GeodesicIntegratorFunc m_theIntegrator`
This is the function that will integrate the geodesic equation one step.

6.24.1 Detailed Description

[Geodesic](#) class: an instance of this class is created for each [Geodesic](#) that is integrated.

The [Geodesic](#) is in charge of integrating itself until termination, updating its Diagnostics accordingly, and (after termination) returning the appropriate output

6.24.2 Constructor & Destructor Documentation**6.24.2.1 [Geodesic\(\)](#) [1/3]**

```
Geodesic::Geodesic () [delete]
```

6.24.2.2 [Geodesic\(\)](#) [2/3]

```
Geodesic::Geodesic (
    const Geodesic & ) [delete]
```

6.24.2.3 [Geodesic\(\)](#) [3/3]

```
Geodesic::Geodesic (
    const Metric *const theMetric,
    const Source *const theSource,
    DiagBitflag diagbit,
    DiagBitflag valdiagbit,
    TermBitflag termbit,
    GeodesicIntegratorFunc theIntegrator) [inline]
```

Construct a new [Geodesic](#) object. Arguments initialize private member variables that remain the same over all geodesics.

Parameters

<i>theMetric</i>	Pointer to the Metric object.
<i>theSource</i>	Pointer to the Source object.
<i>diagbit</i>	Diagnostic bitflag.
<i>valdiagbit</i>	Value diagnostic bitflag.
<i>termbit</i>	Termination bitflag.
<i>theIntegrator</i>	Geoodesic integrator function to use for integrating geodesic equation.

6.24.3 Member Function Documentation

6.24.3.1 getAllOutputStr()

```
std::vector< std::string > Geodesic::getAllOutputStr () const
```

Get the complete output that should be written to the output files.

There is one string more than the count of Diagnostics: one string per [Diagnostic](#), PLUS the first string is the screen index.

Returns

```
std::vector<std::string>
```

6.24.3.2 getCurrentLambda()

```
real Geodesic::getCurrentLambda () const
```

Get the current value of the affine parameter.

Returns

```
real
```

6.24.3.3 getCurrentPos()

```
Point Geodesic::getCurrentPos () const
```

Get the current position.

Returns

```
Point
```

6.24.3.4 `getCurrentVel()`

```
OneIndex Geodesic::getCurrentVel () const
```

Get the current velocity.

Returns

[OneIndex](#)

6.24.3.5 `getDiagnosticFinalValue()`

```
std::vector< real > Geodesic::getDiagnosticFinalValue () const
```

Get the "value" (from the [Diagnostic](#) that was set to the value [Diagnostic](#)) that is associated to the [Geodesic](#).

Will be used to determine "distance" between Geodesics which is used in [Mesh](#) refinement.

Returns

`std::vector<real>`

6.24.3.6 `getScreenIndex()`

```
ScreenIndex Geodesic::getScreenIndex () const
```

Get the screen index.

Returns

[ScreenIndex](#)

6.24.3.7 `getTermCondition()`

```
Term Geodesic::getTermCondition () const
```

Get the current termination condition.

Returns

[Term](#)

6.24.3.8 `operator=()`

```
Geodesic & Geodesic::operator= (
    const Geodesic & ) [delete]
```

6.24.3.9 `Reset()`

```
void Geodesic::Reset (
    ScreenIndex scrindex,
    Point initpos,
    OneIndex initvel)
```

[Geodesic](#) (and descendant classes) functions.

This initializes/resets the geodesic with a given [ScreenIndex](#), initial position, and initial velocity

Also resets all Diagnostics and Terminations, resets the TermCondition to [Term::Continue](#), and puts the [Geodesic](#) back to $\lambda = 0.0$.

Parameters

<i>scrindex</i>	ScreenIndex
<i>initpos</i>	Initial position
<i>initvel</i>	Initial velocity

6.24.3.10 Update()

`Term Geodesic::Update ()`

This makes the [Geodesic](#) integrate itself one step; then the [Geodesic](#) loops through all Terminations and Diagnostics to update.

Returns

[Term](#)

6.24.4 Member Data Documentation**6.24.4.1 m_AllDiagnostics**

`const DiagnosticUniqueVector Geodesic::m_AllDiagnostics [private]`

Vector of unique pointers to Diagnostics. An instance of each [Diagnostic](#) is created for the [Geodesic](#), so the [Geodesic](#) is the owner of these objects.

6.24.4.2 m_AllTerminations

`const TerminationUniqueVector Geodesic::m_AllTerminations [private]`

Vector of unique pointers to Terminations. An instance of each [Termination](#) is created for the [Geodesic](#), so the [Geodesic](#) is the owner of these objects.

6.24.4.3 m_curLambda

`real Geodesic::m_curLambda {0.0} [private]`

Current value of affine parameter (starts at 0.0)

6.24.4.4 m_CurrentPos

`Point Geodesic::m_CurrentPos {} [private]`

Current position.

6.24.4.5 m_CurrentVel

```
OneIndex Geodesic::m_CurrentVel {} [private]
```

Current proper velocity.

6.24.4.6 m_ScreenIndex

```
ScreenIndex Geodesic::m_ScreenIndex {} [private]
```

The [Geodesic](#) keeps track of what index it has been assigned; it outputs this information in its final output string

6.24.4.7 m_TermCond

```
Term Geodesic::m_TermCond {Term::Uninitialized} [private]
```

As long as this is [Term::Continue](#), not done integrating yet.

6.24.4.8 m_theIntegrator

```
const GeodesicIntegratorFunc Geodesic::m_theIntegrator [private]
```

This is the function that will integrate the geodesic equation one step.

6.24.4.9 m_theMetric

```
const Metric* const Geodesic::m_theMetric [private]
```

const pointer to the [Metric](#)

6.24.4.10 m_theSource

```
const Source* const Geodesic::m_theSource [private]
```

const pointer to the [Source](#) for the rhs of the geodesic equation

The documentation for this class was generated from the following files:

- FOORT/src/[Geodesic.h](#)
- FOORT/src/[Geodesic.cpp](#)

6.25 GeodesicOutputHandler Class Reference

// [GeodesicOutputHandler](#) handles all of the output to file. It gets passed all of the output strings for every [Geodesic](#), it then stores this data until it eventually writes all data to the appropriate files

```
#include <InputOutput.h>
```

Public Member Functions

- [GeodesicOutputHandler](#) ()=delete
- [GeodesicOutputHandler](#) (std::string FilePrefix, std::string TimeStamp, std::string FileExtension, std::vector< std::string > DiagNames, [largecounter](#) nroutputstocache=LARGECOUNTER_MAX - 1, [largecounter](#) geodperfile=LARGECOUNTER_MAX, std::string firstlineinfo="")
Construct a new [Geodesic](#) Output Handler object and initialize all const member variables.
- void [PrepareForOutput](#) ([largecounter](#) nrOutputToCome)
prepare the cache for output
- void [NewGeodesicOutput](#) ([largecounter](#) index, std::vector< std::string > theOutput)
- void [OutputFinished](#) ()
There is no more output, so we write anything that is cached to file to clean up and finalize.
- std::string [getFullDescriptionStr](#) () const
full description string getter for the geodesic output handler

Private Member Functions

- void [WriteCachedOutputToFile](#) ()
Write cached output to a file.
- void [OpenForFirstTime](#) (const std::string &filename)
We check here to see if the files are being put in a (sub)directory; if so, we create the directory/directories first to make sure creating/opening the file will succeed.
- std::string [GetFileName](#) (int diagnr, unsigned short filenr) const
Get the file name based on the various parts of the file name stored in the member variables.

Private Attributes

- const std::string [m_FilePrefix](#)
Const string for file prefix.
- const std::string [m_TimeStamp](#)
Const string for timestamp.
- const std::string [m_FileExtension](#)
Const string for file extension.
- const std::vector< std::string > [m_DiagNames](#)
Const string for the diagnostic names.
- const bool [m_PrintFirstLineInfo](#)
Const variable that control whether we write a descriptive first line in every output file or not.
- const std::string [m_FirstLineInfoString](#)
Const string for the first line info.
- const [largecounter](#) [m_nrOutputsToCache](#) {}
- const [largecounter](#) [m_nrGeodesicsPerFile](#) {}
Const setting the max number of geodesics per file.
- bool [m_WriteToConsole](#) {false}
If this is false, then we are writing to file(s). At any time, if a file I/O error occurs, the output handler switches to outputting everything to the console.
- [largecounter](#) [m_PrevCached](#) {0}
How many outputs are already cached in m_nrOutputsToCache before the current iteration of output.
- [largecounter](#) [m_CurrentGeodesicsInFile](#) {0}
The number of geodesics already written to the current file (once this hits m_nrGeodesicsPerFile, this file is full)
- unsigned short [m_CurrentFullFiles](#) {0}
The current counter of completely full files (this is kept track of so that it knows what the next file to write output to is). (We had better not have more than 60k files!)
- std::vector< std::vector< std::string > > [m_AllCachedData](#) {}
Cached data that has not been written to a file yet (once this hits a size of > m_nrOutputsToCache, this must be written to file(s))

6.25.1 Detailed Description

// [GeodesicOutputHandler](#) handles all of the output to file. It gets passed all of the output strings for every [Geodesic](#), it then stores this data until it eventually writes all data to the appropriate files

6.25.2 Constructor & Destructor Documentation

6.25.2.1 GeodesicOutputHandler() [1/2]

```
GeodesicOutputHandler::GeodesicOutputHandler () [delete]
```

6.25.2.2 GeodesicOutputHandler() [2/2]

```
GeodesicOutputHandler::GeodesicOutputHandler (
    std::string FilePrefix,
    std::string TimeStamp,
    std::string FileExtension,
    std::vector< std::string > DiagNames,
    largecounter nrouterstocache = LARGE_COUNTER_MAX - 1,
    largecounter geodperfile = LARGE_COUNTER_MAX,
    std::string firstlineinfo = "")
```

Construct a new [Geodesic](#) Output Handler object and initialize all const member variables.

Parameters

<i>FilePrefix</i>	file prefix
<i>TimeStamp</i>	time stamp
<i>FileExtension</i>	file extensions
<i>DiagNames</i>	diagnostics names
<i>nrouterstocache</i>	number of geodesics to put to the cache
<i>geodperfile</i>	max number of geodesics per output file
<i>firstlineinfo</i>	the info to print on the first line

6.25.3 Member Function Documentation

6.25.3.1 GetFileName()

```
std::string GeodesicOutputHandler::GetFileName (
    int diagnr,
    unsigned short filenr) const [private]
```

Get the file name based on the various parts of the file name stored in the member variables.

Parameters

<i>diagnr</i>	
<i>filenr</i>	

Returns

std::string

6.25.3.2 getFullDescriptionStr()

```
std::string GeodesicOutputHandler::getFullDescriptionStr () const
```

full description string getter for the geodesic output handler

Returns

std::string

6.25.3.3 NewGeodesicOutput()

```
void GeodesicOutputHandler::NewGeodesicOutput (
    largecounter index,
    std::vector< std::string > theOutput)
```

Parameters

<i>index</i>	
<i>theOutput</i>	

6.25.3.4 OpenForFirstTime()

```
void GeodesicOutputHandler::OpenForFirstTime (
    const std::string & filename) [private]
```

We check here to see if the files are being put in a (sub)directory; if so, we create the directory/directories first to make sure creating/opening the file will succeed.

Parameters

<i>filename</i>	Filename to be found / created
-----------------	--------------------------------

6.25.3.5 OutputFinished()

```
void GeodesicOutputHandler::OutputFinished ()
```

There is no more output, so we write anything that is cached to file to clean up and finalize.

6.25.3.6 PrepareForOutput()

```
void GeodesicOutputHandler::PrepareForOutput (
    largecounter nrOutputToCome)
```

prepare the cache for output

Parameters

<i>nrOutputToCome</i>	number of outputs to come into the cache
-----------------------	--

6.25.3.7 WriteCachedOutputToFile()

```
void GeodesicOutputHandler::WriteCachedOutputToFile () [private]
```

Write cached output to a file.

6.25.4 Member Data Documentation**6.25.4.1 m_AllCachedData**

```
std::vector<std::vector<std::string> > GeodesicOutputHandler::m_AllCachedData {} [private]
```

Cached data that has not been written to a file yet (once this hits a size of > m_nrOutputsToCache, this must be written to file(s))

6.25.4.2 m_CurrentFullFiles

```
unsigned short GeodesicOutputHandler::m_CurrentFullFiles {0} [private]
```

The current counter of completely full files (this is kept track of so that it knows what the next file to write output to is). (We had better not have more than 60k files!)

6.25.4.3 m_CurrentGeodesicsInFile

```
largecounter GeodesicOutputHandler::m_CurrentGeodesicsInFile {0} [private]
```

The number of geodesics already written to the current file (once this hits m_nrGeodesicsPerFile, this file is full)

6.25.4.4 m_DiagNames

```
const std::vector<std::string> GeodesicOutputHandler::m_DiagNames [private]
```

Const string for the diagnostic names.

6.25.4.5 m_FileExtension

```
const std::string GeodesicOutputHandler::m_FileExtension [private]
```

Const string for file extension.

6.25.4.6 m_FilePrefix

```
const std::string GeodesicOutputHandler::m_FilePrefix [private]
```

Const string for file prefix.

6.25.4.7 m_FirstLineInfoString

```
const std::string GeodesicOutputHandler::m_FirstLineInfoString [private]
```

Const string for the first line info.

6.25.4.8 m_nrGeodesicsPerFile

```
const largecounter GeodesicOutputHandler::m_nrGeodesicsPerFile {} [private]
```

Const setting the max number of geodesics per file.

6.25.4.9 m_nrOutputsToCache

```
const largecounter GeodesicOutputHandler::m_nrOutputsToCache {} [private]
```

6.25.4.10 m_PrevCached

```
largecounter GeodesicOutputHandler::m_PrevCached {0} [private]
```

How many outputs are already cached in m_nrOutputsToCache before the current iteration of output.

6.25.4.11 m_PrintFirstLineInfo

```
const bool GeodesicOutputHandler::m_PrintFirstLineInfo [private]
```

Const variable that control whether we write a descriptive first line in every output file or not.

6.25.4.12 m_TimeStamp

```
const std::string GeodesicOutputHandler::m_TimeStamp [private]
```

Const string for timestamp.

6.25.4.13 m_WriteToConsole

```
bool GeodesicOutputHandler::m_WriteToConsole {false} [private]
```

If this is false, then we are writing to file(s). At any time, if a file I/O error occurs, the output handler switches to outputting everything to the console.

The documentation for this class was generated from the following files:

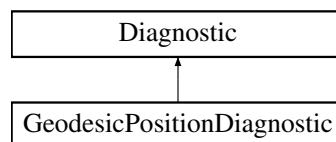
- FOORT/src/InputOutput.h
- FOORT/src/InputOutput.cpp

6.26 GeodesicPositionDiagnostic Class Reference

[Geodesic](#) position tracker.

```
#include <Diagnostics.h>
```

Inheritance diagram for GeodesicPositionDiagnostic:



Public Member Functions

- [GeodesicPositionDiagnostic](#) ([Geodesic](#) *const theGeodesic)
- void [Reset](#) () final
[GeodesicPositionDiagnostic](#) functions.
- void [UpdateData](#) () final
Update the saved points vector with the current position of the geodesic.
- std::string [getFullDataStr](#) () const final
Return full data string of all saved points.
- std::vector< [real](#) > [getFinalDataVal](#) () const final
Return the final (theta, phi) value of the geodesic.
- [real](#) [FinalDataValDistance](#) (const std::vector< [real](#) > &val1, const std::vector< [real](#) > &val2) const final
Return the angular distance between two geodesics.
- std::string [getNameStr](#) () const final
Simple name without spaces.
- std::string [getFullDescriptionStr](#) () const final
More descriptive string (with spaces)

Public Member Functions inherited from [Diagnostic](#)

- [Diagnostic](#) ()=delete
- [Diagnostic](#) ([Geodesic](#) *const theGeodesic)
- virtual [~Diagnostic](#) ()=default

Static Public Attributes

- static std::unique_ptr< [GeodesicPositionOptions](#) > [DiagOptions](#)

Private Attributes

- std::vector< [Point](#) > [m_AllSavedPoints](#) {}

Additional Inherited Members

Protected Member Functions inherited from [Diagnostic](#)

- bool [DecideUpdate](#) (const [UpdateFrequency](#) &myUpdateFrequency)
Returns true if the [Diagnostic](#) should update its initial status.

Protected Attributes inherited from [Diagnostic](#)

- [Geodesic](#) *const [m_OwnerGeodesic](#)
- [largecounter](#) [m_StepsSinceUpdated](#) {}

6.26.1 Detailed Description

[Geodesic](#) position tracker.

6.26.2 Constructor & Destructor Documentation

6.26.2.1 [GeodesicPositionDiagnostic\(\)](#)

```
GeodesicPositionDiagnostic::GeodesicPositionDiagnostic (
    Geodesic *const theGeodesic) [inline]
```

6.26.3 Member Function Documentation

6.26.3.1 [FinalDataValDistance\(\)](#)

```
real GeodesicPositionDiagnostic::FinalDataValDistance (
    const std::vector< real > & val1,
    const std::vector< real > & val2) const [final], [virtual]
```

Return the angular distance between two geodesics.

This is based on their final angles on the boundary sphere (theta, phi)

Parameters

<i>val1</i>	first geodesic final position
<i>val2</i>	second geodesic final position

Returns

real

Implements [Diagnostic](#).

6.26.3.2 getFinalDataVal()

```
std::vector< real > GeodesicPositionDiagnostic::getFinalDataVal () const [final], [virtual]
```

Return the final (theta, phi) value of the geodesic.

Returns

std::vector<real>

Implements [Diagnostic](#).

6.26.3.3 getFullDataStr()

```
std::string GeodesicPositionDiagnostic::getFullDataStr () const [final], [virtual]
```

Return full data string of all saved points.

The full output string looks like this: "(total nr steps) ;; (step 1) (step 2) (step 3) ..." where each step is a (space-separated) output of the geodesic coordinates at that step

Returns

std::string

Implements [Diagnostic](#).

6.26.3.4 getFullDescriptionStr()

```
std::string GeodesicPositionDiagnostic::getFullDescriptionStr () const [final], [virtual]
```

More descriptive string (with spaces)

Returns

std::string

Reimplemented from [Diagnostic](#).

6.26.3.5 getNameStr()

```
std::string GeodesicPositionDiagnostic::getNameStr () const [final], [virtual]
```

Simple name without spaces.

Returns

std::string

Implements [Diagnostic](#).

6.26.3.6 Reset()

```
void GeodesicPositionDiagnostic::Reset () [final], [virtual]
```

[GeodesicPositionDiagnostic](#) functions.

Empty out vector of points and call base Reset function

Reimplemented from [Diagnostic](#).

6.26.3.7 UpdateData()

```
void GeodesicPositionDiagnostic::UpdateData () [final], [virtual]
```

Update the saved points vector with the current position of the geodesic.

Implements [Diagnostic](#).

6.26.4 Member Data Documentation

6.26.4.1 DiagOptions

```
std::unique_ptr< GeodesicPositionOptions > GeodesicPositionDiagnostic::DiagOptions [static]
```

6.26.4.2 m_AllSavedPoints

```
std::vector<Point> GeodesicPositionDiagnostic::m_AllSavedPoints {} [private]
```

The documentation for this class was generated from the following files:

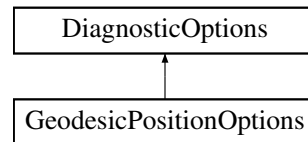
- FOORT/src/[Diagnostics.h](#)
- FOORT/src/[Config.cpp](#)
- FOORT/src/[Diagnostics.cpp](#)

6.27 GeodesicPositionOptions Struct Reference

Options for [GeodesicPositionDiagnostic](#).

```
#include <Diagnostics.h>
```

Inheritance diagram for GeodesicPositionOptions:



Public Member Functions

- [GeodesicPositionOptions](#) ([largecounter](#) outputsteps, [UpdateFrequency](#) thefrequency)

Public Member Functions inherited from [DiagnosticOptions](#)

- [DiagnosticOptions](#) ([UpdateFrequency](#) thefrequency)
- virtual [~DiagnosticOptions](#) ()=default

Public Attributes

- const [largecounter](#) OutputNrSteps

Public Attributes inherited from [DiagnosticOptions](#)

- const [UpdateFrequency](#) theUpdateFrequency

6.27.1 Detailed Description

Options for [GeodesicPositionDiagnostic](#).

Inherits from [DiagnosticOptions](#), and adds the number of steps to output.

6.27.2 Constructor & Destructor Documentation

6.27.2.1 GeodesicPositionOptions()

```
GeodesicPositionOptions::GeodesicPositionOptions (
    largecounter outputsteps,
    UpdateFrequency thefrequency) [inline]
```

6.27.3 Member Data Documentation

6.27.3.1 OutputNrSteps

```
const largecounter GeodesicPositionOptions::OutputNrSteps
```

The documentation for this struct was generated from the following file:

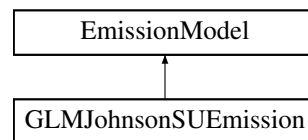
- FOORT/src/[Diagnostics.h](#)

6.28 GLMJohnsonSUEmission Struct Reference

The Johnson SU emission model used in GLM 2020, PRD 102(12):124004.

```
#include <DiagnosticsEmission.h>
```

Inheritance diagram for GLMJohnsonSUEmission:



Public Member Functions

- [GLMJohnsonSUEmission](#) ([real](#) mu, [real](#) gamma, [real](#) sigma)
- [real](#) [GetEmission](#) (const [Point](#) &p) const final
Get the emission at a point according to the Johnson SU model used in GLM.
- std::string [getFullDescriptionStr](#) () const final
Description string getter for the Johnson SU emission model.

Public Member Functions inherited from [EmissionModel](#)

- virtual [~EmissionModel](#) ()=default

Private Attributes

- const [real](#) [m_mu](#)
mu parameter
- const [real](#) [m_gamma](#)
gamma parameter
- const [real](#) [m_sigma](#)
sigma parameter

6.28.1 Detailed Description

The Johnson SU emission model used in GLM 2020, PRD 102(12):124004.

6.28.2 Constructor & Destructor Documentation

6.28.2.1 GLMJohnsonSUEmission()

```
GLMJohnsonSUEmission::GLMJohnsonSUEmission (  
    real mu,  
    real gamma,  
    real sigma) [inline]
```

6.28.3 Member Function Documentation

6.28.3.1 GetEmission()

```
real GLMJohnsonSUEmission::GetEmission (  
    const Point & p) const [final], [virtual]
```

Get the emission at a point according to the Johnson SU model used in GLM.

Parameters

<i>p</i>	Point at which to calculate the emission
----------	--

Returns

real Emission at the point

Implements [EmissionModel](#).

6.28.3.2 getFullDescriptionStr()

```
std::string GLMJohnsonSUEmission::getFullDescriptionStr () const [final], [virtual]
```

Description string getter for the Johnson SU emission model.

Returns

std::string

Reimplemented from [EmissionModel](#).

6.28.4 Member Data Documentation

6.28.4.1 m_gamma

```
const real GLMJohnsonSUEmission::m_gamma [private]
```

gamma parameter

6.28.4.2 m_mu

```
const real GLMJohnsonSUEmission::m_mu [private]
```

mu parameter

6.28.4.3 m_sigma

```
const real GLMJohnsonSUEmission::m_sigma [private]
```

sigma parameter

The documentation for this struct was generated from the following files:

- FOORT/src/[DiagnosticsEmission.h](#)
- FOORT/src/[DiagnosticsEmission.cpp](#)

6.29 Grid Class Reference

A class for a 2D grid that contains the values of the necessary functions.

```
#include <Grid.h>
```

Public Member Functions

- [Grid](#) (int [N_row](#), int [N_col](#))
A class for a 2D grid that contains the values of the necessary functions.
- [~Grid](#) ()
Destructor.
- double & [operator](#)() (int i, int j)
Overload the () operator to access the data.
- const double & [operator](#)() (int i, int j) const
Overload the () operator to access the data (const version)
- void [initialize_from_file](#) (std::string file)

Public Attributes

- int [N_row](#)
Number of rows in the grid.
- int [N_col](#)
Number of columns in the grid.
- double * [data](#)
Pointer to the data.
- int [size](#)
Number of rows times the number of columns.

6.29.1 Detailed Description

A class for a 2D grid that contains the values of the necessary functions.

6.29.2 Constructor & Destructor Documentation

6.29.2.1 Grid()

```
Grid::Grid (  
    int N_row,  
    int N_col)
```

A class for a 2D grid that contains the values of the necessary functions.

6.29.2.2 ~Grid()

```
Grid::~Grid () [inline]
```

Destructor.

6.29.3 Member Function Documentation

6.29.3.1 initialize_from_file()

```
void Grid::initialize_from_file (  
    std::string file)
```

6.29.3.2 operator>() [1/2]

```
double & Grid::operator() (  
    int i,  
    int j) [inline]
```

Overload the () operator to access the data.

6.29.3.3 operator() [2/2]

```
const double & Grid::operator() (
    int i,
    int j) const [inline]
```

Overload the () operator to access the data (const version)

6.29.4 Member Data Documentation

6.29.4.1 data

```
double* Grid::data
```

Pointer to the data.

6.29.4.2 N_col

```
int Grid::N_col
```

Number of columns in the grid.

6.29.4.3 N_row

```
int Grid::N_row
```

Number of rows in the grid.

6.29.4.4 size

```
int Grid::size
```

Number of rows times the number of columns.

The documentation for this class was generated from the following files:

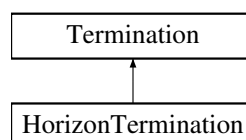
- [FOORT/src/Grid.h](#)
- [FOORT/src/Grid.cpp](#)

6.30 HorizonTermination Class Reference

[HorizonTermination](#): Terminate geodesics if they get too close to the horizon.

```
#include <Terminations.h>
```

Inheritance diagram for HorizonTermination:



Public Member Functions

- [HorizonTermination](#) ([Geodesic](#) *const theGeodesic)
- [Term CheckTermination](#) () final
Termination checker for the horizon termination.
- std::string [getFullDescriptionStr](#) () const final
Description string getter for the Horizon [Termination](#).

Public Member Functions inherited from [Termination](#)

- [Termination](#) ()=delete
- [Termination](#) ([Geodesic](#) *const theGeodesic)
- virtual void [Reset](#) ()
Reset to 0 to start integrating a new geodesic.
- virtual [~Termination](#) ()=default

Static Public Attributes

- static std::unique_ptr< [HorizonTermOptions](#) > [TermOptions](#)
Options (contains horizon radius, if we are using logarithmic r coordinate, and distance allowed from the horizon)

Additional Inherited Members

Protected Member Functions inherited from [Termination](#)

- bool [DecideUpdate](#) ([largecounter](#) UpdateNSteps)
Returns true if the [Termination](#) should update its internal status.

Protected Attributes inherited from [Termination](#)

- [Geodesic](#) *const [m_OwnerGeodesic](#)
The geodesic that owns the [Termination](#) (a const pointer to the [Geodesic](#))
- [largecounter](#) [m_StepsSinceUpdated](#) {}
The termination is itself in charge of keeping track of how many steps it has been since it has been updated. The [Termination](#)'s [TerminationOptions](#) struct tells it how many steps it needs to wait between updates.

6.30.1 Detailed Description

[HorizonTermination](#): Terminate geodesics if they get too close to the horizon.

6.30.2 Constructor & Destructor Documentation

6.30.2.1 HorizonTermination()

```
HorizonTermination::HorizonTermination (
    Geodesic *const theGeodesic) [inline]
```

6.30.3 Member Function Documentation

6.30.3.1 CheckTermination()

`Term` `HorizonTermination::CheckTermination () [final], [virtual]`

`Termination` checker for the horizon termination.

Returns

`Term`

Implements `Termination`.

6.30.3.2 getFullDescriptionStr()

`std::string` `HorizonTermination::getFullDescriptionStr () const [final], [virtual]`

Description string getter for the Horizon `Termination`.

Returns

`std::string`

Implements `Termination`.

6.30.4 Member Data Documentation

6.30.4.1 TermOptions

`std::unique_ptr< HorizonTermOptions >` `HorizonTermination::TermOptions [static]`

Options (contains horizon radius, if we are using logarithmic r coordinate, and distance allowed from the horizon)

The documentation for this class was generated from the following files:

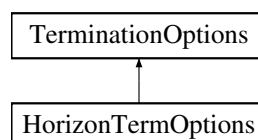
- FOORT/src/[Terminations.h](#)
- FOORT/src/[Config.cpp](#)
- FOORT/src/[Terminations.cpp](#)

6.31 HorizonTermOptions Struct Reference

Options for `HorizonTermination`.

```
#include <Terminations.h>
```

Inheritance diagram for `HorizonTermOptions`:



Public Member Functions

- [HorizonTermOptions](#) ([real](#) theHorizonRadius, [bool](#) therLogScale, [real](#) theAtHorizonEps, [largecounter](#) Nsteps)

Public Member Functions inherited from [TerminationOptions](#)

- [TerminationOptions](#) ([largecounter](#) Nsteps)
- virtual $\sim$ [TerminationOptions](#) ()=default

Public Attributes

- const [real](#) [HorizonRadius](#)
The radius of the horizon.
- const [real](#) [AtHorizonEps](#)
The distance allowed from the horizon.
- const [bool](#) [rLogScale](#)
Whether we are using a logarithmic r coordinate.

Public Attributes inherited from [TerminationOptions](#)

- const [largecounter](#) [UpdateEveryNSteps](#)
Number of steps between updates.

6.31.1 Detailed Description

Options for [HorizonTermination](#).

Keeps track of the horizon radius, whether we are using a logarithmic r coordinate, and the distance allowed from the horizon.

6.31.2 Constructor & Destructor Documentation

6.31.2.1 [HorizonTermOptions\(\)](#)

```
HorizonTermOptions::HorizonTermOptions (
    real theHorizonRadius,
    bool therLogScale,
    real theAtHorizonEps,
    largecounter Nsteps) [inline]
```

6.31.3 Member Data Documentation

6.31.3.1 [AtHorizonEps](#)

```
const real HorizonTermOptions::AtHorizonEps
```

The distance allowed from the horizon.

6.31.3.2 HorizonRadius

```
const real HorizonTermOptions::HorizonRadius
```

The radius of the horizon.

6.31.3.3 rLogScale

```
const bool HorizonTermOptions::rLogScale
```

Whether we are using a logarithmic r coordinate.

The documentation for this struct was generated from the following file:

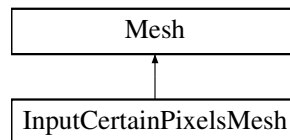
- FOORT/src/[Terminations.h](#)

6.32 InputCertainPixelsMesh Class Reference

[Mesh](#) which integrates only certain user-inputted pixels.

```
#include <Mesh.h>
```

Inheritance diagram for InputCertainPixelsMesh:



Public Member Functions

- [InputCertainPixelsMesh](#) ()=delete
- [InputCertainPixelsMesh](#) (const [InputCertainPixelsMesh](#) &)=delete
- [InputCertainPixelsMesh](#) ([largecounter](#) totalPixels, [DiagBitflag](#) valdiag)
Constructor for [InputCertainPixelsMesh](#).
- [largecounter](#) getCurNrGeodesics () const final
Get the current number of geodesics to be integrated.
- void [getNewInitConds](#) ([largecounter](#) index, [ScreenPoint](#) &newunitpoint, [ScreenIndex](#) &newscreenindex) const final
Get the initial conditions for the next pixel to be integrated.
- void [GeodesicFinished](#) ([largecounter](#) index, std::vector< [real](#) > finalValues) final
Function that is called when a geodesic is finished integrating.
- void [EndCurrentLoop](#) () final
End the current loop of integrating pixels.
- bool [IsFinished](#) () const final
Check whether the [Mesh](#) is finished integrating.
- std::string [getFullDescriptionStr](#) () const final
Description string getter.

Public Member Functions inherited from [Mesh](#)

- [Mesh](#) ([DiagBitflag](#) valdiag)
- virtual [~Mesh](#) ()=default

Private Attributes

- const [pixelcoord](#) [m_RowColumnSize](#)
Total pixel size of the screen (square grid)
- [largecounter](#) [m_TotalPixels](#) {0}
How many pixels have been inputted in total, i.e. need integrating.
- [std::vector](#)< [ScreenIndex](#) > [m_PixelsToIntegrate](#) {}
All pixels' location.
- bool [m_Finished](#) {false}
Are we finished integrating?

Additional Inherited Members

Protected Attributes inherited from [Mesh](#)

- const [std::unique_ptr](#)< const [Diagnostic](#) > [m_DistanceDiagnostic](#)
The [Diagnostic](#) (a const pointer to a const [Diagnostic](#) object) that is used to calculate distances (using [FinalData](#)↔[ValDistance\(\)](#)) between the "values" that are assigned to Geodesics.

6.32.1 Detailed Description

[Mesh](#) which integrates only certain user-inputted pixels.

6.32.2 Constructor & Destructor Documentation

6.32.2.1 [InputCertainPixelsMesh\(\)](#) [1/3]

```
InputCertainPixelsMesh::InputCertainPixelsMesh () [delete]
```

6.32.2.2 [InputCertainPixelsMesh\(\)](#) [2/3]

```
InputCertainPixelsMesh::InputCertainPixelsMesh (
    const InputCertainPixelsMesh & ) [delete]
```

6.32.2.3 [InputCertainPixelsMesh\(\)](#) [3/3]

```
InputCertainPixelsMesh::InputCertainPixelsMesh (
    largecounter totalPixels,
    DiagBitflag valdiag)
```

Constructor for [InputCertainPixelsMesh](#).

Constructor will ask for pixels to be inputted by the user through the console

Parameters

<i>totalPixels</i>	total number of pixels in the grid
<i>valdiag</i>	value diagnostic bitflag

6.32.3 Member Function Documentation

6.32.3.1 EndCurrentLoop()

```
void InputCertainPixelsMesh::EndCurrentLoop () [final], [virtual]
```

End the current loop of integrating pixels.

This [Mesh](#) only has one loop; we are now finished integrating

Implements [Mesh](#).

6.32.3.2 GeodesicFinished()

```
void InputCertainPixelsMesh::GeodesicFinished (
    largecounter index,
    std::vector< real > finalValues) [final], [virtual]
```

Function that is called when a geodesic is finished integrating.

This [Mesh](#) doesn't actually have to do anything with the geodesic (values) when it is done!

Parameters

<i>index</i>	index of the geodesic that is finished
<i>finalValues</i>	final values of the geodesic

Implements [Mesh](#).

6.32.3.3 getCurNrGeodesics()

```
largecounter InputCertainPixelsMesh::getCurNrGeodesics () const [final], [virtual]
```

Get the current number of geodesics to be integrated.

This is the total number of pixels in the grid

Returns

largecounter

Implements [Mesh](#).

6.32.3.4 getFullDescriptionStr()

```
std::string InputCertainPixelsMesh::getFullDescriptionStr () const [final], [virtual]
```

Description string getter.

Returns

std::string

Reimplemented from [Mesh](#).

6.32.3.5 getNewInitConds()

```
void InputCertainPixelsMesh::getNewInitConds (
    largecounter index,
    ScreenPoint & newunitpoint,
    ScreenIndex & newscreenindex) const [final], [virtual]
```

Get the initial conditions for the next pixel to be integrated.

The next pixel is the one with index m_CurrentPixel

Parameters

<i>index</i>	index of the pixel to be initialized
<i>newunitpoint</i>	new unit point to be initialized
<i>newscreenindex</i>	new screen index to be initialized

Implements [Mesh](#).

6.32.3.6 IsFinished()

```
bool InputCertainPixelsMesh::IsFinished () const [final], [virtual]
```

Check whether the [Mesh](#) is finished integrating.

Returns

true
false

Implements [Mesh](#).

6.32.4 Member Data Documentation

6.32.4.1 m_Finished

```
bool InputCertainPixelsMesh::m_Finished {false} [private]
```

Are we finished integrating?

6.32.4.2 m_PixelsToIntegrate

```
std::vector<ScreenIndex> InputCertainPixelsMesh::m_PixelsToIntegrate {} [private]
```

All pixels' location.

6.32.4.3 m_RowColumnSize

```
const pixelcoord InputCertainPixelsMesh::m_RowColumnSize [private]
```

Total pixel size of the screen (square grid)

6.32.4.4 m_TotalPixels

```
largecounter InputCertainPixelsMesh::m_TotalPixels {0} [private]
```

How many pixels have been inputted in total, i.e. need integrating.

The documentation for this class was generated from the following files:

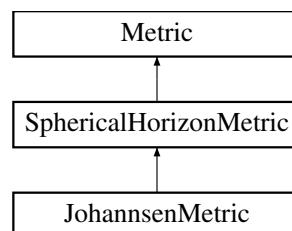
- FOORT/src/[Mesh.h](#)
- FOORT/src/[Mesh.cpp](#)

6.33 JohannsenMetric Class Reference

Johannsen black hole metric.

```
#include <Metric.h>
```

Inheritance diagram for JohannsenMetric:



Public Member Functions

- [JohannsenMetric](#) ()=delete
- [JohannsenMetric](#) (real aParam, real alpha13Param, real alpha22Param, real alpha52Param, real eps3Param, bool rLogScale=false)
Construct a new Johannsen [Metric](#) object. We always have a $M = 1$ BH.
- [TwoIndex getMetric_dd](#) (const [Point](#) &p) const final
Johannsen metric getter, indices down.
- [TwoIndex getMetric_uu](#) (const [Point](#) &p) const final
Johannsen metric getter, indices up.
- std::string [getFullDescriptionStr](#) () const final
Johannsen metric description string getter.

Public Member Functions inherited from [SphericalHorizonMetric](#)

- [SphericalHorizonMetric](#) ()=delete
- [SphericalHorizonMetric](#) (real HorizonRadius, bool rLogScale)
Construct a new Spherical Horizon [Metric](#) object.
- [real getHorizonRadius](#) () const
Get the radius of the horizon.

Public Member Functions inherited from [Metric](#)

- virtual [~Metric](#) ()=default
- [Metric](#) (bool rlogscale=false)
Construct a new [Metric](#) object.
- virtual [ThreeIndex getChristoffel_udd](#) (const [Point](#) &p) const
Return the Christoffel symbols of the metric (indices up, down, down)
- virtual [FourIndex getRiemann_uddd](#) (const [Point](#) &p) const
Riemann tensor of the metric (indices up, down, down, down)
- virtual [real getKretschmann](#) (const [Point](#) &p) const
Get the Kretschmann scalar at a point.
- bool [getrLogScale](#) () const
Get whether we are using a logarithmic radial scale.

Private Attributes

- const [real m_aParam](#)
a parameter for the Johannsen metric
- const [real m_alpha13Param](#)
alpha13 parameter for the Johannsen metric
- const [real m_alpha22Param](#)
alpha22 parameter for the Johannsen metric
- const [real m_alpha52Param](#)
alpha52 parameter for the Johannsen metric
- const [real m_eps3Param](#)
eps3 parameter for the Johannsen metric

Additional Inherited Members

Protected Attributes inherited from [SphericalHorizonMetric](#)

- const [real m_HorizonRadius](#)
Radius of the horizon.

Protected Attributes inherited from [Metric](#)

- std::vector< int > [m_Symmetries](#) {}
The symmetries (coordinate Killing vectors) of the metric. Should be set by descendant constructor.
- const bool [m_rLogScale](#)
Are we using a logarithmic r coordinate?

6.33.1 Detailed Description

Johannsen black hole metric.

Note

Implementation by Seppe Staelens

6.33.2 Constructor & Destructor Documentation

6.33.2.1 JohannsenMetric() [1/2]

```
JohannsenMetric::JohannsenMetric () [delete]
```

6.33.2.2 JohannsenMetric() [2/2]

```
JohannsenMetric::JohannsenMetric (
    real aParam,
    real alpha13Param,
    real alpha22Param,
    real alpha52Param,
    real eps3Param,
    bool rLogScale = false)
```

Construct a new Johannsen [Metric](#) object. We always have a $M = 1$ BH.

Parameters

<i>aParam</i>	a parameter for the Johannsen metric
<i>alpha13Param</i>	alpha13 parameter for the Johannsen metric
<i>alpha22Param</i>	alpha22 parameter for the Johannsen metric
<i>alpha52Param</i>	alpha52 parameter for the Johannsen metric
<i>eps3Param</i>	eps3 parameter for the Johannsen metric
<i>rLogScale</i>	whether we are using a logarithmic radial scale

Note

Implementation by Seppe Staelens

6.33.3 Member Function Documentation

6.33.3.1 getFullDescriptionStr()

```
std::string JohannsenMetric::getFullDescriptionStr () const [final], [virtual]
```

Johannsen metric description string getter.

Returns

std::string

Reimplemented from [Metric](#).

6.33.3.2 getMetric_dd()

```
TwoIndex JohanssenMetric::getMetric_dd (  
    const Point & p) const [final], [virtual]
```

Johanssen metric getter, indices down.

Parameters

p	Point at which to evaluate the metric
-----	---

Returns

[TwoIndex](#)Implements [Metric](#).**6.33.3.3 getMetric_uu()**

```
TwoIndex JohannsenMetric::getMetric_uu (  
    const Point & p) const [final], [virtual]
```

Johannsen metric getter, indices up.

Parameters

p	Point at which to evaluate the metric
-----	---

Returns

[TwoIndex](#)Implements [Metric](#).**6.33.4 Member Data Documentation****6.33.4.1 m_alpha13Param**

```
const real JohannsenMetric::m_alpha13Param [private]
```

alpha13 parameter for the Johannsen metric

6.33.4.2 m_alpha22Param

```
const real JohannsenMetric::m_alpha22Param [private]
```

alpha22 parameter for the Johannsen metric

6.33.4.3 m_alpha52Param

```
const real JohannsenMetric::m_alpha52Param [private]
```

alpha52 parameter for the Johannsen metric

6.33.4.4 m_aParam

```
const real JohannsenMetric::m_aParam [private]
```

a parameter for the Johannsen metric

6.33.4.5 m_eps3Param

```
const real JohannsenMetric::m_eps3Param [private]
```

eps3 parameter for the Johannsen metric

The documentation for this class was generated from the following files:

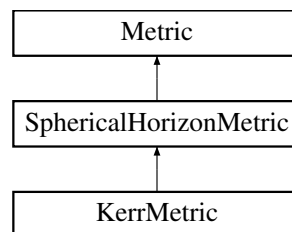
- FOORT/src/[Metric.h](#)
- FOORT/src/[Metric.cpp](#)

6.34 KerrMetric Class Reference

The Kerr metric.

```
#include <Metric.h>
```

Inheritance diagram for KerrMetric:



Public Member Functions

- [KerrMetric](#) ()=delete
- [KerrMetric](#) (real aParam, bool rLogScale=false, real mParam=1.)
Construct a new Kerr [Metric](#) object.
- [TwoIndex getMetric_dd](#) (const [Point](#) &p) const final
Kerr metric getter, indices down.
- [TwoIndex getMetric_uu](#) (const [Point](#) &p) const final
Kerr metric getter, indices up.
- std::string [getFullDescriptionStr](#) () const final
Kerr metric description string getter.

Public Member Functions inherited from [SphericalHorizonMetric](#)

- [SphericalHorizonMetric](#) ()=delete
- [SphericalHorizonMetric](#) (real HorizonRadius, bool rLogScale)
Construct a new Spherical Horizon [Metric](#) object.
- [real getHorizonRadius](#) () const
Get the radius of the horizon.

Public Member Functions inherited from [Metric](#)

- virtual [~Metric](#) ()=default
- [Metric](#) (bool rlogscale=false)
Construct a new [Metric](#) object.
- virtual [ThreeIndex getChristoffel_udd](#) (const [Point](#) &p) const
Return the Christoffel symbols of the metric (indices up, down, down)
- virtual [FourIndex getRiemann_uddd](#) (const [Point](#) &p) const
Riemann tensor of the metric (indices up, down, down, down)
- virtual [real getKretschmann](#) (const [Point](#) &p) const
Get the Kretschmann scalar at a point.
- bool [getrLogScale](#) () const
Get whether we are using a logarithmic radial scale.

Private Attributes

- const [real m_aParam](#)
Mass-rescaled rotation parameter for Kerr. Note that this should be between -1 and 1.
- const [real m_mParam](#)
Mass parameter for Kerr. Default is 1.

Additional Inherited Members

Protected Attributes inherited from [SphericalHorizonMetric](#)

- const [real m_HorizonRadius](#)
Radius of the horizon.

Protected Attributes inherited from [Metric](#)

- [std::vector< int > m_Symmetries](#) {}
The symmetries (coordinate Killing vectors) of the metric. Should be set by descendant constructor.
- const bool [m_rLogScale](#)
Are we using a logarithmic r coordinate?

6.34.1 Detailed Description

The Kerr metric.

6.34.2 Constructor & Destructor Documentation

6.34.2.1 KerrMetric() [1/2]

```
KerrMetric::KerrMetric () [delete]
```

6.34.2.2 KerrMetric() [2/2]

```
KerrMetric::KerrMetric (
    real aParam,
    bool rLogScale = false,
    real mParam = 1.)
```

Construct a new Kerr [Metric](#) object.

Parameters

<i>aParam</i>	a parameter for the Kerr metric
<i>rLogScale</i>	Whether we are using a logarithmic radial scale
<i>mParam</i>	mass parameter for the Kerr metric

6.34.3 Member Function Documentation

6.34.3.1 getFullDescriptionStr()

```
std::string KerrMetric::getFullDescriptionStr () const [final], [virtual]
```

Kerr metric description string getter.

Returns

std::string

Reimplemented from [Metric](#).

6.34.3.2 getMetric_dd()

```
TwoIndex KerrMetric::getMetric_dd (
    const Point & p) const [final], [virtual]
```

Kerr metric getter, indices down.

Parameters

<i>p</i>	Point at which to evaluate the metric
----------	---

Returns

[TwoIndex](#)

Implements [Metric](#).

6.34.3.3 getMetric_uu()

```
TwoIndex KerrMetric::getMetric_uu (
    const Point & p) const [final], [virtual]
```

Kerr metric getter, indices up.

Parameters

p	Point at which to evaluate the metric
-----	---

Returns

[TwoIndex](#)Implements [Metric](#).

6.34.4 Member Data Documentation

6.34.4.1 `m_aParam`

```
const real KerrMetric::m_aParam [private]
```

Mass-rescaled rotation parameter for Kerr. Note that this should be between -1 and 1.

6.34.4.2 `m_mParam`

```
const real KerrMetric::m_mParam [private]
```

Mass parameter for Kerr. Default is 1.

The documentation for this class was generated from the following files:

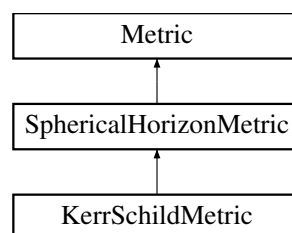
- [FOORT/src/Metric.h](#)
- [FOORT/src/Metric.cpp](#)

6.35 KerrSchildMetric Class Reference

Kerr metric in Kerr-Schild coordinates.

```
#include <Metric.h>
```

Inheritance diagram for `KerrSchildMetric`:



Public Member Functions

- [KerrSchildMetric](#) ()=delete
- [KerrSchildMetric](#) ([real](#) aParam, bool rLogScale=false)
Construct a new Kerr Schild [Metric](#) object. We always have a $M = 1$ BH.
- [TwoIndex getMetric_dd](#) (const [Point](#) &p) const final
Kerr-Schild metric getter, indices down.
- [TwoIndex getMetric_uu](#) (const [Point](#) &p) const final
Kerr-Schild metric getter, indices up.
- std::string [getFullDescriptionStr](#) () const final
Kerr-Schild metric description string getter.

Public Member Functions inherited from [SphericalHorizonMetric](#)

- [SphericalHorizonMetric](#) ()=delete
- [SphericalHorizonMetric](#) ([real](#) HorizonRadius, bool rLogScale)
Construct a new Spherical Horizon [Metric](#) object.
- [real getHorizonRadius](#) () const
Get the radius of the horizon.

Public Member Functions inherited from [Metric](#)

- virtual [~Metric](#) ()=default
- [Metric](#) (bool rlogscale=false)
Construct a new [Metric](#) object.
- virtual [ThreeIndex getChristoffel_udd](#) (const [Point](#) &p) const
Return the Christoffel symbols of the metric (indices up, down, down)
- virtual [FourIndex getRiemann_uddd](#) (const [Point](#) &p) const
Riemann tensor of the metric (indices up, down, down, down)
- virtual [real getKretschmann](#) (const [Point](#) &p) const
Get the Kretschmann scalar at a point.
- bool [getrLogScale](#) () const
Get whether we are using a logarithmic radial scale.

Private Attributes

- const [real](#) m_aParam
Rotation parameter for Kerr. Note that this should be between -1 and 1 since $M=1$.

Additional Inherited Members

Protected Attributes inherited from [SphericalHorizonMetric](#)

- const [real](#) m_HorizonRadius
Radius of the horizon.

Protected Attributes inherited from [Metric](#)

- `std::vector< int > m\_Symmetries {}`
The symmetries (coordinate Killing vectors) of the metric. Should be set by descendant constructor.
- `const bool m\_rLogScale`
Are we using a logarithmic r coordinate?

6.35.1 Detailed Description

Kerr metric in Kerr-Schild coordinates.

Note

Normalized so that $M = 1$

6.35.2 Constructor & Destructor Documentation

6.35.2.1 `KerrSchildMetric()` [1/2]

```
KerrSchildMetric::KerrSchildMetric () [delete]
```

6.35.2.2 `KerrSchildMetric()` [2/2]

```
KerrSchildMetric::KerrSchildMetric (
    real aParam,
    bool rLogScale = false)
```

Construct a new Kerr Schild [Metric](#) object. We always have a $M = 1$ BH.

Parameters

<i>aParam</i>	a parameter for the Kerr-Schild metric
<i>rLogScale</i>	whether we are using a logarithmic radial scale

6.35.3 Member Function Documentation

6.35.3.1 `getFullDescriptionStr()`

```
std::string KerrSchildMetric::getFullDescriptionStr () const [final], [virtual]
```

Kerr-Schild metric description string getter.

Returns

`std::string`

Reimplemented from [Metric](#).

6.35.3.2 `getMetric_dd()`

```
TwoIndex KerrSchildMetric::getMetric_dd (
    const Point & p) const [final], [virtual]
```

Kerr-Schild metric getter, indices down.

Parameters

p	Point at which to evaluate the metric
-----	---

Returns

[TwoIndex](#)Implements [Metric](#).**6.35.3.3 getMetric_uu()**

```
TwoIndex KerrSchildMetric::getMetric_uu (
    const Point & p) const [final], [virtual]
```

Kerr-Schild metric getter, indices up.

Parameters

p	Point at which to evaluate the metric
-----	---

Returns

[TwoIndex](#)Implements [Metric](#).**6.35.4 Member Data Documentation****6.35.4.1 m_aParam**

```
const real KerrSchildMetric::m_aParam [private]
```

Rotation parameter for Kerr. Note that this should be between -1 and 1 since $M=1$.

The documentation for this class was generated from the following files:

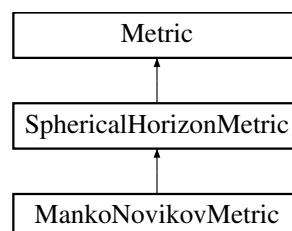
- FOORT/src/[Metric.h](#)
- FOORT/src/[Metric.cpp](#)

6.36 MankoNovikovMetric Class Reference

Mano-Novikov metric (with angular momentum and M3 parameter turned on)

```
#include <Metric.h>
```

Inheritance diagram for MankoNovikovMetric:



Public Member Functions

- [MankoNovikovMetric](#) ()=delete
- [MankoNovikovMetric](#) (real aParam, real alpha3Param, bool rLogScale=false)
Construct a new Manko Novikov [Metric](#) object. We always have a $M = 1$ BH.
- [TwoIndex getMetric_dd](#) (const [Point](#) &p) const final
Manko-Novikov metric getter, indices down.
- [TwoIndex getMetric_uu](#) (const [Point](#) &p) const final
Manko-Novikov metric getter, indices up.
- [std::string getFullDescriptionStr](#) () const final
Manko-Novikov metric description string getter.

Public Member Functions inherited from [SphericalHorizonMetric](#)

- [SphericalHorizonMetric](#) ()=delete
- [SphericalHorizonMetric](#) (real HorizonRadius, bool rLogScale)
Construct a new Spherical Horizon [Metric](#) object.
- [real getHorizonRadius](#) () const
Get the radius of the horizon.

Public Member Functions inherited from [Metric](#)

- virtual [~Metric](#) ()=default
- [Metric](#) (bool rlogscale=false)
Construct a new [Metric](#) object.
- virtual [ThreeIndex getChristoffel_udd](#) (const [Point](#) &p) const
Return the Christoffel symbols of the metric (indices up, down, down)
- virtual [FourIndex getRiemann_uddd](#) (const [Point](#) &p) const
Riemann tensor of the metric (indices up, down, down, down)
- virtual [real getKretschmann](#) (const [Point](#) &p) const
Get the Kretschmann scalar at a point.
- [bool getrLogScale](#) () const
Get whether we are using a logarithmic radial scale.

Private Attributes

- const [real m_aParam](#)
a parameter for the Manko-Novikov metric
- const [real m_alpha3Param](#)
alpha3 parameter for the Manko-Novikov metric
- const [real m_alphaParam](#)
Derived alpha parameter.
- const [real m_kParam](#)
Derived k parameter.

Additional Inherited Members

Protected Attributes inherited from [SphericalHorizonMetric](#)

- const [real m_HorizonRadius](#)
Radius of the horizon.

Protected Attributes inherited from [Metric](#)

- `std::vector< int > m_Symmetries {}`
The symmetries (coordinate Killing vectors) of the metric. Should be set by descendant constructor.
- `const bool m_rLogScale`
Are we using a logarithmic r coordinate?

6.36.1 Detailed Description

Mano-Novikov metric (with angular momentum and M3 parameter turned on)

Note

Implementation by Seppe Staelens

6.36.2 Constructor & Destructor Documentation

6.36.2.1 MankoNovikovMetric() [1/2]

```
MankoNovikovMetric::MankoNovikovMetric () [delete]
```

6.36.2.2 MankoNovikovMetric() [2/2]

```
MankoNovikovMetric::MankoNovikovMetric (
    real aParam,
    real alpha3Param,
    bool rLogScale = false)
```

Construct a new Manko Novikov [Metric](#) object. We always have a $M = 1$ BH.

The horizon is at $r = M + k$

Parameters

<i>aParam</i>	a parameter for the Manko-Novikov metric. The angular momentum $a = J/M^2$
<i>alpha3Param</i>	alpha3 parameter for the Manko-Novikov metric
<i>rLogScale</i>	whether we are using a logarithmic radial scale

6.36.3 Member Function Documentation

6.36.3.1 getFullDescriptionStr()

```
std::string MankoNovikovMetric::getFullDescriptionStr () const [final], [virtual]
```

Manko-Novikov metric description string getter.

Returns

`std::string`

Reimplemented from [Metric](#).

6.36.3.2 getMetric_dd()

```
TwoIndex MankoNovikovMetric::getMetric_dd (
    const Point & p) const [final], [virtual]
```

Manko-Novikov metric getter, indices down.

Parameters

p	Point at which to evaluate the metric
-----	---

Returns

[TwoIndex](#)Implements [Metric](#).**6.36.3.3 getMetric_uu()**

```
TwoIndex MankoNovikovMetric::getMetric_uu (  
    const Point & p) const [final], [virtual]
```

Manko-Novikov metric getter, indices up.

Parameters

p	Point at which to evaluate the metric
-----	---

Returns

[TwoIndex](#)Implements [Metric](#).**6.36.4 Member Data Documentation****6.36.4.1 m_alpha3Param**

```
const real MankoNovikovMetric::m_alpha3Param [private]
```

alpha3 parameter for the Manko-Novikov metric

6.36.4.2 m_alphaParam

```
const real MankoNovikovMetric::m_alphaParam [private]
```

Derived alpha parameter.

6.36.4.3 m_aParam

```
const real MankoNovikovMetric::m_aParam [private]
```

a parameter for the Manko-Novikov metric

6.36.4.4 m_kParam

```
const real MankoNovikovMetric::m_kParam [private]
```

Derived k parameter.

The documentation for this class was generated from the following files:

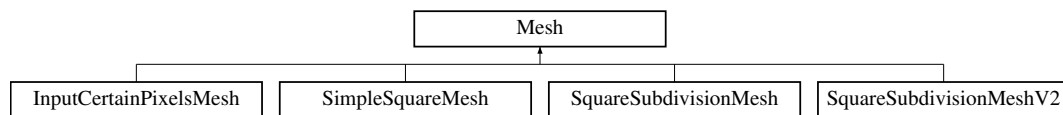
- FOORT/src/[Metric.h](#)
- FOORT/src/[Metric.cpp](#)

6.37 Mesh Class Reference

Abstract base [Mesh](#) class.

```
#include <Mesh.h>
```

Inheritance diagram for Mesh:



Public Member Functions

- [Mesh](#) ([DiagBitflag](#) valdiag)
- virtual [~Mesh](#) ()=default
- virtual [largecounter](#) [getCurNrGeodesics](#) () const =0
- virtual void [getNewInitConds](#) ([largecounter](#) index, [ScreenPoint](#) &newunitpoint, [ScreenIndex](#) &newscreenindex) const =0
- virtual void [GeodesicFinished](#) ([largecounter](#) index, std::vector< [real](#) > finalValues)=0
- virtual void [EndCurrentLoop](#) ()=0
- virtual bool [IsFinished](#) () const =0
- virtual std::string [getFullDescriptionStr](#) () const

Basic description string getter.

Protected Attributes

- const std::unique_ptr< const [Diagnostic](#) > [m_DistanceDiagnostic](#)

The [Diagnostic](#) (a const pointer to a const [Diagnostic](#) object) that is used to calculate distances (using [FinalData](#)↔[ValDistance\(\)](#)) between the "values" that are assigned to Geodesics.

6.37.1 Detailed Description

Abstract base [Mesh](#) class.

6.37.2 Constructor & Destructor Documentation

6.37.2.1 Mesh()

```
Mesh::Mesh (
    DiagBitflag valdiag) [inline]
```

6.37.2.2 ~Mesh()

```
virtual Mesh::~Mesh () [virtual], [default]
```

6.37.3 Member Function Documentation

6.37.3.1 EndCurrentLoop()

```
virtual void Mesh::EndCurrentLoop () [pure virtual]
```

Implemented in [InputCertainPixelsMesh](#), [SimpleSquareMesh](#), [SquareSubdivisionMesh](#), and [SquareSubdivisionMeshV2](#).

6.37.3.2 GeodesicFinished()

```
virtual void Mesh::GeodesicFinished (
    largecounter index,
    std::vector< real > finalValues) [pure virtual]
```

Implemented in [InputCertainPixelsMesh](#), [SimpleSquareMesh](#), [SquareSubdivisionMesh](#), and [SquareSubdivisionMeshV2](#).

6.37.3.3 getCurNrGeodesics()

```
virtual largecounter Mesh::getCurNrGeodesics () const [pure virtual]
```

Implemented in [InputCertainPixelsMesh](#), [SimpleSquareMesh](#), [SquareSubdivisionMesh](#), and [SquareSubdivisionMeshV2](#).

6.37.3.4 getFullDescriptionStr()

```
std::string Mesh::getFullDescriptionStr () const [virtual]
```

Basic description string getter.

Returns

std::string

Reimplemented in [InputCertainPixelsMesh](#), [SimpleSquareMesh](#), [SquareSubdivisionMesh](#), and [SquareSubdivisionMeshV2](#).

6.37.3.5 getNewInitConds()

```
virtual void Mesh::getNewInitConds (
    largecounter index,
    ScreenPoint & newunitpoint,
    ScreenIndex & newscreenindex) const [pure virtual]
```

Implemented in [InputCertainPixelsMesh](#), [SimpleSquareMesh](#), [SquareSubdivisionMesh](#), and [SquareSubdivisionMeshV2](#).

6.37.3.6 IsFinished()

```
virtual bool Mesh::IsFinished () const [pure virtual]
```

Implemented in [InputCertainPixelsMesh](#), [SimpleSquareMesh](#), [SquareSubdivisionMesh](#), and [SquareSubdivisionMeshV2](#).

6.37.4 Member Data Documentation

6.37.4.1 m_DistanceDiagnostic

```
const std::unique_ptr<const Diagnostic> Mesh::m_DistanceDiagnostic [protected]
```

The [Diagnostic](#) (a const pointer to a const [Diagnostic](#) object) that is used to calculate distances (using `FinalData↔ValDistance()`) between the "values" that are assigned to Geodesics.

The documentation for this class was generated from the following files:

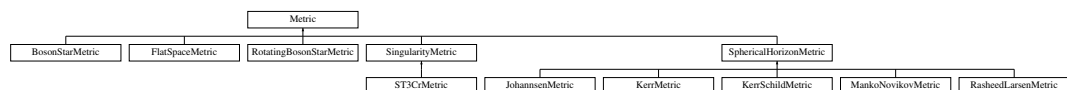
- FOORT/src/[Mesh.h](#)
- FOORT/src/[Mesh.cpp](#)

6.38 Metric Class Reference

The abstract base class for all Metrics.

```
#include <Metric.h>
```

Inheritance diagram for Metric:



Public Member Functions

- virtual `~Metric()`=default
- `Metric` (bool rlogscale=false)
Construct a new `Metric` object.
- virtual `TwoIndex getMetric_dd` (const `Point` &p) const =0
- virtual `TwoIndex getMetric_uu` (const `Point` &p) const =0
- virtual `ThreeIndex getChristoffel_udd` (const `Point` &p) const
Return the Christoffel symbols of the metric (indices up, down, down)
- virtual `FourIndex getRiemann_uddd` (const `Point` &p) const
Riemann tensor of the metric (indices up, down, down, down)
- virtual `real getKretschmann` (const `Point` &p) const
Get the Kretschmann scalar at a point.
- virtual std::string `getFullDescriptionStr` () const
Base class description string getter.
- bool `getrLogScale` () const
Get whether we are using a logarithmic radial scale.

Protected Attributes

- std::vector< int > `m_Symmetries` {}
The symmetries (coordinate Killing vectors) of the metric. Should be set by descendant constructor.
- const bool `m_rLogScale`
Are we using a logarithmic r coordinate?

6.38.1 Detailed Description

The abstract base class for all Metrics.

6.38.2 Constructor & Destructor Documentation

6.38.2.1 ~Metric()

```
virtual Metric::~Metric () [virtual], [default]
```

6.38.2.2 Metric()

```
Metric::Metric (
    bool rlogscale = false)
```

Construct a new `Metric` object.

Parameters

<code>rlogscale</code>	Whether we are using a logarithmic radial scale
------------------------	---

6.38.3 Member Function Documentation

6.38.3.1 getChristoffel_udd()

```
ThreeIndex Metric::getChristoffel_udd (
    const Point & p) const [virtual]
```

Return the Christoffel symbols of the metric (indices up, down, down)

Parameters

p	Point at which to evaluate the Christoffel symbols
-----	--

Returns

[ThreeIndex](#)**6.38.3.2 getFullDescriptionStr()**

```
std::string Metric::getFullDescriptionStr () const [virtual]
```

Base class description string getter.

Returns

std::string

Reimplemented in [BosonStarMetric](#), [FlatSpaceMetric](#), [JohannsenMetric](#), [KerrMetric](#), [KerrSchildMetric](#), [MankoNovikovMetric](#), [RasheedLarsenMetric](#), [RotatingBosonStarMetric](#), and [ST3CrMetric](#).

6.38.3.3 getKretschmann()

```
real Metric::getKretschmann (
    const Point & p) const [virtual]
```

Get the Kretschmann scalar at a point.

The Kretschmann scalar is defined as $R_{\{abcd\}} R^{\{abcd\}}$

Note

This needs to be implemented still.

Parameters

p	Point at which to evaluate the Kretschmann scalar
-----	---

Returns

real

6.38.3.4 getMetric_dd()

```
virtual TwoIndex Metric::getMetric_dd (
    const Point & p) const [pure virtual]
```

Implemented in [BosonStarMetric](#), [FlatSpaceMetric](#), [JohannsenMetric](#), [KerrMetric](#), [KerrSchildMetric](#), [MankoNovikovMetric](#), [RasheedLarsenMetric](#), [RotatingBosonStarMetric](#), and [ST3CrMetric](#).

6.38.3.5 getMetric_uu()

```
virtual TwoIndex Metric::getMetric_uu (
    const Point & p) const [pure virtual]
```

Implemented in [BosonStarMetric](#), [FlatSpaceMetric](#), [JohannsenMetric](#), [KerrMetric](#), [KerrSchildMetric](#), [MankoNovikovMetric](#), [RasheedLarsenMetric](#), [RotatingBosonStarMetric](#), and [ST3CrMetric](#).

6.38.3.6 getRiemann_uddd()

```
FourIndex Metric::getRiemann_uddd (
    const Point & p) const [virtual]
```

Riemann tensor of the metric (indices up, down, down, down)

Parameters

p	Point at which to evaluate the Riemann tensor
-----	---

Note

This needs to be implemented still.

Returns

[FourIndex](#)

6.38.3.7 getrLogScale()

```
bool Metric::getrLogScale () const
```

Get whether we are using a logarithmic radial scale.

Returns

true

false

6.38.4 Member Data Documentation

6.38.4.1 m_rLogScale

```
const bool Metric::m_rLogScale [protected]
```

Are we using a logarithmic r coordinate?

6.38.4.2 m_Symmetries

```
std::vector<int> Metric::m_Symmetries {} [protected]
```

The symmetries (coordinate Killing vectors) of the metric. Should be set by descendant constructor.

The documentation for this class was generated from the following files:

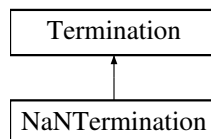
- FOORT/src/[Metric.h](#)
- FOORT/src/[Metric.cpp](#)

6.39 NaNTermination Class Reference

[NaNTermination](#): Terminate geodesics if they contain a NaN in position or velocity.

```
#include <Terminations.h>
```

Inheritance diagram for NaNTermination:



Public Member Functions

- [NaNTermination](#) ([Geodesic](#) *const theGeodesic)
- [Term CheckTermination](#) () final
Check if the geodesic contains a NaN.
- std::string [getFullDescriptionStr](#) () const final
Description string getter for the NaN [Termination](#).

Public Member Functions inherited from [Termination](#)

- [Termination](#) ()=delete
- [Termination](#) ([Geodesic](#) *const theGeodesic)
- virtual void [Reset](#) ()
Reset to 0 to start integrating a new geodesic.
- virtual [~Termination](#) ()=default

Static Public Attributes

- static std::unique_ptr< [NaNTermOptions](#) > [TermOptions](#)
Options (contains horizon radius, if we are using logarithmic r coordinate, and distance allowed from the horizon)

Additional Inherited Members

Protected Member Functions inherited from [Termination](#)

- bool [DecideUpdate](#) ([largecounter](#) UpdateNSteps)
Returns true if the [Termination](#) should update its internal status.

Protected Attributes inherited from [Termination](#)

- [Geodesic](#) *const [m_OwnerGeodesic](#)
The geodesic that owns the [Termination](#) (a const pointer to the [Geodesic](#))
- [largecounter](#) [m_StepsSinceUpdated](#) {}
The termination is itself in charge of keeping track of how many steps it has been since it has been updated. The [Termination](#)'s [TerminationOptions](#) struct tells it how many steps it needs to wait between updates.

6.39.1 Detailed Description

[NaNTermination](#): Terminate geodesics if they contain a NaN in position or velocity.

6.39.2 Constructor & Destructor Documentation

6.39.2.1 NaNTermination()

```
NaNTermination::NaNTermination (
    Geodesic *const theGeodesic) [inline]
```

6.39.3 Member Function Documentation

6.39.3.1 CheckTermination()

```
Term NaNTermination::CheckTermination () [final], [virtual]
```

Check if the geodesic contains a NaN.

Returns

[Term](#)

Implements [Termination](#).

6.39.3.2 getFullDescriptionStr()

```
std::string NaNTermination::getFullDescriptionStr () const [final], [virtual]
```

Description string getter for the NaN [Termination](#).

Returns

std::string

Implements [Termination](#).

6.39.4 Member Data Documentation

6.39.4.1 TermOptions

```
std::unique_ptr< NaNTermOptions > NaNTermination::TermOptions [static]
```

Options (contains horizon radius, if we are using logarithmic r coordinate, and distance allowed from the horizon)

The documentation for this class was generated from the following files:

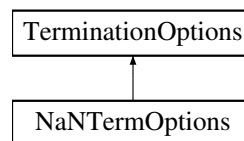
- FOORT/src/Terminations.h
- FOORT/src/Config.cpp
- FOORT/src/Terminations.cpp

6.40 NaNTermOptions Struct Reference

Options for [NaNTermination](#).

```
#include <Terminations.h>
```

Inheritance diagram for NaNTermOptions:



Public Member Functions

- [NaNTermOptions](#) (bool consoleoutputon, [largecounter](#) Nsteps)

Public Member Functions inherited from [TerminationOptions](#)

- [TerminationOptions](#) ([largecounter](#) Nsteps)
- virtual [~TerminationOptions](#) ()=default

Public Attributes

- const bool [OutputToConsole](#)
Whether to output to the console when a NaN is encountered.

Public Attributes inherited from [TerminationOptions](#)

- const [largecounter](#) [UpdateEveryNSteps](#)
Number of steps between updates.

6.40.1 Detailed Description

Options for [NaNTermination](#).

6.40.2 Constructor & Destructor Documentation

6.40.2.1 NaNTermOptions()

```
NaNTermOptions::NaNTermOptions (
    bool consoleoutputon,
    largecounter Nsteps) [inline]
```

6.40.3 Member Data Documentation

6.40.3.1 OutputToConsole

```
const bool NaNTermOptions::OutputToConsole
```

Whether to output to the console when a NaN is encountered.

The documentation for this struct was generated from the following file:

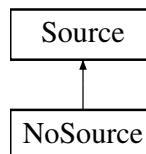
- FOORT/src/[Terminations.h](#)

6.41 NoSource Class Reference

[NoSource](#) class.

```
#include <Geodesic.h>
```

Inheritance diagram for NoSource:



Public Member Functions

- [NoSource](#) (const [Metric](#) *const theMetric)
- [OneIndex getSource](#) ([Point](#) pos, [OneIndex](#) vel) const final
Returns zero source for the geodesic equation: no force felt by geodesic.
- std::string [getFullDescriptionStr](#) () const final
Full description string getter for [NoSource](#) class.

Public Member Functions inherited from [Source](#)

- [Source](#) (const [Metric](#) *const theMetric)
- virtual [~Source](#) ()=default

Additional Inherited Members

Protected Attributes inherited from [Source](#)

- const [Metric](#) *const [m_theMetric](#)
A const pointer to a const metric.

6.41.1 Detailed Description

[NoSource](#) class.

There is no source, i.e. the geodesic is indeed a geodesic (and feels no force)

6.41.2 Constructor & Destructor Documentation

6.41.2.1 NoSource()

```
NoSource::NoSource (
    const Metric *const theMetric) [inline]
```

6.41.3 Member Function Documentation

6.41.3.1 getFullDescriptionStr()

```
std::string NoSource::getFullDescriptionStr () const [final], [virtual]
```

Full description string getter for [NoSource](#) class.

Returns

std::string

Reimplemented from [Source](#).

6.41.3.2 getSource()

```
OneIndex NoSource::getSource (
    Point pos,
    OneIndex vel) const [final], [virtual]
```

Returns zero source for the geodesic equation: no force felt by geodesic.

Parameters

<i>pos</i>	Position of the geodesic
<i>vel</i>	Velocity of the geodesic

Returns

[OneIndex Source](#) for the geodesic

Implements [Source](#).

The documentation for this class was generated from the following files:

- FOORT/src/[Geodesic.h](#)
- FOORT/src/[Geodesic.cpp](#)

6.42 SquareSubdivisionMesh::PixelInfo Struct Reference

A struct the [Mesh](#) uses to keep all information about a given pixel.

Public Member Functions

- [PixelInfo](#) ([ScreenIndex](#) ind, int subdiv)

Public Attributes

- [ScreenIndex](#) [Index](#) {}
The pixel's screenindex.
- int [SubdivideLevel](#) {}
The level at which the pixel has been subdivided. Note: initial grid pixels are at 1; pixels at 0 are pixels that cannot be subdivided (for example, at the right or lower edges)
- [real](#) [Weight](#) {-1}
Weight of the pixel: if negative, this signifies that it needs to be updated/calculated! The weight is determined as the max of the distance (as calculated by the value [Diagnostic](#)) between its values and those of its right, lower, and right-lower neighbors.
- `std::vector< real >` [DiagValue](#) {}
The values associated to this pixel (as calculated by the value [Diagnostic](#))
- `largecounter` [LowerNbrIndex](#) {0}
Where its lower neighbor is located in `m_AllPixels`. Note: the pixel with index 0 is (0,0) and can never be the lower or right neighbor of any other pixel!
- `largecounter` [RightNbrIndex](#) {0}
Where its right neighbor is located in `m_AllPixels`. Note: the pixel with index 0 is (0,0) and can never be the lower or right neighbor of any other pixel!

6.42.1 Detailed Description

A struct the [Mesh](#) uses to keep all information about a given pixel.

6.42.2 Constructor & Destructor Documentation

6.42.2.1 PixelInfo()

```
SquareSubdivisionMesh::PixelInfo::PixelInfo (
    ScreenIndex ind,
    int subdiv) [inline]
```

6.42.3 Member Data Documentation

6.42.3.1 DiagValue

```
std::vector<real> SquareSubdivisionMesh::PixelInfo::DiagValue {}
```

The values associated to this pixel (as calculated by the value [Diagnostic](#))

6.42.3.2 Index

```
ScreenIndex SquareSubdivisionMesh::PixelInfo::Index {}
```

The pixel's screenindex.

6.42.3.3 LowerNbrIndex

```
largecounter SquareSubdivisionMesh::PixelInfo::LowerNbrIndex {0}
```

Where its lower neighbor is located in m_AllPixels. Note: the pixel with index 0 is (0,0) and can never be the lower or right neighbor of any other pixel!

6.42.3.4 RightNbrIndex

```
largecounter SquareSubdivisionMesh::PixelInfo::RightNbrIndex {0}
```

Where its right neighbor is located in m_AllPixels. Note: the pixel with index 0 is (0,0) and can never be the lower or right neighbor of any other pixel!

6.42.3.5 SubdivideLevel

```
int SquareSubdivisionMesh::PixelInfo::SubdivideLevel {}
```

The level at which the pixel has been subdivided. Note: initial grid pixels are at 1; pixels at 0 are pixels that cannot be subdivided (for example, at the right or lower edges)

6.42.3.6 Weight

```
real SquareSubdivisionMesh::PixelInfo::Weight {-1}
```

Weight of the pixel: if negative, this signifies that it needs to be updated/calculated! The weight is determined as the max of the distance (as calculated by the value [Diagnostic](#)) between its values and those of its right, lower, and right-lower neighbors.

The documentation for this struct was generated from the following file:

- FOORT/src/[Mesh.h](#)

6.43 SquareSubdivisionMeshV2::PixelInfo Struct Reference

A struct the [Mesh](#) uses to keep all information about a given pixel.

Public Member Functions

- [PixelInfo](#) ([ScreenIndex](#) ind, int subdiv)

Public Attributes

- const [ScreenIndex](#) Index {}
The pixel's screenindex: this gets set by the constructor and cannot change anymore.
- int [SubdivideLevel](#) {}
The level at which the pixel has been subdivided. Note: initial grid pixels are at 1; pixels at 0 are pixels that cannot be subdivided (for example, at the right or lower edges)
- [real](#) Weight {-1}
Weight of the pixel: if negative, this signifies that it needs to be updated/calculated! The weight is determined as the max of the distance (as calculated by the value [Diagnostic](#)) between its values and those of its right, lower, and right-lower neighbors.
- std::vector< [real](#) > [DiagValue](#) {}
The values associated to this pixel (as calculated by the value [Diagnostic](#))
- [PixelInfo](#) * [LeftNbr](#) {nullptr}
Pointer to its left neighbor.
- [PixelInfo](#) * [RightNbr](#) {nullptr}
Pointer to its right neighbor.
- [PixelInfo](#) * [UpNbr](#) {nullptr}
Pointer to its upper neighbor.
- [PixelInfo](#) * [DownNbr](#) {nullptr}
Pointer to its lower neighbor.
- [PixelInfo](#) * [SEdiagNbr](#) {nullptr}
Pointer to its lower-right neighbor.

6.43.1 Detailed Description

A struct the [Mesh](#) uses to keep all information about a given pixel.

6.43.2 Constructor & Destructor Documentation

6.43.2.1 PixelInfo()

```
SquareSubdivisionMeshV2::PixelInfo::PixelInfo (
    ScreenIndex ind,
    int subdiv) [inline]
```

6.43.3 Member Data Documentation

6.43.3.1 DiagValue

```
std::vector<real> SquareSubdivisionMeshV2::PixelInfo::DiagValue {}
```

The values associated to this pixel (as calculated by the value [Diagnostic](#))

6.43.3.2 DownNbr

```
PixelInfo* SquareSubdivisionMeshV2::PixelInfo::DownNbr {nullptr}
```

Pointer to its lower neighbor.

6.43.3.3 Index

```
const ScreenIndex SquareSubdivisionMeshV2::PixelInfo::Index {}
```

The pixel's screenindex: this gets set by the constructor and cannot change anymore.

6.43.3.4 LeftNbr

```
PixelInfo* SquareSubdivisionMeshV2::PixelInfo::LeftNbr {nullptr}
```

Pointer to its left neighbor.

6.43.3.5 RightNbr

```
PixelInfo* SquareSubdivisionMeshV2::PixelInfo::RightNbr {nullptr}
```

Pointer to its right neighbor.

6.43.3.6 SEdiagNbr

```
PixelInfo* SquareSubdivisionMeshV2::PixelInfo::SEdiagNbr {nullptr}
```

Pointer to its lower-righttt neighbor.

6.43.3.7 SubdivideLevel

```
int SquareSubdivisionMeshV2::PixelInfo::SubdivideLevel {}
```

The level at which the pixel has been subdivided. Note: initial grid pixels are at 1; pixels at 0 are pixels that cannot be subdivided (for example, at the right or lower edges)

6.43.3.8 UpNbr

```
PixelInfo* SquareSubdivisionMeshV2::PixelInfo::UpNbr {nullptr}
```

Pointer to its upper neighbor.

6.43.3.9 Weight

```
real SquareSubdivisionMeshV2::PixelInfo::Weight {-1}
```

Weight of the pixel: if negative, this signifies that it needs to be updated/calculated! The weight is determined as the max of the distance (as calculated by the value [Diagnostic](#)) between its values and those of its right, lower, and right-lower neighbors.

The documentation for this struct was generated from the following file:

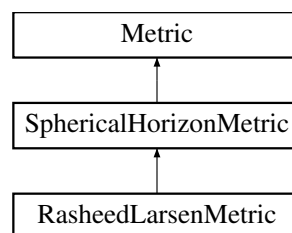
- FOORT/src/[Mesh.h](#)

6.44 RasheedLarsenMetric Class Reference

Rasheed-Larsen black hole.

```
#include <Metric.h>
```

Inheritance diagram for RasheedLarsenMetric:



Public Member Functions

- [RasheedLarsenMetric](#) ()=delete
- [RasheedLarsenMetric](#) (real mParam, real aParam, real pParam, real qParam, bool rLogScale=false)
Construct a new Rasheed Larsen [Metric](#) object. All parameters are rescaled by the mass to end up with a $M = 1$ BH.
- [TwoIndex getMetric_dd](#) (const [Point](#) &p) const final
Rasheed-Larsen metric getter, indices down.
- [TwoIndex getMetric_uu](#) (const [Point](#) &p) const final
Rasheed-Larsen metric getter, indices up.
- std::string [getFullDescriptionStr](#) () const final
Rasheed-Larsen metric description string getter.

Public Member Functions inherited from SphericalHorizonMetric

- [SphericalHorizonMetric](#) ()=delete
- [SphericalHorizonMetric](#) (real HorizonRadius, bool rLogScale)
Construct a new Spherical Horizon Metric object.
- [real getHorizonRadius](#) () const
Get the radius of the horizon.

Public Member Functions inherited from Metric

- virtual [~Metric](#) ()=default
- [Metric](#) (bool rlogscale=false)
Construct a new Metric object.
- virtual [ThreeIndex getChristoffel_udd](#) (const [Point](#) &p) const
Return the Christoffel symbols of the metric (indices up, down, down)
- virtual [FourIndex getRiemann_uddd](#) (const [Point](#) &p) const
Riemann tensor of the metric (indices up, down, down, down)
- virtual [real getKretschmann](#) (const [Point](#) &p) const
Get the Kretschmann scalar at a point.
- bool [getrLogScale](#) () const
Get whether we are using a logarithmic radial scale.

Private Attributes

- const [real m_aParam](#)
a parameter for the Rasheed-Larsen metric
- const [real m_mParam](#)
m parameter for the Rasheed-Larsen metric
- const [real m_pParam](#)
p parameter for the Rasheed-Larsen metric
- const [real m_qParam](#)
q parameter for the Rasheed-Larsen metric

Additional Inherited Members

Protected Attributes inherited from SphericalHorizonMetric

- const [real m_HorizonRadius](#)
Radius of the horizon.

Protected Attributes inherited from Metric

- [std::vector< int > m_Symmetries](#) {}
The symmetries (coordinate Killing vectors) of the metric. Should be set by descendant constructor.
- const bool [m_rLogScale](#)
Are we using a logarithmic r coordinate?

6.44.1 Detailed Description

Rasheed-Larsen black hole.

6.44.2 Constructor & Destructor Documentation

6.44.2.1 RasheedLarsenMetric() [1/2]

```
RasheedLarsenMetric::RasheedLarsenMetric () [delete]
```

6.44.2.2 RasheedLarsenMetric() [2/2]

```
RasheedLarsenMetric::RasheedLarsenMetric (
    real mParam,
    real aParam,
    real pParam,
    real qParam,
    bool rLogScale = false)
```

Construct a new Rasheed Larsen [Metric](#) object. All parameters are rescaled by the mass to end up with a $M = 1$ BH.

Parameters

<i>mParam</i>	m parameter for the Rasheed-Larsen metric
<i>aParam</i>	a parameter for the Rasheed-Larsen metric
<i>pParam</i>	p parameter for the Rasheed-Larsen metric
<i>qParam</i>	q parameter for the Rasheed-Larsen metric
<i>rLogScale</i>	whether we are using a logarithmic radial scale

6.44.3 Member Function Documentation

6.44.3.1 getFullDescriptionStr()

```
std::string RasheedLarsenMetric::getFullDescriptionStr () const [final], [virtual]
```

Rasheed-Larsen metric description string getter.

Returns

std::string

Reimplemented from [Metric](#).

6.44.3.2 getMetric_dd()

```
TwoIndex RasheedLarsenMetric::getMetric_dd (
    const Point & p) const [final], [virtual]
```

Rasheed-Larsen metric getter, indices down.

Parameters

p	Point at which to evaluate the metric
-----	---------------------------------------

Returns

TwoIndex

Implements [Metric](#).

6.44.3.3 getMetric_uu()

```
TwoIndex RasheedLarsenMetric::getMetric_uu (
    const Point & p) const [final], [virtual]
```

Rasheed-Larsen metric getter, indices up.

Parameters

p	Point at which to evaluate the metric
-----	---------------------------------------

Returns

TwoIndex

Implements [Metric](#).

6.44.4 Member Data Documentation

6.44.4.1 m_aParam

```
const real RasheedLarsenMetric::m_aParam [private]
```

a parameter for the Rasheed-Larsen metric

6.44.4.2 m_mParam

```
const real RasheedLarsenMetric::m_mParam [private]
```

m parameter for the Rasheed-Larsen metric

6.44.4.3 m_pParam

```
const real RasheedLarsenMetric::m_pParam [private]
```

p parameter for the Rasheed-Larsen metric

6.44.4.4 m_qParam

```
const real RasheedLarsenMetric::m_qParam [private]
```

q parameter for the Rasheed-Larsen metric

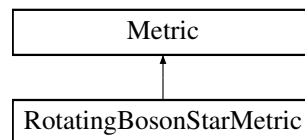
The documentation for this class was generated from the following files:

- FOORT/src/[Metric.h](#)
- FOORT/src/[Metric.cpp](#)

6.45 RotatingBosonStarMetric Class Reference

```
#include <Metric.h>
```

Inheritance diagram for RotatingBosonStarMetric:



Public Member Functions

- [RotatingBosonStarMetric](#) (bool rLogScale=false, std::string MetricFolder="RotatingBosonStar/data_Will/", int num_x=500, int num_th=399, bool FlipAngularMomentum=false)
Construct a new Rotating Boson Star [Metric](#) object.
- [~RotatingBosonStarMetric](#) ()=default
- [TwoIndex getMetric_dd](#) (const [Point](#) &p) const final
RotatingBosonStar metric getter, indices down.
- [TwoIndex getMetric_uu](#) (const [Point](#) &p) const final
RotatingBosonStar metric getter, indices up.
- std::string [getFullDescriptionStr](#) () const final
RotatingBosonStar metric description string getter.

Public Member Functions inherited from [Metric](#)

- virtual [~Metric](#) ()=default
- [Metric](#) (bool rlogscale=false)
Construct a new [Metric](#) object.
- virtual [ThreeIndex getChristoffel_udd](#) (const [Point](#) &p) const
Return the Christoffel symbols of the metric (indices up, down, down)
- virtual [FourIndex getRiemann_uddd](#) (const [Point](#) &p) const
Riemann tensor of the metric (indices up, down, down, down)
- virtual [real getKretschmann](#) (const [Point](#) &p) const
Get the Kretschmann scalar at a point.
- bool [getrLogScale](#) () const
Get whether we are using a logarithmic radial scale.

Public Attributes

- `std::unique_ptr< BicubicSplineInterpolator > m_fInterpolator`
The grid interpolators.
- `std::unique_ptr< BicubicSplineInterpolator > m_lInterpolator`
- `std::unique_ptr< BicubicSplineInterpolator > m_gInterpolator`
- `std::unique_ptr< BicubicSplineInterpolator > m_OmegaInterpolator`
- `const int m\_OmegaSign`

Private Attributes

- `std::unique_ptr< Grid > m\_grid\_f`
The grids with the metric functions.
- `std::unique_ptr< Grid > m\_grid\_l`
- `std::unique_ptr< Grid > m\_grid\_g`
- `std::unique_ptr< Grid > m\_grid\_Omega`

Additional Inherited Members

Protected Attributes inherited from [Metric](#)

- `std::vector< int > m\_Symmetries {}`
The symmetries (coordinate Killing vectors) of the metric. Should be set by descendant constructor.
- `const bool m\_rLogScale`
Are we using a logarithmic r coordinate?

6.45.1 Constructor & Destructor Documentation

6.45.1.1 RotatingBosonStarMetric()

```
RotatingBosonStarMetric::RotatingBosonStarMetric (
    bool rLogScale = false,
    std::string MetricFolder = "RotatingBosonStar/data_Will/",
    int num_x = 500,
    int num_th = 399,
    bool FlipAngularMomentum = false)
```

Construct a new Rotating Boson Star [Metric](#) object.

Parameters

<i>rLogScale</i>	whether we are using a logarithmic radial scale
<i>FlipAngularMomentum</i>	whether to flip the sign of Omega (reverse rotation direction)

6.45.1.2 ~RotatingBosonStarMetric()

```
RotatingBosonStarMetric::~~RotatingBosonStarMetric () [default]
```

6.45.2 Member Function Documentation

6.45.2.1 getFullDescriptionStr()

```
std::string RotatingBosonStarMetric::getFullDescriptionStr () const [final], [virtual]
```

RotatingBosonStar metric description string getter.

Returns

std::string

Reimplemented from [Metric](#).

6.45.2.2 getMetric_dd()

```
TwoIndex RotatingBosonStarMetric::getMetric_dd (  
    const Point & p) const [final], [virtual]
```

RotatingBosonStar metric getter, indices down.

Parameters

p	Point at which to evaluate the metric
-----	---

Returns

[TwoIndex](#)

Implements [Metric](#).

6.45.2.3 getMetric_uu()

```
TwoIndex RotatingBosonStarMetric::getMetric_uu (  
    const Point & p) const [final], [virtual]
```

RotatingBosonStar metric getter, indices up.

Parameters

p	Point at which to evaluate the metric
-----	---

Returns

[TwoIndex](#)

Implements [Metric](#).

6.45.3 Member Data Documentation

6.45.3.1 m_fInterpolator

`std::unique_ptr<BicubicSplineInterpolator> RotatingBosonStarMetric::m_fInterpolator`

The grid interpolators.

6.45.3.2 m_gInterpolator

`std::unique_ptr<BicubicSplineInterpolator> RotatingBosonStarMetric::m_gInterpolator`

6.45.3.3 m_grid_f

`std::unique_ptr<Grid> RotatingBosonStarMetric::m_grid_f [private]`

The grids with the metric functions.

6.45.3.4 m_grid_g

`std::unique_ptr<Grid> RotatingBosonStarMetric::m_grid_g [private]`

6.45.3.5 m_grid_l

`std::unique_ptr<Grid> RotatingBosonStarMetric::m_grid_l [private]`

6.45.3.6 m_grid_Omega

`std::unique_ptr<Grid> RotatingBosonStarMetric::m_grid_Omega [private]`

6.45.3.7 m_lInterpolator

`std::unique_ptr<BicubicSplineInterpolator> RotatingBosonStarMetric::m_lInterpolator`

6.45.3.8 m_OmegaInterpolator

`std::unique_ptr<BicubicSplineInterpolator> RotatingBosonStarMetric::m_OmegaInterpolator`

6.45.3.9 m_OmegaSign

`const int RotatingBosonStarMetric::m_OmegaSign`

The documentation for this class was generated from the following files:

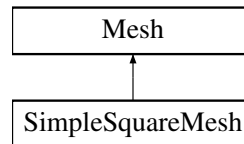
- FOORT/src/[Metric.h](#)
- FOORT/src/[Metric.cpp](#)

6.46 SimpleSquareMesh Class Reference

A simple square mesh that will integrate a square of evenly spaced pixels.

```
#include <Mesh.h>
```

Inheritance diagram for SimpleSquareMesh:



Public Member Functions

- [SimpleSquareMesh](#) ()=delete
- [SimpleSquareMesh](#) ([largecounter](#) totalPixels, [DiagBitflag](#) valdiag)
- [largecounter](#) getCurNrGeodesics () const final
Get the current number of geodesics to be integrated.
- void [getNewInitConds](#) ([largecounter](#) index, [ScreenPoint](#) &newunitpoint, [ScreenIndex](#) &newscreenindex) const final
Get the initial conditions for the next pixel to be integrated.
- void [GeodesicFinished](#) ([largecounter](#) index, std::vector< [real](#) > finalValues) final
Function that is called when a geodesic is finished integrating.
- void [EndCurrentLoop](#) () final
End the current loop of integrating pixels.
- bool [IsFinished](#) () const final
Check whether the [Mesh](#) is finished integrating.
- std::string [getFullDescriptionStr](#) () const final
Description string getter.

Public Member Functions inherited from [Mesh](#)

- [Mesh](#) ([DiagBitflag](#) valdiag)
- virtual [~Mesh](#) ()=default

Private Attributes

- const [largecounter](#) [m_TotalPixels](#)
Total amount of pixels in grid (is the square of m_RowColumnSize)
- const [pixelcoord](#) [m_RowColumnSize](#)
Amount of pixels per row or column (square grid)
- bool [m_Finished](#) {false}
Are we done integrating or not?

Additional Inherited Members

Protected Attributes inherited from [Mesh](#)

- `const std::unique_ptr< const Diagnostic > m_DistanceDiagnostic`
The [Diagnostic](#) (a const pointer to a const [Diagnostic](#) object) that is used to calculate distances (using `FinalData←ValDistance()`) between the "values" that are assigned to Geodesics.

6.46.1 Detailed Description

A simple square mesh that will integrate a square of evenly spaced pixels.

6.46.2 Constructor & Destructor Documentation

6.46.2.1 SimpleSquareMesh() [1/2]

```
SimpleSquareMesh::SimpleSquareMesh () [delete]
```

6.46.2.2 SimpleSquareMesh() [2/2]

```
SimpleSquareMesh::SimpleSquareMesh (
    largecounter totalPixels,
    DiagBitflag valdiag) [inline]
```

6.46.3 Member Function Documentation

6.46.3.1 EndCurrentLoop()

```
void SimpleSquareMesh::EndCurrentLoop () [final], [virtual]
```

End the current loop of integrating pixels.

We are done integrating after one loop

Implements [Mesh](#).

6.46.3.2 GeodesicFinished()

```
void SimpleSquareMesh::GeodesicFinished (
    largecounter index,
    std::vector< real > finalValues) [final], [virtual]
```

Function that is called when a geodesic is finished integrating.

This [Mesh](#) doesn't actually have to do anything with the geodesic (values) when it is done!

Parameters

<i>index</i>	index of the geodesic that is finished
<i>finalValues</i>	final values of the geodesic

Implements [Mesh](#).

6.46.3.3 getCurNrGeodesics()

```
largecounter SimpleSquareMesh::getCurNrGeodesics () const [final], [virtual]
```

Get the current number of geodesics to be integrated.

This is the total number of pixels in the grid

Returns

largecounter

Implements [Mesh](#).

6.46.3.4 getFullDescriptionStr()

```
std::string SimpleSquareMesh::getFullDescriptionStr () const [final], [virtual]
```

Description string getter.

Returns

std::string

Reimplemented from [Mesh](#).

6.46.3.5 getNewInitConds()

```
void SimpleSquareMesh::getNewInitConds (
    largecounter index,
    ScreenPoint & newunitpoint,
    ScreenIndex & newscreenindex) const [final], [virtual]
```

Get the initial conditions for the next pixel to be integrated.

The next pixel is the one with index m_CurrentPixel

Parameters

<i>index</i>	index of the pixel to be initialized
<i>newunitpoint</i>	new unit point to be initialized
<i>newscreenindex</i>	new screen index to be initialized

Implements [Mesh](#).

6.46.3.6 IsFinished()

```
bool SimpleSquareMesh::IsFinished () const [final], [virtual]
```

Check whether the [Mesh](#) is finished integrating.

We are done if we have sent all pixels to be integrated (there is only one iteration of pixels)

Returns

```
true
false
```

Implements [Mesh](#).

6.46.4 Member Data Documentation

6.46.4.1 m_Finished

```
bool SimpleSquareMesh::m_Finished {false} [private]
```

Are we done integrating or not?

6.46.4.2 m_RowColumnSize

```
const pixelcoord SimpleSquareMesh::m_RowColumnSize [private]
```

Amount of pixels per row or column (square grid)

6.46.4.3 m_TotalPixels

```
const largecounter SimpleSquareMesh::m_TotalPixels [private]
```

Total amount of pixels in grid (is the square of m_RowColumnSize)

The documentation for this class was generated from the following files:

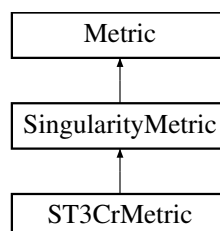
- FOORT/src/[Mesh.h](#)
- FOORT/src/[Mesh.cpp](#)

6.47 SingularityMetric Class Reference

Abstract base class for a metric with an arbitrary number of singularities (of arbitrary codimension)

```
#include <Metric.h>
```

Inheritance diagram for SingularityMetric:



Public Member Functions

- [SingularityMetric](#) (std::vector< [Singularity](#) > thesings, bool rLogScale)
Construct a new [Singularity Metric](#) object.
- std::vector< [Singularity](#) > [getSingularities](#) () const
Getters for the singularities.

Public Member Functions inherited from [Metric](#)

- virtual [~Metric](#) ()=default
- [Metric](#) (bool rlogscale=false)
Construct a new [Metric](#) object.
- virtual [TwoIndex](#) [getMetric_dd](#) (const [Point](#) &p) const =0
- virtual [TwoIndex](#) [getMetric_uu](#) (const [Point](#) &p) const =0
- virtual [ThreeIndex](#) [getChristoffel_udd](#) (const [Point](#) &p) const
Return the Christoffel symbols of the metric (indices up, down, down)
- virtual [FourIndex](#) [getRiemann_uddd](#) (const [Point](#) &p) const
Riemann tensor of the metric (indices up, down, down, down)
- virtual [real](#) [getKretschmann](#) (const [Point](#) &p) const
Get the Kretschmann scalar at a point.
- virtual std::string [getFullDescriptionStr](#) () const
Base class description string getter.
- bool [getrLogScale](#) () const
Get whether we are using a logarithmic radial scale.

Protected Attributes

- const std::vector< [Singularity](#) > [m_AllSingularities](#)
All singularities of the metric.

Protected Attributes inherited from [Metric](#)

- std::vector< int > [m_Symmetries](#) {}
The symmetries (coordinate Killing vectors) of the metric. Should be set by descendant constructor.
- const bool [m_rLogScale](#)
Are we using a logarithmic r coordinate?

6.47.1 Detailed Description

Abstract base class for a metric with an arbitrary number of singularities (of arbitrary codimension)

6.47.2 Constructor & Destructor Documentation

6.47.2.1 SingularityMetric()

```
SingularityMetric::SingularityMetric (
    std::vector< Singularity > thesings,
    bool rLogScale)
```

Construct a new [Singularity Metric](#) object.

Parameters

<i>thesings</i>	The singularities to include in the metric
<i>rLogScale</i>	whether we are using a logarithmic radial scale

6.47.3 Member Function Documentation

6.47.3.1 getSingularities()

```
std::vector< Singularity > SingularityMetric::getSingularities () const
```

Getters for the singularities.

Returns

```
std::vector<Singularity>
```

6.47.4 Member Data Documentation

6.47.4.1 m_AllSingularities

```
const std::vector<Singularity> SingularityMetric::m_AllSingularities [protected]
```

All singularities of the metric.

The documentation for this class was generated from the following files:

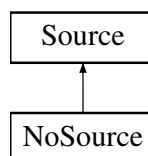
- FOORT/src/[Metric.h](#)
- FOORT/src/[Metric.cpp](#)

6.48 Source Class Reference

Abstract [Source](#) base class.

```
#include <Geodesic.h>
```

Inheritance diagram for Source:



Public Member Functions

- [Source](#) (const [Metric](#) *const theMetric)
- virtual [~Source](#) ()=default
- virtual [OneIndex](#) [getSource](#) ([Point](#) pos, [OneIndex](#) vel) const =0
- virtual std::string [getFullDescriptionStr](#) () const

Basic full description string getter for [Source](#) base class.

Protected Attributes

- const [Metric](#) *const [m_theMetric](#)

A const pointer to a const metric.

6.48.1 Detailed Description

Abstract [Source](#) base class.

6.48.2 Constructor & Destructor Documentation

6.48.2.1 Source()

```
Source::Source (  
    const Metric *const theMetric) [inline]
```

6.48.2.2 ~Source()

```
virtual Source::~Source () [virtual], [default]
```

6.48.3 Member Function Documentation

6.48.3.1 getFullDescriptionStr()

```
std::string Source::getFullDescriptionStr () const [virtual]
```

Basic full description string getter for [Source](#) base class.

Returns

std::string

Reimplemented in [NoSource](#).

6.48.3.2 getSource()

```
virtual OneIndex Source::getSource (
    Point pos,
    OneIndex vel) const [pure virtual]
```

Implemented in [NoSource](#).

6.48.4 Member Data Documentation

6.48.4.1 m_theMetric

```
const Metric* const Source::m_theMetric [protected]
```

A const pointer to a const metric.

The documentation for this class was generated from the following files:

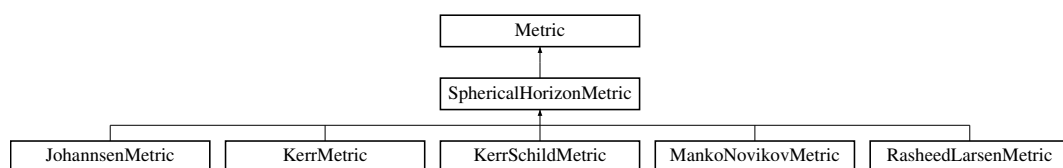
- [FOORT/src/Geodesic.h](#)
- [FOORT/src/Geodesic.cpp](#)

6.49 SphericalHorizonMetric Class Reference

Abstract base class for a metric with a spherical horizon (i.e. horizon at constant radius r)

```
#include <Metric.h>
```

Inheritance diagram for SphericalHorizonMetric:



Public Member Functions

- [SphericalHorizonMetric](#) ()=delete
- [SphericalHorizonMetric](#) (real HorizonRadius, bool rLogScale)
Construct a new Spherical Horizon [Metric](#) object.
- [real getHorizonRadius](#) () const
Get the radius of the horizon.

Public Member Functions inherited from [Metric](#)

- virtual [~Metric](#) ()=default
- [Metric](#) (bool rlogscale=false)
Construct a new [Metric](#) object.
- virtual [TwoIndex](#) [getMetric_dd](#) (const [Point](#) &p) const =0
- virtual [TwoIndex](#) [getMetric_uu](#) (const [Point](#) &p) const =0
- virtual [ThreeIndex](#) [getChristoffel_udd](#) (const [Point](#) &p) const
Return the Christoffel symbols of the metric (indices up, down, down)
- virtual [FourIndex](#) [getRiemann_uddd](#) (const [Point](#) &p) const
Riemann tensor of the metric (indices up, down, down, down)
- virtual [real](#) [getKretschmann](#) (const [Point](#) &p) const
Get the Kretschmann scalar at a point.
- virtual std::string [getFullDescriptionStr](#) () const
Base class description string getter.
- bool [getrLogScale](#) () const
Get whether we are using a logarithmic radial scale.

Protected Attributes

- const [real](#) [m_HorizonRadius](#)
Radius of the horizon.

Protected Attributes inherited from [Metric](#)

- std::vector< int > [m_Symmetries](#) {}
The symmetries (coordinate Killing vectors) of the metric. Should be set by descendant constructor.
- const bool [m_rLogScale](#)
Are we using a logarithmic r coordinate?

6.49.1 Detailed Description

Abstract base class for a metric with a spherical horizon (i.e. horizon at constant radius r)

6.49.2 Constructor & Destructor Documentation

6.49.2.1 [SphericalHorizonMetric](#)() [1/2]

```
SphericalHorizonMetric::SphericalHorizonMetric () [delete]
```

6.49.2.2 [SphericalHorizonMetric](#)() [2/2]

```
SphericalHorizonMetric::SphericalHorizonMetric (
    real HorizonRadius,
    bool rLogScale)
```

Construct a new Spherical Horizon [Metric](#) object.

Parameters

<i>HorizonRadius</i>	Radius of the horizon
<i>rLogScale</i>	Whether we are using a logarithmic radial scale

6.49.3 Member Function Documentation

6.49.3.1 getHorizonRadius()

```
real SphericalHorizonMetric::getHorizonRadius () const
```

Get the radius of the horizon.

Returns

real

6.49.4 Member Data Documentation

6.49.4.1 m_HorizonRadius

```
const real SphericalHorizonMetric::m_HorizonRadius [protected]
```

Radius of the horizon.

The documentation for this class was generated from the following files:

- FOORT/src/[Metric.h](#)
- FOORT/src/[Metric.cpp](#)

6.50 tk::spline Class Reference

```
#include <Spline.h>
```

Public Types

- enum [spline_type](#) { [linear](#) = 10 , [cspline](#) = 30 , [cspline_hermite](#) = 31 }
- enum [bd_type](#) { [first_deriv](#) = 1 , [second_deriv](#) = 2 }

Public Member Functions

- [spline](#) ()
- [spline](#) (const std::vector< double > &X, const std::vector< double > &Y, [spline_type](#) type=[cspline](#), bool [make_monotonic](#)=false, [bd_type](#) left=[second_deriv](#), double left_value=0.0, [bd_type](#) right=[second_deriv](#), double right_value=0.0)
- void [set_boundary](#) ([bd_type](#) left, double left_value, [bd_type](#) right, double right_value)
- void [set_points](#) (const std::vector< double > &x, const std::vector< double > &y, [spline_type](#) type=[cspline](#))
- bool [make_monotonic](#) ()
- double [operator\(\)](#) (double x) const
- double [deriv](#) (int order, double x) const
- std::vector< double > [get_x](#) () const
- std::vector< double > [get_y](#) () const
- double [get_x_min](#) () const
- double [get_x_max](#) () const

Protected Member Functions

- void [set_coeffs_from_b](#) ()
- size_t [find_closest](#) (double x) const

Protected Attributes

- std::vector< double > [m_x](#)
- std::vector< double > [m_y](#)
- std::vector< double > [m_b](#)
- std::vector< double > [m_c](#)
- std::vector< double > [m_d](#)
- double [m_c0](#)
- [spline_type](#) [m_type](#)
- [bd_type](#) [m_left](#)
- [bd_type](#) [m_right](#)
- double [m_left_value](#)
- double [m_right_value](#)
- bool [m_made_monotonic](#)

6.50.1 Member Enumeration Documentation

6.50.1.1 [bd_type](#)

```
enum tk::spline::bd_type
```

Enumerator

first_deriv	
second_deriv	

6.50.1.2 [spline_type](#)

```
enum tk::spline::spline_type
```

Enumerator

linear	
cspline	
cspline_hermite	

6.50.2 Constructor & Destructor Documentation

6.50.2.1 spline() [1/2]

```
tk::spline::spline ()
```

6.50.2.2 spline() [2/2]

```
tk::spline::spline (
    const std::vector< double > & X,
    const std::vector< double > & Y,
    spline_type type = cspline,
    bool make_monotonic = false,
    bd_type left = second_deriv,
    double left_value = 0.0,
    bd_type right = second_deriv,
    double right_value = 0.0)
```

6.50.3 Member Function Documentation

6.50.3.1 deriv()

```
double tk::spline::deriv (
    int order,
    double x) const
```

6.50.3.2 find_closest()

```
size_t tk::spline::find_closest (
    double x) const [protected]
```

6.50.3.3 get_x()

```
std::vector< double > tk::spline::get_x () const
```

6.50.3.4 get_x_max()

```
double tk::spline::get_x_max () const
```

6.50.3.5 get_x_min()

```
double tk::spline::get_x_min () const
```

6.50.3.6 get_y()

```
std::vector< double > tk::spline::get_y () const
```

6.50.3.7 make_monotonic()

```
bool tk::spline::make_monotonic ()
```

6.50.3.8 operator()()

```
double tk::spline::operator() (
    double x) const
```

6.50.3.9 set_boundary()

```
void tk::spline::set_boundary (
    spline::bd_type left,
    double left_value,
    spline::bd_type right,
    double right_value)
```

6.50.3.10 set_coeffs_from_b()

```
void tk::spline::set_coeffs_from_b () [protected]
```

6.50.3.11 set_points()

```
void tk::spline::set_points (
    const std::vector< double > & x,
    const std::vector< double > & y,
    spline_type type = cspline)
```

6.50.4 Member Data Documentation

6.50.4.1 m_b

```
std::vector<double> tk::spline::m_b [protected]
```

6.50.4.2 m_c

`std::vector<double> tk::spline::m_c` [protected]

6.50.4.3 m_c0

`double tk::spline::m_c0` [protected]

6.50.4.4 m_d

`std::vector<double> tk::spline::m_d` [protected]

6.50.4.5 m_left

`bd_type tk::spline::m_left` [protected]

6.50.4.6 m_left_value

`double tk::spline::m_left_value` [protected]

6.50.4.7 m_made_monotonic

`bool tk::spline::m_made_monotonic` [protected]

6.50.4.8 m_right

`bd_type tk::spline::m_right` [protected]

6.50.4.9 m_right_value

`double tk::spline::m_right_value` [protected]

6.50.4.10 m_type

`spline_type tk::spline::m_type` [protected]

6.50.4.11 m_x

`std::vector<double> tk::spline::m_x` [protected]

6.50.4.12 m_y

```
std::vector<double> tk::spline::m_y [protected]
```

The documentation for this class was generated from the following files:

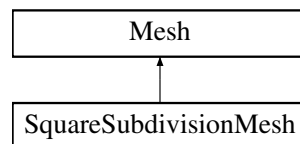
- FOORT/src/[Spline.h](#)
- FOORT/src/[Spline.cpp](#)

6.51 SquareSubdivisionMesh Class Reference

Adaptive subdivision [Mesh](#).

```
#include <Mesh.h>
```

Inheritance diagram for SquareSubdivisionMesh:



Classes

- struct [PixelInfo](#)
A struct the [Mesh](#) uses to keep all information about a given pixel.

Public Member Functions

- [SquareSubdivisionMesh](#) ()=delete
- [SquareSubdivisionMesh](#) ([largecounter](#) maxPixels, [largecounter](#) initialPixels, int maxSubdivide, [largecounter](#) iterationPixels, bool initialSubToFinal, [DiagBitflag](#) valdiag)
- [largecounter](#) getCurNrGeodesics () const final
Return the current number of geodesics to be integrated.
- void [getNewInitConds](#) ([largecounter](#) index, [ScreenPoint](#) &newunitpoint, [ScreenIndex](#) &newscreenindex) const final
Get the initial conditions for the next pixel to be integrated.
- void [GeodesicFinished](#) ([largecounter](#) index, std::vector< [real](#) > finalValues) final
Called when a geodesic is finished integrating: the final values are stored.
- void [EndCurrentLoop](#) () final
End the current loop of integrating pixels.
- bool [IsFinished](#) () const final
Check whether the [Mesh](#) is finished integrating.
- std::string [getFullDescriptionStr](#) () const final
Description string getter.

Public Member Functions inherited from [Mesh](#)

- [Mesh](#) ([DiagBitflag](#) valdiag)
- virtual [~Mesh](#) ()=default

Private Member Functions

- void [InitializeFirstGrid](#) ()
Helper function to initialize the first grid of pixels to integrate.
- void [UpdateAllNeighbors](#) ()
Helper function to update all pixel neighbors (if needed)
- void [UpdateAllWeights](#) ()
Helper function to update all pixel weights.
- void [SubdivideAndQueue](#) ([largecounter](#) ind)
Helper function to subdivide a pixel and add up to 5 new pixels to the queue.
- [pixelcoord](#) [Explnt](#) (int base, int exp)
Helper function to exponentiate ints.

Private Attributes

- const [largecounter](#) [m_InitialPixels](#)
How many initial pixels (spread uniformly over the grid) do we integrate?
- const int [m_MaxSubdivide](#)
How many times are we allowed to subdivide a square? Note: the initial grid is already at 1.
- const [pixelcoord](#) [m_RowColumnSize](#)
The total size in pixels of a row or column (square grid)
- const [largecounter](#) [m_IterationPixels](#)
How many pixels per iteration can we subdivide?
- const [largecounter](#) [m_MaxPixels](#)
How many pixels can we integrate in total over all iterations?
- const bool [m_InitialSubDivideToFinal](#)
If we decide to subdivide a square, do we automatically subdivide it further to the max level?
- const bool [m_InfinitePixels](#)
Are we allowed to integrate as many pixels as we want? (m_MaxPixels == 0)
- [largecounter](#) [m_PixelsLeft](#)
How many pixels are we still allowed to integrate (if !m_InfinitePixels)?
- std::vector< [PixelInfo](#) > [m_CurrentPixelQueue](#) {}
The current queue of pixels to be integrated.
- std::vector< bool > [m_CurrentPixelQueueDone](#) {}
A bool for every pixel in the current queue: gets set to true when the pixel is done integrating and gets its values returned.
- std::vector< [PixelInfo](#) > [m_AllPixels](#) {}
All pixels that have been integrated already (so does not include the pixels in the current queue)

Additional Inherited Members

Protected Attributes inherited from [Mesh](#)

- const std::unique_ptr< const [Diagnostic](#) > [m_DistanceDiagnostic](#)
The [Diagnostic](#) (a const pointer to a const [Diagnostic](#) object) that is used to calculate distances (using [FinalData](#)↔[ValDistance\(\)](#)) between the "values" that are assigned to Geodesics.

6.51.1 Detailed Description

Adaptive subdivision [Mesh](#).

starts with evenly spaced, square [Mesh](#), then decides to subdivide certain squares of pixels into smaller squares, based on which pixels have a bigger "weight", which is defined as the maximum "distance" (using the [Diagnostic](#) value distance) between the upper-left vertex of the square with the other three vertices of the square.

6.51.2 Constructor & Destructor Documentation

6.51.2.1 SquareSubdivisionMesh() [1/2]

```
SquareSubdivisionMesh::SquareSubdivisionMesh () [delete]
```

6.51.2.2 SquareSubdivisionMesh() [2/2]

```
SquareSubdivisionMesh::SquareSubdivisionMesh (
    largecounter maxPixels,
    largecounter initialPixels,
    int maxSubdivide,
    largecounter iterationPixels,
    bool initialSubToFinal,
    DiagBitflag valdiag) [inline]
```

6.51.3 Member Function Documentation

6.51.3.1 EndCurrentLoop()

```
void SquareSubdivisionMesh::EndCurrentLoop () [final], [virtual]
```

End the current loop of integrating pixels.

This function is called at the end of each integration iteration loop. We must wrap up the current iteration and initialize the next one.

Implements [Mesh](#).

6.51.3.2 ExpInt()

```
pixelcoord SquareSubdivisionMesh::ExpInt (
    int base,
    int exp) [private]
```

Helper function to exponentiate ints.

Note that the result may be larger than fits in an int, but this is only called with int arguments

Parameters

<i>base</i>	base
<i>exp</i>	exponent

Returns

pixelcoord

6.51.3.3 GeodesicFinished()

```
void SquareSubdivisionMesh::GeodesicFinished (
    largecounter index,
    std::vector< real > finalValues) [final], [virtual]
```

Called when a geodesic is finished integrating: the final values are stored.

Parameters

<i>index</i>	index of the geodesic that is finished
<i>finalValues</i>	final values of the diagnostics

Implements [Mesh](#).

6.51.3.4 getCurNrGeodesics()

```
largecounter SquareSubdivisionMesh::getCurNrGeodesics () const [final], [virtual]
```

Return the current number of geodesics to be integrated.

Returns

largecounter

Implements [Mesh](#).

6.51.3.5 getFullDescriptionStr()

```
std::string SquareSubdivisionMesh::getFullDescriptionStr () const [final], [virtual]
```

Description string getter.

Returns

std::string

Reimplemented from [Mesh](#).

6.51.3.6 getNewInitConds()

```
void SquareSubdivisionMesh::getNewInitConds (
    largecounter index,
    ScreenPoint & newunitpoint,
    ScreenIndex & newscreenindex) const [final], [virtual]
```

Get the initial conditions for the next pixel to be integrated.

The next pixel is the one with index `m_CurrentPixel`

Parameters

<i>index</i>	index of the pixel to be initialized
<i>newunitpoint</i>	new unit point to be initialized
<i>newscreenindex</i>	new screen index to be initialized

Implements [Mesh](#).

6.51.3.7 InitializeFirstGrid()

```
void SquareSubdivisionMesh::InitializeFirstGrid () [private]
```

Helper function to initialize the first grid of pixels to integrate.

6.51.3.8 IsFinished()

```
bool SquareSubdivisionMesh::IsFinished () const [final], [virtual]
```

Check whether the [Mesh](#) is finished integrating.

Returns

true
false

Implements [Mesh](#).

6.51.3.9 SubdivideAndQueue()

```
void SquareSubdivisionMesh::SubdivideAndQueue (  
    largecounter ind) [private]
```

Helper function to subdivide a pixel and add up to 5 new pixels to the queue.

This will take the pixel `m_AllPixels[ind]` and subdivide it, adding up to ≤ 5 pixels to the `CurrentPixelQueue`

Parameters

<i>ind</i>	index of the pixel to subdivide
------------	---------------------------------

6.51.3.10 UpdateAllNeighbors()

```
void SquareSubdivisionMesh::UpdateAllNeighbors () [private]
```

Helper function to update all pixel neighbors (if needed)

6.51.3.11 UpdateAllWeights()

```
void SquareSubdivisionMesh::UpdateAllWeights () [private]
```

Helper function to update all pixel weights.

Updates all weights of the pixels in `m_AllPixels` with `weight < 0` and `subdiv > 0` and `subdiv < m_MaxSubdivide`

Note

Assumes all squares have neighbors assigned correctly!

6.51.4 Member Data Documentation

6.51.4.1 m_AllPixels

```
std::vector<PixelInfo> SquareSubdivisionMesh::m_AllPixels {} [private]
```

All pixels that have been integrated already (so does not include the pixels in the current queue)

6.51.4.2 m_CurrentPixelQueue

```
std::vector<PixelInfo> SquareSubdivisionMesh::m_CurrentPixelQueue {} [private]
```

The current queue of pixels to be integrated.

6.51.4.3 m_CurrentPixelQueueDone

```
std::vector<bool> SquareSubdivisionMesh::m_CurrentPixelQueueDone {} [private]
```

A bool for every pixel in the current queue: gets set to true when the pixel is done integrating and gets its values returned.

6.51.4.4 m_InfinitePixels

```
const bool SquareSubdivisionMesh::m_InfinitePixels [private]
```

Are we allowed to integrate as many pixels as we want? (`m_MaxPixels == 0`)

6.51.4.5 m_InitialPixels

```
const largecounter SquareSubdivisionMesh::m_InitialPixels [private]
```

How many initial pixels (spread uniformly over the grid) do we integrate?

6.51.4.6 m_InitialSubDividideToFinal

```
const bool SquareSubdivisionMesh::m_InitialSubDividideToFinal [private]
```

If we decide to subdivide a square, do we automatically subdivide it further to the max level?

6.51.4.7 m_IterationPixels

```
const largecounter SquareSubdivisionMesh::m_IterationPixels [private]
```

How many pixels per iteration can we subdivide?

6.51.4.8 m_MaxPixels

```
const largecounter SquareSubdivisionMesh::m_MaxPixels [private]
```

How many pixels can we integrate in total over all iterations?

6.51.4.9 m_MaxSubdivide

```
const int SquareSubdivisionMesh::m_MaxSubdivide [private]
```

How many times are we allowed to subdivide a square? Note: the initial grid is already at 1.

6.51.4.10 m_PixelsLeft

```
largecounter SquareSubdivisionMesh::m_PixelsLeft [private]
```

How many pixels are we still allowed to integrate (if !m_InfinitePixels)?

6.51.4.11 m_RowColumnSize

```
const pixelcoord SquareSubdivisionMesh::m_RowColumnSize [private]
```

The total size in pixels of a row or column (square grid)

The documentation for this class was generated from the following files:

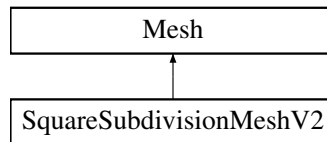
- FOORT/src/[Mesh.h](#)
- FOORT/src/[Mesh.cpp](#)

6.52 SquareSubdivisionMeshV2 Class Reference

Adaptive subdivision [Mesh](#).

```
#include <Mesh.h>
```

Inheritance diagram for SquareSubdivisionMeshV2:



Classes

- struct [PixelInfo](#)
A struct the [Mesh](#) uses to keep all information about a given pixel.

Public Member Functions

- [SquareSubdivisionMeshV2](#) ()=delete
- [SquareSubdivisionMeshV2](#) ([largecounter](#) maxPixels, [largecounter](#) initialPixels, int maxSubdivide, [largecounter](#) iterationPixels, bool initialSubToFinal, [DiagBitflag](#) valdiag)
- [largecounter](#) [getCurNrGeodesics](#) () const final
Return the current number of geodesics to be integrated.
- void [getNewInitConds](#) ([largecounter](#) index, [ScreenPoint](#) &newunitpoint, [ScreenIndex](#) &newscreenindex) const final
Get the initial conditions for the next pixel to be integrated.
- void [GeodesicFinished](#) ([largecounter](#) index, std::vector< [real](#) > finalValues) final
Called when a geodesic is finished integrating: the final values are stored.
- void [EndCurrentLoop](#) () final
Called at the end of each integration iteration loop. Wraps up the current iteration and initializes the next one.
- bool [IsFinished](#) () const final
Check whether the [Mesh](#) is finished integrating.
- std::string [getFullDescriptionStr](#) () const final
Description string getter.

Public Member Functions inherited from [Mesh](#)

- [Mesh](#) ([DiagBitflag](#) valdiag)
- virtual [~Mesh](#) ()=default

Private Member Functions

- void [InitializeFirstGrid](#) ()
Helper function to initialize the first grid of pixels to integrate.
- void [UpdateAllWeights](#) ()
Helper function to update all pixel weights in `m_CurrentPixelUpdating`.
- [PixelInfo](#) * [GetUp](#) ([PixelInfo](#) *p, int subdiv) const
Get the upper neighbor of a pixel at a certain subdivision level.
- [PixelInfo](#) * [GetDown](#) ([PixelInfo](#) *p, int subdiv) const
Get the lower neighbor of a pixel at a certain subdivision level.
- [PixelInfo](#) * [GetRight](#) ([PixelInfo](#) *p, int subdiv) const
Get the right neighbor of a pixel at a certain subdivision level.
- [PixelInfo](#) * [GetLeft](#) ([PixelInfo](#) *p, int subdiv) const
Get the left neighbor of a pixel at a certain subdivision level.
- void [SubdivideAndQueue](#) ([largecounter](#) ind)
Subdivide the pixel at index ind and add up to 5 new pixels to the integration queue.
- [pixelcoord](#) [Explnt](#) (int base, int exp) const
Helper function to exponentiate ints.

Private Attributes

- const [largecounter](#) [m_InitialPixels](#)
How many initial pixels (spread uniformly over the grid) do we integrate?
- const int [m_MaxSubdivide](#)
How many times are we allowed to subdivide a square? Note: the initial grid is already at 1.
- const [pixelcoord](#) [m_RowColumnSize](#)
The total size in pixels of a row or column (square grid)
- const [largecounter](#) [m_IterationPixels](#)
How many pixels per iteration can we subdivide?
- const [largecounter](#) [m_MaxPixels](#)
How many pixels can we integrate in total over all iterations?
- const bool [m_InitialSubDivideToFinal](#)
If we decide to subdivide a square, do we automatically subdivide it further to the max level?
- const bool [m_InfinitePixels](#)
Are we allowed to integrate as many pixels as we want? (`m_MaxPixels == 0`)
- [largecounter](#) [m_PixelsLeft](#)
How many pixels are we still allowed to integrate (if `!m_InfinitePixels`)?
- [largecounter](#) [m_PixelsIntegrated](#) {0}
How many pixels we have integrated so far.
- std::forward_list< std::unique_ptr< [PixelInfo](#) > > [m_AllPixels](#) {}
Master list of all pixels. `std::forward_list` is more space-efficient than `std::list`, bidirectional iteration is not needed, no random access supported. This list is only used to store the owner pointers (and thus the objects) of all pixels. The other pixel vectors are used to iterate through (and need random access)
- std::vector< [PixelInfo](#) * > [m_ActivePixels](#) {}
List of active pixels, i.e. those that can be subdivided and have non-zero weight.
- std::vector< [PixelInfo](#) * > [m_CurrentPixelQueue](#) {}
List of current queue of pixels to be sent to be integrated.
- std::vector< bool > [m_CurrentPixelQueueDone](#) {}
A bool for every pixel in the current queue: gets set to true when the pixel is done integrating and gets its values returned.
- std::vector< [PixelInfo](#) * > [m_CurrentPixelUpdating](#) {}
List of pixels that are already integrated but need updating weights after current queue is all integrated.

Additional Inherited Members

Protected Attributes inherited from [Mesh](#)

- `const std::unique_ptr< const Diagnostic > m_DistanceDiagnostic`

The [Diagnostic](#) (a `const` pointer to a `const Diagnostic` object) that is used to calculate distances (using `FinalData↔ValDistance()`) between the "values" that are assigned to Geodesics.

6.52.1 Detailed Description

Adaptive subdivision [Mesh](#).

starts with evenly spaced, square [Mesh](#), then decides to subdivide certain squares of pixels into smaller squares, based on which pixels have a bigger "weight", which is defined as the maximum "distance" (using the [Diagnostic](#) value distance) between the upper-left vertex of the square with the other three vertices of the square. V2: new way of dealing with neighbors and looping over pixels

6.52.2 Constructor & Destructor Documentation

6.52.2.1 SquareSubdivisionMeshV2() [1/2]

```
SquareSubdivisionMeshV2::SquareSubdivisionMeshV2 () [delete]
```

6.52.2.2 SquareSubdivisionMeshV2() [2/2]

```
SquareSubdivisionMeshV2::SquareSubdivisionMeshV2 (
    largecounter maxPixels,
    largecounter initialPixels,
    int maxSubdivide,
    largecounter iterationPixels,
    bool initialSubToFinal,
    DiagBitflag valdiag) [inline]
```

6.52.3 Member Function Documentation

6.52.3.1 EndCurrentLoop()

```
void SquareSubdivisionMeshV2::EndCurrentLoop () [final], [virtual]
```

Called at the end of each integration iteration loop. Wraps up the current iteration and initializes the next one.

Implements [Mesh](#).

6.52.3.2 ExpInt()

```
pixelcoord SquareSubdivisionMeshV2::ExpInt (
    int base,
    int exp) const [private]
```

Helper function to exponentiate ints.

Note

The result may be larger than fits in an int, but this is only called with int arguments

Parameters

<i>base</i>	base
<i>exp</i>	exponent

Returns

pixelcoord

6.52.3.3 GeodesicFinished()

```
void SquareSubdivisionMeshV2::GeodesicFinished (
    largecounter index,
    std::vector< real > finalValues) [final], [virtual]
```

Called when a geodesic is finished integrating: the final values are stored.

Parameters

<i>index</i>	index of the geodesic that is finished
<i>finalValues</i>	final values of the diagnostics

Implements [Mesh](#).

6.52.3.4 getCurNrGeodesics()

```
largecounter SquareSubdivisionMeshV2::getCurNrGeodesics () const [final], [virtual]
```

Return the current number of geodesics to be integrated.

Returns

largecounter

Implements [Mesh](#).

6.52.3.5 GetDown()

```
SquareSubdivisionMeshV2::PixelInfo * SquareSubdivisionMeshV2::GetDown (
    PixelInfo * p,
    int subdiv) const [private]
```

Get the lower neighbor of a pixel at a certain subdivision level.

Parameters

<i>p</i>	PixelInfo pointer to the pixel
<i>subdiv</i>	subdivision level of the neighbor

Returns

[SquareSubdivisionMeshV2::PixelInfo*](#)

6.52.3.6 getFullDescriptionStr()

```
std::string SquareSubdivisionMeshV2::getFullDescriptionStr () const [final], [virtual]
```

Description string getter.

Returns

std::string

Reimplemented from [Mesh](#).

6.52.3.7 GetLeft()

```
SquareSubdivisionMeshV2::PixelInfo * SquareSubdivisionMeshV2::GetLeft (
    PixelInfo * p,
    int subdiv) const [private]
```

Get the left neighbor of a pixel at a certain subdivision level.

Parameters

<i>p</i>	PixelInfo pointer to the pixel
<i>subdiv</i>	subdivision level of the neighbor

Returns

[SquareSubdivisionMeshV2::PixelInfo*](#)

6.52.3.8 getNewInitConds()

```
void SquareSubdivisionMeshV2::getNewInitConds (
    largecounter index,
    ScreenPoint & newunitpoint,
    ScreenIndex & newscreenindex) const [final], [virtual]
```

Get the initial conditions for the next pixel to be integrated.

The next pixel is the one with index m_CurrentPixel

Parameters

<i>index</i>	index of the pixel to be initialized
<i>newunitpoint</i>	new unit point to be initialized
<i>newscreenindex</i>	new screen index to be initialized

Implements [Mesh](#).

6.52.3.9 GetRight()

```
SquareSubdivisionMeshV2::PixelInfo * SquareSubdivisionMeshV2::GetRight (
    PixelInfo * p,
    int subdiv) const [private]
```

Get the right neighbor of a pixel at a certain subdivision level.

Parameters

<i>p</i>	PixelInfo pointer to the pixel
<i>subdiv</i>	subdivision level of the neighbor

Returns

[SquareSubdivisionMeshV2::PixelInfo*](#)

6.52.3.10 GetUp()

```

SquareSubdivisionMeshV2::PixelInfo * SquareSubdivisionMeshV2::GetUp (
    PixelInfo * p,
    int subdiv) const [private]

```

Get the upper neighbor of a pixel at a certain subdivision level.

Parameters

<i>p</i>	PixelInfo pointer to the pixel
<i>subdiv</i>	subdivision level of the neighbor

Returns

[SquareSubdivisionMeshV2::PixelInfo*](#)

6.52.3.11 InitializeFirstGrid()

```

void SquareSubdivisionMeshV2::InitializeFirstGrid () [private]

```

Helper function to initialize the first grid of pixels to integrate.

6.52.3.12 IsFinished()

```

bool SquareSubdivisionMeshV2::IsFinished () const [final], [virtual]

```

Check whether the [Mesh](#) is finished integrating.

Returns

true
false

Implements [Mesh](#).

6.52.3.13 SubdivideAndQueue()

```

void SquareSubdivisionMeshV2::SubdivideAndQueue (
    largecounter ind) [private]

```

Subdivide the pixel at index *ind* and add up to 5 new pixels to the integration queue.

The pixels are numbered as 1 2 3 4 5 6 7 8 9 i.e. the initial square is given by (1, 3, 7, 9), and the new pixels are (2, 4, 5, 6, 8).

Parameters

<i>ind</i>	index of the pixel to subdivide
------------	---------------------------------

6.52.3.14 UpdateAllWeights()

```
void SquareSubdivisionMeshV2::UpdateAllWeights () [private]
```

Helper function to update all pixel weights in m_CurrentPixelUpdating.

All pixels with weight > 0 will be added to m_ActivePixels

Note

Assumes all pixels have subdiv > 0 and subdiv < m_MaxSubdivide, and all their neighbors assigned correctly
Assumes all pixels have their (and their neighbor's) values assigned correctly

6.52.4 Member Data Documentation**6.52.4.1 m_ActivePixels**

```
std::vector<PixelInfo *> SquareSubdivisionMeshV2::m_ActivePixels {} [private]
```

List of active pixels, i.e. those that can be subdivided and have non-zero weight.

6.52.4.2 m_AllPixels

```
std::forward_list<std::unique_ptr<PixelInfo> > SquareSubdivisionMeshV2::m_AllPixels {} [private]
```

Master list of all pixels. std::forward_list is more space-efficient than std::list, bidirectional iteration is not needed, no random access supported. This list is only used to store the owner pointers (and thus the objects) of all pixels. The other pixel vectors are used to iterate through (and need random access)

6.52.4.3 m_CurrentPixelQueue

```
std::vector<PixelInfo *> SquareSubdivisionMeshV2::m_CurrentPixelQueue {} [private]
```

List of current queue of pixels to be sent to be integrated.

6.52.4.4 m_CurrentPixelQueueDone

```
std::vector<bool> SquareSubdivisionMeshV2::m_CurrentPixelQueueDone {} [private]
```

A bool for every pixel in the current queue: gets set to true when the pixel is done integrating and gets its values returned.

6.52.4.5 m_CurrentPixelUpdating

```
std::vector<PixelInfo *> SquareSubdivisionMeshV2::m_CurrentPixelUpdating {} [private]
```

List of pixels that are already integrated but need updating weights after current queue is all integrated.

6.52.4.6 m_InfinitePixels

```
const bool SquareSubdivisionMeshV2::m_InfinitePixels [private]
```

Are we allowed to integrate as many pixels as we want? (m_MaxPixels == 0)

6.52.4.7 m_InitialPixels

```
const largecounter SquareSubdivisionMeshV2::m_InitialPixels [private]
```

How many initial pixels (spread uniformly over the grid) do we integrate?

6.52.4.8 m_InitialSubDividideToFinal

```
const bool SquareSubdivisionMeshV2::m_InitialSubDividideToFinal [private]
```

If we decide to subdivide a square, do we automatically subdivide it further to the max level?

6.52.4.9 m_IterationPixels

```
const largecounter SquareSubdivisionMeshV2::m_IterationPixels [private]
```

How many pixels per iteration can we subdivide?

6.52.4.10 m_MaxPixels

```
const largecounter SquareSubdivisionMeshV2::m_MaxPixels [private]
```

How many pixels can we integrate in total over all iterations?

6.52.4.11 m_MaxSubdivide

```
const int SquareSubdivisionMeshV2::m_MaxSubdivide [private]
```

How many times are we allowed to subdivide a square? Note: the initial grid is already at 1.

6.52.4.12 m_PixelsIntegrated

```
largecounter SquareSubdivisionMeshV2::m_PixelsIntegrated {0} [private]
```

How many pixels we have integrated so far.

6.52.4.13 m_PixelsLeft

`largecounter` SquareSubdivisionMeshV2::m_PixelsLeft [private]

How many pixels are we still allowed to integrate (if !m_InfinitePixels)?

6.52.4.14 m_RowColumnSize

`const pixelcoord` SquareSubdivisionMeshV2::m_RowColumnSize [private]

The total size in pixels of a row or column (square grid)

The documentation for this class was generated from the following files:

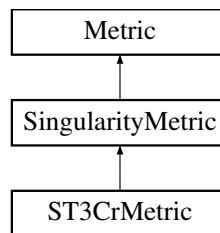
- FOORT/src/[Mesh.h](#)
- FOORT/src/[Mesh.cpp](#)

6.53 ST3CrMetric Class Reference

Ring fuzzball metric.

```
#include <Metric.h>
```

Inheritance diagram for ST3CrMetric:



Public Member Functions

- [ST3CrMetric](#) (`real` P, `real` q0, `real` lambda, `bool` rlogscale=false)
Construct a new ST3Cr [Metric](#) object.
- `TwoIndex` [getMetric_dd](#) (`const` [Point](#) &p) `const` final
ST3Cr metric getter, indices down.
- `TwoIndex` [getMetric_uu](#) (`const` [Point](#) &p) `const` final
ST3Cr metric getter, indices up.
- `std::string` [getFullDescriptionStr](#) () `const` final
ST3Cr metric description string getter.

Public Member Functions inherited from [SingularityMetric](#)

- [SingularityMetric](#) (`std::vector`< [Singularity](#) > thesings, `bool` rLogScale)
Construct a new [Singularity Metric](#) object.
- `std::vector`< [Singularity](#) > [getSingularities](#) () `const`
Getters for the singularities.

Public Member Functions inherited from [Metric](#)

- virtual [~Metric](#) ()=default
- [Metric](#) (bool rlogscale=false)
Construct a new [Metric](#) object.
- virtual [ThreeIndex](#) [getChristoffel_udd](#) (const [Point](#) &p) const
Return the Christoffel symbols of the metric (indices up, down, down)
- virtual [FourIndex](#) [getRiemann_uddd](#) (const [Point](#) &p) const
Riemann tensor of the metric (indices up, down, down, down)
- virtual [real](#) [getKretschmann](#) (const [Point](#) &p) const
Get the Kretschmann scalar at a point.
- bool [getrLogScale](#) () const
Get whether we are using a logarithmic radial scale.

Private Member Functions

- [real](#) [get_omega](#) ([real](#) r, [real](#) theta, [real](#) l) const
Get omega for the ST3Cr metric.
- [real](#) [f_phi](#) ([real](#) phi, [real](#) r, [real](#) theta, [real](#) l, [real](#) R) const
*function for prefactor * dphi' for ring*
- [real](#) [f_om_phi](#) ([real](#) phi, [real](#) r, [real](#) theta, [real](#) l, [real](#) R) const
omega_phi ring functions

Private Attributes

- const [real](#) [m_P](#)
- const [real](#) [m_q0](#)
- const [real](#) [m_lambda](#)

Additional Inherited Members

Protected Attributes inherited from [SingularityMetric](#)

- const std::vector< [Singularity](#) > [m_AllSingularities](#)
All singularities of the metric.

Protected Attributes inherited from [Metric](#)

- std::vector< int > [m_Symmetries](#) {}
The symmetries (coordinate Killing vectors) of the metric. Should be set by descendant constructor.
- const bool [m_rLogScale](#)
Are we using a logarithmic r coordinate?

6.53.1 Detailed Description

Ring fuzzball metric.

Note

Implementation by Lies Van Dael

6.53.2 Constructor & Destructor Documentation

6.53.2.1 ST3CrMetric()

```
ST3CrMetric::ST3CrMetric (
    real P,
    real q0,
    real lambda,
    bool rlogscale = false)
```

Construct a new ST3Cr [Metric](#) object.

Parameters

<i>P</i>	P parameter for the ST3Cr metric
<i>q0</i>	q0 parameter for the ST3Cr metric
<i>lambda</i>	lambda parameter for the ST3Cr metric
<i>rlogscale</i>	whether we are using a logarithmic radial scale

6.53.3 Member Function Documentation

6.53.3.1 f_om_phi()

```
real ST3CrMetric::f_om_phi (
    real phi,
    real r,
    real theta,
    real l,
    real R) const [private]
```

omega_phi ring functions

Parameters

<i>phi</i>	
<i>r</i>	
<i>theta</i>	
<i>l</i>	
<i>R</i>	

Returns

real

6.53.3.2 f_phi()

```
real ST3CrMetric::f_phi (
    real phi,
    real r,
    real theta,
    real l,
    real R) const [private]
```

function for prefactor * dphi' for ring

Parameters

ϕ	
r	
θ	
l	
R	

Returns

real

6.53.3.3 get_omega()

```
real ST3CrMetric::get_omega (
    real r,
    real theta,
    real l) const [private]
```

Get omega for the ST3Cr metric.

Parameters

r	r_{ij}
θ	$\theta_{ij} \leftrightarrow$
l	l_{ij}

Returns

real

6.53.3.4 getFullDescriptionStr()

```
std::string ST3CrMetric::getFullDescriptionStr () const [final], [virtual]
```

ST3Cr metric description string getter.

Returns

std::string

Reimplemented from [Metric](#).

6.53.3.5 getMetric_dd()

```
TwoIndex ST3CrMetric::getMetric_dd (
    const Point & p) const [final], [virtual]
```

ST3Cr metric getter, indices down.

Parameters

p	Point at which to evaluate the metric
-----	---

Returns

[TwoIndex](#)Implements [Metric](#).**6.53.3.6 getMetric_uu()**

```
TwoIndex ST3CrMetric::getMetric_uu (  
    const Point & p) const [final], [virtual]
```

ST3Cr metric getter, indices up.

Parameters

p	Point at which to evaluate the metric
-----	---

Returns

[TwoIndex](#)Implements [Metric](#).**6.53.4 Member Data Documentation****6.53.4.1 m_lambda**

```
const real ST3CrMetric::m_lambda [private]
```

6.53.4.2 m_P

```
const real ST3CrMetric::m_P [private]
```

6.53.4.3 m_q0

```
const real ST3CrMetric::m_q0 [private]
```

The documentation for this class was generated from the following files:

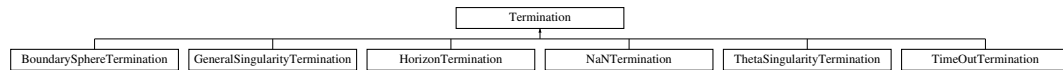
- FOORT/src/[Metric.h](#)
- FOORT/src/[Metric.cpp](#)

6.54 Termination Class Reference

Abstract base class for all Terminations.

```
#include <Terminations.h>
```

Inheritance diagram for Termination:



Public Member Functions

- [Termination](#) ()=delete
- [Termination](#) ([Geodesic](#) *const theGeodesic)
- virtual void [Reset](#) ()
Reset to 0 to start integrating a new geodesic.
- virtual [~Termination](#) ()=default
- virtual [Term CheckTermination](#) ()=0
- virtual std::string [getFullDescriptionStr](#) () const =0

Protected Member Functions

- bool [DecideUpdate](#) ([largecounter](#) UpdateNSteps)
Returns true if the [Termination](#) should update its internal status.

Protected Attributes

- [Geodesic](#) *const [m_OwnerGeodesic](#)
The geodesic that owns the [Termination](#) (a const pointer to the [Geodesic](#))
- [largecounter](#) [m_StepsSinceUpdated](#) {}
The termination is itself in charge of keeping track of how many steps it has been since it has been updated. The [Termination](#)'s [TerminationOptions](#) struct tells it how many steps it needs to wait between updates.

6.54.1 Detailed Description

Abstract base class for all Terminations.

6.54.2 Constructor & Destructor Documentation

6.54.2.1 Termination() [1/2]

```
Termination::Termination () [delete]
```

6.54.2.2 Termination() [2/2]

```
Termination::Termination (
    Geodesic *const theGeodesic) [inline]
```

6.54.2.3 ~Termination()

```
virtual Termination::~Termination () [virtual], [default]
```

6.54.3 Member Function Documentation

6.54.3.1 CheckTermination()

```
virtual Term Termination::CheckTermination () [pure virtual]
```

Implemented in [BoundarySphereTermination](#), [GeneralSingularityTermination](#), [HorizonTermination](#), [NaNTermination](#), [ThetaSingularityTermination](#), and [TimeOutTermination](#).

6.54.3.2 DecideUpdate()

```
bool Termination::DecideUpdate (
    largecounter UpdateNSteps) [protected]
```

Returns true if the [Termination](#) should update its internal status.

Should be called from within [CheckTermination\(\)](#) with the appropriate [TermOptions::UpdateEveryNSteps](#)

Parameters

<i>UpdateNSteps</i>	the number of steps after which the terminations should be updated
---------------------	--

Returns

true

false

6.54.3.3 getFullDescriptionStr()

```
virtual std::string Termination::getFullDescriptionStr () const [pure virtual]
```

Implemented in [BoundarySphereTermination](#), [GeneralSingularityTermination](#), [HorizonTermination](#), [NaNTermination](#), [ThetaSingularityTermination](#), and [TimeOutTermination](#).

6.54.3.4 Reset()

```
void Termination::Reset () [virtual]
```

Reset to 0 to start integrating a new geodesic.

Reimplemented in [TimeOutTermination](#).

6.54.4 Member Data Documentation

6.54.4.1 m_OwnerGeodesic

```
Geodesic* const Termination::m_OwnerGeodesic [protected]
```

The geodesic that owns the [Termination](#) (a const pointer to the [Geodesic](#))

6.54.4.2 m_StepsSinceUpdated

```
largecounter Termination::m_StepsSinceUpdated {} [protected]
```

The termination is itself in charge of keeping track of how many steps it has been since it has been updated. The [Termination](#)'s [TerminationOptions](#) struct tells it how many steps it needs to wait between updates.

The documentation for this class was generated from the following files:

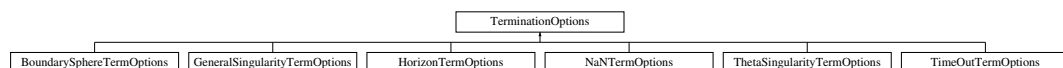
- FOORT/src/[Terminations.h](#)
- FOORT/src/[Terminations.cpp](#)

6.55 TerminationOptions Struct Reference

Base class for all [TerminationOptions](#) structs. Other [TerminationOptions](#) can inherit from here if they require more options.

```
#include <Terminations.h>
```

Inheritance diagram for TerminationOptions:



Public Member Functions

- [TerminationOptions](#) ([largecounter](#) Nsteps)
- virtual [~TerminationOptions](#) ()=default

Public Attributes

- const [largecounter UpdateEveryNSteps](#)
Number of steps between updates.

6.55.1 Detailed Description

Base class for all [TerminationOptions](#) structs. Other [TerminationOptions](#) can inherit from here if they require more options.

6.55.2 Constructor & Destructor Documentation

6.55.2.1 TerminationOptions()

```
TerminationOptions::TerminationOptions (
    largecounter Nsteps) [inline]
```

6.55.2.2 ~TerminationOptions()

```
virtual TerminationOptions::~~TerminationOptions () [virtual], [default]
```

6.55.3 Member Data Documentation

6.55.3.1 UpdateEveryNSteps

```
const largecounter TerminationOptions::UpdateEveryNSteps
```

Number of steps between updates.

The documentation for this struct was generated from the following file:

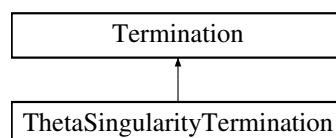
- FOORT/src/[Terminations.h](#)

6.56 ThetaSingularityTermination Class Reference

[ThetaSingularityTermination](#): Terminate geodesics if they get too close to a polar coordinate singularity.

```
#include <Terminations.h>
```

Inheritance diagram for ThetaSingularityTermination:



Public Member Functions

- [ThetaSingularityTermination](#) ([Geodesic](#) *const theGeodesic)
- [Term CheckTermination](#) () final
Check if the geodesic is too close to a pole.
- std::string [getFullDescriptionStr](#) () const final
Description string getter for the Theta [Singularity Termination](#).

Public Member Functions inherited from [Termination](#)

- [Termination](#) ()=delete
- [Termination](#) ([Geodesic](#) *const theGeodesic)
- virtual void [Reset](#) ()
Reset to 0 to start integrating a new geodesic.
- virtual [~Termination](#) ()=default

Static Public Attributes

- static std::unique_ptr< [ThetaSingularityTermOptions](#) > [TermOptions](#)
The options that the [Termination](#) keeps (will probably be a descendant struct instead, which specifies any additional options the [Termination](#) needs)

Additional Inherited Members

Protected Member Functions inherited from [Termination](#)

- bool [DecideUpdate](#) ([largecounter](#) UpdateNSteps)
Returns true if the [Termination](#) should update its internal status.

Protected Attributes inherited from [Termination](#)

- [Geodesic](#) *const [m_OwnerGeodesic](#)
The geodesic that owns the [Termination](#) (a const pointer to the [Geodesic](#))
- [largecounter](#) [m_StepsSinceUpdated](#) {}
The termination is itself in charge of keeping track of how many steps it has been since it has been updated. The [Termination](#)'s [TerminationOptions](#) struct tells it how many steps it needs to wait between updates.

6.56.1 Detailed Description

[ThetaSingularityTermination](#): Terminate geodesics if they get too close to a polar coordinate singularity.

6.56.2 Constructor & Destructor Documentation

6.56.2.1 [ThetaSingularityTermination](#)()

```
ThetaSingularityTermination::ThetaSingularityTermination (
    Geodesic *const theGeodesic) [inline]
```


6.56.3 Member Function Documentation

6.56.3.1 CheckTermination()

`Term` ThetaSingularityTermination::CheckTermination () [final], [virtual]

Check if the geodesic is too close to a pole.

Returns

`Term`

Implements [Termination](#).

6.56.3.2 getFullDescriptionStr()

`std::string` ThetaSingularityTermination::getFullDescriptionStr () const [final], [virtual]

Description string getter for the Theta [Singularity Termination](#).

Returns

`std::string`

Implements [Termination](#).

6.56.4 Member Data Documentation

6.56.4.1 TermOptions

`std::unique_ptr< ThetaSingularityTermOptions >` ThetaSingularityTermination::TermOptions [static]

The options that the [Termination](#) keeps (will probably be a descendant struct instead, which specifies any additional options the [Termination](#) needs)

The documentation for this class was generated from the following files:

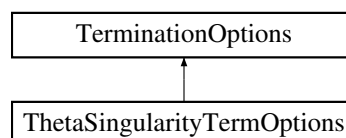
- FOORT/src/[Terminations.h](#)
- FOORT/src/[Config.cpp](#)
- FOORT/src/[Terminations.cpp](#)

6.57 ThetaSingularityTermOptions Struct Reference

Options for [ThetaSingularityTermination](#).

```
#include <Terminations.h>
```

Inheritance diagram for ThetaSingularityTermOptions:



Public Member Functions

- [ThetaSingularityTermOptions](#) ([real](#) epsilon, [largecounter](#) Nsteps)

Public Member Functions inherited from [TerminationOptions](#)

- [TerminationOptions](#) ([largecounter](#) Nsteps)
- virtual [~TerminationOptions](#) ()=default

Public Attributes

- const [real](#) [ThetaSingEpsilon](#)
Minimum difference between theta and 0 or pi.

Public Attributes inherited from [TerminationOptions](#)

- const [largecounter](#) [UpdateEveryNSteps](#)
Number of steps between updates.

6.57.1 Detailed Description

Options for [ThetaSingularityTermination](#).

Keeps track of the minimum required distance from the singularity

6.57.2 Constructor & Destructor Documentation

6.57.2.1 [ThetaSingularityTermOptions\(\)](#)

```
ThetaSingularityTermOptions::ThetaSingularityTermOptions (
    real epsilon,
    largecounter Nsteps) [inline]
```

6.57.3 Member Data Documentation

6.57.3.1 [ThetaSingEpsilon](#)

```
const real ThetaSingularityTermOptions::ThetaSingEpsilon
```

Minimum difference between theta and 0 or pi.

The documentation for this struct was generated from the following file:

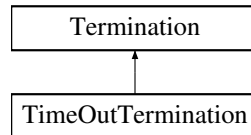
- FOORT/src/[Terminations.h](#)

6.58 TimeOutTermination Class Reference

TimeOutTermination: Terminate geodesics if they take too many steps.

```
#include <Terminations.h>
```

Inheritance diagram for TimeOutTermination:



Public Member Functions

- **TimeOutTermination** (**Geodesic** *const theGeodesic)
- void **Reset** () final
Reset m_CurNrSteps to 0 for new geodesic, and call base class implementation to reset base class member variables.
- **Term CheckTermination** () final
Check if the geodesic has taken too many steps.
- std::string **getFullDescriptionStr** () const final
*Description string getter for the Time Out **Termination**.*

Public Member Functions inherited from **Termination**

- **Termination** ()=delete
- **Termination** (**Geodesic** *const theGeodesic)
- virtual **~Termination** ()=default

Static Public Attributes

- static std::unique_ptr< **TimeOutTermOptions** > **TermOptions**
*The options that the **TimeOutTermination** keeps (contains max number of steps allowed)*

Private Attributes

- **largecounter m_CurNrSteps** {0}
Keep track of the number of steps that the geodesic has taken so far.

Additional Inherited Members

Protected Member Functions inherited from **Termination**

- bool **DecideUpdate** (**largecounter** UpdateNSteps)
*Returns true if the **Termination** should update its internal status.*

Protected Attributes inherited from [Termination](#)

- [Geodesic](#) *const [m_OwnerGeodesic](#)

The geodesic that owns the [Termination](#) (a const pointer to the [Geodesic](#))

- [largecounter](#) [m_StepsSinceUpdated](#) {}

The termination is itself in charge of keeping track of how many steps it has been since it has been updated. The [Termination](#)'s [TerminationOptions](#) struct tells it how many steps it needs to wait between updates.

6.58.1 Detailed Description

[TimeOutTermination](#): Terminate geodesics if they take too many steps.

6.58.2 Constructor & Destructor Documentation

6.58.2.1 TimeOutTermination()

```
TimeOutTermination::TimeOutTermination (
    Geodesic *const theGeodesic) [inline]
```

6.58.3 Member Function Documentation

6.58.3.1 CheckTermination()

```
Term TimeOutTermination::CheckTermination () [final], [virtual]
```

Check if the geodesic has taken too many steps.

Returns

[Term](#)

Implements [Termination](#).

6.58.3.2 getFullDescriptionStr()

```
std::string TimeOutTermination::getFullDescriptionStr () const [final], [virtual]
```

Description string getter for the Time Out [Termination](#).

Returns

std::string

Implements [Termination](#).

6.58.3.3 Reset()

```
void TimeOutTermination::Reset () [final], [virtual]
```

Reset m_CurNrSteps to 0 for new geodesic, and call base class implementation to reset base class member variables.

Reimplemented from [Termination](#).

6.58.4 Member Data Documentation

6.58.4.1 m_CurNrSteps

```
largecounter TimeOutTermination::m_CurNrSteps {0} [private]
```

Keep track of the number of steps that the geodesic has taken so far.

6.58.4.2 TermOptions

```
std::unique_ptr< TimeOutTermOptions > TimeOutTermination::TermOptions [static]
```

The options that the [TimeOutTermination](#) keeps (contains max number of steps allowed)

The documentation for this class was generated from the following files:

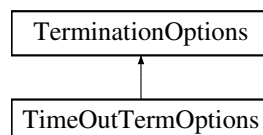
- FOORT/src/[Terminations.h](#)
- FOORT/src/[Config.cpp](#)
- FOORT/src/[Terminations.cpp](#)

6.59 TimeOutTermOptions Struct Reference

Options for [TimeOutTermination](#).

```
#include <Terminations.h>
```

Inheritance diagram for TimeOutTermOptions:



Public Member Functions

- [TimeOutTermOptions](#) ([largecounter](#) MaxStepsAllowed, [largecounter](#) Nsteps)

Public Member Functions inherited from [TerminationOptions](#)

- [TerminationOptions](#) ([largecounter](#) Nsteps)
- virtual [~TerminationOptions](#) ()=default

Public Attributes

- const [largecounter](#) [MaxSteps](#)
Max. number of integration steps allowed.

Public Attributes inherited from [TerminationOptions](#)

- const [largecounter](#) [UpdateEveryNSteps](#)
Number of steps between updates.

6.59.1 Detailed Description

Options for [TimeOutTermination](#).

Keeps track of the max. number of integration steps allowed

6.59.2 Constructor & Destructor Documentation

6.59.2.1 TimeOutTermOptions()

```
TimeOutTermOptions::TimeOutTermOptions (
    largecounter MaxStepsAllowed,
    largecounter Nsteps) [inline]
```

6.59.3 Member Data Documentation

6.59.3.1 MaxSteps

```
const largecounter TimeOutTermOptions::MaxSteps
```

Max. number of integration steps allowed.

The documentation for this struct was generated from the following file:

- FOORT/src/[Terminations.h](#)

6.60 Utilities::Timer Class Reference

[Timer](#) class to keep track of elapsed time.

```
#include <Utilities.h>
```

Public Member Functions

- void `reset()`
Reset begin time.
- double `elapsed()` const
Returns time elapsed since begin time.

Private Types

- using `Clock` = std::chrono::steady_clock
Type alias to make accessing nested type easier.
- using `Second` = std::chrono::duration<double, std::ratio<1>>
Type alias to make accessing nested type easier.

Private Attributes

- std::chrono::time_point< `Clock` > `m_beg` {Clock::now()}
begin time

6.60.1 Detailed Description

`Timer` class to keep track of elapsed time.

6.60.2 Member Typedef Documentation

6.60.2.1 Clock

```
using Utilities::Timer::Clock = std::chrono::steady_clock [private]
```

Type alias to make accessing nested type easier.

6.60.2.2 Second

```
using Utilities::Timer::Second = std::chrono::duration<double, std::ratio<1>> [private]
```

Type alias to make accessing nested type easier.

6.60.3 Member Function Documentation

6.60.3.1 elapsed()

```
double Utilities::Timer::elapsed () const
```

Returns time elapsed since begin time.

Returns

double

6.60.3.2 reset()

```
void Utilities::Timer::reset ()
```

Reset begin time.

6.60.4 Member Data Documentation

6.60.4.1 m_beg

```
std::chrono::time_point<Clock> Utilities::Timer::m_beg {Clock::now()} [private]
```

begin time

The documentation for this class was generated from the following files:

- FOORT/src/[Utilities.h](#)
- FOORT/src/[Utilities.cpp](#)

6.61 UpdateFrequency Struct Reference

Struct to hold the information for a [Diagnostic](#) to update itself.

```
#include <Diagnostics.h>
```

Public Attributes

- [largecounter](#) [UpdateNSteps](#) {0}
- bool [UpdateStart](#) {false}
- bool [UpdateFinish](#) {false}

6.61.1 Detailed Description

Struct to hold the information for a [Diagnostic](#) to update itself.

If `UpdateNSteps > 0`, then every so many steps. If `UpdateNSteps == 0`, then it only updates at the start and/or finish of integration if the appropriate bool is set.

6.61.2 Member Data Documentation

6.61.2.1 UpdateFinish

```
bool UpdateFrequency::UpdateFinish {false}
```


6.61.2.2 UpdateNSteps

```
largecounter UpdateFrequency::UpdateNSteps {0}
```

6.61.2.3 UpdateStart

```
bool UpdateFrequency::UpdateStart {false}
```

The documentation for this struct was generated from the following file:

- FOORT/src/[Diagnostics.h](#)

6.62 ViewScreen Class Reference

This class is in charge of converting a pixel on the screen into physical initial conditions for a geodesic.

```
#include <ViewScreen.h>
```

Public Member Functions

- [ViewScreen](#) ()=delete
- [ViewScreen](#) ([Point](#) pos, [OneIndex](#) dir, [ScreenPoint](#) screensize, [ScreenPoint](#) screencenter, std::unique_ptr< [Mesh](#) > theMesh, const [Metric](#) *const theMetric, [GeodesicType](#) thegeodtype=[GeodesicType::Null](#))
- void [SetNewInitialConditions](#) ([largecounter](#) index, [Point](#) &pos, [OneIndex](#) &vel, [ScreenIndex](#) &scrIndex) const
Set new initial conditions for geodesic nr index.
- bool [IsFinished](#) () const
Check if the [ViewScreen](#) is finished.
- [largecounter](#) [getCurNrGeodesics](#) () const
Get the current number of geodesics.
- void [EndCurrentLoop](#) ()
End the current loop of geodesics.
- void [GeodesicFinished](#) ([largecounter](#) index, std::vector< [real](#) > finalValues)
Pass on information that a geodesic has finished.
- std::string [getFullDescriptionStr](#) () const
Get the full description string of the [ViewScreen](#).

Private Member Functions

- void [ConstructVielbein](#) ()
Construct the vielbein at the position of the camera.

Private Attributes

- [TwoIndex m_Metric_dd](#) {}
The metric at the position of the viewscreen (we only need indices down)
- [TwoIndex m_Vielbein](#) {}
The vielbein used to transform from the curved spacetime at the viewscreen to a locally flat frame.
- const [Point m_Pos](#)
The position of the camera.
- const [OneIndex m_Direction](#)
The looking direction of the camera (always pointing towards the origin)
- const [ScreenPoint m_ScreenSize](#)
The screensize (in physical units of length)
- const [ScreenPoint m_ScreenCenter](#)
The screen center.
- const bool [m_rLogScale](#)
Whether the metric uses a logarithmic r coordinate or not.
- const [Metric](#) *const [m_theMetric](#)
const pointer to const [Metric](#)
- const [GeodesicType m_GeodType](#) {[GeodesicType::Null](#)}
The geodesic type to be integrated.
- const std::unique_ptr< [Mesh](#) > [m_theMesh](#)
The const pointer to the [Mesh](#) we are using to determine pixels to be integrated.

6.62.1 Detailed Description

This class is in charge of converting a pixel on the screen into physical initial conditions for a geodesic.

The [ViewScreen](#) class is in charge of converting a pixel on the screen (which the [Mesh](#) wants to integrate) into physical initial conditions for the position and velocity of a geodesic. It owns a [Mesh](#) instance, which will tell it which pixels to integrate etc.

6.62.2 Constructor & Destructor Documentation

6.62.2.1 ViewScreen() [1/2]

```
ViewScreen::ViewScreen () [delete]
```

6.62.2.2 ViewScreen() [2/2]

```
ViewScreen::ViewScreen (
    Point pos,
    OneIndex dir,
    ScreenPoint screensize,
    ScreenPoint screencenter,
    std::unique_ptr< Mesh > theMesh,
    const Metric *const theMetric,
    GeodesicType thegeodtype = GeodesicType::Null) [inline]
```

6.62.3 Member Function Documentation

6.62.3.1 ConstructVielbein()

```
void ViewScreen::ConstructVielbein () [private]
```

Construct the vielbein at the position of the camera.

6.62.3.2 EndCurrentLoop()

```
void ViewScreen::EndCurrentLoop ()
```

End the current loop of geodesics.

6.62.3.3 GeodesicFinished()

```
void ViewScreen::GeodesicFinished (  
    largecounter index,  
    std::vector< real > finalValues)
```

Pass on information that a geodesic has finished.

Parameters

<i>index</i>	index of the geodesic
<i>finalValues</i>	final values of the geodesic

6.62.3.4 getCurNrGeodesics()

```
largecounter ViewScreen::getCurNrGeodesics () const
```

Get the current number of geodesics.

Returns

largecounter

6.62.3.5 getFullDescriptionStr()

```
std::string ViewScreen::getFullDescriptionStr () const
```

Get the full description string of the [ViewScreen](#).

Returns

std::string

6.62.3.6 IsFinished()

```
bool ViewScreen::IsFinished () const
```

Check if the [ViewScreen](#) is finished.

Returns

true if the [ViewScreen](#) is finished, false otherwise

6.62.3.7 SetNewInitialConditions()

```
void ViewScreen::SetNewInitialConditions (
    largecounter index,
    Point & pos,
    OneIndex & vel,
    ScreenIndex & scrIndex) const
```

Set new initial conditions for geodesic nr index.

Parameters

<i>index</i>	index of the geodesic
<i>pos</i>	pointer to the starting position of the geodesic - which is the camera position
<i>vel</i>	pointer to the starting velocity of the geodesic
<i>scrIndex</i>	pointer to the screen index of the geodesic

6.62.4 Member Data Documentation

6.62.4.1 m_Direction

```
const OneIndex ViewScreen::m_Direction [private]
```

The looking direction of the camera (always pointing towards the origin)

6.62.4.2 m_GeodType

```
const GeodesicType ViewScreen::m_GeodType {GeodesicType::Null} [private]
```

The geodesic type to be integrated.

6.62.4.3 m_Metric_dd

```
TwoIndex ViewScreen::m_Metric_dd {} [private]
```

The metric at the position of the viewscreen (we only need indices down)

6.62.4.4 m_Pos

```
const Point ViewScreen::m_Pos [private]
```

The position of the camera.

6.62.4.5 m_rLogScale

```
const bool ViewScreen::m_rLogScale [private]
```

Whether the metric uses a logarithmic r coordinate or not.

6.62.4.6 m_ScreenCenter

```
const ScreenPoint ViewScreen::m_ScreenCenter [private]
```

The screen center.

6.62.4.7 m_ScreenSize

```
const ScreenPoint ViewScreen::m_ScreenSize [private]
```

The screensize (in physical units of length)

6.62.4.8 m_theMesh

```
const std::unique_ptr<Mesh> ViewScreen::m_theMesh [private]
```

The const pointer to the [Mesh](#) we are using to determine pixels to be integrated.

6.62.4.9 m_theMetric

```
const Metric* const ViewScreen::m_theMetric [private]
```

const pointer to const [Metric](#)

6.62.4.10 m_Vielbein

```
TwoIndex ViewScreen::m_Vielbein {} [private]
```

The vielbein used to transform from the curved spacetime at the viewscreen to a locally flat frame.

The documentation for this class was generated from the following files:

- [FOORT/src/ViewScreen.h](#)
- [FOORT/src/ViewScreen.cpp](#)

Chapter 7

File Documentation

7.1 FOORT/src/Config.cpp File Reference

```
#include "Config.h"
#include "Utilities.h"
#include <algorithm>
#include <cctype>
#include <utility>
```

7.1.1 Detailed Description

Author

Daniel R. Mayerson

Version

1.0

Date

2024-12-12

Copyright

Copyright (c) 2024

7.2 FOORT/src/Config.h File Reference

Functions that read the configuration file and initialize all objects.

```
#include "Geometry.h"
#include "Metric.h"
#include "Diagnostics.h"
#include "Terminations.h"
#include "ViewScreen.h"
#include "Geodesic.h"
#include "Integrators.h"
#include <memory>
#include <string>
#include <exception>
#include "ConfigReader.h"
```

Namespaces

- namespace [Config](#)

Namespace for all configuration functions that initialize objects based on configuration file.

Typedefs

- using [Config::ConfigCollection](#) = [ConfigReader::ConfigCollection](#)
- using [Config::SettingError](#) = `std::invalid_argument`

Functions

- void [Config::InitializeScreenOutput](#) (const [ConfigCollection](#) &theCfg)
Initialize screen output options.
- std::unique_ptr< [Metric](#) > [Config::GetMetric](#) (const [ConfigCollection](#) &theCfg)
Use configuration to create the correct [Metric](#) with specified parameters.
- std::unique_ptr< [Source](#) > [Config::GetSource](#) (const [ConfigCollection](#) &theCfg, const [Metric](#) *const theMetric)
Use configuration to create the correct [Source](#) with specified parameters.
- void [Config::InitializeDiagnostics](#) (const [ConfigCollection](#) &theCfg, [DiagBitflag](#) &alldiags, [DiagBitflag](#) &valdiag, const [Metric](#) *const theMetric)
Use configuration to set the [Diagnostics](#) bitflag appropriately; initialize all [DiagnosticOptions](#) for all [Diagnostics](#) that are turned on; and set bitflag for diagnostic to be used for coarseness evaluating in [Mesh](#).
- void [Config::InitializeTerminations](#) (const [ConfigCollection](#) &theCfg, [TermBitflag](#) &allterms, const [Metric](#) *const theMetric)
Use configuration to set the [Termination](#) bitflag appropriately; and initialize all [TerminationOptions](#) for all [Terminations](#) that are turned on.
- std::unique_ptr< [ViewScreen](#) > [Config::GetViewScreen](#) (const [ConfigCollection](#) &theCfg, [DiagBitflag](#) valdiag, const [Metric](#) *const theMetric)
Use configuration to create the [ViewScreen](#) object with specified parameters.
- std::unique_ptr< [Mesh](#) > [Config::GetMesh](#) (const [ConfigCollection](#) &theCfg, [DiagBitflag](#) valdiag)
Use configuration to create the [Mesh](#) object with specified parameters.
- [GeodesicIntegratorFunc](#) [Config::GetGeodesicIntegrator](#) (const [ConfigCollection](#) &theCfg)
Use configuration to set the [GeodesicIntegratorFunc](#) pointer to the correct integrator function.
- std::unique_ptr< [GeodesicOutputHandler](#) > [Config::GetOutputHandler](#) (const [ConfigCollection](#) &theCfg, [DiagBitflag](#) alldiags, [DiagBitflag](#) valdiag, std::string FirstLineInfo)
Use configuration to create the [GeodesicOutputHandler](#) object with specified parameters.

Variables

- constexpr auto [Config::Output_Important_Default](#) = [OutputLevel::Level_0_WARNING](#)
- constexpr auto [Config::Output_Other_Default](#) = [OutputLevel::Level_1_PROC](#)

7.2.1 Detailed Description

Functions that read the configuration file and initialize all objects.

Author

Daniel R. Mayerson

Version

1.0

Date

2024-12-12

Copyright

Copyright (c) 2024

7.3 Config.h

[Go to the documentation of this file.](#)

```
00001 #ifndef _FOORT_CONFIG_H
00002 #define _FOORT_CONFIG_H
00003
00012
00013 #include "Geometry.h"
00014 #include "Metric.h"
00015 #include "Diagnostics.h"
00016 #include "Terminations.h"
00017 #include "ViewScreen.h"
00018 #include "Geodesic.h"
00019 #include "Integrators.h"
00020
00021 #include <memory> // std::unique_ptr
00022 #include <string> // std::string
00023
00024 #include <exception> // needed to define our own configuration error
00025
00026 #include "ConfigReader.h"
00027
00031 namespace Config
00032 {
00033     // Output level for important missing information that will default
00034     // (e.g. no metric selected, no diagnostics selected, ...)
00035     constexpr auto Output_Important_Default = OutputLevel::Level_0_WARNING;
00036     // Output level for less important information that will default
00037     // (e.g. Kerr metric a parameter not specified)
00038     constexpr auto Output_Other_Default = OutputLevel::Level_1_PROC;
00039
00040     // Total configuration object, as loaded from configuration file
00041     using ConfigCollection = ConfigReader::ConfigCollection;
00042
00043     // An exception to throw whenever an important setting is not found
00044     // Always should be caught and then reverted to default settings
00045     using SettingError = std::invalid_argument;
00046
00049
00050     // Use configuration to initialize the screen output level
00051     void InitializeScreenOutput(const ConfigCollection &theCfg);
00052
00053     // Use configuration to create the correct Metric with specified parameters
00054     std::unique_ptr<Metric> GetMetric(const ConfigCollection &theCfg);
00055
00056     // Use configuration to create the correct Source with specified parameters
00057     std::unique_ptr<Source> GetSource(const ConfigCollection &theCfg, const Metric *const theMetric);
00058 }
```

```

00059     // Use configuration to set the Diagnostics bitflag appropriately;
00060     // initialize all DiagnosticOptions for all Diagnostics that are turned on;
00061     // and set bitflag for diagnostic to be used for coarseness evaluating in Mesh
00062     void InitializeDiagnostics(const ConfigCollection &theCfg, DiagBitflag &alldiags, DiagBitflag
&valdiag, const Metric *const theMetric);
00063
00064     // Use configuration to set the Terminations bitflag appropriately;
00065     // initialize all TerminationOptions for all Terminations that are turned on;
00066     void InitializeTerminations(const ConfigCollection &theCfg, TermBitflag &allterms, const Metric
*const theMetric);
00067
00068     // Use configuration to create ViewScreen appropriately
00069     std::unique_ptr<ViewScreen> GetViewScreen(const ConfigCollection &theCfg, DiagBitflag valdiag,
const Metric *const theMetric);
00070     // GetViewScreen calls GetMesh to create the correct Mesh
00071     std::unique_ptr<Mesh> GetMesh(const ConfigCollection &theCfg, DiagBitflag valdiag);
00072
00073     // Use configuration to return a pointer to the correct integration function to use
00074     GeodesicIntegratorFunc GetGeodesicIntegrator(const ConfigCollection &theCfg);
00075
00076     // Use configuration to initialize the output handler
00077     std::unique_ptr<GeodesicOutputHandler> GetOutputHandler(const ConfigCollection &theCfg,
DiagBitflag alldiags, DiagBitflag valdiag,
00078 std::string FirstLineInfo);
00079
00080 } // end namespace Config
00081
00082 #endif

```

7.4 FOORT/src/ConfigReader.cpp File Reference

```

#include "ConfigReader.h"
#include <ios>
#include <type_traits>

```

7.4.1 Detailed Description

Author

Daniel R. Mayerson

Version

1.0

Date

2024-12-12

Copyright

Copyright (c) 2024

7.5 FOORT/src/ConfigReader.h File Reference

Classes used to read in a configuration file and store the settings.

```
#include <string_view>
#include <string>
#include <iostream>
#include <fstream>
#include <vector>
#include <variant>
#include <stdexcept>
#include <limits>
#include <memory>
```

Classes

- class [ConfigReader::ConfigReaderException](#)
- class [ConfigReader::ConfigCollection](#)
- struct [ConfigReader::ConfigCollection::ConfigSetting](#)

Namespaces

- namespace [ConfigReader](#)
Namespace to put all [ConfigReader](#) objects.

Variables

- const long long [ConfigReader::MaxStreamSize](#) {std::numeric_limits<std::streamsize>::max()}

7.5.1 Detailed Description

Classes used to read in a configuration file and store the settings.

Author

Daniel R. Mayerson

Version

1.0

Date

2024-12-12

Copyright

Copyright (c) 2024

7.6 ConfigReader.h

[Go to the documentation of this file.](#)

```

00001 #ifndef _CONFIGREADER_H
00002 #define _CONFIGREADER_H
00003
00012
00013 #include <string_view> // std::string_view
00014 #include <string>      // std::string
00015 #include <iostream>    // needed for file and console output
00016 #include <fstream>     // needed for file input
00017 #include <vector>      // needed to create vectors of Settings
00018 #include <variant>     // needed for std::variant
00019 #include <stdexcept>   // needed for exceptions
00020 #include <limits>      // for std::numeric_limits
00021 #include <memory>      // for std::unique_ptr
00022
00026 namespace ConfigReader
00027 {
00028     // Shorthand for the maximum number of characters we can ignore in a stream
00029     const long long MaxStreamSize{std::numeric_limits<std::streamsize>::max()};
00030
00031     // Exception that is thrown if anything bad happens when reading in configuration file
00032     class ConfigReaderException : public std::runtime_error
00033     {
00034     public:
00035         ConfigReaderException(const std::string &error, std::vector<int> settingtrace = {})
00036             : std::runtime_error(error.c_str()), m_settingtrace{settingtrace}
00037         {
00038         }
00039         // no need to override what() since we can just use std::runtime_error::what()
00040
00041         std::vector<int> trace() const
00042         {
00043             return m_settingtrace;
00044         };
00045
00046     private:
00047         std::vector<int> m_settingtrace;
00048     };
00049
00050     // A collection of configuration settings
00051     class ConfigCollection
00052     {
00053     public:
00054         // Returns true if the collection contains a setting with the given name
00055         bool Exists(std::string_view SettingName) const;
00056         // Returns total number of settings in this collection
00057         int NrSettings() const;
00058
00059         // Returns true if the setting with the given name (resp. index) is a collection of
00060         subsettings
00061         // (Returns false if this setting name does not exist, or index is out of bounds)
00062         bool IsCollection(std::string_view SettingName) const;
00063         bool IsCollection(int SettingIndex) const;
00064         // If CollectionName (resp. CollectionIndex) is the name (resp. index)
00065         // of a setting that is a subcollection, then this returns
00066         // a const reference to this subcollection; otherwise throws ConfigReaderException
00067         const ConfigCollection &operator[](std::string_view CollectionName) const;
00068         const ConfigCollection &operator[](int CollectionIndex) const;
00069
00070         // Generic functions that look up the value of a given setting
00071         // and put it in the output variable
00072         // Returns true if successful, false if unsuccessful
00073         // (either will due to type mismatch of output and setting value, or if setting doesn't exist)
00074         // Templated functions implemented below in .h file!
00075         template <class OutputType>
00076         bool LookupValue(std::string_view SettingName, OutputType &theOutput) const;
00077         template <class OutputType>
00078         bool LookupValue(int SettingIndex, OutputType &theOutput) const;
00079
00080         // Specific functions for looking up signed integral types
00081         // These will first see if the setting value is of the given type, then
00082         // they will try smaller types and then recast to the larger type
00083         // (so e.g. for long long, first long long is tried, then long, then int)
00084         bool LookupValueInteger(std::string_view SettingName, int &theOutput) const;
00085         bool LookupValueInteger(std::string_view SettingName, long &theOutput) const;
00086         bool LookupValueInteger(std::string_view SettingName, long long &theOutput) const;
00087         // Specific functions for looking up unsigned integral types
00088         // These will first test if the setting value is of the signed case, if it is and it is >=0
00089         then it always fits in
00090         // the unsigned version; then it will test larger (signed) types and see if the result still
00091         fits in the unsigned version
00092         bool LookupValueInteger(std::string_view SettingName, unsigned int &theOutput) const;
00093         bool LookupValueInteger(std::string_view SettingName, unsigned long &theOutput) const;

```

```

00093     bool LookupValueInteger(std::string_view SettingName, unsigned long long &theOutput) const;
00094     // Templated functions implemented below in .h file!
00095     template <class OutputType>
00096     bool LookupValueInteger(int SettingIndex, OutputType &theOutput) const;
00097
00098     void DisplayCollection(std::ostream &OutputStream, int Indent = 0) const;
00099
00100     // this overwrites the current collection with the contents of the config file
00101     // Will throw ConfigReaderException if parse/syntax error in configuration file
00102     // Returns true if successful
00103     bool ReadFile(const std::string &FileName);
00104
00105 private:
00106     // A configuration setting value can be an integral type, bool, double, string, or a
00107     (sub)collection of settings
00108     // Note: when reading in a configuration file, an integral value will always be stored in the
00109     smallest possible type!
00110     using ConfigSettingValue = std::variant<bool,
00111                                             int, long, long long,
00112                                             double,
00113                                             std::string,
00114                                             std::unique_ptr<ConfigCollection>;
00115     // A setting is a name and setting value
00116     struct ConfigSetting
00117     {
00118         std::string SettingName;
00119         ConfigSettingValue SettingValue;
00120     };
00121     // The vector of all settings in this collection
00122     std::vector<ConfigSetting> m_Settings{};
00123
00124     // Helper function that returns the index of the setting with the given name
00125     // returns -1 if setting not found
00126     int GetSettingIndex(std::string_view SettingName) const;
00127
00128     // Helper functions for DisplayCollection
00129     // DisplaySetting displays one setting; if it's a subcollection then it calls
00130     // DisplayCollection on the subcollection
00131     void DisplaySetting(std::ostream &OutputStream, int SettingIndex, int Indent) const;
00132     // Output given number of tabs
00133     void DisplayTabs(std::ostream &OutputStream, int NrTabs) const;
00134
00135     // Helper functions for ReadFile()
00136     // These all take an ifstream at a given location and will read in one specific object
00137     //
00138     // Read in entire collection (including subcollections)
00139     // Pre: stream is positioned right after { (or at beginning of file)
00140     // Post: stream is positioned right after } (or at EOF)
00141     void ReadCollection(std::ifstream &InputFile);
00142     // Read in a setting name
00143     // Pre: next non-whitespace character in stream is beginning of name
00144     // Post: stream has just read in all characters of name (but no more);
00145     // returns "" if there is no more setting to read in the current collection, in this case
00146     // '}' has been read in (for subcollections)
00147     void ReadSettingName(std::ifstream &InputFile, std::string &theName);
00148     // Read in one specific character only (not '!!')
00149     // Pre: next non-whitespace character in stream is this character
00150     // Post: stream is just after character
00151     void ReadSettingSpecificChar(std::ifstream &InputFile, char theChar) const;
00152     // Read in setting value (could be subcollection)
00153     // Pre: next non-whitespace character in stream is beginning of value
00154     // (can be '{' for subcollection, digit for number, '"' for string, or 't'/'f' for boolean)
00155     // Post: Value has entirely been read in (i.e. next character should be ';')
00156     void ReadSettingValue(std::ifstream &InputFile, ConfigSettingValue &theValue);
00157 };
00158 } // end namespace declarations
00159
00160
00161 template <class OutputType>
00162 bool ConfigReader::ConfigCollection::LookupValue(int SettingIndex, OutputType &theOutput) const
00163 {
00164     // Get the setting value, if it is indeed of this type
00165     const OutputType *OutputPointer{std::get_if<OutputType>(&m_Settings[SettingIndex].SettingValue)};
00166     if (OutputPointer)
00167     {
00168         // Success, the setting is indeed of this type; set the output accordingly
00169         theOutput = *OutputPointer;
00170         return true;
00171     }
00172     return false;
00173 }
00174
00175 template <class OutputType>
00176 bool ConfigReader::ConfigCollection::LookupValue(std::string_view SettingName, OutputType &theOutput)
00177 const
00178 {
00179     {

```

```

00199 // Look up setting index and then defer to index-based implementation, if setting exists
00200 int SettingIndex{GetSettingIndex(SettingName)};
00201 if (SettingIndex >= 0)
00202 {
00203     return LookupValue(SettingIndex, theOutput);
00204 }
00205 return false;
00206 }
00207
00218 template <class OutputType>
00219 bool ConfigReader::ConfigCollection::LookupValueInteger(int SettingIndex, OutputType &theOutput) const
00220 {
00221     return LookupValueInteger(m_Settings[SettingIndex].SettingName, theOutput);
00222 }
00223
00224 #endif

```

7.7 FOORT/src/Diagnostics.cpp File Reference

```

#include "Diagnostics.h"
#include "DiagnosticsEmission.h"
#include "Geodesic.h"
#include "InputOutput.h"
#include "Integrators.h"
#include <algorithm>
#include <cmath>

```

Functions

- [DiagnosticUniqueVector CreateDiagnosticVector](#) ([DiagBitflag](#) diagflags, [DiagBitflag](#) valdiag, [Geodesic](#) *const theGeodesic)
Diagnostic helper function.

7.7.1 Detailed Description

Author

Daniel R. Mayerson

Version

1.0

Date

2024-12-16

Copyright

Copyright (c) 2024

7.7.2 Function Documentation

7.7.2.1 CreateDiagnosticVector()

```
DiagnosticUniqueVector CreateDiagnosticVector (  
    DiagBitflag diagflags,  
    DiagBitflag valdiag,  
    Geodesic *const theGeodesic)
```

[Diagnostic](#) helper function.

Create a [Diagnostic](#) Vector object

Parameters

<i>diagflags</i>	Diagnostic bitflags
<i>valdiag</i>	Value Diagnostic bitflag
<i>theGeodesic</i>	Pointer to the Geodesic object

Returns

DiagnosticUniqueVector

7.8 FOORT/src/Diagnostics.h File Reference

Declarations of abstract base [Diagnostic](#) class and all its descendants. All Definitions in [Diagnostics.cpp](#).

```
#include "Geometry.h"
#include "Metric.h"
#include "InputOutput.h"
#include "DiagnosticsEmission.h"
#include <stdint>
#include <string>
#include <memory>
#include <vector>
```

Classes

- struct [UpdateFrequency](#)
Struct to hold the information for a [Diagnostic](#) to update itself.
- class [Diagnostic](#)
Abstract base class for all diagnostics.
- class [FourColorScreenDiagnostic](#)
The four color screen diagnostic.
- class [GeodesicPositionDiagnostic](#)
Geodesic position tracker.
- class [EquatorialPassesDiagnostic](#)
Diagnostic that counts the number of passes through the equatorial plane.
- class [ClosestRadiusDiagnostic](#)
Diagnostic that keeps track of the closest (true, not log) radius that the geodesic passes through.
- class [EquatorialEmissionDiagnostic](#)
Diagnostic that calculates brightness intensity for the geodesic, based on a specified equatorial disc emission model.
- struct [DiagnosticOptions](#)
Base class for [DiagnosticOptions](#). Other Diagnostics can inherit from here if they require more options.
- struct [GeodesicPositionOptions](#)
Options for [GeodesicPositionDiagnostic](#).
- struct [EquatorialPassesOptions](#)
Options for [EquatorialPassesDiagnostic](#).
- struct [ClosestRadiusOptions](#)
Options for [ClosestRadiusDiagnostic](#).
- struct [EquatorialEmissionOptions](#)
Options for [EquatorialEmissionDiagnostic](#).

Typedefs

- using [DiagBitflag](#) = std::uint16_t
Diagnostic bitflags.
- using [DiagnosticUniqueVector](#) = std::vector<std::unique_ptr<[Diagnostic](#)>>
Owner vector of derived Diagnostics classes.

Functions

- [DiagnosticUniqueVector CreateDiagnosticVector](#) ([DiagBitflag](#) diagflags, [DiagBitflag](#) valdiag, [Geodesic](#) *const theGeodesic)
Diagnostic helper function.

Variables

- constexpr [DiagBitflag](#) [Diag_None](#) {0b0000'0000'0000'0000}
- constexpr [DiagBitflag](#) [Diag_GeodesicPosition](#) {0b0000'0000'0000'0001}
- constexpr [DiagBitflag](#) [Diag_FourColorScreen](#) {0b0000'0000'0000'0010}
- constexpr [DiagBitflag](#) [Diag_EquatorialPasses](#) {0b0000'0000'0000'0100}
- constexpr [DiagBitflag](#) [Diag_ClosestRadius](#) {0b0000'0000'0000'1000}
- constexpr [DiagBitflag](#) [Diag_EquatorialEmission](#) {0b0000'0000'0001'0000}

7.8.1 Detailed Description

Declarations of abstract base [Diagnostic](#) class and all its descendants. All Definitions in [Diagnostics.cpp](#).

Author

Daniel R. Mayerson

Version

1.0

Date

2024-12-16

Copyright

Copyright (c) 2024

7.8.2 Typedef Documentation

7.8.2.1 DiagBitflag

using [DiagBitflag](#) = std::uint16_t

[Diagnostic](#) bitflags.

Used for constructing vector of Diagnostics ...

7.8.2.2 DiagnosticUniqueVector

```
using DiagnosticUniqueVector = std::vector<std::unique_ptr<Diagnostic>>
```

Owner vector of derived Diagnostics classes.

7.8.3 Function Documentation

7.8.3.1 CreateDiagnosticVector()

```
DiagnosticUniqueVector CreateDiagnosticVector (
    DiagBitflag diagflags,
    DiagBitflag valdiag,
    Geodesic *const theGeodesic)
```

[Diagnostic](#) helper function.

Helper to create a new vector of [Diagnostic](#) options, based on the bitflag The first diagnostic is the value diagnostic

Create a [Diagnostic](#) Vector object

Parameters

<i>diagflags</i>	Diagnostic bitflags
<i>valdiag</i>	Value Diagnostic bitflag
<i>theGeodesic</i>	Pointer to the Geodesic object

Returns

[DiagnosticUniqueVector](#)

7.8.4 Variable Documentation

7.8.4.1 Diag_ClosestRadius

```
DiagBitflag Diag_ClosestRadius {0b0000'0000'0000'1000} [constexpr]
```

7.8.4.2 Diag_EquatorialEmission

```
DiagBitflag Diag_EquatorialEmission {0b0000'0000'0001'0000} [constexpr]
```

7.8.4.3 Diag_EquatorialPasses

```
DiagBitflag Diag_EquatorialPasses {0b0000'0000'0000'0100} [constexpr]
```

7.8.4.4 Diag_FourColorScreen

```
DiagBitflag Diag_FourColorScreen {0b0000'0000'0000'0010} [constexpr]
```

7.8.4.5 Diag_GeodesicPosition

```
DiagBitflag Diag_GeodesicPosition {0b0000'0000'0000'0001} [constexpr]
```

7.8.4.6 Diag_None

```
DiagBitflag Diag_None {0b0000'0000'0000'0000} [constexpr]
```

7.9 Diagnostics.h

[Go to the documentation of this file.](#)

```
00001 #ifndef _FOORT_DIAGNOSTICS_H
00002 #define _FOORT_DIAGNOSTICS_H
00003
00012
00013 #include "Geometry.h" // for tensors
00014 #include "Metric.h" // for metric
00015 #include "InputOutput.h" // for ScreenOutput
00016
00017 #include "DiagnosticsEmission.h" // Emission models
00018
00019 #include <stdint> // for std::uint16_t
00020 #include <string> // for strings
00021 #include <memory> // for std::unique_ptr
00022 #include <vector> // for std::vector
00023
00024 // Forward declaration of Geodesic class needed here, since Diagnostics are passed a pointer to their
00025 // owner Geodesic
00026 // (note "Geodesic.h" is NOT included to avoid header loop, and we do not need Geodesic member
00027 // functions here!)
00028 class Geodesic;
00029
00030
00031 // Diagnostic bitflags
00032 // Used for constructing vector of Diagnostics
00033 // Note that this means every diagnostic is either "on" or "off";
00034 // it is not possible to have a Diagnostic "on" more than once
00035 // Note: why not std::bitset<size>? Because in this way we can use expressions with DiagBitflag in
00036 // conditional expressions,
00037 // e.g. if ( mydiagbitflag & Diag_FourColorScreen)
00038 using DiagBitflag = std::uint16_t;
00039
00040
00041 // Define a bitflag per existing diagnostic
00042 constexpr DiagBitflag Diag_None{0b0000'0000'0000'0000};
00043 constexpr DiagBitflag Diag_GeodesicPosition{0b0000'0000'0000'0001};
00044 constexpr DiagBitflag Diag_FourColorScreen{0b0000'0000'0000'0010};
00045 constexpr DiagBitflag Diag_EquatorialPasses{0b0000'0000'0000'0100};
00046 constexpr DiagBitflag Diag_ClosestRadius{0b0000'0000'0000'1000};
00047 constexpr DiagBitflag Diag_EquatorialEmission{0b0000'0000'0001'0000};
00048
00049 // Add a DiagBitflag for your new diagnostic. Make sure you use a bitflag that has not been used
00050 // before!
00051 // Sample code:
00052 /*
00053 constexpr DiagBitflag Diag_MyDiag { 0b0000'0000'0000'1000 };
00054 */
00055
00056 struct UpdateFrequency
00057 {
00058     largecounter UpdateNSteps{0};
00059     bool UpdateStart{false};
00060     bool UpdateFinish{false};
00061 };
00062
00063 class Diagnostic
00064 {
```

```

00076 public:
00077     // Constructor must initialize the pointer to its owner Geodesic
00078     Diagnostic() = delete;
00079     Diagnostic(Geodesic *const theGeodesic) : m_OwnerGeodesic{theGeodesic}
00080     {
00081     }
00082
00083     // Resets Diagnostic object. This is called when the owner Geodesic is reset in order to start
    integrating
00084     // a new geodesic.
00085     // The base class implementation only resets m_StepsSinceUpdated
00086     // Descendants can override this if they need to reset additional internal variables
00087     virtual void Reset();
00088
00089     // virtual destructor to ensure correct destruction of descendants
00090     virtual ~Diagnostic() = default;
00091
00092     // This is the heart of the Diagnostic. It calls the helper function DecideUpdate(), and if this
00093     // returns true, will update its internal status based on the current (new) state of its owner
    geodesic.
00094     virtual void UpdateData() = 0;
00095
00096     // These functions are for use at the end of integration of a geodesic.
00097     // getFullData() returns all the data stored in the Diagnostic as a string (for output to file)
00098     virtual std::string getFullDataStr() const = 0;
00099     // getFinalDataVal() associates a value to the geodesic corresponding to the final value of this
    diagnostic
00100     // (used for determining coarseness of nearby geodesics)
00101     virtual std::vector<real> getFinalDataVal() const = 0;
00102
00103     // Function used to determine distance between two values obtained from getFinalDataVal()
00104     // (for determining coarseness of nearby geodesics)
00105     // This should return a number >=0
00106     virtual real FinalDataValDistance(const std::vector<real> &val1, const std::vector<real> &val2)
    const = 0;
00107
00108     // Getters for descriptions
00109     // This returns the name (only) of the Diagnostic, as a string without spaces that will be
    appended
00110     // to an output file (e.g. prefix_DiagName.ext). Must be implemented!
00111     virtual std::string getNameStr() const = 0;
00112     // This returns the full description of the Diagnostic. Default implementation
00113     // returns GetDiagNameStr()
00114     virtual std::string getFullDescriptionStr() const;
00115
00116 protected:
00117     // The geodesic that owns the Diagnostic (a const pointer to the Geodesic)
00118     Geodesic *const m_OwnerGeodesic;
00119
00120     // Helper function to decide if the Diagnostic should indeed update its status, based on
00121     // its UpdateFrequency struct information (which will come from the Diagnostics's
    DiagnosticOptions)
00122     bool DecideUpdate(const UpdateFrequency &myUpdateFrequency);
00123
00124     // The diagnostic is itself in charge of keeping track of how many steps it has been since it has
    been updated
00125     // The Diagnostic's DiagnosticOptions struct tells it how many steps it needs to wait between
    updates
00126     largecounter m_StepsSinceUpdated{};
00127 };
00128
00129 using DiagnosticUniqueVector = std::vector<std::unique_ptr<Diagnostic>;
00130
00131 DiagnosticUniqueVector CreateDiagnosticVector(DiagBitflag diagflags, DiagBitflag valdiag, Geodesic
    *const theGeodesic);
00132
00133
00134 class FourColorScreenDiagnostic final : public Diagnostic
00135 {
00136 public:
00137     // Basic constructor only passes on Geodesic pointer to base class constructor
00138     FourColorScreenDiagnostic(Geodesic *const theGeodesic) : Diagnostic(theGeodesic) {}
00139
00140     // Reset needs to be overridden in order to also reset m_quadrant
00141     void Reset() final;
00142
00143     // Sets the quadrant to the appropriate value, IF the boundary sphere has been reached
00144     void UpdateData() override;
00145
00146     // Both of these output functions simply returns the quadrant number associated with the
    geodesic's end position
00147     std::string getFullDataStr() const final;
00148     std::vector<real> getFinalDataVal() const final;
00149
00150     // Discrete metric for distance: returns 0 if the quadrants are the same, 1 if they are not
00151     real FinalDataValDistance(const std::vector<real> &val1, const std::vector<real> &val2) const
    final;

```

```

00162
00163     // Description string getters
00164     std::string getNameStr() const final;
00165     std::string getFullDescriptionStr() const final;
00166
00167     // FourColorScreen does not need any (static) options!
00168
00169 private:
00171     int m_quadrant{0};
00172 };
00173
00175 struct GeodesicPositionOptions;
00179 class GeodesicPositionDiagnostic final : public Diagnostic
00180 {
00181 public:
00182     // Basic constructor only passes on Geodesic pointer to base class constructor
00183     GeodesicPositionDiagnostic(Geodesic *const theGeodesic) : Diagnostic(theGeodesic) {}
00184
00185     // Override Reset to also clear m_AllSavedPoints
00186     void Reset() final;
00187
00188     // Stores the current position of the geodesic
00189     void UpdateData() final;
00190
00191     // This returns as many stored positions as is specified in the options struct
00192     std::string getFullDataStr() const final;
00193     // This returns the final (theta,phi) value of the geodesic
00194     std::vector<real> getFinalDataVal() const final;
00195
00196     // This determines the angular distance between two geodesic
00197     // (based on their final angles on the boundary sphere (theta, phi))
00198     real FinalDataValDistance(const std::vector<real> &vall, const std::vector<real> &val2) const
00199     final;
00200
00201     // Description string getters
00202     std::string getNameStr() const final;
00203     std::string getFullDescriptionStr() const final;
00204
00205     // The options: specifies the update frequency but also how many points we want to output in the
00206     end
00207     static std::unique_ptr<GeodesicPositionOptions> DiagOptions;
00208
00209 private:
00210     // Keeps track of the points that are saved
00211     std::vector<Point> m_AllSavedPoints{};
00212 };
00213
00215 struct EquatorialPassesOptions;
00217 class EquatorialPassesDiagnostic : public Diagnostic
00218 {
00219 public:
00220     // Basic constructor only passes on Geodesic pointer to base class constructor
00221     EquatorialPassesDiagnostic(Geodesic *const theGeodesic) : Diagnostic(theGeodesic) {}
00222
00223     // Override Reset to also reset m_EquatPasses and m_PrevTheta
00224     void Reset() override;
00225
00226     // Checks to see if we have a new cross over the equatorial plane
00227     void UpdateData() override;
00228
00229     // Returns the number of passes over the equatorial plane
00230     std::string getFullDataStr() const override;
00231     std::vector<real> getFinalDataVal() const override;
00232
00233     // Simple absolute value of difference of passes
00234     real FinalDataValDistance(const std::vector<real> &vall, const std::vector<real> &val2) const
00235     override;
00236
00237     // Description string getters
00238     std::string getNameStr() const override;
00239     std::string getFullDescriptionStr() const override;
00240
00241     static std::unique_ptr<EquatorialPassesOptions> DiagOptions;
00242
00243 protected:
00244     int m_EquatPasses{0};
00245
00246 private:
00247     real m_PrevTheta{-1};
00248 };
00249
00251 // Forward declaration needed before Diagnostic
00252 struct ClosestRadiusOptions;
00253 class ClosestRadiusDiagnostic final : public Diagnostic
00254 {
00255 public:
00256     // Basic constructor only passes on Geodesic pointer to base class constructor

```

```

00261     ClosestRadiusDiagnostic(Geodesic *const theGeodesic) : Diagnostic(theGeodesic) {}
00262
00263     // re-initialize closest radius
00264     void Reset() final;
00265
00266     // Check to see if we have travelled closer
00267     void UpdateData() final;
00268
00269     // Return closest radius travelled
00270     std::string getFullDataStr() const final;
00271     std::vector<real> getFinalDataVal() const final;
00272
00273     // Returns the absolute value of the difference between closest radii
00274     real FinalDataValDistance(const std::vector<real> &vall, const std::vector<real> &val2) const
    final;
00275
00276     // Description string getters
00277     std::string getNameStr() const final;
00278     std::string getFullDescriptionStr() const final;
00279
00280     // Option struct to keep track of RLogScale of Metric
00281     static std::unique_ptr<ClosestRadiusOptions> DiagOptions;
00282
00283 private:
00284     // Keeps track of closest radius travelled
00285     real m_ClosestRadius{-1};
00286 };
00287
00288 // Forward declaration needed before Diagnostic
00289 struct EquatorialEmissionOptions;
00290 class EquatorialEmissionDiagnostic final : public EquatorialPassesDiagnostic
00291 {
00292 public:
00293     // Basic constructor only passes on Geodesic pointer to parent class constructor
00294     EquatorialEmissionDiagnostic(Geodesic *const theGeodesic) :
    EquatorialPassesDiagnostic(theGeodesic) {}
00301
00302     // Reset intensity back to 0.0, and call parent class Reset()
00303     void Reset() final;
00304
00305     // Use parent class UpdateData() to decide whether the update the intensity
00306     // using the specified emission model
00307     void UpdateData() final;
00308
00309     // Returns total intensity and total number of equatorial passes (i.e. normal output from
    EquatorialPassesDiagnostic)
00310     std::string getFullDataStr() const final;
00311     std::vector<real> getFinalDataVal() const final;
00312
00313     // Distance is difference in intensities multiplied by a factor magnifying difference in
    equatorial passes
00314     real FinalDataValDistance(const std::vector<real> &vall, const std::vector<real> &val2) const
    final;
00315
00316     // Description string getters
00317     std::string getNameStr() const final;
00318     std::string getFullDescriptionStr() const final;
00319
00320     // Option struct to keep track emission model specifics
00321     static std::unique_ptr<EquatorialEmissionOptions> DiagOptions;
00322
00323 private:
00324     // Brightness intensity of the geodesic
00325     real m_Intensity{0.0};
00326 };
00327
00328 // Declare your Diagnostic class here, inheriting from Diagnostic.
00329 // Sample code:
00330 /*
00331 // Forward declaration needed before Diagnostic (if using base class DiagnosticOptions, this is
    strictly speaking not necessary)
00332 struct DiagnosticOptions;
00333 // class definition
00334 class MyDiagnostic final : public Diagnostic // good practice to make the class final unless
    descendant classes are possible
00335 {
00336 public:
00337     // constructor must at least take and pass along the const pointer to the owner Geodesic
00338     MyDiagnostic(Geodesic* const theGeodesic) : Diagnostic(theGeodesic) {}
00339
00340     // If you have MyDiagnostic-specific member variables, then they probably need to be reset at the
    beginning
00341
00342     // of integration for each new geodesic; in that case, you need to override Reset() to do so.
00343     // Note: make sure to call the base class implementation Diagnostic::Reset()
00344     // from within your implementation of MyDiagnostic::Reset(), so that the base class internal
    variable is also reset!
00345     void Reset() final;

```

```

00346
00347 // This is the heart of the Diagnostic: here the Diagnostic updates its internal state according
to
00348 // the owner Geodesic's current state
00349 void UpdateData() final;
00350
00351 // This should return the string that is to be outputted to the file as the final output of this
Diagnostic
00352 // for its owner Geodesic
00353 std::string getFullDataStr() const final;
00354 // This should return a vector of real numbers that indicates the final "value" that should be
associated to
00355 // the owner Geodesic --- this is then used in FinalDataValDistance() to find "distances" between
geodesics.
00356 std::vector<real> getFinalDataVal() const final;
00357
00358 // This should return a (positive) distance of two values returned by getFinalDataVal(), indicated
the
00359 // "distance" of two geodesics (this is used for Mesh refinement)
00360 real FinalDataValDistance(const std::vector<real>& val1, const std::vector<real>& val2) const
final;
00361
00362 // Must implement getNameStr() (and recommended also to implement getFullDescriptionStr)
00363 // getNameStr() is a simple, short string (without spaces) that will be appended to the file name
00364 // where this Diagnostic's output is written. getFullDescriptionStr() is a descriptive string that
should list
00365 // all relevant options set; this is outputted to e.g. the screen at runtime.
00366 std::string getNameStr() const final;
00367 std::string getFullDescriptionStr() const final;
00368
00369 // Diagnostic options: basic DiagnosticOptions only contains UpdateFrequency information,
00370 // if needed, can use a descendant class with more options (see e.g. GeodesicPositionDiagnostic)
00371 static std::unique_ptr<DiagnosticOptions> DiagOptions;
00372
00373 private:
00374 // will probably want some private member variable(s) to keep track of whatever geodesic property
is desired
00375 };
00376 */
00377
00378
00381
00385 struct DiagnosticOptions
00386 {
00387 public:
00388 // Basic constructor only sets the number of steps between updates
00389 DiagnosticOptions(UpdateFrequency thefrequency) : theUpdateFrequency{thefrequency}
00390 {
00391 }
00392
00393 virtual ~DiagnosticOptions() = default;
00394
00395 const UpdateFrequency theUpdateFrequency;
00396 };
00402 struct GeodesicPositionOptions : public DiagnosticOptions
00403 {
00404 public:
00405 GeodesicPositionOptions(largecounter outputsteps, UpdateFrequency thefrequency) :
OutputNrSteps{outputsteps},
00406 DiagnosticOptions(thefrequency)
00407 {
00408 }
00409
00410 const largecounter OutputNrSteps;
00411 };
00412
00418 struct EquatorialPassesOptions : public DiagnosticOptions
00419 {
00420 public:
00421 EquatorialPassesOptions(real thethreshold, UpdateFrequency thefrequency) :
Threshold{thethreshold},
00422 DiagnosticOptions(thefrequency)
00423 {
00424 }
00425
00426 const real Threshold;
00427 };
00428
00434 struct ClosestRadiusOptions : public DiagnosticOptions
00435 {
00436 public:
00437 ClosestRadiusOptions(bool rlog, UpdateFrequency thefrequency) : RLogScale{rlog},
DiagnosticOptions(thefrequency)
00438 {
00439 }
00440 }
00441

```

```

00442     const bool RLogScale;
00443 };
00444
00445 // Forward declarations of abstract EmissionModel and FluidVelocityModel classes.
00446 // See Diagnostics_Emission.h and .cpp for declarations and definitions of these and their descendant
00447 // classes!
00447 struct EmissionModel;
00448 struct FluidVelocityModel;
00454 struct EquatorialEmissionOptions : public EquatorialPassesOptions
00455 {
00456 public:
00457     EquatorialEmissionOptions(real thefudgefactor, int equatupper,
00458                             std::unique_ptr<EmissionModel> theemission,
00459                             std::unique_ptr<FluidVelocityModel> thefluidmodel,
00459                             bool rlog, int theredshiftpower, real thethreshold, UpdateFrequency
00460                             thefrequency) : RedShiftPower{theredshiftpower}, RLogScale{rlog},
00461                             GeometricFudgeFactor{thefudgefactor},
00462                             EquatPassUpperBound{equatupper},
00463                             TheEmissionModel{std::move(theemission)}, TheFluidVelocityModel{std::move(thefluidmodel)},
00464                             EquatorialPassesOptions(thethreshold, thefrequency) // constructor for parent class
00465     {
00466         // Geometric fudge factor for n>0 passes
00467         const real GeometricFudgeFactor;
00468         // Upper bound allowed for contribution to intensity
00469         const int EquatPassUpperBound;
00470
00471         // Logarithmic radius scale used or not
00472         const bool RLogScale;
00473
00474         // Power of redshift in intensity contribution (should be 3 or 4)
00475         const int RedShiftPower;
00476
00477         // The emission model used (see Diagnostics_Emission.h and .cpp for specific models)
00478         const std::unique_ptr<EmissionModel> TheEmissionModel;
00479         // The fluid velocity model used (see Diagnostics_Emission.h and .cpp for specific models)
00480         const std::unique_ptr<FluidVelocityModel> TheFluidVelocityModel;
00481     };
00482
00484 // if necessary, define your new DiagnosticOptions class here
00485 // Sample code:
00486 /*
00487 struct MyDiagnosticOptions : public DiagnosticOptions
00488 {
00489 public:
00490     // Constructor should pass along UpdateFrequency information to base class
00491     MyDiagnosticOptions(UpdateFrequency thefrequency) : DiagnosticOptions(thefrequency) //, (...)
00492     other initializations
00493     {}
00494
00495     // other member variables here - make them const!
00496     // ...
00497 };
00498 */
00499
00500 #endif

```

7.10 FOORT/src/DiagnosticsEmission.cpp File Reference

```

#include "DiagnosticsEmission.h"
#include "Integrators.h"
#include <cmath>

```

7.10.1 Detailed Description

Author

Daniel R. Mayerson

Version

1.0

Date

2024-12-16

Copyright

Copyright (c) 2024

7.11 FOORT/src/DiagnosticsEmission.h File Reference

Declarations of emission and fluid velocity models used for equatorial disc emission.

```
#include "Geometry.h"
#include "Metric.h"
#include "InputOutput.h"
#include <string>
#include <cmath>
#include <stdexcept>
```

Classes

- struct [EmissionModel](#)
Emission model abstract base class.
- struct [GLMJohnsonSUEmission](#)
The Johnson SU emission model used in GLM 2020, PRD 102(12):124004.
- struct [FluidVelocityModel](#)
Fluid velocity model abstract base class.
- struct [GeneralCircularRadialFluid](#)
General circular radial fluid velocity model.

7.11.1 Detailed Description

Declarations of emission and fluid velocity models used for equatorial disc emission.

Author

Daniel R. Mayerson

Version

1.0

Date

2024-12-16

Copyright

Copyright (c) 2024

7.12 DiagnosticsEmission.h

[Go to the documentation of this file.](#)

```

00001 #ifndef _FOORT_DIAGNOSTICS_EMISSION_H
00002 #define _FOORT_DIAGNOSTICS_EMISSION_H
00003
00004 #include "Geometry.h" // Tensor objects
00005 #include "Metric.h" // for Metric functions
00006 #include "InputOutput.h" // for ScreenOutput
00007
00008 #include <string> // for strings
00009 #include <cmath> // for fmax, fmin
00010 #include <stdexcept> // for std::runtime_error
00011
00020
00025 struct EmissionModel
00026 {
00027 public:
00028     // Virtual destructor to ensure correct destruction
00029     virtual ~EmissionModel() = default;
00030
00031     // Function which returns the emitted brightness intensity at Point p
00032     // Note: always pass the true radius (not log(r)) to EmissionModel!
00033     virtual real GetEmission(const Point &p) const = 0;
00034
00035     // Description string getter
00036     virtual std::string getFullDescriptionStr() const;
00037 };
00038
00042 struct GLMJohnsonSUEmission final : public EmissionModel
00043 {
00044 public:
00045     // Constructor
00046     GLMJohnsonSUEmission(real mu, real gamma, real sigma) : m_mu{mu}, m_gamma{gamma}, m_sigma{sigma}
00047     {
00048     }
00049
00050     // Emitted brightness (only depends on radius)
00051     real GetEmission(const Point &p) const final;
00052
00053     // Description string getter
00054     std::string getFullDescriptionStr() const final;
00055 private:
00056     const real m_mu;
00058     const real m_gamma;
00060     const real m_sigma;
00062 };
00063
00064
00066
00071 struct FluidVelocityModel
00072 {
00073 public:
00074     // Constructor is passed Metric pointer
00075     FluidVelocityModel(const Metric *const theMetric) : m_theMetric{theMetric} {}
00076
00077     // Virtual destructor to ensure correct destruction
00078     virtual ~FluidVelocityModel() = default;
00079
00080     // Get the local four-velocity(with index down!) of the fluid at Point p
00081     virtual OneIndex GetFourVelocityd(const Point &p) const = 0;
00082
00083     // Description string getter
00084     virtual std::string getFullDescriptionStr() const;
00085 protected:
00086     // Metric pointer, used for e.g. calculating geodesic orbits
00087     const Metric *const m_theMetric;
00088 };
00089
00090
00096 struct GeneralCircularRadialFluid final : public FluidVelocityModel
00097 {
00098     // Constructor with three parameters and Metric pointer (which is passed to base class
00099     // constructor)
00100     GeneralCircularRadialFluid(real subKeplerParam, real betar, real betaphi, const Metric *const
00101     theMetric,
00102     real ISCO_lowerbound, real ISCO_upperbound) :
00103     m_subKeplerParam{fmin(fmax(subKeplerParam, 0.0), 1.0)}, m_betaR{fmin(fmax(betar, 0.0), 1.0)},
00104     m_betaPhi{fmin(fmax(betaphi, 0.0), 1.0)}, FluidVelocityModel(theMetric),
00105     m_ISCOlowerbound{ISCO_lowerbound}, m_ISCOupperbound{ISCO_upperbound}
00106     {
00107     // Do some checks on three params, which must lie between 0.0 and 1.0 (note that they are
00108     // adjusted as such in

```

```

00105         // initializer above)
00106         if (subKeplerParam < 0.0)
00107             ScreenOutput("Sub-Keplerian parameter must be between 0 and 1; adjusting to 0",
OutputLevel::Level_0_WARNING);
00108         if (subKeplerParam > 1.0)
00109             ScreenOutput("Sub-Keplerian parameter must be between 0 and 1; adjusting to 1",
OutputLevel::Level_0_WARNING);
00110
00111         if (betar < 0.0)
00112             ScreenOutput("beta_r parameter must be between 0 and 1; adjusting to 0",
OutputLevel::Level_0_WARNING);
00113         if (betar > 1.0)
00114             ScreenOutput("beta_r parameter must be between 0 and 1; adjusting to 1",
OutputLevel::Level_0_WARNING);
00115
00116         if (betaphi < 0.0)
00117             ScreenOutput("beta_phi parameter must be between 0 and 1; adjusting to 0",
OutputLevel::Level_0_WARNING);
00118         if (betaphi > 1.0)
00119             ScreenOutput("beta_phi parameter must be between 0 and 1; adjusting to 1",
OutputLevel::Level_0_WARNING);
00120
00121         // ISCO search bounds must be positive and correctly ordered, or the binary search in
00122         // FindISCO() silently produces a wrong or unconverged ISCO (NaN bounds under log(r)
00123         // coordinates, or an empty search range)
00124         if (ISCO_lowerbound <= 0.0 || ISCO_upperbound <= 0.0 || ISCO_lowerbound >= ISCO_upperbound)
00125             throw std::runtime_error("ISCOLowerBound and ISCOUpperBound must be positive with
ISCOLowerBound < ISCOUpperBound (got lower=" +
00126                                     std::to_string(ISCO_lowerbound) + ", upper=" +
std::to_string(ISCO_upperbound) + ")");
00127
00128         // Find the (equatorial) ISCO for this Metric
00129         FindISCO();
00130     }
00131
00132     // Get the local four-velocity of the fluid according to this model
00133     // Note: will always calculate with Point p exactly on the equator theta=pi/2, despite what p[2]
may be passed
00134     OneIndex GetFourVelocityd(const Point &p) const final;
00135
00136     // Description string getter
00137     std::string getFullDescriptionStr() const final;
00138
00139 private:
00140     // Three parameters determining the flow
00142     const real m_subKeplerParam;
00144     const real m_betaR;
00146     const real m_betaPhi;
00147
00148     // Helper function to get circular (sub)Keplerian velocity outside the ISCO
00149     OneIndex GetCircularVelocityd(const Point &p, bool subKeplerianOn = true) const;
00150     // Helper function to get circular and infalling (sub)Keplerian velocity inside ISCO
00151     // (according to prescription of Cunningham)
00152     OneIndex GetInsideISCOCircularVelocityd(const Point &p) const;
00153
00154     // Helper function to get radial infalling velocity
00155     OneIndex GetRadialVelocityd(const Point &p) const;
00156
00157     // Helper function to find the ISCO (called in Constructor)
00158     void FindISCO();
00160     bool m_ISCOexists{false};
00162     real m_ISCO{ -1.0 };
00164     real m_ISCOLowerbound;
00166     real m_ISCOUpperbound;
00168     real m_ISCOpt{};
00170     real m_ISCOpPhi{};
00171
00172     // Helper function which returns \partial_r(g^{ab})\Gamma^r_{bc}g^{cd}),
00173     // whose sign (after contracted with p_a p_d of the corresponding circular orbit)
00174     // tells us if the circular orbit considered is stable or not
00175     TwoIndex GetChrstrRaisedDer(real r) const;
00176 };
00177
00178 #endif

```

7.13 FOORT/src/Geodesic.cpp File Reference

```

#include "Geodesic.h"
#include "InputOutput.h"

```

7.13.1 Detailed Description

Author

Daniel R. Mayerson

Version

1.0

Date

2024-12-16

Copyright

Copyright (c) 2024

7.14 FOORT/src/Geodesic.h File Reference

Declarations of abstract base [Source](#) class and all its descendants. Declaration of [Geodesic](#) class.

```
#include "Geometry.h"
#include "Metric.h"
#include "Diagnostics.h"
#include "Terminations.h"
#include "Integrators.h"
#include <string>
#include <vector>
```

Classes

- class [Source](#)
Abstract [Source](#) base class.
- class [NoSource](#)
[NoSource](#) class.
- class [Geodesic](#)
[Geodesic](#) class: an instance of this class is created for each [Geodesic](#) that is integrated.

7.14.1 Detailed Description

Declarations of abstract base [Source](#) class and all its descendants. Declaration of [Geodesic](#) class.

Author

Daniel R. Mayerson

Version

1.0

Date

2024-12-16

Copyright

Copyright (c) 2024

7.15 Geodesic.h

[Go to the documentation of this file.](#)

```

00001 #ifndef _FOORT_GEODESIC_H
00002 #define _FOORT_GEODESIC_H
00003
00004 #include "Geometry.h" // for tensor objects
00005 #include "Metric.h" // for the metric
00006 #include "Diagnostics.h" // Geodesics own Diagnostics
00007 #include "Terminations.h" // Geodesics own Terminations
00008 #include "Integrators.h" // Geodesics use an GeodesicIntegratorFunc to integrate itself
00009
00010 #include <string> // for strings
00011 #include <vector> // for std::vector
00012
00021
00024
00029 class Source
00030 {
00031 public:
00032     // Constructor initializes Metric
00033     Source(const Metric *const theMetric) : m_theMetric{theMetric} {}
00034
00035     // Virtual destructor to ensure correct descendant destruction
00036     virtual ~Source() = default;
00037
00038     // Get the source for the current geodesic position and velocity
00039     virtual OneIndex getSource(Point pos, OneIndex vel) const = 0;
00040
00041     // Full description string (space allowed), to be outputted to file
00042     virtual std::string getFullDescriptionStr() const;
00043
00044 protected:
00045     const Metric *const m_theMetric;
00046 };
00047
00048
00054 class NoSource final : public Source
00055 {
00056 public:
00057     // Simple constructor passes on the Metric pointer to the base constructor
00058     NoSource(const Metric *const theMetric) : Source(theMetric) {}
00059
00060     // Returns zero source
00061     OneIndex getSource(Point pos, OneIndex vel) const final;
00062
00063     // Description string getter
00064     std::string getFullDescriptionStr() const final;
00065 };
00066
00069
00076 class Geodesic
00077 {
00078 public:
00079     // Default constructor not allowed
00080     Geodesic() = delete;
00081     // Copy constructor or copy assignment not allowed
00082     Geodesic(const Geodesic &) = delete;
00083     Geodesic &operator=(const Geodesic &) = delete;
00084
00085     Geodesic(const Metric *const theMetric, const Source *const theSource,
00086             DiagBitflag diagbit, DiagBitflag valdiagbit,
00087             TermBitflag termbit, GeodesicIntegratorFunc theIntegrator) : m_theMetric{theMetric},
00088             m_theSource{theSource},
00089             m_AllDiagnostics{CreateDiagnosticVector(diagbit, valdiagbit, this)},
00090             m_AllTerminations{CreateTerminationVector(termbit, this)},
00091             m_theIntegrator{theIntegrator}
00092     {
00093     }
00094
00095     // This initializes/resets the geodesic with a given ScreenIndex, initial position, and initial
00096     // velocity
00097     // Also resets all Diagnostics and Terminations, resets the TermCondition to Term::Continue,
00098     // and puts the Geodesic back to lambda = 0.0.
00099     void Reset(ScreenIndex scrindex, Point initpos, OneIndex initvel);
00100
00101     // This makes the Geodesic integrate itself one step; then the Geodesic loops through all
00102     // Terminations and Diagnostics to update
00103     Term Update();
00104
00105     // Getters for properties of its internal state
00106     Term getTermCondition() const; // Current termination condition (Term::Continue if not done
00107     // integrating)

```

```

00114     Point getCurrentPos() const;           // Current position
00115     OneIndex getCurrentVel() const;        // Current velocity
00116     real getCurrentLambda() const;        // Current value of affine parameter
00117     ScreenIndex getScreenIndex() const;    // screen index
00118
00119     // Output getters, to be called after the Geodesic terminates
00120     // This gets the complete output that should be written to the output files;
00121     // there is one string more than the count of Diagnostics: one string per Diagnostic,
00122     // PLUS the first string is the screen index.
00123     std::vector<std::string> getAllOutputStr() const;
00124     // This returns the "value" (from the Diagnostic that was set to the value Diagnostic) that is
associated
00125     // to the Geodesic. Will be used to determine "distance" between Geodesics which is used in Mesh
refinement.
00126     std::vector<real> getDiagnosticFinalValue() const;
00127
00128 private:
00129     // These variables define its internal state
00131     Term m_TermCond{Term::Uninitialized};
00133     Point m_CurrentPos{};
00135     OneIndex m_CurrentVel{};
00137     real m_curLambda{0.0};
00138
00141     ScreenIndex m_ScreenIndex{};
00142
00143     // These are const pointers (or const vectors of pointers) that contain all the information the
Geodesic needs
00145     const Metric *const m_theMetric;
00147     const Source *const m_theSource;
00149     const DiagnosticUniqueVector m_AllDiagnostics;
00151     const TerminationUniqueVector m_AllTerminations;
00153     const GeodesicIntegratorFunc m_theIntegrator;
00154 };
00155
00156 #endif

```

7.16 FOORT/src/Geometry.h File Reference

Definitions and some operations with geometric objects.alignas.

```

#include <limits>
#include <string>
#include <array>
#include <utility>
#include <vector>

```

Macros

- #define `LARGECOUNTER_MAX` `std::numeric_limits<largecounter>::max()`
Macro definition of maximum value that can be held in this large counter.
- #define `PIXEL_MAX` `std::numeric_limits<pixelcoord>::max()`
Macro definition of maximum value that can be held in this large counter.

Typedefs

- using `real` = double
A real number (could be changed to use arbitrary precision in the future).
- using `largecounter` = unsigned long
This type is used to count geodesics integrated.
- using `pixelcoord` = `largecounter`
A pixel coordinate: always ≥ 0 and integer; we use the largecounter type.
- using `Point` = `std::array<real, dimension>`

- *A point in spacetime. Note that coordinates are always assumed to be (t, r, theta, phi)*
using `ScreenPoint = std::array<real, dimension - 2>`
A point on the `ViewScreen`; this does not have a time or radial extent.
- using `ScreenIndex = std::array<pixelcoord, dimension - 2>`
An index on the `ViewScreen` (row, column)
- using `OneIndex = Point`
Object with one index has the same structure as a `Point`.
- using `TwoIndex = std::array<OneIndex, dimension>`
Object with two indices is an array of `OneIndex` objects.
- using `ThreeIndex = std::array<TwoIndex, dimension>`
Object with three indices is an array of `TwoIndex` objects.
- using `FourIndex = std::array<ThreeIndex, dimension>`
Object with four indices is an array of `ThreeIndex` objects.
- using `SingularityCoord = std::pair<int, real>`
SingularityCoord: pair of (coordinate number, coordinate value)
- using `Singularity = std::vector<SingularityCoord>`
Singularity: a number of SingularityCoords together that define a arbitrary codimension surface/line/point.

Functions

- `template<size_t TensorDim>`
`std::string toString (const std::array< largecounter, TensorDim > &theTensor)`
Base case for single index tensor of unsigned integers (`ScreenIndex`).
- `template<size_t TensorDim>`
`std::string toString (const std::array< real, TensorDim > &theTensor)`
Base case for single index tensor of reals (`Point`, `OneIndex`, `ScreenPoint`).
- `template<typename Tensor, size_t TensorDim>`
`std::string toString (const std::array< Tensor, TensorDim > &theTensor)`
General case for tensors with more than one index (`TwoIndex`, `ThreeIndex`, `FourIndex`).
- `template<typename t, size_t TensorDim>`
`std::array< t, TensorDim > operator+ (const std::array< t, TensorDim > &a1, const std::array< t, TensorDim > &a2)`
Function to recursively call + on the lower rank tensor (OR the underlying reals/ints, if the tensor is rank 1)
- `template<typename t, size_t TensorDim>`
`std::array< t, TensorDim > operator- (const std::array< t, TensorDim > &a1, const std::array< t, TensorDim > &a2)`
Function to recursively call - on the lower rank tensor (OR the underlying reals/ints, if the tensor is rank 1)
- `template<typename t, size_t TensorDim>`
`std::array< t, TensorDim > operator* (const std::array< t, TensorDim > &t1, real lambda)`
Function to recursively scalar multiply the lower-rank tensors (OR the underlying reals/ints for the rank-1 tensor)
- `template<typename t, size_t TensorDim>`
`std::array< t, TensorDim > operator* (real lambda, const std::array< t, TensorDim > &t1)`
Function to recursively scalar multiply the lower-rank tensors (OR the underlying reals/ints for the rank-1 tensor)
- `template<typename t, size_t TensorDim>`
`std::array< t, TensorDim > operator/ (const std::array< t, TensorDim > &t1, real lambda)`
Function to recursively scalar divide the lower-rank tensors (OR the underlying reals/ints for the rank-1 tensor)

Variables

- `constexpr real pi {3.1415926535}`
The value of pi.
- `constexpr int dimension {4}`
The spacetime dimension.

7.16.1 Detailed Description

Definitions and some operations with geometric objects.

Author

Daniel R. Mayerson

Defines [Point](#), tensors with 1-4 indices, and the operator `toString` for them. Also defines basic tensor arithmetic (+, -, *, /). No .cpp with implementations; all functions are inline.

Version

0.1

Date

2025-01-14

Copyright

Copyright (c) 2025

7.16.2 Macro Definition Documentation

7.16.2.1 LARGE_COUNTER_MAX

```
#define LARGE_COUNTER_MAX std::numeric_limits<largecounter>::max()
```

Macro definition of maximum value that can be held in this large counter.

7.16.2.2 PIXEL_MAX

```
#define PIXEL_MAX std::numeric_limits<pixelcoord>::max()
```

Macro definition of maximum value that can be held in this large counter.

7.16.3 Typedef Documentation

7.16.3.1 FourIndex

```
using FourIndex = std::array<ThreeIndex, dimension>
```

Object with four indices is an array of [ThreeIndex](#) objects.

7.16.3.2 largecounter

```
using largecounter = unsigned long
```

This type is used to count geodesics integrated.

7.16.3.3 OneIndex

```
using OneIndex = Point
```

Object with one index has the same structure as a [Point](#).

7.16.3.4 pixelcoord

```
using pixelcoord = largecounter
```

A pixel coordinate: always ≥ 0 and integer; we use the largecounter type.

7.16.3.5 Point

```
using Point = std::array<real, dimension>
```

A point in spacetime. Note that coordinates are always assumed to be (t, r, theta, phi)

7.16.3.6 real

```
using real = double
```

A real number (could be changed to use arbitrary precision in the future).

7.16.3.7 ScreenIndex

```
using ScreenIndex = std::array<pixelcoord, dimension - 2>
```

An index on the [ViewScreen](#) (row, column)

7.16.3.8 ScreenPoint

```
using ScreenPoint = std::array<real, dimension - 2>
```

A point on the [ViewScreen](#); this does not have a time or radial extent.

7.16.3.9 Singularity

```
using Singularity = std::vector<SingularityCoord>
```

[Singularity](#): a number of SingularityCoords together that define an arbitrary codimension surface/line/point.

7.16.3.10 SingularityCoord

```
using SingularityCoord = std::pair<int, real>
```

SingularityCoord: pair of (coordinate number, coordinate value)

7.16.3.11 ThreeIndex

```
using ThreeIndex = std::array<TwoIndex, dimension>
```

Object with three indices is an array of **TwoIndex** objects.

7.16.3.12 TwoIndex

```
using TwoIndex = std::array<OneIndex, dimension>
```

Object with two indices is an array of **OneIndex** objects.

7.16.4 Function Documentation

7.16.4.1 operator*() [1/2]

```
template<typename t, size_t TensorDim>
std::array< t, TensorDim > operator* (
    const std::array< t, TensorDim > & t1,
    real lambda)
```

Function to recursively scalar multiply the lower-rank tensors (OR the underlying reals/ints for the rank-1 tensor)

Template Parameters

<i>t</i>	Type of the tensor
<i>TensorDim</i>	Dimension of the tensor

Parameters

<i>t1</i>	Tensor
<i>lambda</i>	Scalar

Returns

`std::array<t, TensorDim>` Scalar product of t1 and lambda

7.16.4.2 operator*() [2/2]

```
template<typename t, size_t TensorDim>
std::array< t, TensorDim > operator* (
    real lambda,
    const std::array< t, TensorDim > & t1)
```

Function to recursively scalar multiply the lower-rank tensors (OR the underlying reals/ints for the rank-1 tensor)

This function calls the right multiplication operator.

Template Parameters

<i>t</i>	Type of the tensor
<i>TensorDim</i>	Dimension of the tensor

Parameters

<i>lambda</i>	Scalar
<i>t1</i>	Tensor

Returns

std::array<t, TensorDim> Scalar product of lambda and t1

7.16.4.3 operator+()

```
template<typename t, size_t TensorDim>
std::array< t, TensorDim > operator+ (
    const std::array< t, TensorDim > & a1,
    const std::array< t, TensorDim > & a2)
```

Function to recursively call + on the lower rank tensor (OR the underlying reals/ints, if the tensor is rank 1)

Template Parameters

<i>t</i>	data type of the tensor
<i>TensorDim</i>	Dimension of the tensor

Parameters

<i>a1</i>	Tensor 1
<i>a2</i>	Tensor 2

Returns

std::array<t, TensorDim> Tensor sum of a1 and a2

7.16.4.4 operator-()

```
template<typename t, size_t TensorDim>
std::array< t, TensorDim > operator- (
    const std::array< t, TensorDim > & a1,
    const std::array< t, TensorDim > & a2)
```

Function to recursively call - on the lower rank tensor (OR the underlying reals/ints, if the tensor is rank 1)

Template Parameters

<i>t</i>	data type of the tensor
<i>TensorDim</i>	Dimension of the tensor

Parameters

<i>a1</i>	Tensor 1
<i>a2</i>	Tensor 2

Returns

std::array<t, TensorDim> Tensor difference of a1 and a2

7.16.4.5 operator/()

```
template<typename t, size_t TensorDim>
std::array< t, TensorDim > operator/ (
    const std::array< t, TensorDim > & t1,
    real lambda)
```

Function to recursively scalar divide the lower-rank tensors (OR the underlying reals/ints for the rank-1 tensor)

This function calls the right multiplication operator.

Template Parameters

<i>t</i>	Type of the tensor
<i>TensorDim</i>	Dimension of the tensor

Parameters

<i>t1</i>	Tensor
<i>lambda</i>	Scalar

Returns

std::array<t, TensorDim> Scalar division of t1 by lambda

7.16.4.6 toString() [1/3]

```
template<size_t TensorDim>
std::string toString (
    const std::array< largecounter, TensorDim > & theTensor)
```

Base case for single index tensor of unsigned integers ([ScreenIndex](#)).

We do not want toString(ScreenIndex) to convert its entries to reals and use the implementation for a single index tensor of reals, because we do not want decimal points in our string for the ints!

Template Parameters

<i>TensorDim</i>	Dimension of the tensor
------------------	-------------------------

Parameters

<i>theTensor</i>	Tensor object
------------------	---------------

Returns

std::string

7.16.4.7 toString() [2/3]

```
template<size_t TensorDim>
std::string toString (
    const std::array< real, TensorDim > & theTensor)
```

Base case for single index tensor of reals ([Point](#), [OneIndex](#), [ScreenPoint](#)).

Template Parameters

<i>TensorDim</i>	Dimension of the tensor
------------------	-------------------------

Parameters

<i>theTensor</i>	Tensor object
------------------	---------------

Returns

std::string

7.16.4.8 toString() [3/3]

```
template<typename Tensor, size_t TensorDim>
std::string toString (
    const std::array< Tensor, TensorDim > & theTensor)
```

General case for tensors with more than one index ([TwoIndex](#), [ThreeIndex](#), [FourIndex](#)).

This function recursively calls the lower rank tensor to print itself.

Template Parameters

<i>Tensor</i>	The type of the tensor
<i>TensorDim</i>	Dimension of the tensor

Parameters

<i>theTensor</i>	Tensor object
------------------	---------------

Returns

std::string

7.16.5 Variable Documentation

7.16.5.1 dimension

```
int dimension {4} [inline], [constexpr]
```

The spacetime dimension.

7.16.5.2 pi

```
real pi {3.1415926535} [inline], [constexpr]
```

The value of pi.

7.17 Geometry.h

[Go to the documentation of this file.](#)

```
00001 #ifndef _FOORT_GEOMETRY_H
00002 #define _FOORT_GEOMETRY_H
00003
00004
00005 #include <limits> // for std::numeric_limits
00006 #include <string> // needed for toString(...) to convert tensors to strings
00007 #include <array> // needed to define tensors as fixed-size arrays of real or pixelcoord
00008 #include <utility> // needed for std::pair
00009 #include <vector> // for std::vector
00010
00011 using real = double;
00012
00013
00014 // Note: An unsigned long is guaranteed to be able to hold at least 4 294 967 295 (4.10^10).
00015 // An unsigned int is only guaranteed to be able to hold 65 535, although
00016 // many modern-day implementations will actually make the int 32-bit and so much larger
00017
00018 using largecounter = unsigned long;
00019 using pixelcoord = largecounter;
00020
00021 #ifndef LARGE_COUNTER_MAX
00022 #define LARGE_COUNTER_MAX std::numeric_limits<largecounter>::max()
00023 #endif
00024 #ifndef PIXEL_MAX
00025 #define PIXEL_MAX std::numeric_limits<pixelcoord>::max()
00026 #endif
00027
00028 // CONSTANTS
00029
00030 inline constexpr real pi{3.1415926535};
00031
00032 inline constexpr int dimension{4};
00033
00034 // TENSOR DEFINITIONS
00035
00036 using Point = std::array<real, dimension>;
00037
00038 using ScreenPoint = std::array<real, dimension - 2>;
```

```

00057
00059 using ScreenIndex = std::array<pixelcoord, dimension - 2>;
00060
00062 using OneIndex = Point;
00064 using TwoIndex = std::array<OneIndex, dimension>;
00066 using ThreeIndex = std::array<TwoIndex, dimension>;
00068 using FourIndex = std::array<ThreeIndex, dimension>;
00069
00070 // Definition used to define singularity of arbitrary codimension
00072 using SingularityCoord = std::pair<int, real>;
00074 using Singularity = std::vector<SingularityCoord>;
00075
00076 // PRINTING TENSORS TO STRING
00077
00087 template <size_t TensorDim>
00088 std::string toString(const std::array<largecounter, TensorDim> &theTensor)
00089 {
00090     std::string theStr{"("}; // no spaces for the innermost brackets
00091     for (int i = 0; i < TensorDim - 1; ++i)
00092     {
00093         // Here we use the std::to_string to convert the real to string
00094         theStr += std::to_string(theTensor[i]);
00095         theStr += ", ";
00096     }
00097     theStr += std::to_string(theTensor[TensorDim - 1]);
00098     theStr += ")"; // no spaces for the innermost brackets
00099
00100     return theStr;
00101 }
00102
00110 template <size_t TensorDim>
00111 std::string toString(const std::array<real, TensorDim> &theTensor)
00112 {
00113     std::string theStr{"("}; // no spaces for the innermost brackets
00114     for (int i = 0; i < TensorDim - 1; ++i)
00115     {
00116         // Here we use the std::to_string to convert the real to string
00117         theStr += std::to_string(theTensor[i]);
00118         theStr += ", ";
00119     }
00120     theStr += std::to_string(theTensor[TensorDim - 1]);
00121     theStr += ")"; // no spaces for the innermost brackets
00122
00123     return theStr;
00124 }
00125
00135 template <typename Tensor, size_t TensorDim>
00136 std::string toString(const std::array<Tensor, TensorDim> &theTensor)
00137 {
00138     std::string theStr{" ("}; // All but the innermost brackets have an extra space padding the
    bracket
00139     for (int i = 0; i < TensorDim - 1; ++i)
00140     {
00141         theStr += toString(theTensor[i]);
00142         theStr += ", ";
00143     }
00144     theStr += toString(theTensor[TensorDim - 1]); // the last element doesn't have a comma after it
00145     theStr += ")"; // All but the innermost brackets have an extra space padding the bracket
00146
00147     return theStr;
00148 }
00149
00150 }
00151
00152 // TENSOR ARITHMETIC: addition/subtraction of tensors, scalar multiplication/division
00153
00163 template <typename t, size_t TensorDim>
00164 std::array<t, TensorDim> operator+(const std::array<t, TensorDim> &a1, const std::array<t, TensorDim>
    &a2)
00165 {
00166     std::array<t, TensorDim> temp{a1};
00167     for (int i = 0; i < TensorDim; ++i)
00168         temp[i] = temp[i] + a2[i];
00169     return temp;
00170 }
00171
00172 template <typename t, size_t TensorDim>
00183 std::array<t, TensorDim> operator-(const std::array<t, TensorDim> &a1, const std::array<t, TensorDim>
    &a2)
00184 {
00185     std::array<t, TensorDim> temp{a1};
00186     for (int i = 0; i < TensorDim; ++i)
00187         temp[i] = temp[i] - a2[i];
00188     return temp;
00189 }

```

```

00190 }
00191
00201 template <typename t, size_t TensorDim>
00202 std::array<t, TensorDim> operator*(const std::array<t, TensorDim> &t1, real lambda)
00203 {
00204     std::array<t, TensorDim> temp{t1};
00205     for (int i = 0; i < TensorDim; ++i)
00206         temp[i] = static_cast<t>(temp[i] * lambda); // static_cast necessary if t is integral type!
00207     return temp;
00208 }
00209
00219 template <typename t, size_t TensorDim>
00220 std::array<t, TensorDim> operator*(real lambda, const std::array<t, TensorDim> &t1)
00221 {
00222     return t1 * lambda;
00223 }
00224
00234 template <typename t, size_t TensorDim>
00235 std::array<t, TensorDim> operator/(const std::array<t, TensorDim> &t1, real lambda)
00236 {
00237     return t1 * (1 / lambda);
00238 }
00239
00240 #endif

```

7.18 FOORT/src/Grid.cpp File Reference

```

#include <fstream>
#include <iostream>
#include <sstream>
#include <stdexcept>
#include "Grid.h"

```

7.19 FOORT/src/Grid.h File Reference

```

#include <fstream>
#include <iostream>

```

Classes

- class [Grid](#)

A class for a 2D grid that contains the values of the necessary functions.

7.20 Grid.h

[Go to the documentation of this file.](#)

```

00001 #include <fstream>
00002 #include <iostream>
00003
00004 #ifndef GRID_H
00005 #define GRID_H
00006
00008 class Grid
00009 {
00010 public:
00012     int N_row;
00014     int N_col;
00016     double *data;

```



```

00018     int size;
00019     Grid(int N_row, int N_col);
00020
00022     ~Grid() { delete[] this->data; }
00023
00025     double &operator()(int i, int j) { return this->data[i * N_col + j]; }
00027     const double &operator()(int i, int j) const { return this->data[i * N_col + j]; }
00028
00029     void initialize_from_file(std::string file);
00030 };
00031
00032 #endif // GRID_H

```

7.21 FOORT/src/InOutOutput.cpp File Reference

```

#include "InOutOutput.h"
#include <algorithm>
#include <filesystem>

```

Functions

- void [SetOutputLevel](#) ([OutputLevel](#) theLvl)
Set the Output Level.
- void [SetLoopMessageFrequency](#) ([largecounter](#) thefreq)
Set the Loop Message Frequency.
- [largecounter](#) [GetLoopMessageFrequency](#) ()
Get the Loop Message Frequency during integration loops.
- void [ScreenOutput](#) (std::string_view theOutput, [OutputLevel](#) lvl, bool newLine)
Outputs line to screen console, contingent on it being allowed by the set outputlevel.

7.21.1 Detailed Description

Author

Daniel R. Mayerson

Version

0.1

Date

2025-01-14

Copyright

Copyright (c) 2025

7.21.2 Function Documentation

7.21.2.1 GetLoopMessageFrequency()

```
largecounter GetLoopMessageFrequency ()
```

Get the Loop Message Frequency during integration loops.

Returns

largecounter

7.21.2.2 ScreenOutput()

```
void ScreenOutput (
    std::string_view theOutput,
    OutputLevel lvl,
    bool newLine)
```

Outputs line to screen console, contingent on it being allowed by the set outputlevel.

Defaults are lvl = `OutputLevel::Level_3_ALLDETAIL` and newLine = true

Parameters

<i>theOutput</i>	output to be put on the screen
<i>lvl</i>	the output level
<i>newLine</i>	indicates whether we want a new line after the output

7.21.2.3 SetLoopMessageFrequency()

```
void SetLoopMessageFrequency (
    largecounter thefreq)
```

Set the Loop Message Frequency.

Parameters

<i>thefreq</i>	the desired frequency
----------------	-----------------------

7.21.2.4 SetOutputLevel()

```
void SetOutputLevel (
    OutputLevel theLvl)
```

Set the Output Level.

Parameters

<i>theLvl</i>	The desired level
---------------	-------------------

7.22 FOORT/src/InOutOutput.h File Reference

Declarations of functions & classes that deal with the output to files and to the screen.

```
#include "Geometry.h"
#include <string_view>
#include <iostream>
#include <fstream>
#include <string>
#include <vector>
```

Classes

- class [GeodesicOutputHandler](#)
// [GeodesicOutputHandler](#) handles all of the output to file. It gets passed all of the output strings for every [Geodesic](#), it then stores this data until it eventually writes all data to the appropriate files

Enumerations

- enum class [OutputLevel](#) {
[Level_0_WARNING](#) = 0 , [Level_1_PROC](#) = 1 , [Level_2_SUBPROC](#) = 2 , [Level_3_ALLDDETAIL](#) = 3 ,
[Level_4_DEBUG](#) = 4 , [MaxLevel](#) }

The output level determines at what priority level the output is generated to the console.

Functions

- void [SetOutputLevel](#) ([OutputLevel](#) theLvl)
Set the Output Level.
- void [ScreenOutput](#) (std::string_view theOutput, [OutputLevel](#) lvl=[OutputLevel::Level_3_ALLDDETAIL](#), bool newLine=true)
Outputs line to screen console, contingent on it being allowed by the set outputlevel.
- void [SetLoopMessageFrequency](#) ([largecounter](#) thefreq)
Set the Loop Message Frequency.
- [largecounter](#) [GetLoopMessageFrequency](#) ()
Get the Loop Message Frequency during integration loops.

7.22.1 Detailed Description

Declarations of functions & classes that deal with the output to files and to the screen.

Author

Daniel R. Mayerson

Version

0.1

Date

2025-01-14

Copyright

Copyright (c) 2025

7.22.2 Enumeration Type Documentation

7.22.2.1 OutputLevel

```
enum class OutputLevel [strong]
```

The output level determines at what priority level the output is generated to the console.

Level_0_WARNING: Only warnings are outputted. Level_1_PROC: Coarsest level output; only the major procedures produce output. Level_2_SUBPROC: Subprocedures can also produce output. Level_3_ALLDetail: Finest level output; all details are shown. Level_4_DEBUG: Finest level output AND debug messages as well.

Enumerator

Level_0_WARNING	
Level_1_PROC	
Level_2_SUBPROC	
Level_3_ALLDetail	
Level_4_DEBUG	
MaxLevel	

7.22.3 Function Documentation

7.22.3.1 GetLoopMessageFrequency()

```
largecounter GetLoopMessageFrequency ()
```

Get the Loop Message Frequency during integration loops.

Returns

largecounter

7.22.3.2 ScreenOutput()

```
void ScreenOutput (
    std::string_view theOutput,
    OutputLevel lvl,
    bool newLine)
```

Outputs line to screen console, contingent on it being allowed by the set outputlevel.

Defaults are lvl = `OutputLevel::Level_3_ALLDETAIL` and newLine = true

Parameters

<i>theOutput</i>	output to be put on the screen
<i>lvl</i>	the output level
<i>newLine</i>	indicates whether we want a new line after the output

7.22.3.3 SetLoopMessageFrequency()

```
void SetLoopMessageFrequency (
    largecounter thefreq)
```

Set the Loop Message Frequency.

Parameters

<i>thefreq</i>	the desired frequency
----------------	-----------------------

7.22.3.4 SetOutputLevel()

```
void SetOutputLevel (
    OutputLevel theLvl)
```

Set the Output Level.

Parameters

<i>theLvl</i>	The desired level
---------------	-------------------

7.23 InputOutput.h

[Go to the documentation of this file.](#)

```

00001 #ifndef _FOORT_INPUTOUTPUT_H
00002 #define _FOORT_INPUTOUTPUT_H
00003
00012
00013 #include "Geometry.h" // for basic tensor objects
00014
00015 #include <string_view> // std::string_view
00016 #include <iostream> // needed for file and console output
00017 #include <fstream> // needed for file output
00018 #include <string> // std::string used in various places
00019 #include <vector> // needed to create vectors of strings
00020
00021 // SCREENOUTPUT
00022
00031 enum class OutputLevel
00032 {
00033     Level_0_WARNING = 0,
00034     Level_1_PROC = 1,
00035     Level_2_SUBPROC = 2,
00036     Level_3_ALLDDETAIL = 3,
00037     Level_4_DEBUG = 4,
00038
00039     MaxLevel // (unused)
00040 };
00041
00042 // Set the output level used; note: the output level itself is a static variable in InputOutput.cpp
00043 void SetOutputLevel(OutputLevel theLvl);
00044
00045 // Outputs line to screen console, contingent on it being allowed by the set outputlevel
00046 void ScreenOutput(std::string_view theOutput, OutputLevel lvl = OutputLevel::Level_3_ALLDDETAIL, bool
    newline = true);
00047
00048 // Set and Get for the loop message frequency (messages indicating progress during each integration
    loop)
00049 void SetLoopMessageFrequency(largecounter thefreq);
00050 largecounter GetLoopMessageFrequency();
00051
00052 // FILE OUTPUT
00053
00059 class GeodesicOutputHandler
00060 {
00061 public:
00062     // No default constructor possible
00063     GeodesicOutputHandler() = delete;
00064     // Constructor must pass the following strings that are used to construct the file names of the
    output file:
00065     // FilePrefix, TimeStamp, FileExtension, and a vector of strings DiagNames (the names of each of
    the Diagnostics that will
00066     // be outputting). It must also specify how many outputs to cache before outputting to a file, and
00067     // how many geodesics are allowed per file created
00068     GeodesicOutputHandler(std::string FilePrefix, std::string TimeStamp, std::string FileExtension,
        std::vector<std::string> DiagNames,
00069         largecounter nroutputstocache = LARGE_COUNTER_MAX - 1, // note -1,
00070         // since we will
00071         actually cache one more then this number before outputting everything
00072         largecounter geodperfile = LARGE_COUNTER_MAX,
00073         std::string firstlineinfo = "");
00074
00075     // This tells the OutputHandler to prepare for this many geodesic outputs to arrive;
00076     // the internal state needs to be prepared such that they can come in without providing a data
    race
00077     void PrepareForOutput(largecounter nrOutputToCome);
00078
00079     // A new vector of output strings from a (single) Geodesic;
00080     // the length of the vector should be m_DiagNames.size()+1, since the first entry
00081     // is the screen index
00082     // NOTE: this procedure needs to be thread-safe!
00083     void NewGeodesicOutput(largecounter index, std::vector<std::string> theOutput);
00084
00085     // Calling this indicates that there is no further output to be expected;
00086     // this means we will write all remaining cached output to file
00087     void OutputFinished();
00088
00089     // Returns full description string of output handler
00090     std::string getFullDescriptionStr() const;
00091
00092 private:
00093     // Helper function: write everything that is cached to file now (clear the cache)
00094     void WriteCachedOutputToFile();
00095
00096     // Helper function: open the file with name filename for the first time, preparing it to write
00097     // This will effectively clear this file of any pre-existing content.

```

```

00098     void OpenForFirstTime(const std::string &filename);
00099
00100     // Helper function: return the full file name for the n-th output file
00101     // for the Diagnostic diagnr (this is an entry in m_DiagNames)
00102     std::string GetFileName(int diagnr, unsigned short filenr) const;
00103
00104     // The const strings that are used to construct the output file names
00105     const std::string m_FilePrefix;
00106     const std::string m_TimeStamp;
00107     const std::string m_FileExtension;
00108     const std::vector<std::string> m_DiagNames;
00109
00110     const bool m_PrintFirstLineInfo;
00111     const std::string m_FirstLineInfoString;
00112
00113     // Const setting the maximum number of outputs that can be cached before writing output to file.
00114     const largecounter m_nrOutputsToCache{};
00115     const largecounter m_nrGeodesicsPerFile{};
00116
00117     bool m_WriteToConsole{false};
00118
00119     largecounter m_PrevCached{0};
00120
00121     largecounter m_CurrentGeodesicsInFile{0};
00122
00123     unsigned short m_CurrentFullFiles{0};
00124
00125     std::vector<std::vector<std::string> m_AllCachedData{};
00126 };
00127
00128 #endif

```

7.24 FOORT/src/Integrators.cpp File Reference

```

#include "Integrators.h"
#include "Geodesic.h"
#include <algorithm>
#include <cmath>
#include <sstream>
#include <iostream>

```

7.24.1 Detailed Description

Author

Daniel R. Mayerson

Version

0.1

Date

2025-01-14

Copyright

Copyright (c) 2025

7.25 FOORT/src/Integrators.h File Reference

Declarations of integrator functions that integrate geodesic equation.

```
#include "Geometry.h"
#include "Metric.h"
#include <string>
```

Namespaces

- namespace [Integrators](#)

Typedefs

- using [GeodesicIntegratorFunc](#) = void (*)([Point](#), [OneIndex](#), [Point](#) &, [OneIndex](#) &, [real](#) &, const [Metric](#) *, const [Source](#) *)

Functions

- std::string [Integrators::GetFullIntegratorDescription](#) ()
Description string getter for the Integrator.
- [real](#) [Integrators::GetAdaptiveStep](#) ([Point](#) curpos, [OneIndex](#) curvel)
Determine the (affine parameter) step size to take.
- void [Integrators::IntegrateGeodesicStep_RK4](#) ([Point](#) curpos, [OneIndex](#) curvel, [Point](#) &nextpos, [OneIndex](#) &nextvel, [real](#) &stepsize, const [Metric](#) *theMetric, const [Source](#) *theSource)
Integrate the geodesic equation by one step using Runge-Kutta-4.
- void [Integrators::IntegrateGeodesicStep_Verlet](#) ([Point](#) curpos, [OneIndex](#) curvel, [Point](#) &nextpos, [OneIndex](#) &nextvel, [real](#) &stepsize, const [Metric](#) *theMetric, const [Source](#) *theSource)
Integrate the geodesic equation by one step using the Verlet algorithm.

Variables

- constexpr [real](#) [Integrators::delta_nodiv0](#) = 1e-20
This is used to avoid dividing by zero.
- [real](#) [Integrators::Derivative_hval](#) {1e-7}
The spacing used in stencils for the calculation of numerical derivatives (e.g. in Christoffel symbols)
- std::string [Integrators::IntegratorDescription](#) {"RK4"}
The name of the integrator selected.
- [real](#) [Integrators::epsilon](#) {0.03}
This is the base step size to be taken (the integrator will adapt this if necessary)
- [real](#) [Integrators::SmallestPossibleStepsize](#) {1e-12}
The affine parameter must always go forward by at least this amount.
- [real](#) [Integrators::VerletVelocityTolerance](#) {0.001}
Tolerance on the change in velocity in the iteration loop of the Verlet integration algorithm.

7.25.1 Detailed Description

Declarations of integrator functions that integrate geodesic equation.

Author

Daniel R. Mayerson

Version

0.1

Date

2025-01-14

Copyright

Copyright (c) 2025

7.25.2 Typedef Documentation

7.25.2.1 GeodesicIntegratorFunc

```
using GeodesicIntegratorFunc = void (*)(Point, OneIndex, Point &, OneIndex &, real &, const
Metric *, const Source *)
```

7.26 Integrators.h

[Go to the documentation of this file.](#)

```
00001 #ifndef _FOORT_INTEGRATORS_H
00002 #define _FOORT_INTEGRATORS_H
00003
00012
00013 #include "Geometry.h" // for basic tensor objects
00014 #include "Metric.h"   // for the metric
00015
00016 #include <string> // std::string
00017
00019 class Source;
00020
00021 // This is the structure of a function that integrates the geodesic equation one step
00022 // It takes as arguments:
00023 // - current position, current velocity
00024 // - references to: next position, next velocity, affine parameter step size (which the functions
    sets)
00025 // - pointers to: the Metric, the Source that are used to evaluate the geodesic equation
00026 using GeodesicIntegratorFunc = void (*)(Point, OneIndex, Point &, OneIndex &, real &, const Metric *,
    const Source *);
00027
00028 // Namespace for integrator constants and functions
00029 namespace Integrators
00030 {
00032     constexpr real delta_nodiv0 = 1e-20;
00033
00035     inline real Derivative_hval{1e-7};
00036
00038     inline std::string IntegratorDescription{"RK4"};
00039 }
```

```

00040 // Full descriptive string of integrator and all integrator options
00041 std::string GetFullIntegratorDescription();
00042
00044 inline real epsilon{0.03};
00045
00047 inline real SmallestPossibleStepsize{1e-12};
00048
00049 // Function to get (adaptive) step size
00050 real GetAdaptiveStep(Point curpos, OneIndex curvel);
00051
00052 // This is a GeodesicIntegratorFunc
00053 // Using the Runge-Kutta-4 algorithm to integrate the geodesic equation
00054 void IntegrateGeodesicStep_RK4(Point curpos, OneIndex curvel,
00055                                Point &nextpos, OneIndex &nextvel, real &stepsize, const Metric
00056                                *theMetric, const Source *theSource);
00057
00058 inline real VerletVelocityTolerance{0.001};
00059
00060 // This is a GeodesicIntegratorFunc
00061 // Using the velocity Verlet algorithm to integrate the geodesic equation
00062 void IntegrateGeodesicStep_Verlet(Point curpos, OneIndex curvel,
00063                                    Point &nextpos, OneIndex &nextvel, real &stepsize, const Metric
00064                                    *theMetric, const Source *theSource);
00065 }
00066 #endif

```

7.27 FOORT/src/Interpolator.cpp File Reference

```

#include <fstream>
#include <iomanip>
#include <iostream>
#include <sstream>
#include <string>
#include "Interpolator.h"
#include "Grid.h"

```

7.28 FOORT/src/Interpolator.h File Reference

```

#include <fstream>
#include <iostream>
#include <sstream>
#include <string>
#include <vector>
#include <array>
#include <stdexcept>
#include <algorithm>
#include <cmath>
#include "Grid.h"

```

Classes

- class [BicubicSplineInterpolator](#)

7.29 Interpolator.h

[Go to the documentation of this file.](#)

```

00001 #include <fstream>
00002 #include <iostream>
00003 #include <sstream>
00004 #include <string>
00005 #include <vector>
00006 #include <array>
00007 #include <stdexcept>
00008 #include <algorithm>
00009 #include <cmath>
00010 #include "Grid.h"
00011
00012 #ifndef INTERPOLATOR_H
00013 #define INTERPOLATOR_H
00014
00015 class BicubicSplineInterpolator
00016 {
00017 public:
00018     BicubicSplineInterpolator(const std::string x_file = "RotatingBosonStar/x.txt",
00019                             const std::string th_file = "RotatingBosonStar/theta.txt")
00020     {
00021         // Load x and theta coordinates from files
00022         std::ifstream x_input(x_file);
00023         std::ifstream th_input(th_file);
00024
00025         if (!th_input || !x_input)
00026         {
00027             std::cerr << "Error opening file!" << std::endl;
00028             throw std::runtime_error("File not found");
00029         }
00030
00031         double val;
00032         while (x_input >> val)
00033         {
00034             m_x.push_back(val);
00035         }
00036         while (th_input >> val)
00037         {
00038             m_theta.push_back(val);
00039         }
00040
00041         // Explicitly close files
00042         x_input.close();
00043         th_input.close();
00044     }
00045
00046     // BicubicSplineInterpolator(const std::vector<double> &x_coords,
00047     //                          const std::vector<double> &theta_coords,
00048     //                          const Grid *grid)
00049     // : m_x(x_coords), m_theta(theta_coords)
00050     // {
00051     //     compute_spline_coefficients(grid);
00052     // }
00053
00054     void set_grid(const Grid *grid)
00055     {
00056         // Precompute spline coefficients for better performance
00057         // This is more complex but provides C2 continuity
00058         compute_spline_coefficients(grid);
00059         std::cout << "Interpolation domain set: x in [" << m_x.front() << ", " << m_x.back() << "], theta
in [" << m_theta.front() << ", " << m_theta.back() << "].\n";
00060     }
00061
00062     double interpolate(double p_x, double p_theta,
00063                       bool allow_extrapolation = false) const;
00064
00065 private:
00066     std::vector<double> m_x;
00067     std::vector<double> m_theta;
00068     std::vector<std::array<double, 16> > m_coeffs; // 16 coefficients per cell
00069
00070     void compute_spline_coefficients(const Grid *grid);
00071     double estimate_dx(const Grid *grid, size_t j, size_t i) const;
00072     double estimate_dy(const Grid *grid, size_t j, size_t i) const;
00073     double estimate_dxdy(const Grid *grid, size_t j, size_t i) const;
00074     void compute_cell_coefficients(std::array<double, 16> &coeffs,
00075                                   double f00, double f10, double f01, double f11,
00076                                   double fx00, double fx10, double fx01, double fx11,
00077                                   double fy00, double fy10, double fy01, double fy11,
00078                                   double fxy00, double fxy10, double fxy01, double fxy11) const;
00079 };
00080
00081 #endif // INTERPOLATOR_H

```

7.30 FOORT/src/Main.cpp File Reference

Main function for FOORT.

```
#include "Config.h"
#include "Diagnostics.h"
#include "Geodesic.h"
#include "Geometry.h"
#include "InputOutput.h"
#include "Integrators.h"
#include "Metric.h"
#include "Terminations.h"
#include "Utilities.h"
#include "ViewScreen.h"
#include <omp.h>
#include <iostream>
```

Functions

- int [main](#) (int argc, char *argv[])

7.30.1 Detailed Description

Main function for FOORT.

Author

Daniel R. Mayerson

Version

1.0

Date

2025-1-21

Copyright

Copyright (c) 2025

7.30.2 Function Documentation

7.30.2.1 main()

```
int main (
    int argc,
    char * argv[])
```

7.31 FOORT/src/Mesh.cpp File Reference

```
#include "Mesh.h"  
#include "Utilities.h"  
#include <algorithm>  
#include <limits>  
#include <iostream>
```

7.31.1 Detailed Description

Author

Daniel R. Mayerson

Version

0.1

Date

2025-01-14

Copyright

Copyright (c) 2025

7.32 FOORT/src/Mesh.h File Reference

Declarations of abstract base [Mesh](#) class and all its descendants.

```
#include "Geometry.h"  
#include "Diagnostics.h"  
#include "InputOutput.h"  
#include <cmath>  
#include <utility>  
#include <forward_list>  
#include <memory>  
#include <vector>  
#include <array>  
#include <string>
```

Classes

- class [Mesh](#)
Abstract base [Mesh](#) class.
- class [SimpleSquareMesh](#)
A simple square mesh that will integrate a square of evenly spaced pixels.
- class [InputCertainPixelsMesh](#)
[Mesh](#) which integrates only certain user-inputted pixels.
- class [SquareSubdivisionMesh](#)
Adaptive subdivision [Mesh](#).
- struct [SquareSubdivisionMesh::PixelInfo](#)
A struct the [Mesh](#) uses to keep all information about a given pixel.
- class [SquareSubdivisionMeshV2](#)
Adaptive subdivision [Mesh](#).
- struct [SquareSubdivisionMeshV2::PixelInfo](#)
A struct the [Mesh](#) uses to keep all information about a given pixel.

7.32.1 Detailed Description

Declarations of abstract base [Mesh](#) class and all its descendants.

Author

Daniel R. Mayerson

Version

0.1

Date

2025-01-14

Copyright

Copyright (c) 2025

7.33 Mesh.h

[Go to the documentation of this file.](#)

```
00001 #ifndef _FOORT_MESH_H
00002 #define _FOORT_MESH_H
00003
00012
00013 #include "Geometry.h" // needed for basic tensor objects
00014 #include "Diagnostics.h" // needed for Diagnostic "value" and "distance" functions
00015 #include "InputOutput.h" // needed for ScreenOutput
00016
00017 #include <cmath> // needed for sqrt (only on Linux)
00018 #include <utility> // std::move
00019 #include <forward_list> // std::forward_list
00020 #include <memory> // std::unique_ptr
00021 #include <vector> // std::vector
00022 #include <array> // std::array
```

```

00023 #include <string>          // for strings
00024
00028 class Mesh
00029 {
00030 public:
00031     // Basic constructor constructs Diagnostic that is used for determinines "values" and "distances"
    between values
00032     Mesh(DiagBitflag valdiag)
00033     { // Calling CreateDiagnosticVector in this way will create a vector with exactly one element in
        it,
00034         // i.e. the Diagnostic we need!
00035         : m_DistanceDiagnostic{std::move((CreateDiagnosticVector(valdiag, valdiag, nullptr))[0])}
00036     {
00037     }
00038
00039     // virtual destructor to ensure correct destruction of descendants
00040     virtual ~Mesh() = default;
00041
00042     // Getter for how many geodesics the Mesh currently wants to integrate in this iteration
00043     virtual largecounter getCurNrGeodesics() const = 0;
00044
00045     // This sets a new initial conditions (in the form of a ScreenPoint and ScreenIndex)
00046     // for a next pixel to be integrated in the current iteration
00047     virtual void getNewInitConds(largecounter index, ScreenPoint &newunitpoint, ScreenIndex
    &newscreenindex) const = 0;
00048
00049     // When a geodesic is finished integrating, it tells the Mesh and passes on its final "value"
00050     // NOTE: despite being a non-const member function, this must be designed to be thread-safe!
00051     virtual void GeodesicFinished(largecounter index, std::vector<real> finalValues) = 0;
00052
00053     // This is called when the current iteration is finished. The Mesh can now evaluate whether to
    continue or not
00054     virtual void EndCurrentLoop() = 0;
00055
00056     // Returns false if the Mesh wants another iteration of pixels to integrate
00057     virtual bool IsFinished() const = 0;
00058
00059     // Returns a string description of the Mesh (spaces allowed), describing its options
00060     virtual std::string getFullDescriptionStr() const;
00061
00062 protected:
00063     const std::unique_ptr<const Diagnostic> m_DistanceDiagnostic;
00064 };
00065
00066 class SimpleSquareMesh final : public Mesh
00067 {
00068 public:
00069     // Default constructor not possible
00070     SimpleSquareMesh() = delete;
00071     // Constructor initializes total number of pixels and passes valdiag to base constructor
00072     // Note that we static_cast the sqrt() to round off the row/column size to an integer number
00073     SimpleSquareMesh(largecounter totalPixels, DiagBitflag valdiag)
00074     : m_TotalPixels{static_cast<pixelcoord>(sqrt(totalPixels)) *
    static_cast<pixelcoord>(sqrt(totalPixels))},
00075       m_RowColumnSize{static_cast<pixelcoord>(sqrt(totalPixels))},
00076       Mesh(valdiag)
00077     {
00078         if constexpr (dimension != 4)
00079             ScreenOutput("SimpleSquareMesh only defined in 4D!", OutputLevel::Level_0_WARNING);
00080     }
00081
00082     // Declarations of overriding virtual functions
00083     largecounter getCurNrGeodesics() const final;
00084
00085     void getNewInitConds(largecounter index, ScreenPoint &newunitpoint, ScreenIndex &newscreenindex)
    const final;
00086
00087     void GeodesicFinished(largecounter index, std::vector<real> finalValues) final;
00088
00089     void EndCurrentLoop() final;
00090
00091     bool IsFinished() const final;
00092
00093     // Description string getter
00094     std::string getFullDescriptionStr() const final;
00095
00096 private:
00097     const largecounter m_TotalPixels;
00098     const pixelcoord m_RowColumnSize;
00099     bool m_Finished{false};
00100 };
00101
00102 class InputCertainPixelsMesh : public Mesh
00103 {
00104 public:
00105     // Default constructor not possible

```

```

00117     InputCertainPixelsMesh() = delete;
00118     // Copy constructor not possible
00119     InputCertainPixelsMesh(const InputCertainPixelsMesh &) = delete;
00120     // Constructor given in Mesh.cpp file; constructor asks for input of pixels
00121     InputCertainPixelsMesh(largecounter totalPixels, DiagBitflag valdiag);
00122
00123     // Declarations of overriding virtual functions
00124
00125     largecounter getCurNrGeodesics() const final;
00126
00127     void getNewInitConds(largecounter index, ScreenPoint &newunitpoint, ScreenIndex &newscreenindex)
const final;
00128
00129     void GeodesicFinished(largecounter index, std::vector<real> finalValues) final;
00130
00131     void EndCurrentLoop() final;
00132
00133     bool IsFinished() const final;
00134
00135     // Description string getter
00136     std::string getFullDescriptionStr() const final;
00137
00138 private:
00139     const pixelcoord m_RowColumnSize;
00140
00141     largecounter m_TotalPixels{0};
00142     std::vector<ScreenIndex> m_PixelsToIntegrate{};
00143     bool m_Finished{false};
00144 };
00145
00146 class SquareSubdivisionMesh : public Mesh
00147 {
00148 public:
00149     // default constructor not possible
00150     SquareSubdivisionMesh() = delete;
00151     // Constructor must be called with arguments:
00152     // - maxPixels: max. nr of pixels that can be integrated in TOTAL, over all iterations (if 0, then
this is infinite,
00153     // i.e. we keep integrating until all squares are maximally subdivided or have weight 0)
00154     // - initialPixels: initial number of pixels to integrate (spaced equally over the screen)
00155     // - maxSubdivide: maximum number of times that we can subdivide squares (note: 1 denotes the
00156     // initial grid, so 2 would be subdividing the squares once)
00157     // - iterationPixels: maximum number of pixels to subdivide in each integration iteration
00158     // (max number of pixels that will be integrates is then 5*iterationPixels)
00159     // - initialSubToFinal: once we decide to subdivide a square, do we automatically keep subdividing
it
00160     // until we reach maxSubdivision?
00161     // - valdiag: the "value" and "distance" Diagnostic to use
00162     SquareSubdivisionMesh(largecounter maxPixels, largecounter initialPixels, int maxSubdivide,
largecounter iterationPixels, bool initialSubToFinal,
00163     DiagBitflag valdiag)
00164     : m_InitialPixels(static_cast<pixelcoord>(sqrt(initialPixels)) *
static_cast<pixelcoord>(sqrt(initialPixels))),
00165     m_MaxSubdivide{maxSubdivide},
00166     m_RowColumnSize(static_cast<pixelcoord>((sqrt(initialPixels) - 1) * ExpInt(2, maxSubdivide -
1) + 1)),
00167     m_PixelsLeft{maxPixels}, m_MaxPixels{maxPixels}, m_InfinitePixels{maxPixels == 0},
m_IterationPixels{iterationPixels},
00168     m_InitialSubDivideToFinal{initialSubToFinal}, Mesh(valdiag)
00169     {
00170         if constexpr (dimension != 4)
00171             ScreenOutput("SquareSubdivisionMesh only defined in 4D!", OutputLevel::Level_0_WARNING);
00172
00173         // DEBUG message for constructor (can delete)
00174         ScreenOutput("SquareSubdivisionMesh constructed: maxPixels: " + (m_InfinitePixels ? "infinite"
: std::to_string(maxPixels)) + "; m_InitialPixels: " + std::to_string(m_InitialPixels) + ";
m_RowColumnSize: " + std::to_string(m_RowColumnSize), OutputLevel::Level_4_DEBUG);
00175
00176         // Initialize the initial square, equally spaced grid
00177         InitializeFirstGrid();
00178     }
00179
00180     // Declarations of overriding virtual functions
00181
00182     largecounter getCurNrGeodesics() const final;
00183
00184     void getNewInitConds(largecounter index, ScreenPoint &newunitpoint, ScreenIndex &newscreenindex)
const final;
00185
00186     void GeodesicFinished(largecounter index, std::vector<real> finalValues) final;
00187
00188     void EndCurrentLoop() final;
00189
00190     bool IsFinished() const final;
00191
00192     // Description string getter
00193     std::string getFullDescriptionStr() const final;

```



```

00205
00206 private:
00208     const largecounter m_InitialPixels;
00210     const int m_MaxSubdivide;
00212     const pixelcoord m_RowColumnSize;
00214     const largecounter m_IterationPixels;
00216     const largecounter m_MaxPixels;
00218     const bool m_InitialSubDividideToFinal;
00220     const bool m_InfinitePixels;
00221
00223     largecounter m_PixelsLeft;
00224
00228     struct PixelInfo
00229     {
00230         // Constructor with its ScreenIndex and current subdivision level
00231         PixelInfo(ScreenIndex ind, int subdiv) : Index{ind}, SubdivideLevel{subdiv} {}
00232
00234         ScreenIndex Index{};
00235
00237         int SubdivideLevel{};
00238
00240         real Weight{-1};
00241
00243         std::vector<real> DiagValue{};
00244
00246         largecounter LowerNbrIndex{0};
00248         largecounter RightNbrIndex{0};
00249     };
00251     std::vector<PixelInfo> m_CurrentPixelQueue{};
00253     std::vector<bool> m_CurrentPixelQueueDone{};
00255     std::vector<PixelInfo> m_AllPixels{};
00256
00257     // Initializes the first nxn screen in m_CurrentPixelQueue
00258     void InitializeFirstGrid();
00259
00260     // Updates the neighbors of all pixels (that should have neighbors and don't yet) in m_AllPixels
00261     void UpdateAllNeighbors();
00262
00263     // Updates all weights of the pixels in m_AllPixels with weight < 0 and subdiv > 0 and subdiv <
00264     m_MaxSubdivide
00265     // Assumes all squares have neighbors assigned correctly
00266     void UpdateAllWeights();
00267
00268     // This will take the pixel m_AllPixels[ind] and subdivide it,
00269     // adding up to <=5 pixels to the CurrentPixelQueue
00270     void SubdivideAndQueue(largecounter ind);
00271
00272     // Helper function to exponentiate an int to an int
00273     // Note: the result can be larger than fits in an int, but the base is always 2 and the exp is
00274     always
00275     // a number <=m_MaxSubdivide (which is an int)
00276     pixelcoord ExpInt(int base, int exp);
00277 };
00278
00286 class SquareSubdivisionMeshV2 : public Mesh
00287 {
00288 public:
00289     // default constructor not possible
00290     SquareSubdivisionMeshV2() = delete;
00291     // Constructor must be called with arguments:
00292     // - maxPixels: max. nr of pixels that can be integrated in TOTAL, over all iterations (if 0, then
00293     this is infinite,
00294     // i.e. we keep integrating until all squares are maximally subdivided or have weight 0)
00295     // - initialPixels: initial number of pixels to integrate (spaced equally over the screen)
00296     // - maxSubdivide: maximum number of times that we can subdivide squares (note: 1 denotes the
00297     // initial grid, so 2 would be subdividing the squares once)
00298     // - iterationPixels: maximum number of pixels to subdivide in each integration iteration
00299     // (max number of pixels that will be integrates is then 5*iterationPixels)
00300     // - initialSubToFinal: once we decide to subdivide a square, do we automatically keep subdividing
00301     it
00302     // until we reach maxSubdivision?
00303     // - valdiag: the "value" and "distance" Diagnostic to use
00304     SquareSubdivisionMeshV2(largecounter maxPixels, largecounter initialPixels, int maxSubdivide,
00305     largecounter iterationPixels, bool initialSubToFinal,
00306     DiagBitflag valdiag)
00307     : m_InitialPixels{static_cast<pixelcoord>(sqrt(initialPixels)) *
00308     static_cast<pixelcoord>(sqrt(initialPixels))},
00309     m_MaxSubdivide{maxSubdivide},
00310     m_RowColumnSize{static_cast<pixelcoord>((sqrt(initialPixels) - 1) * ExpInt(2, maxSubdivide -
00311     1) + 1)},
00312     m_PixelsLeft{maxPixels}, m_MaxPixels{maxPixels}, m_InfinitePixels{maxPixels == 0},
00313     m_IterationPixels{iterationPixels},
00314     m_InitialSubDividideToFinal{initialSubToFinal}, Mesh(valdiag)
00315     {
00316         if constexpr (dimension != 4)
00317             ScreenOutput("SquareSubdivisionMeshV2 only defined in 4D!", OutputLevel::Level_0_WARNING);
00318     }
00319

```

```

00313         // Initialize the initial square, equally spaced grid
00314         InitializeFirstGrid();
00315     }
00316
00317     // Declarations of overriding virtual functions
00318
00319     largecounter getCurNrGeodesics() const final;
00320
00321     void getNewInitConds(largecounter index, ScreenPoint &newunitpoint, ScreenIndex &newscreenindex)
const final;
00322
00323     void GeodesicFinished(largecounter index, std::vector<real> finalValues) final;
00324
00325     void EndCurrentLoop() final;
00326
00327     bool IsFinished() const final;
00328
00329     // Description string getter
00330     std::string getFullDescriptionStr() const final;
00331
00332 private:
00333     const largecounter m_InitialPixels;
00334     const int m_MaxSubdivide;
00335     const pixelcoord m_RowColumnSize;
00336     const largecounter m_IterationPixels;
00337     const largecounter m_MaxPixels;
00338     const bool m_InitialSubDivideToFinal;
00339     const bool m_InfinitePixels;
00340
00341     largecounter m_PixelsLeft;
00342
00343     largecounter m_PixelsIntegrated{0};
00344
00345     struct PixelInfo
00346     {
00347         // Constructor with its ScreenIndex and current subdivision level
00348         PixelInfo(ScreenIndex ind, int subdiv) : Index{ind}, SubdivideLevel{subdiv} {}
00349
00350         const ScreenIndex Index{};
00351
00352         int SubdivideLevel{};
00353
00354         real Weight{-1};
00355
00356         std::vector<real> DiagValue{};
00357
00358         PixelInfo *LeftNbr{nullptr};
00359         PixelInfo *RightNbr{nullptr};
00360         PixelInfo *UpNbr{nullptr};
00361         PixelInfo *DownNbr{nullptr};
00362         PixelInfo *SEdiagNbr{nullptr};
00363     };
00364
00365     std::forward_list<std::unique_ptr<PixelInfo>> m_AllPixels{};
00366
00367     std::vector<PixelInfo *> m_ActivePixels{};
00368     std::vector<PixelInfo *> m_CurrentPixelQueue{};
00369     std::vector<bool> m_CurrentPixelQueueDone{};
00370     std::vector<PixelInfo *> m_CurrentPixelUpdating{};
00371
00372     // Initializes the first nxn screen and puts them in m_CurrentPixelQueue
00373     void InitializeFirstGrid();
00374
00375     // Updates all weights of the pixels in m_CurrentPixelUpdating;
00376     // these should have subdiv > 0 and subdiv < m_MaxSubdivide, and all their neighbors assigned
correctly
00377     // All pixels with weight > 0 will be added to m_ActivePixels
00378     void UpdateAllWeights();
00379
00380     // Helper functions that return the appropriate neighbor of p, ONLY if this neighbor exists at the
subdivision level specified
00381     // Returns nullptr otherwise; also return nullptr if p==nullptr
00382     PixelInfo *GetUp(PixelInfo *p, int subdiv) const;
00383     PixelInfo *GetDown(PixelInfo *p, int subdiv) const;
00384     PixelInfo *GetRight(PixelInfo *p, int subdiv) const;
00385     PixelInfo *GetLeft(PixelInfo *p, int subdiv) const;
00386
00387     // This will take the pixel m_AllPixels[ind] and subdivide it,
00388     // adding up to <=5 pixels to the CurrentPixelQueue
00389     void SubdivideAndQueue(largecounter ind);
00390
00391     // Helper function to exponentiate an int to an int
00392     // Note: the result can be larger than fits in an int, but the base is always 2 and the exp is
always
00393     // a number <=m_MaxSubdivide (which is an int)
00394     pixelcoord ExpInt(int base, int exp) const;
00395 };

```

```
00422
00423 #endif
```

7.34 FOORT/src/Metric.cpp File Reference

```
#include "Metric.h"
#include "InputOutput.h"
#include "Integrators.h"
#include <cmath>
#include <algorithm>
#include <iomanip>
#include "Spline.h"
#include "Grid.h"
#include "Interpolator.h"
#include <sstream>
```

7.35 FOORT/src/Metric.h File Reference

Declarations of abstract base [Metric](#) class and all its descendants.

```
#include "Geometry.h"
#include "Spline.h"
#include "Grid.h"
#include "Interpolator.h"
#include <string>
#include <vector>
#include <memory>
```

Classes

- class [Metric](#)
The abstract base class for all Metrics.
- class [SphericalHorizonMetric](#)
Abstract base class for a metric with a spherical horizon (i.e. horizon at constant radius r)
- class [KerrMetric](#)
The Kerr metric.
- class [FlatSpaceMetric](#)
Flat space metric in 4D.
- class [RasheedLarsenMetric](#)
Rasheed-Larsen black hole.
- class [JohannsenMetric](#)
Johannsen black hole metric.
- class [MankoNovikovMetric](#)
Mano-Novikov metric (with angular momentum and $M3$ parameter turned on)
- class [KerrSchildMetric](#)
Kerr metric in Kerr-Schild coordinates.
- class [SingularityMetric](#)
Abstract base class for a metric with an arbitrary number of singularities (of arbitrary codimension)
- class [ST3CrMetric](#)
Ring fuzzball metric.
- class [BosonStarMetric](#)
Boson star metric with solitonic potential.
- class [RotatingBosonStarMetric](#)

7.35.1 Detailed Description

Declarations of abstract base [Metric](#) class and all its descendants.

Author

Daniel R. Mayerson

Version

0.1

Date

2025-01-14

Copyright

Copyright (c) 2025

7.36 Metric.h

[Go to the documentation of this file.](#)

```

00001 #ifndef _FOORT_METRIC_H
00002 #define _FOORT_METRIC_H
00003
00004 #include "Geometry.h" // Needed for basic tensor objects etc.
00005
00006 #include "Spline.h" // needed for spline interpolation, for the Boson star metric
00007 #include "Grid.h" // needed for the grid class, for the Rotating Boson star metric
00008 #include "Interpolator.h" // needed for the Interpolator class, for the Rotating Boson star metric
00009 #include <string> // for strings
00010 #include <vector> // needed for the (non-fixed size) vector of symmetries in the metric
00011 #include <memory> // for std::unique_ptr, for the Rotating Boson star metric's
    grids/interpolators
00012
00021
00026 class Metric
00027 {
00028 public:
00029     // Virtual destructor to ensure correct destruction of descendants
00030     virtual ~Metric() = default;
00031
00032     Metric(bool rlogscale = false);
00033
00034     // Basic functions that return the metric with indices down or up:
00035     // pure virtual as they must be defined in the descendant classes.
00036     //
00037     // Get the metric at Point p, indices down
00038     virtual TwoIndex getMetric_dd(const Point &p) const = 0;
00039     // Get the metric at Point p, indices up
00040     virtual TwoIndex getMetric_uu(const Point &p) const = 0;
00041
00042     // The following functions return the Christoffel and other derivative quantities of the metric.
00043     // They are implemented for this base class, BUT are left as virtual functions to allow for
00044     // other metrics to implement their own (more efficient)
00045     // way of calculating them, if so desired.
00046     //
00047     // Get the Christoffel symbol, indices up-down-down
00048     virtual ThreeIndex getChristoffel_udd(const Point &p) const;
00049     // Get the Riemann tensor, indices up-down-down-down
00050     virtual FourIndex getRiemann_uddd(const Point &p) const;
00051     // Get the Kretschmann scalar
00052     virtual real getKretschmann(const Point &p) const;
00053
00054     // Function to get the description of the metric
00055     // (used for outputting to the screen while running and possibly to the output files)
00056     // There is a base class implementation of this function returning an un-descriptive string

```

```

00057     virtual std::string getFullDescriptionStr() const;
00058
00059     bool getrLogScale() const;
00060
00061 protected:
00062     std::vector<int> m_Symmetries{};
00063     const bool m_rLogScale;
00064 };
00065
00066 class SphericalHorizonMetric : public Metric
00067 {
00068 public:
00069     // No default construction allowed, must specify horizon radius
00070     SphericalHorizonMetric() = delete;
00071     // Constructor that initializes horizon radius
00072     SphericalHorizonMetric(real HorizonRadius, bool rLogScale);
00073
00074     // Getter functions for the two member variables
00075     real getHorizonRadius() const;
00076
00077 protected:
00078     const real m_HorizonRadius;
00079 };
00080
00081 class KerrMetric final : public SphericalHorizonMetric
00082 {
00083 private:
00084     const real m_aParam;
00085     const real m_mParam;
00086
00087 public:
00088     // No default constructor allowed, must specify a
00089     KerrMetric() = delete;
00090
00091     // Constructor setting parameter a
00092     KerrMetric(real aParam, bool rLogScale = false, real mParam = 1.);
00093
00094     // The override of the basic metric getter functions
00095     TwoIndex getMetric_dd(const Point &p) const final;
00096     TwoIndex getMetric_uu(const Point &p) const final;
00097
00098     // The override of the description string getter
00099     std::string getFullDescriptionStr() const final;
00100 };
00101
00102 class FlatSpaceMetric final : public Metric
00103 {
00104 public:
00105     // Simple (default) constructor is all that is needed
00106     FlatSpaceMetric(bool rlogscale = false);
00107
00108     // The override of the basic metric getter functions
00109     TwoIndex getMetric_dd(const Point &p) const final;
00110     TwoIndex getMetric_uu(const Point &p) const final;
00111
00112     // The override of the description string getter
00113     std::string getFullDescriptionStr() const final;
00114 };
00115
00116 class RasheedLarsenMetric final : public SphericalHorizonMetric
00117 {
00118 private:
00119     const real m_aParam;
00120     const real m_mParam;
00121     const real m_pParam;
00122     const real m_qParam;
00123
00124 public:
00125     // No default constructor allowed, must specify parameters
00126     RasheedLarsenMetric() = delete;
00127
00128     // Constructor setting parameter a
00129     RasheedLarsenMetric(real mParam, real aParam, real pParam, real qParam, bool rLogScale = false);
00130
00131     // The override of the basic metric getter functions
00132     TwoIndex getMetric_dd(const Point &p) const final;
00133     TwoIndex getMetric_uu(const Point &p) const final;
00134
00135     // The override of the description string getter
00136     std::string getFullDescriptionStr() const final;
00137 };
00138
00139 class JohannsenMetric final : public SphericalHorizonMetric
00140 {
00141 private:
00142     // Johannsen up to first order in deviation function is specified by five parameters (if M=1)

```

```

00170     const real m_aParam;
00172     const real m_alpha13Param;
00174     const real m_alpha22Param;
00176     const real m_alpha52Param;
00178     const real m_eps3Param;
00179
00180 public:
00181     // No default constructor allowed, must specify parameters
00182     JohannsenMetric() = delete;
00183
00184     // Constructor setting parameter a
00185     JohannsenMetric(real aParam, real alpha13Param, real alpha22Param, real alpha52Param, real
eps3Param, bool rLogScale = false);
00186
00187     // The override of the basic metric getter functions
00188     TwoIndex getMetric_dd(const Point &p) const final;
00189     TwoIndex getMetric_uu(const Point &p) const final;
00190
00191     // The override of the description string getter
00192     std::string getFullDescriptionStr() const final;
00193 };
00194
00199 class MankoNovikovMetric final : public SphericalHorizonMetric
00200 {
00201 private:
00202     // Manko-Novikov metric with only alpha3 as symmetry breaking parameter
00204     const real m_aParam;
00206     const real m_alpha3Param;
00207
00208     // These are convenient derived quantities from a
00210     const real m_alphaParam;
00212     const real m_kParam;
00213
00214 public:
00215     // No default constructor allowed, must specify parameters
00216     MankoNovikovMetric() = delete;
00217
00218     // Constructor setting parameter a and alpha3
00219     MankoNovikovMetric(real aParam, real alpha3Param, bool rLogScale = false);
00220
00221     // The override of the basic metric getter functions
00222     TwoIndex getMetric_dd(const Point &p) const final;
00223     TwoIndex getMetric_uu(const Point &p) const final;
00224
00225     // The override of the description string getter
00226     std::string getFullDescriptionStr() const final;
00227 };
00228
00233 class KerrSchildMetric final : public SphericalHorizonMetric
00234 {
00235 private:
00237     const real m_aParam;
00238
00239 public:
00240     // No default constructor allowed, must specify a
00241     KerrSchildMetric() = delete;
00242
00243     // Constructor setting parameter a
00244     KerrSchildMetric(real aParam, bool rLogScale = false);
00245
00246     // The override of the basic metric getter functions
00247     TwoIndex getMetric_dd(const Point &p) const final;
00248     TwoIndex getMetric_uu(const Point &p) const final;
00249
00250     // The override of the description string getter
00251     std::string getFullDescriptionStr() const final;
00252 };
00253
00257 class SingularityMetric : public Metric
00258 {
00259 public:
00260     // Constructor that initializes singularities
00261     SingularityMetric(std::vector<Singularity> thesings, bool rLogScale);
00262
00263     // Getter functions for the two member variables
00264     std::vector<Singularity> getSingularities() const;
00265
00266 protected:
00268     const std::vector<Singularity> m_AllSingularities;
00269 };
00270
00275 class ST3CrMetric final : public SingularityMetric
00276 {
00277 public:
00278     // Constructor which will be called to initialize all parameters of the metric
00279     ST3CrMetric(real P, real q0, real lambda, bool rlogscale = false);
00280

```

```

00281 // The basic getter functions
00282 TwoIndex getMetric_dd(const Point &p) const final;
00283 TwoIndex getMetric_uu(const Point &p) const final;
00284
00285 // The description string getter
00286 std::string getFullDescriptionStr() const final;
00287
00288 private:
00289     const real m_P;
00290     const real m_q0;
00291     const real m_lambda;
00292
00293     real get_omega(real r, real theta, real l) const;
00294     real f_phi(real phi, real r, real theta, real l, real R) const;
00295     real f_om_phi(real phi, real r, real theta, real l, real R) const;
00296 };
00297
00303 class BosonStarMetric final : public Metric
00304 {
00305 public:
00306     // Simple (default) constructor is all that is needed
00307     BosonStarMetric(double Phi_infinity, int num_lines, bool rLogScale = false,
00308                     std::string Phi_filename = "Phi.dat", std::string m_filename = "m.dat");
00309     // The override of the basic metric getter functions
00310     TwoIndex getMetric_dd(const Point &p) const final;
00311     TwoIndex getMetric_uu(const Point &p) const final;
00312     // The override of the description string getter
00313     std::string getFullDescriptionStr() const final;
00314
00315 protected:
00317     const double m_Phi_infinity;
00320     const int m_num_lines;
00321
00323     std::string m_Phi_filename;
00325     std::string m_m_filename;
00326
00328     tk::spline m_PhiSpline;
00330     tk::spline m_mSpline;
00331
00333     void read_data();
00334 };
00335
00336 class RotatingBosonStarMetric final : public Metric
00337 {
00338 public:
00339     // Simple (default) constructor is all that is needed
00340     RotatingBosonStarMetric(bool rLogScale = false, std::string MetricFolder =
00341                             "RotatingBosonStar/data_Will/", int num_x = 500, int num_th = 399, bool FlipAngularMomentum = false);
00342     ~RotatingBosonStarMetric() = default;
00343
00344     // The override of the basic metric getter functions
00345     TwoIndex getMetric_dd(const Point &p) const final;
00346     TwoIndex getMetric_uu(const Point &p) const final;
00347     // The override of the description string getter
00348     std::string getFullDescriptionStr() const final;
00349
00350 private:
00351     std::unique_ptr<Grid> m_grid_f;
00352     std::unique_ptr<Grid> m_grid_l;
00353     std::unique_ptr<Grid> m_grid_g;
00354     std::unique_ptr<Grid> m_grid_Omega;
00355
00356 public:
00358     std::unique_ptr<BicubicSplineInterpolator> m_fInterpolator;
00359     std::unique_ptr<BicubicSplineInterpolator> m_lInterpolator;
00360     std::unique_ptr<BicubicSplineInterpolator> m_gInterpolator;
00361     std::unique_ptr<BicubicSplineInterpolator> m_OmegaInterpolator;
00362
00363     // Sign multiplier for Omega: +1 for normal rotation, -1 for flipped rotation
00364     const int m_OmegaSign;
00365 };
00366
00368 // Declare your new Metric class here, publically inheriting from the base class Metric
00369 // (or SphericalHorizonMetric if your Metric has a horizon, or SingularityMetric if your Metric has
00370 // other, arbitrary singularities)
00371 // Give definitions (implementation) of these functions in Metric.cpp (or other source code file)
00372 // Don't forget to set m_Symmetries appropriately (in the constructor),
00373 // if your metric has any symmetry (e.g. stationarity, axisymmetry)!
00374 // Sample code:
00375 /*
00376 class MyMetric final : public Metric // good practice to make the class final unless descendant
00377                                     // classes are possible
00378 {
00379 public:
00380     // Constructor which will be called to initialize all parameters of the metric
00381     MyMetric(args...);
00382 };

```

```
00381 // The basic getter functions
00382 // These MUST be implemented
00383 TwoIndex getMetric_dd(const Point& p) const final;
00384 TwoIndex getMetric_uu(const Point& p) const final;
00385
00386 // The description string getter
00387 // This is optional (but recommended) to implement; if not implemented,
00388 // the base class Metric::getFullDescriptionStr() will be called instead
00389 std::string getFullDescriptionStr() const final;
00390
00391 private:
00392 // good practice to have all const params (initialized in the constructor)
00393 // since the metric cannot change after initialization
00394 // const params...;
00395
00396 };
00397 */
00399
00400 #endif
```

7.37 FOORT/src/Spline.cpp File Reference

Simple cubic spline interpolation library without external dependencies.

```
#include "Spline.h"
#include <cstdio>
#include <cassert>
#include <cmath>
#include <vector>
#include <algorithm>
```

7.37.1 Detailed Description

Simple cubic spline interpolation library without external dependencies.

This library is not made by the authors of FOORT, but implemented as found in the following link: <https://kluge.in-chemnitz.de/opensource/spline/>. The main body has been separated into `Spline.cpp`, however.

Author

Tino Kluge

Date

2021

Copyright

GNU General Public License

7.38 FOORT/src/Spline.h File Reference

Simple cubic spline interpolation library without external dependencies.

```
#include <cstdio>
#include <cassert>
#include <cmath>
#include <vector>
#include <algorithm>
```


Classes

- class `tk::spline`
- class `tk::internal::band_matrix`

Namespaces

- namespace `tk`
- namespace `tk::internal`

7.38.1 Detailed Description

Simple cubic spline interpolation library without external dependencies.

This library is not made by the authors of FOORT, but implemented as found in the following link: <https://kluge.in-chemnitz.de/opensource/spline/>. The main body has been separated into `Spline.cpp`, however.

Author

Tino Kluge

Date

2021

Copyright

GNU General Public License

7.39 Spline.h

[Go to the documentation of this file.](#)

```
00001 /*
00002  * spline.h
00003  *
00004  * simple cubic spline interpolation library without external
00005  * dependencies
00006  *
00007  * -----
00008  * Copyright (C) 2011, 2014, 2016, 2021 Tino Kluge (ttk448 at gmail.com)
00009  *
00010  * This program is free software; you can redistribute it and/or
00011  * modify it under the terms of the GNU General Public License
00012  * as published by the Free Software Foundation; either version 2
00013  * of the license, or (at your option) any later version.
00014  *
00015  * This program is distributed in the hope that it will be useful,
00016  * but WITHOUT ANY WARRANTY; without even the implied warranty of
00017  * MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
00018  * GNU General Public License for more details.
00019  *
00020  * You should have received a copy of the GNU General Public License
00021  * along with this program. If not, see <http://www.gnu.org/licenses/>.
00022  * -----
00023  *
00024  */
00025
00035
00036 #ifndef TK_SPLINE_H
```

```

00037 #define TK_SPLINE_H
00038
00039 #include <cstdio>
00040 #include <cassert>
00041 #include <cmath>
00042 #include <vector>
00043 #include <algorithm>
00044 #ifndef HAVE_SSTREAM
00045 #include <sstream>
00046 #include <string>
00047 #endif // HAVE_SSTREAM
00048
00049 // not ideal but disable unused-function warnings
00050 // (we get them because we have implementations in the header file,
00051 // and this is because we want to be able to quickly separate them
00052 // into a cpp file if necessary)
00053 #pragma GCC diagnostic push
00054 #pragma GCC diagnostic ignored "-Wunused-function"
00055 namespace tk
00056 {
00057     // spline interpolation
00058     class spline
00059     {
00060     public:
00061         // spline types
00062         enum spline_type
00063         {
00064             linear = 10,          // linear interpolation
00065             cspline = 30,         // cubic splines (classical C^2)
00066             cspline_hermite = 31 // cubic hermite splines (local, only C^1)
00067         };
00068
00069         // boundary condition type for the spline end-points
00070         enum bd_type
00071         {
00072             first_deriv = 1,
00073             second_deriv = 2
00074         };
00075
00076     protected:
00077         std::vector<double> m_x, m_y; // x,y coordinates of points
00078         // interpolation parameters
00079         //  $f(x) = a_i + b_i(x-x_i) + c_i(x-x_i)^2 + d_i(x-x_i)^3$ 
00080         // where  $a_i = y_i$ , or else it won't go through grid points
00081         std::vector<double> m_b, m_c, m_d; // spline coefficients
00082         double m_c0; // for left extrapolation
00083         spline_type m_type;
00084         bd_type m_left, m_right;
00085         double m_left_value, m_right_value;
00086         bool m_made_monotonic;
00087         void set_coeffs_from_b(); // calculate  $c_i, d_i$  from  $b_i$ 
00088         size_t find_closest(double x) const; // closest idx so that  $m_x[idx] \leq x$ 
00089
00090     public:
00091         // default constructor: set boundary condition to be zero curvature
00092         // at both ends, i.e. natural splines
00093         spline();
00094
00095         spline(const std::vector<double> &X, const std::vector<double> &Y,
00096               spline_type type = cspline,
00097               bool make_monotonic = false,
00098               bd_type left = second_deriv, double left_value = 0.0,
00099               bd_type right = second_deriv, double right_value = 0.0);
00100
00101         // modify boundary conditions: if called it must be before set_points()
00102         void set_boundary(bd_type left, double left_value,
00103                          bd_type right, double right_value);
00104
00105         // set all data points (cubic_spline=false means linear interpolation)
00106         void set_points(const std::vector<double> &x,
00107                        const std::vector<double> &y,
00108                        spline_type type = cspline);
00109
00110         // adjust coefficients so that the spline becomes piecewise monotonic
00111         // where possible
00112         // this is done by adjusting slopes at grid points by a non-negative
00113         // factor and this will break C^2
00114         // this can also break boundary conditions if adjustments need to
00115         // be made at the boundary points
00116         // returns false if no adjustments have been made, true otherwise
00117         bool make_monotonic();
00118
00119         // evaluates the spline at point x
00120         double operator()(double x) const;
00121         double deriv(int order, double x) const;
00122
00123         // returns the input data points

```

```

00124         std::vector<double> get_x() const;
00125         std::vector<double> get_y() const;
00126         double get_x_min() const;
00127         double get_x_max() const;
00128
00129 #ifdef HAVE_SSTREAM
00130         // spline info string, i.e. spline type, boundary conditions etc.
00131         std::string info() const;
00132 #endif // HAVE_SSTREAM
00133     };
00134
00135     namespace internal
00136     {
00137
00138         // band matrix solver
00139         class band_matrix
00140         {
00141         private:
00142             std::vector<std::vector<double>> m_upper; // upper band
00143             std::vector<std::vector<double>> m_lower; // lower band
00144         public:
00145             band_matrix() {}; // constructor
00146             band_matrix(int dim, int n_u, int n_l); // constructor
00147             ~band_matrix() {}; // destructor
00148             void resize(int dim, int n_u, int n_l); // init with dim,n_u,n_l
00149             int dim() const; // matrix dimension
00150             int num_upper() const;
00151             int num_lower() const;
00152             // access operator
00153             double &operator()(int i, int j); // write
00154             double operator()(int i, int j) const; // read
00155             // we can store an additional diagonal (in m_lower)
00156             double &saved_diag(int i);
00157             double saved_diag(int i) const;
00158             void lu_decompose();
00159             std::vector<double> r_solve(const std::vector<double> &b) const;
00160             std::vector<double> l_solve(const std::vector<double> &b) const;
00161             std::vector<double> lu_solve(const std::vector<double> &b,
00162                                         bool is_lu_decomposed = false);
00163         };
00164
00165     } // namespace internal
00166 } // namespace tk
00167 #pragma GCC diagnostic pop
00168
00169 #endif /* TK_SPLINE_H */

```

7.40 FOORT/src/Terminations.cpp File Reference

```

#include "Terminations.h"
#include "Geodesic.h"
#include "InputOutput.h"
#include <cmath>

```

Functions

- [TerminationUniqueVector](#) [CreateTerminationVector](#) ([TermBitflag](#) termflags, [Geodesic](#) *const theGeodesic)
Create a [Termination](#) Vector object.

7.40.1 Detailed Description

Author

Daniel R. Mayerson

Version

0.1

Date

2025-01-14

Copyright

Copyright (c) 2025

7.40.2 Function Documentation

7.40.2.1 CreateTerminationVector()

```
TerminationUniqueVector CreateTerminationVector (  
    TermBitflag termflags,  
    Geodesic *const theGeodesic)
```

Create a [Termination](#) Vector object.

Parameters

<i>termflags</i>	bitflags for the terminations
<i>theGeodesic</i>	the geodesic

Returns[TerminationUniqueVector](#)

7.41 FOORT/src/Terminations.h File Reference

Declarations of abstract base [Termination](#) class and all its descendants.

```
#include "Geometry.h"  
#include <cstdint>  
#include <memory>  
#include <vector>  
#include <utility>
```

Classes

- class [Termination](#)
Abstract base class for all Terminations.
- class [HorizonTermination](#)
HorizonTermination: Terminate geodesics if they get too close to the horizon.
- class [BoundarySphereTermination](#)
BoundarySphereTermination: Terminate geodesics if they reach outside of a boundary sphere.
- class [TimeOutTermination](#)
TimeOutTermination: Terminate geodesics if they take too many steps.
- class [ThetaSingularityTermination](#)
ThetaSingularityTermination: Terminate geodesics if they get too close to a polar coordinate singularity.
- class [NaNTermination](#)
NaNTermination: Terminate geodesics if they contain a NaN in position or velocity.
- class [GeneralSingularityTermination](#)
GeneralSingularityTermination: Terminate geodesics if they get too close to one of a number of singularities (of arbitrary codimension)
- struct [TerminationOptions](#)
Base class for all TerminationOptions structs. Other TerminationOptions can inherit from here if they require more options.
- struct [HorizonTermOptions](#)
Options for HorizonTermination.
- struct [BoundarySphereTermOptions](#)
Options for BoundarySphereTermination.
- struct [TimeOutTermOptions](#)
Options for TimeOutTermination.
- struct [ThetaSingularityTermOptions](#)
Options for ThetaSingularityTermination.
- struct [NaNTermOptions](#)
Options for NaNTermination.
- struct [GeneralSingularityTermOptions](#)
Options for GeneralSingularityTermination.

Typedefs

- using [TermBitflag](#) = std::uint16_t
Used for constructing vector of Terminations. Note that this means every Termination is either "on" or "off"; it is not possible to have a Termination "on" more than once.
- using [TerminationUniqueVector](#) = std::vector<std::unique_ptr<[Termination](#)>>
The owner vector of derived Termination classes.

Enumerations

- enum class [Term](#) {
 [Uninitialized](#) = -1 , [Continue](#) = 0 , [Horizon](#) , [BoundarySphere](#) ,
 [TimeOut](#) , [ThetaSingularity](#) , [NaN](#) , [GeneralSingularity](#) ,
 [Maxterms](#) }
Possible termination conditions that can be set by Terminations.

Functions

- [TerminationUniqueVector](#) [CreateTerminationVector](#) ([TermBitflag](#) termflags, [Geodesic](#) *const theGeodesic)
Create a [Termination](#) Vector object.

Variables

- constexpr [TermBitflag](#) [Term_None](#) {0b0000'0000'0000'0000}
No terminations bitflag.
- constexpr [TermBitflag](#) [Term_BoundarySphere](#) {0b0000'0000'0000'0001}
Boundary Sphere termination bitflag.
- constexpr [TermBitflag](#) [Term_TimeOut](#) {0b0000'0000'0000'0010}
Time Out termination bitflag.
- constexpr [TermBitflag](#) [Term_Horizon](#) {0b0000'0000'0000'0100}
Horizon termination bitflag.
- constexpr [TermBitflag](#) [Term_ThetaSingularity](#) {0b0000'0000'0000'1000}
Theta [Singularity](#) termination bitflag.
- constexpr [TermBitflag](#) [Term_NaN](#) {0b0000'0000'0001'0000}
NaN termination bitflag.
- constexpr [TermBitflag](#) [Term_GeneralSingularity](#) {0b0000'0000'0010'0000}
General [Singularity](#) termination bitflag.

7.41.1 Detailed Description

Declarations of abstract base [Termination](#) class and all its descendants.

Author

Daniel R. Mayerson

Version

0.1

Date

2025-01-14

Copyright

Copyright (c) 2025

7.41.2 Typedef Documentation

7.41.2.1 TermBitflag

```
using TermBitflag = std::uint16_t
```

Used for constructing vector of Terminations. Note that this means every [Termination](#) is either "on" or "off"; it is not possible to have a [Termination](#) "on" more than once.

7.41.2.2 TerminationUniqueVector

```
using TerminationUniqueVector = std::vector<std::unique_ptr<Termination>>
```

The owner vector of derived [Termination](#) classes.

7.41.3 Enumeration Type Documentation

7.41.3.1 Term

```
enum class Term [strong]
```

Possible termination conditions that can be set by Terminations.

Enumerator

Uninitialized	Geodesic has not been properly initialized yet with initial position/velocity.
Continue	All is right, continue integrating geodesic.
Horizon	STOP, encountered horizon (set by HorizonTermination)
BoundarySphere	STOP, encountered boundary sphere (set by BoundarySphereTermination)
TimeOut	STOP, taken too many steps (set by TimeOutTermination)
ThetaSingularity	STOP, too close to polar coordinate singularity ($\theta = 0$ or $\theta = \pi/2$) (set by ThetaSingularityTermination)
NaN	STOP, NaN encountered in geodesic position or velocity (set by NaNTermination)
GeneralSingularity	STOP, singularity encountered (of any codimension) (set by GeneralSingularityTermination)
Maxterms	

7.41.4 Function Documentation

7.41.4.1 CreateTerminationVector()

```
TerminationUniqueVector CreateTerminationVector (
    TermBitflag termflags,
    Geodesic *const theGeodesic)
```

Create a [Termination](#) Vector object.

Parameters

<i>termflags</i>	bitflags for the terminations
<i>theGeodesic</i>	the geodesic

Returns

[TerminationUniqueVector](#)

7.41.5 Variable Documentation

7.41.5.1 Term_BoundarySphere

```
TermBitflag Term_BoundarySphere {0b0000'0000'0000'0001} [constexpr]
```

Boundary Sphere termination bitflag.

7.41.5.2 Term_GeneralSingularity

```
TermBitflag Term_GeneralSingularity {0b0000'0000'0010'0000} [constexpr]
```

General [Singularity](#) termination bitflag.

7.41.5.3 Term_Horizon

```
TermBitflag Term_Horizon {0b0000'0000'0000'0100} [constexpr]
```

Horizon termination bitflag.

7.41.5.4 Term_NaN

```
TermBitflag Term_NaN {0b0000'0000'0001'0000} [constexpr]
```

NaN termination bitflag.

7.41.5.5 Term_None

```
TermBitflag Term_None {0b0000'0000'0000'0000} [constexpr]
```

No terminations bitflag.

7.41.5.6 Term_ThetaSingularity

```
TermBitflag Term_ThetaSingularity {0b0000'0000'0000'1000} [constexpr]
```

Theta [Singularity](#) termination bitflag.

7.41.5.7 Term_TimeOut

```
TermBitflag Term_TimeOut {0b0000'0000'0000'0010} [constexpr]
```

Time Out termination bitflag.

7.42 Terminations.h

[Go to the documentation of this file.](#)

```

00001 #ifndef _FOORT_TERMINATIONS_H
00002 #define _FOORT_TERMINATIONS_H
00003
00012
00013 #include "Geometry.h" // for basic tensor objects
00014
00015 #include <stdint> // for std::uint16_t
00016 #include <memory> // for std::unique_ptr
00017 #include <vector> // for std::vector
00018 #include <utility> // for std::pair
00019
00020 // Forward declaration of Geodesic class needed here, since Diagnostics are passed a pointer to their
    owner Geodesic
00021 // (note "Geodesic.h" is NOT included to avoid header loop, and we do not need Geodesic member
    functions here!)
00022 class Geodesic;
00023
00024 // TERMINATION BITFLAGS
00026 using TermBitflag = std::uint16_t;
00027
00028 // Define a bitflag per existing Termination
00030 constexpr TermBitflag Term_None{0b0000'0000'0000'0000};
00032 constexpr TermBitflag Term_BoundarySphere{0b0000'0000'0000'0001};
00034 constexpr TermBitflag Term_TimeOut{0b0000'0000'0000'0010};
00036 constexpr TermBitflag Term_Horizon{0b0000'0000'0000'0100};
00038 constexpr TermBitflag Term_ThetaSingularity{0b0000'0000'0000'1000};
00040 constexpr TermBitflag Term_NaN{0b0000'0000'0001'0000};
00042 constexpr TermBitflag Term_GeneralSingularity{0b0000'0000'0010'0000};
00043
00045 // Add a TermBitflag for your new Termination. Make sure you use a bitflag that has not been used
    before!
00046 // Sample code:
00047 /*
00048 constexpr TermBitflag Term_MyTerm { 0b0000'0000'0000'1000 };
00049 */
00051
00052 // TERMINATION CONDITIONS
00053
00057 enum class Term
00058 {
00060     Uninitialized = -1,
00062     Continue = 0,
00064     Horizon,
00066     BoundarySphere,
00068     TimeOut,
00070     ThetaSingularity,
00072     NaN,
00074     GeneralSingularity,
00075
00077     // Add a new Termination condition that your new Termination can set
00078     // Sample code:
00079     /*
00080     MyTermCond, // STOP, encountered (...)
00081     */
00083
00084     Maxterms // Number of termination conditions that exist
00085 };
00086
00087 // GENERAL DECLARATIONS OF ABSTRACT BASE CLASS
00088
00092 class Termination
00093 {
00094 public:
00095     // Constructor must initialize the pointer to its owner Geodesic
00096     Termination() = delete;
00097     Termination(Geodesic *const theGeodesic) : m_OwnerGeodesic{theGeodesic}
00098     {
00099     }
00100
00101     // Resets Termination object. This is called when the owner Geodesic is reset in order to start
    integrating
00102     // a new geodesic.
00103     // The base class implementation only resets m_StepsSinceUpdated
00104     // Descendants can override this if they need to reset additional internal variables
00105     virtual void Reset();
00106
00107     // virtual destructor to ensure correct destruction of descendants
00108     virtual ~Termination() = default;
00109
00110     // Function that is called to determine whether Termination wants to
00111     // terminate the Geodesic. Returns Term::Continue if no termination wanted,
00112     // otherwise it returns the appropriate Term condition

```

```

00113     virtual Term CheckTermination() = 0;
00114
00115     // This returns the full description of the Termination
00116     virtual std::string getFullDescriptionStr() const = 0;
00117
00118 protected:
00119     Geodesic *const m_OwnerGeodesic;
00120
00121     // Helper function to decide if the Termination should indeed update its status, based on
00122     // UpdateNSteps (which is set to 0 if we always update)
00123     bool DecideUpdate(largecounter UpdateNSteps);
00124
00125     largecounter m_StepsSinceUpdated{};
00126 };
00127
00128 using TerminationUniqueVector = std::vector<std::unique_ptr<Termination>;
00129
00130 // Helper to create a new vector of Termination options, based on the bitflag
00131 TerminationUniqueVector CreateTerminationVector(TermBitflag termflags, Geodesic *const theGeodesic);
00132
00133 // DECLARATIONS FOR DERIVED TERMINATION CLASSES (DESCENDANTS)
00134
00135 // Forward declaration needed before Termination
00136 struct HorizonTermOptions;
00137
00138 class HorizonTermination final : public Termination
00139 {
00140 public:
00141     // Basic constructor only passes on Geodesic pointer to base class constructor
00142     HorizonTermination(Geodesic *const theGeodesic) : Termination(theGeodesic) {}
00143
00144     // Check if we are too close to the horizon
00145     Term CheckTermination() final;
00146
00147     // Description string
00148     std::string getFullDescriptionStr() const final;
00149
00150     static std::unique_ptr<HorizonTermOptions> TermOptions;
00151 };
00152
00153 // Forward declaration needed before Termination
00154 struct BoundarySphereTermOptions;
00155
00156 class BoundarySphereTermination final : public Termination
00157 {
00158 public:
00159     // Basic constructor only passes on Geodesic pointer to base class constructor
00160     BoundarySphereTermination(Geodesic *const theGeodesic) : Termination(theGeodesic) {}
00161
00162     // Check if we have passed the boundary sphere
00163     Term CheckTermination() final;
00164
00165     // Description string
00166     std::string getFullDescriptionStr() const final;
00167
00168     static std::unique_ptr<BoundarySphereTermOptions> TermOptions;
00169 };
00170
00171 // Forward declaration needed before Termination
00172 struct TimeOutTermOptions;
00173
00174 class TimeOutTermination final : public Termination
00175 {
00176 public:
00177     // Basic constructor only passes on Geodesic pointer to base class constructor
00178     TimeOutTermination(Geodesic *const theGeodesic) : Termination(theGeodesic) {}
00179
00180     // This descendant needs to override Reset in order to also reset m_CurNrSteps
00181     void Reset() final;
00182
00183     // Check if we have already taken too many steps
00184     Term CheckTermination() final;
00185
00186     // Description string
00187     std::string getFullDescriptionStr() const final;
00188
00189     static std::unique_ptr<TimeOutTermOptions> TermOptions;
00190 private:
00191     largecounter m_CurNrSteps{0};
00192 };
00193
00194 // Forward declaration needed before Termination
00195 struct ThetaSingularityTermOptions;
00196
00197 class ThetaSingularityTermination final : public Termination
00198 {

```

```

00219 public:
00220     ThetaSingularityTermination(Geodesic *const theGeodesic) : Termination(theGeodesic) {}
00221
00222     // No override of Reset() necessary
00223
00224     // Check the specific termination condition
00225     Term CheckTermination() final;
00226
00227     // Description string
00228     std::string getFullDescriptionStr() const final;
00229
00231     static std::unique_ptr<ThetaSingularityTermOptions> TermOptions;
00232 };
00233
00234 // Forward declaration needed before Termination
00235 struct NaNTermOptions;
00236
00240 class NaNTermination final : public Termination
00241 {
00242 public:
00243     // Basic constructor only passes on Geodesic pointer to base class constructor
00244     NaNTermination(Geodesic *const theGeodesic) : Termination(theGeodesic) {}
00245
00246     // Check if we are too close to the horizon
00247     Term CheckTermination() final;
00248
00249     // Description string
00250     std::string getFullDescriptionStr() const final;
00251
00253     static std::unique_ptr<NaNTermOptions> TermOptions;
00254 };
00255
00256 // Forward declaration needed before Termination
00257 struct GeneralSingularityTermOptions;
00258
00262 class GeneralSingularityTermination final : public Termination
00263 {
00264 public:
00265     // Basic constructor only passes on Geodesic pointer to base class constructor
00266     GeneralSingularityTermination(Geodesic *const theGeodesic) : Termination(theGeodesic) {}
00267
00268     // Check if we are too close to the horizon
00269     Term CheckTermination() final;
00270
00271     // Description string
00272     std::string getFullDescriptionStr() const final;
00273
00275     static std::unique_ptr<GeneralSingularityTermOptions> TermOptions;
00276
00277 private:
00278     std::string SingularityToString(int singnr) const;
00279 };
00280
00282 // Declare your Termination class here, inheriting from Termination.
00283 // Sample code:
00284 /*
00285 // Forward declaration needed before Termination
00286 struct TerminationOptions; // possibly will instead need to declare descendant options struct
00287 class MyTermination final : public Termination // good practice to make the class final unless
00288     descendant classes are possible
00289 {
00289 public:
00290     // Constructor must at least pass on Geodesic pointer to base class constructor
00291     MyTermination(Geodesic* const theGeodesic) : Termination(theGeodesic) {}
00292
00293     // Do you need to reset any internal variables specific to MyTermination? If so, override Reset()
00294     (This is not mandatory)
00295     // Note: make sure to call the base class implementation Termination::Reset()
00296     // from within your implementation of MyTermination::Reset(), so that the base class internal
00297     variable is also reset!
00298     void Reset() final;
00299
00300     // Check the specific termination condition
00301     Term CheckTermination() final;
00302
00303     // Description string
00304     std::string getFullDescriptionStr() const final;
00305
00306     // The options that the Termination keeps (will probably be a descendant struct instead, which
00307     specifies
00308     // any additional options the Termination needs)
00309     static std::unique_ptr<TerminationOptions> TermOptions;
00310
00311 private:
00312     // any private member variables that are needed to keep track of things
00313 };
00314 */

```

```

00313
00314 // ALL TERMINATIONOPTIONS STRUCTS
00315
00319 struct TerminationOptions
00320 {
00321 public:
00322     // Basic constructor only sets the number of steps between updates
00323     TerminationOptions(largecounter Nsteps) : UpdateEveryNSteps{Nsteps}
00324     {
00325     }
00326
00327     // virtual destructor to ensure correct destruction of descendants
00328     virtual ~TerminationOptions() = default;
00329
00331     const largecounter UpdateEveryNSteps;
00332 };
00333
00338 struct HorizonTermOptions : public TerminationOptions
00339 {
00340 public:
00341     HorizonTermOptions(real theHorizonRadius, bool therLogScale, real theAtHorizonEps, largecounter
Nsteps) : HorizonRadius{theHorizonRadius}, AtHorizonEps{theAtHorizonEps}, rLogScale{therLogScale},
TerminationOptions(Nsteps)
00342     {
00343     }
00344
00346     const real HorizonRadius;
00348     const real AtHorizonEps;
00350     const bool rLogScale;
00351 };
00352
00357 struct BoundarySphereTermOptions : public TerminationOptions
00358 {
00359 public:
00360     BoundarySphereTermOptions(real theRadius, bool therLogScale, largecounter Nsteps) :
SphereRadius{theRadius}, rLogScale{therLogScale},
00361     TerminationOptions(Nsteps)
00362     {
00363     }
00364
00366     const real SphereRadius;
00368     const bool rLogScale;
00369 };
00370
00375 struct TimeOutTermOptions : public TerminationOptions
00376 {
00377 public:
00378     TimeOutTermOptions(largecounter MaxStepsAllowed, largecounter Nsteps) : MaxSteps{MaxStepsAllowed},
TerminationOptions(Nsteps)
00379     {
00380     }
00381
00383     const largecounter MaxSteps;
00384 };
00385
00386 // Options class for ThetaSingularityTermination
00391 struct ThetaSingularityTermOptions : public TerminationOptions
00392 {
00393 public:
00394     ThetaSingularityTermOptions(real epsilon, largecounter Nsteps) : ThetaSingEpsilon{epsilon},
TerminationOptions(Nsteps)
00395     {
00396     }
00397
00399     const real ThetaSingEpsilon;
00400 };
00401
00405 struct NaNTermOptions : public TerminationOptions
00406 {
00407 public:
00408     NaNTermOptions(bool consoleoutputon, largecounter Nsteps) : OutputToConsole{consoleoutputon},
TerminationOptions(Nsteps)
00409     {
00410     }
00411
00413     const bool OutputToConsole;
00414 };
00415
00420 struct GeneralSingularityTermOptions : public TerminationOptions
00421 {
00422 public:
00423     GeneralSingularityTermOptions(std::vector<Singularity> sings,
real eps, bool consoleoutputon, bool therlogscale, largecounter
00424     Nsteps)
00425     : Singularities{std::move(sings)}, Epsilon{eps}, OutputToConsole{consoleoutputon},
00426     rLogScale{therlogscale},

```

```

00427         TerminationOptions(Nsteps) {}
00428
00430     const std::vector<Singularity> Singularities;
00432     const real Epsilon;
00434     const bool OutputToConsole;
00436     const bool rLogScale;
00437 };
00438
00440 // Add your new TerminationOptions struct here, inheriting from TerminationOptions (if needed)
00441 // Sample code:
00442 /*
00443 struct MyTermOptions : public TerminationOptions
00444 {
00445     public:
00446         MyTermOptions(..., largecounter Nsteps) : TerminationOptions(Nsteps) //, other initialization
00447         {}
00448
00449         // member variables (const!) here
00450 };
00451 */
00453
00454 #endif

```

7.43 FOORT/src/Utilities.cpp File Reference

```

#include "Utilities.h"
#include <sstream>
#include <iomanip>

```

7.43.1 Detailed Description

Author

Daniel R. Mayerson

Version

0.1

Date

2025-01-14

Copyright

Copyright (c) 2025

7.44 FOORT/src/Utilities.h File Reference

Declarations of various utility functions, in a [Utilities](#) namespace.

```

#include "Metric.h"
#include "Diagnostics.h"
#include "Terminations.h"
#include "Geodesic.h"
#include "ViewScreen.h"
#include "Integrators.h"
#include <chrono>
#include <string>
#include <vector>

```

Classes

- class [Utilities::Timer](#)
Timer class to keep track of elapsed time.

Namespaces

- namespace [Utilities](#)
Namespace that contains our utility functions.

Functions

- `std::string Utilities::GetTimeStampString ()`
Get a string of the current time (in a format that can be used to append to file names)
- `std::vector< std::string > Utilities::GetDiagNameStrings (DiagBitflag alldiags, DiagBitflag valdiag)`
Helper function to get all [Diagnostic](#) Names (for outputting to files)
- `std::string Utilities::GetFirstLineInfoString (const Metric *theMetric, const Source *theSource, DiagBitflag alldiags, DiagBitflag valdiag, TermBitflag allterms, const ViewScreen *theView)`
This returns the full string to be written to every output file as its first line.

7.44.1 Detailed Description

Declarations of various utility functions, in a [Utilities](#) namespace.

Author

Daniel R. Mayerson

Version

0.1

Date

2025-01-14

Copyright

Copyright (c) 2025

7.45 Utilities.h

[Go to the documentation of this file.](#)

```

00001 #ifndef _FOORT_UTILITIES_H
00002 #define _FOORT_UTILITIES_H
00003
00012
00013 // We use Metric, Diagnostic, Termination, Geodesic, ViewScreen, and Integrator declarations here
00014 #include "Metric.h"
00015 #include "Diagnostics.h"
00016 #include "Terminations.h"
00017 #include "Geodesic.h"
00018 #include "ViewScreen.h"
00019 #include "Integrators.h"
00020
00021 #include <chrono> // for timer functionality
00022 #include <string> // for strings
00023 #include <vector> // for std::vector
00024
00026 namespace Utilities
00027 {
00031     class Timer
00032     {
00033     private:
00035         using Clock = std::chrono::steady_clock;
00037         using Second = std::chrono::duration<double, std::ratio<1>;
00038
00040         std::chrono::time_point<Clock> m_beg{Clock::now()};
00041
00042     public:
00043         // reset begin time
00044         void reset();
00045
00046         // returns time elapsed since begin time
00047         double elapsed() const;
00048     };
00049
00050     // Returns a string of the current time (in a format that can be used to append to file names)
00051     std::string GetTimeStampString();
00052
00053     // Helper function to get all Diagnostic Names (for outputting to files)
00054     std::vector<std::string> GetDiagNameStrings(DiagBitflag alldiags, DiagBitflag valdiag);
00055
00056     // This returns the full string to be written to every output file as its first line
00057     // It contains information about all the settings used to produce the output
00058     std::string GetFirstLineInfoString(const Metric *theMetric, const Source *theSource,
00059                                       DiagBitflag alldiags, DiagBitflag valdiag, TermBitflag
allterms, const ViewScreen *theView);
00060 }
00061
00062 #endif

```

7.46 FOORT/src/ViewScreen.cpp File Reference

```

#include "ViewScreen.h"
#include <algorithm>

```

7.46.1 Detailed Description

Author

Daniel R. Mayerson

Version

0.1

Date

2025-01-14

Copyright

Copyright (c) 2025

7.47 FOORT/src/ViewScreen.h File Reference

Declarations of [ViewScreen](#) class.

```
#include "Geometry.h"
#include "Metric.h"
#include "Mesh.h"
#include <memory>
#include <utility>
#include <array>
#include <string>
```

Classes

- class [ViewScreen](#)

This class is in charge of converting a pixel on the screen into physical initial conditions for a geodesic.

Enumerations

- enum class [GeodesicType](#) { [Null](#) = 0 , [Timelike](#) = -1 , [Spacelike](#) = 1 }

Type of geodesic being integrated.

7.47.1 Detailed Description

Declarations of [ViewScreen](#) class.

Author

Daniel R. Mayerson

This class is in charge of converting a pixel on the screen (which the [Mesh](#) wants to integrate) into physical initial conditions for the position and velocity of a geodesic.

Version

0.1

Date

2025-01-14

Copyright

Copyright (c) 2025

7.47.2 Enumeration Type Documentation

7.47.2.1 GeodesicType

```
enum class GeodesicType [strong]
```

Type of geodesic being integrated.

Null is the default, Timelike is -1, Spacelike is 1. Only Null is supported/implemented at the moment.

Enumerator

Null	
Timelike	
Spacelike	

7.48 ViewScreen.h

[Go to the documentation of this file.](#)

```
00001 #ifndef _FOORT_VIEWSCREEN_H
00002 #define _FOORT_VIEWSCREEN_H
00003
00013
00014 #include "Geometry.h" // For basic tensor objects
00015 #include "Metric.h" // For the Metric object
00016 #include "Mesh.h" // For the Mesh object
00017
00018 #include <memory> // std::unique_ptr
00019 #include <utility> // std::move
00020 #include <array> // std::array
00021 #include <string> // strings
00022
00027 enum class GeodesicType
00028 {
00029     Null = 0,
00030     Timelike = -1,
00031     Spacelike = 1,
00032 };
00033
00039 class ViewScreen
00040 {
00041 public:
00042     // No default constructor possible
00043     ViewScreen() = delete;
00044     // constructor must pass following arguments along:
00045     // - physical position and looking direction;
00046     // - screen dimensions (in dimensions of length)
00047     // - the Mesh used (ViewScreen must become a owner of this object!)
00048     // - the Metric used (ViewScreen is NOT the owner of the Metric)
00049     // - the geodesic type to be integrated (null, timelike, spacelike)
00050     ViewScreen(Point pos, OneIndex dir, ScreenPoint screensize, ScreenPoint screencenter,
00051         std::unique_ptr<Mesh> theMesh, const Metric *const theMetric, GeodesicType thegeodtype
00052     = GeodesicType::Null)
00053     : m_Pos{pos}, m_Direction{dir}, m_ScreenSize{screensize}, m_ScreenCenter{screencenter},
00054       m_theMesh{std::move(theMesh)},
00055       m_theMetric{theMetric}, m_GeodType{thegeodtype},
00056       m_rLogScale{theMetric->getrLogScale()}
00057     {
00058         // At the moment, we don't even use the direction; we are always pointed towards the origin
00059         if (m_Direction != Point{0, -1, 0, 0})
00060         {
00061             ScreenOutput("ViewScreen is only supported pointing inwards at the moment; Direction = {0,
-1, 0, 0} will be used",
00062                 OutputLevel::Level_0_WARNING);
00063         }
00064         // At the moment, we are only integrating null geodesics
00065         if (m_GeodType != GeodesicType::Null)
00066         {
00067             ScreenOutput("ViewScreen only supports null geodesics at the moment; geodesics integrated
will be null.",
```

```

00067             OutputLevel::Level_0_WARNING);
00068         }
00069
00070         ScreenOutput("ViewScreen initialized with position " + toString(m_Pos) + ", looking direction
" + toString(m_Direction) +
00071             ", screen size " + toString(m_ScreenSize) + ", screen center " +
toString(m_ScreenCenter) +
00072             ", geodesic type " + (m_GeodType == GeodesicType::Null ? "Null" : (m_GeodType
== GeodesicType::Timelike ? "Timelike" : "Spacelike"))) +
00073             ", and metric: " + m_theMetric->getFullDescriptionStr(),
00074             OutputLevel::Level_4_DEBUG);
00075         // Construct the vielbein now
00076         ConstructVielbein();
00077     }
00078
00079     // Heart of the ViewScreen: here, the ViewScreen is asked to provide initial conditions
00080     // for the geodesic nr index of the current iteration; based on the screen index
00081     // that the Mesh gives, it sets up these physical initial conditions.
00082     void SetNewInitialConditions(largecounter index, Point &pos, OneIndex &vel, ScreenIndex &scrIndex)
const;
00083
00084     // These member functions essentially pass on information to/from the Mesh
00085     bool IsFinished() const; // Does the ViewScreen (i.e. the Mesh) want to integrate
more geodesics or not?
00086     largecounter getCurNrGeodesics() const; // Current number of geodesics in this iteration
00087     void EndCurrentLoop(); // The current iteration of geodesics is finished;
prepare the next one
00088     // NOTE: despite not being const, this function has been designed to be threadsafe!
00089     void GeodesicFinished(largecounter index, std::vector<real> finalValues); // This geodesic has
been integrated, returning its final "values"
00090
00091     // Description string getter (spaces allowed), also will contain information about the Mesh
00092     std::string getFullDescriptionStr() const;
00093
00094 private:
00095     TwoIndex m_Metric_dd{};
00096     TwoIndex m_Vielbein{};
00097
00098     // Helper function to construct the vielbein given the metric
00099     void ConstructVielbein();
00100
00101     const Point m_Pos;
00102     const OneIndex m_Direction;
00103     const ScreenPoint m_ScreenSize;
00104     const ScreenPoint m_ScreenCenter;
00105
00106     const bool m_rLogScale;
00107
00108     const Metric *const m_theMetric;
00109
00110     const GeodesicType m_GeodType{GeodesicType::Null};
00111
00112     const std::unique_ptr<Mesh> m_theMesh;
00113 };
00114
00115 #endif

```

Index

- ~Diagnostic
 - Diagnostic, [51](#)
- ~DiagnosticOptions
 - DiagnosticOptions, [54](#)
- ~EmissionModel
 - EmissionModel, [55](#)
- ~FluidVelocityModel
 - FluidVelocityModel, [70](#)
- ~Grid
 - Grid, [103](#)
- ~Mesh
 - Mesh, [128](#)
- ~Metric
 - Metric, [130](#)
- ~RotatingBosonStarMetric
 - RotatingBosonStarMetric, [147](#)
- ~Source
 - Source, [156](#)
- ~Termination
 - Termination, [185](#)
- ~TerminationOptions
 - TerminationOptions, [187](#)
- ~band_matrix
 - tk::internal::band_matrix, [22](#)
- AtHorizonEps
 - HorizonTermOptions, [107](#)
- band_matrix
 - tk::internal::band_matrix, [21](#)
- bd_type
 - tk::spline, [160](#)
- BicubicSplineInterpolator, [24](#)
 - BicubicSplineInterpolator, [24](#)
 - compute_cell_coefficients, [24](#)
 - compute_spline_coefficients, [24](#)
 - estimate_dx, [25](#)
 - estimate_dxdy, [25](#)
 - estimate_dy, [25](#)
 - interpolate, [25](#)
 - m_coeffs, [25](#)
 - m_theta, [25](#)
 - m_x, [26](#)
 - set_grid, [25](#)
- BosonStarMetric, [26](#)
 - BosonStarMetric, [27](#)
 - getFullDescriptionStr, [28](#)
 - getMetric_dd, [28](#)
 - getMetric_uu, [28](#)
 - m_m_filename, [29](#)
 - m_mSpline, [29](#)
 - m_num_lines, [29](#)
 - m_Phi_filename, [29](#)
 - m_Phi_infinity, [29](#)
 - m_PhiSpline, [30](#)
 - read_data, [29](#)
- BoundarySphere
 - Terminations.h, [267](#)
- BoundarySphereTermination, [30](#)
 - BoundarySphereTermination, [31](#)
 - CheckTermination, [31](#)
 - getFullDescriptionStr, [31](#)
 - TermOptions, [32](#)
- BoundarySphereTermOptions, [32](#)
 - BoundarySphereTermOptions, [33](#)
 - rLogScale, [33](#)
 - SphereRadius, [33](#)
- CheckTermination
 - BoundarySphereTermination, [31](#)
 - GeneralSingularityTermination, [81](#)
 - HorizonTermination, [106](#)
 - NaNTermination, [134](#)
 - Termination, [185](#)
 - ThetaSingularityTermination, [189](#)
 - TimeOutTermination, [192](#)
- Clock
 - Utilities::Timer, [195](#)
- ClosestRadiusDiagnostic, [34](#)
 - ClosestRadiusDiagnostic, [35](#)
 - DiagOptions, [37](#)
 - FinalDataValDistance, [35](#)
 - getFinalDataVal, [35](#)
 - getFullDataStr, [36](#)
 - getFullDescriptionStr, [36](#)
 - getNameStr, [36](#)
 - m_ClosestRadius, [37](#)
 - Reset, [36](#)
 - UpdateData, [37](#)
- ClosestRadiusOptions, [37](#)
 - ClosestRadiusOptions, [38](#)
 - RLogScale, [38](#)
- compute_cell_coefficients
 - BicubicSplineInterpolator, [24](#)
- compute_spline_coefficients
 - BicubicSplineInterpolator, [24](#)
- Config, [11](#)
 - ConfigCollection, [12](#)
 - GetGeodesicIntegrator, [12](#)
 - GetMesh, [12](#)

- GetMetric, [13](#)
- GetOutputHandler, [13](#)
- GetSource, [13](#)
- GetViewScreen, [14](#)
- InitializeDiagnostics, [14](#)
- InitializeScreenOutput, [14](#)
- InitializeTerminations, [15](#)
- Output_Important_Default, [15](#)
- Output_Other_Default, [15](#)
- SettingError, [12](#)
- ConfigCollection
 - Config, [12](#)
- ConfigReader, [15](#)
 - MaxStreamSize, [16](#)
- ConfigReader::ConfigCollection, [38](#)
 - ConfigSettingValue, [40](#)
 - DisplayCollection, [40](#)
 - DisplaySetting, [41](#)
 - DisplayTabs, [41](#)
 - Exists, [41](#)
 - GetSettingIndex, [41](#)
 - IsCollection, [42](#)
 - LookupValue, [42](#), [43](#)
 - LookupValueInteger, [43–45](#)
 - m_Settings, [48](#)
 - NrSettings, [46](#)
 - operator[], [46](#)
 - ReadCollection, [47](#)
 - ReadFile, [47](#)
 - ReadSettingName, [47](#)
 - ReadSettingSpecificChar, [48](#)
 - ReadSettingValue, [48](#)
- ConfigReader::ConfigCollection::ConfigSetting, [50](#)
 - SettingName, [50](#)
 - SettingValue, [50](#)
- ConfigReader::ConfigReaderException, [49](#)
 - ConfigReaderException, [49](#)
 - m_settingtrace, [49](#)
 - trace, [49](#)
- ConfigReaderException
 - ConfigReader::ConfigReaderException, [49](#)
- ConfigSettingValue
 - ConfigReader::ConfigCollection, [40](#)
- ConstructVielbein
 - ViewScreen, [199](#)
- Continue
 - Terminations.h, [267](#)
- CreateDiagnosticVector
 - Diagnostics.cpp, [211](#)
 - Diagnostics.h, [214](#)
- CreateTerminationVector
 - Terminations.cpp, [264](#)
 - Terminations.h, [267](#)
- cspline
 - tk::spline, [161](#)
- cspline_hermite
 - tk::spline, [161](#)
- data
 - Grid, [104](#)
- DecideUpdate
 - Diagnostic, [51](#)
 - Termination, [185](#)
- delta_nodiv0
 - Integrators, [18](#)
- deriv
 - tk::spline, [161](#)
- Derivative_hval
 - Integrators, [18](#)
- Diag_ClosestRadius
 - Diagnostics.h, [214](#)
- Diag_EquatorialEmission
 - Diagnostics.h, [214](#)
- Diag_EquatorialPasses
 - Diagnostics.h, [214](#)
- Diag_FourColorScreen
 - Diagnostics.h, [214](#)
- Diag_GeodesicPosition
 - Diagnostics.h, [215](#)
- Diag_None
 - Diagnostics.h, [215](#)
- DiagBitflag
 - Diagnostics.h, [213](#)
- Diagnostic, [50](#)
 - ~Diagnostic, [51](#)
 - DecideUpdate, [51](#)
 - Diagnostic, [51](#)
 - FinalDataValDistance, [51](#)
 - getFinalDataVal, [52](#)
 - getFullDataStr, [52](#)
 - getFullDescriptionStr, [52](#)
 - getNameStr, [52](#)
 - m_OwnerGeodesic, [53](#)
 - m_StepsSinceUpdated, [53](#)
 - Reset, [52](#)
 - UpdateData, [53](#)
- DiagnosticOptions, [53](#)
 - ~DiagnosticOptions, [54](#)
 - DiagnosticOptions, [54](#)
 - theUpdateFrequency, [54](#)
- Diagnostics.cpp
 - CreateDiagnosticVector, [211](#)
- Diagnostics.h
 - CreateDiagnosticVector, [214](#)
 - Diag_ClosestRadius, [214](#)
 - Diag_EquatorialEmission, [214](#)
 - Diag_EquatorialPasses, [214](#)
 - Diag_FourColorScreen, [214](#)
 - Diag_GeodesicPosition, [215](#)
 - Diag_None, [215](#)
 - DiagBitflag, [213](#)
 - DiagnosticUniqueVector, [213](#)
- DiagnosticUniqueVector
 - Diagnostics.h, [213](#)
- DiagOptions
 - ClosestRadiusDiagnostic, [37](#)
 - EquatorialEmissionDiagnostic, [59](#)

- EquatorialPassesDiagnostic, 65
- GeodesicPositionDiagnostic, 98
- DiagValue
 - SquareSubdivisionMesh::PixelInfo, 139
 - SquareSubdivisionMeshV2::PixelInfo, 141
- dim
 - tk::internal::band_matrix, 22
- dimension
 - Geometry.h, 234
- DisplayCollection
 - ConfigReader::ConfigCollection, 40
- DisplaySetting
 - ConfigReader::ConfigCollection, 41
- DisplayTabs
 - ConfigReader::ConfigCollection, 41
- DownNbr
 - SquareSubdivisionMeshV2::PixelInfo, 141
- elapsed
 - Utilities::Timer, 195
- EmissionModel, 54
 - ~EmissionModel, 55
 - GetEmission, 55
 - getFullDescriptionStr, 55
- EndCurrentLoop
 - InputCertainPixelsMesh, 110
 - Mesh, 128
 - SimpleSquareMesh, 151
 - SquareSubdivisionMesh, 166
 - SquareSubdivisionMeshV2, 173
 - ViewScreen, 199
- Epsilon
 - GeneralSingularityTermOptions, 83
- epsilon
 - Integrators, 18
- EquatorialEmissionDiagnostic, 56
 - DiagOptions, 59
 - EquatorialEmissionDiagnostic, 58
 - FinalDataValDistance, 58
 - getFinalDataVal, 58
 - getFullDataStr, 58
 - getFullDescriptionStr, 58
 - getNameStr, 59
 - m_Intensity, 59
 - Reset, 59
 - UpdateData, 59
- EquatorialEmissionOptions, 60
 - EquatorialEmissionOptions, 61
 - EquatPassUpperBound, 61
 - GeometricFudgeFactor, 61
 - RedShiftPower, 61
 - RLogScale, 61
 - TheEmissionModel, 61
 - TheFluidVelocityModel, 62
- EquatorialPassesDiagnostic, 62
 - DiagOptions, 65
 - EquatorialPassesDiagnostic, 63
 - FinalDataValDistance, 63
 - getFinalDataVal, 64
 - getFullDataStr, 64
 - getFullDescriptionStr, 64
 - getNameStr, 64
 - m_EquatPasses, 65
 - m_PrevTheta, 65
 - Reset, 65
 - UpdateData, 65
- EquatorialPassesOptions, 66
 - EquatorialPassesOptions, 67
 - Threshold, 67
- EquatPassUpperBound
 - EquatorialEmissionOptions, 61
- estimate_dx
 - BicubicSplineInterpolator, 25
- estimate_dxdy
 - BicubicSplineInterpolator, 25
- estimate_dy
 - BicubicSplineInterpolator, 25
- Exists
 - ConfigReader::ConfigCollection, 41
- ExpInt
 - SquareSubdivisionMesh, 166
 - SquareSubdivisionMeshV2, 173
- f_om_phi
 - ST3CrMetric, 181
- f_phi
 - ST3CrMetric, 181
- FinalDataValDistance
 - ClosestRadiusDiagnostic, 35
 - Dagnostic, 51
 - EquatorialEmissionDiagnostic, 58
 - EquatorialPassesDiagnostic, 63
 - FourColorScreenDiagnostic, 72
 - GeodesicPositionDiagnostic, 96
- find_closest
 - tk::spline, 161
- FindISCO
 - GeneralCircularRadialFluid, 76
- first_deriv
 - tk::spline, 160
- FlatSpaceMetric, 67
 - FlatSpaceMetric, 68
 - getFullDescriptionStr, 68
 - getMetric_dd, 68
 - getMetric_uu, 69
- FluidVelocityModel, 69
 - ~FluidVelocityModel, 70
 - FluidVelocityModel, 70
 - GetFourVelocityd, 70
 - getFullDescriptionStr, 70
 - m_theMetric, 71
- FOORT/src/Config.cpp, 203
- FOORT/src/Config.h, 203, 205
- FOORT/src/ConfigReader.cpp, 206
- FOORT/src/ConfigReader.h, 207, 208
- FOORT/src/Diagnostics.cpp, 210
- FOORT/src/Diagnostics.h, 212, 215
- FOORT/src/DiagnosticsEmission.cpp, 220

- FOORT/src/DiagnosticsEmission.h, 221, 222
- FOORT/src/Geodesic.cpp, 223
- FOORT/src/Geodesic.h, 224, 225
- FOORT/src/Geometry.h, 226, 234
- FOORT/src/Grid.cpp, 236
- FOORT/src/Grid.h, 236
- FOORT/src/InputOutput.cpp, 237
- FOORT/src/InputOutput.h, 239, 242
- FOORT/src/Integrators.cpp, 243
- FOORT/src/Integrators.h, 244, 245
- FOORT/src/Interpolator.cpp, 246
- FOORT/src/Interpolator.h, 246, 247
- FOORT/src/Main.cpp, 248
- FOORT/src/Mesh.cpp, 249
- FOORT/src/Mesh.h, 249, 250
- FOORT/src/Metric.cpp, 255
- FOORT/src/Metric.h, 255, 256
- FOORT/src/Spline.cpp, 260
- FOORT/src/Spline.h, 260, 261
- FOORT/src/Terminations.cpp, 263
- FOORT/src/Terminations.h, 264, 269
- FOORT/src/Utilities.cpp, 273
- FOORT/src/Utilities.h, 273, 275
- FOORT/src/ViewScreen.cpp, 275
- FOORT/src/ViewScreen.h, 276, 277
- FourColorScreenDiagnostic, 71
 - FinalDataValDistance, 72
 - FourColorScreenDiagnostic, 72
 - getFinalDataVal, 72
 - getFullDataStr, 73
 - getFullDescriptionStr, 73
 - getNameStr, 73
 - m_quadrant, 74
 - Reset, 73
 - UpdateData, 74
- FourIndex
 - Geometry.h, 228
- GeneralCircularRadialFluid, 74
 - FindISCO, 76
 - GeneralCircularRadialFluid, 76
 - GetChrstrRaisedDer, 76
 - GetCircularVelocityd, 77
 - GetFourVelocityd, 77
 - getFullDescriptionStr, 77
 - GetInsideISCOCircularVelocityd, 78
 - GetRadialVelocityd, 78
 - m_betaPhi, 78
 - m_betaR, 78
 - m_ISCOexists, 79
 - m_ISCOlowerbound, 79
 - m_ISCOpphi, 79
 - m_ISCOpt, 79
 - m_ISCO, 79
 - m_ISCOupperbound, 79
 - m_subKeplerParam, 79
- GeneralSingularity
 - Terminations.h, 267
- GeneralSingularityTermination, 80
 - CheckTermination, 81
 - GeneralSingularityTermination, 81
 - getFullDescriptionStr, 81
 - SingularityToString, 82
 - TermOptions, 82
- GeneralSingularityTermOptions, 82
 - Epsilon, 83
 - GeneralSingularityTermOptions, 83
 - OutputToConsole, 83
 - rLogScale, 84
 - Singularities, 84
- Geodesic, 84
 - Geodesic, 85
 - getAllOutputStr, 86
 - getCurrentLambda, 86
 - getCurrentPos, 86
 - getCurrentVel, 86
 - getDiagnosticFinalValue, 87
 - getScreenIndex, 87
 - getTermCondition, 87
 - m_AllDiagnostics, 88
 - m_AllTerminations, 88
 - m_curLambda, 88
 - m_CurrentPos, 88
 - m_CurrentVel, 88
 - m_ScreenIndex, 89
 - m_TermCond, 89
 - m_theIntegrator, 89
 - m_theMetric, 89
 - m_theSource, 89
 - operator=, 87
 - Reset, 87
 - Update, 88
- GeodesicFinished
 - InputCertainPixelsMesh, 110
 - Mesh, 128
 - SimpleSquareMesh, 151
 - SquareSubdivisionMesh, 167
 - SquareSubdivisionMeshV2, 174
 - ViewScreen, 199
- GeodesicIntegratorFunc
 - Integrators.h, 245
- GeodesicOutputHandler, 89
 - GeodesicOutputHandler, 91
 - GetFileName, 91
 - getFullDescriptionStr, 91
 - m_AllCachedData, 93
 - m_CurrentFullFiles, 93
 - m_CurrentGeodesicsInFile, 93
 - m_DiagNames, 93
 - m_FileExtension, 93
 - m_FilePrefix, 93
 - m_FirstLineInfoString, 94
 - m_nrGeodesicsPerFile, 94
 - m_nrOutputsToCache, 94
 - m_PrevCached, 94
 - m_PrintFirstLineInfo, 94
 - m_TimeStamp, 94

- m_WriteToConsole, 94
 - NewGeodesicOutput, 92
 - OpenForFirstTime, 92
 - OutputFinished, 92
 - PrepareForOutput, 92
 - WriteCachedOutputToFile, 93
- GeodesicPositionDiagnostic, 95
 - DiagOptions, 98
 - FinalDataValDistance, 96
 - GeodesicPositionDiagnostic, 96
 - getFinalDataVal, 97
 - getFullDataStr, 97
 - getFullDescriptionStr, 97
 - getNameStr, 97
 - m_AllSavedPoints, 98
 - Reset, 98
 - UpdateData, 98
- GeodesicPositionOptions, 99
 - GeodesicPositionOptions, 99
 - OutputNrSteps, 100
- GeodesicType
 - ViewScreen.h, 277
- GeometricFudgeFactor
 - EquatorialEmissionOptions, 61
- Geometry.h
 - dimension, 234
 - FourIndex, 228
 - largecounter, 228
 - LARGECOUNTER_MAX, 228
 - OneIndex, 229
 - operator+, 231
 - operator-, 231
 - operator/, 232
 - operator*, 230
 - pi, 234
 - PIXEL_MAX, 228
 - pixelcoord, 229
 - Point, 229
 - real, 229
 - ScreenIndex, 229
 - ScreenPoint, 229
 - Singularity, 229
 - SingularityCoord, 229
 - ThreeIndex, 230
 - toString, 232, 233
 - TwoIndex, 230
- get_omega
 - ST3CrMetric, 182
- get_x
 - tk::spline, 161
- get_x_max
 - tk::spline, 161
- get_x_min
 - tk::spline, 161
- get_y
 - tk::spline, 162
- GetAdaptiveStep
 - Integrators, 16
- getAllOutputStr
 - Geodesic, 86
- getChristoffel_udd
 - Metric, 130
- GetChrstrRaisedDer
 - GeneralCircularRadialFluid, 76
- GetCircularVelocityd
 - GeneralCircularRadialFluid, 77
- getCurNrGeodesics
 - InputCertainPixelsMesh, 110
 - Mesh, 128
 - SimpleSquareMesh, 152
 - SquareSubdivisionMesh, 167
 - SquareSubdivisionMeshV2, 174
 - ViewScreen, 199
- getCurrentLambda
 - Geodesic, 86
- getCurrentPos
 - Geodesic, 86
- getCurrentVel
 - Geodesic, 86
- GetDiagNameStrings
 - Utilities, 19
- getDiagnosticFinalValue
 - Geodesic, 87
- GetDown
 - SquareSubdivisionMeshV2, 174
- GetEmission
 - EmissionModel, 55
 - GLMJohnsonSUEmission, 101
- GetFileName
 - GeodesicOutputHandler, 91
- getFinalDataVal
 - ClosestRadiusDiagnostic, 35
 - Dagnostic, 52
 - EquatorialEmissionDiagnostic, 58
 - EquatorialPassesDiagnostic, 64
 - FourColorScreenDiagnostic, 72
 - GeodesicPositionDiagnostic, 97
- GetFirstLineInfoString
 - Utilities, 20
- GetFourVelocityd
 - FluidVelocityModel, 70
 - GeneralCircularRadialFluid, 77
- getFullDataStr
 - ClosestRadiusDiagnostic, 36
 - Dagnostic, 52
 - EquatorialEmissionDiagnostic, 58
 - EquatorialPassesDiagnostic, 64
 - FourColorScreenDiagnostic, 73
 - GeodesicPositionDiagnostic, 97
- getFullDescriptionStr
 - BosonStarMetric, 28
 - BoundarySphereTermination, 31
 - ClosestRadiusDiagnostic, 36
 - Dagnostic, 52
 - EmissionModel, 55
 - EquatorialEmissionDiagnostic, 58

- EquatorialPassesDiagnostic, 64
- FlatSpaceMetric, 68
- FluidVelocityModel, 70
- FourColorScreenDiagnostic, 73
- GeneralCircularRadialFluid, 77
- GeneralSingularityTermination, 81
- GeodesicOutputHandler, 91
- GeodesicPositionDiagnostic, 97
- GLMJohnsonSUEmission, 101
- HorizonTermination, 106
- InputCertainPixelsMesh, 110
- JohannsenMetric, 114
- KerrMetric, 119
- KerrSchildMetric, 122
- MankoNovikovMetric, 125
- Mesh, 128
- Metric, 131
- NaNTermination, 134
- NoSource, 137
- RasheedLarsenMetric, 144
- RotatingBosonStarMetric, 148
- SimpleSquareMesh, 152
- Source, 156
- SquareSubdivisionMesh, 167
- SquareSubdivisionMeshV2, 174
- ST3CrMetric, 182
- Termination, 185
- ThetaSingularityTermination, 189
- TimeOutTermination, 192
- ViewScreen, 199
- GetFullIntegratorDescription
 - Integrators, 17
- GetGeodesicIntegrator
 - Config, 12
- getHorizonRadius
 - SphericalHorizonMetric, 159
- GetInsideISCOCircularVelocityd
 - GeneralCircularRadialFluid, 78
- getKretschmann
 - Metric, 131
- GetLeft
 - SquareSubdivisionMeshV2, 175
- GetLoopMessageFrequency
 - InputOutput.cpp, 238
 - InputOutput.h, 240
- GetMesh
 - Config, 12
- GetMetric
 - Config, 13
- getMetric_dd
 - BosonStarMetric, 28
 - FlatSpaceMetric, 68
 - JohannsenMetric, 114
 - KerrMetric, 119
 - KerrSchildMetric, 122
 - MankoNovikovMetric, 125
 - Metric, 131
 - RasheedLarsenMetric, 144
 - RotatingBosonStarMetric, 148
 - ST3CrMetric, 182
- getMetric_uu
 - BosonStarMetric, 28
 - FlatSpaceMetric, 69
 - JohannsenMetric, 116
 - KerrMetric, 119
 - KerrSchildMetric, 123
 - MankoNovikovMetric, 126
 - Metric, 131
 - RasheedLarsenMetric, 145
 - RotatingBosonStarMetric, 148
 - ST3CrMetric, 183
- getNameStr
 - ClosestRadiusDiagnostic, 36
 - Diagnostic, 52
 - EquatorialEmissionDiagnostic, 59
 - EquatorialPassesDiagnostic, 64
 - FourColorScreenDiagnostic, 73
 - GeodesicPositionDiagnostic, 97
- getNewInitConds
 - InputCertainPixelsMesh, 111
 - Mesh, 128
 - SimpleSquareMesh, 152
 - SquareSubdivisionMesh, 167
 - SquareSubdivisionMeshV2, 175
- GetOutputHandler
 - Config, 13
- GetRadialVelocityd
 - GeneralCircularRadialFluid, 78
- getRiemann_uddd
 - Metric, 132
- GetRight
 - SquareSubdivisionMeshV2, 175
- getrLogScale
 - Metric, 132
- getScreenIndex
 - Geodesic, 87
- GetSettingIndex
 - ConfigReader::ConfigCollection, 41
- getSingularity
 - SingularityMetric, 155
- GetSource
 - Config, 13
- getSource
 - NoSource, 137
 - Source, 156
- getTermCondition
 - Geodesic, 87
- GetTimeStampString
 - Utilities, 20
- GetUp
 - SquareSubdivisionMeshV2, 176
- GetViewScreen
 - Config, 14
- GLMJohnsonSUEmission, 100
 - GetEmission, 101
 - getFullDescriptionStr, 101

- GLMJohnsonSUEmission, 101
- m_gamma, 102
- m_mu, 102
- m_sigma, 102
- Grid, 102
 - ~Grid, 103
 - data, 104
 - Grid, 103
 - initialize_from_file, 103
 - N_col, 104
 - N_row, 104
 - operator(), 103
 - size, 104
- Horizon
 - Terminations.h, 267
- HorizonRadius
 - HorizonTermOptions, 107
- HorizonTermination, 104
 - CheckTermination, 106
 - getFullDescriptionStr, 106
 - HorizonTermination, 105
 - TermOptions, 106
- HorizonTermOptions, 106
 - AtHorizonEps, 107
 - HorizonRadius, 107
 - HorizonTermOptions, 107
 - rLogScale, 108
- Index
 - SquareSubdivisionMesh::PixelInfo, 139
 - SquareSubdivisionMeshV2::PixelInfo, 141
- initialize_from_file
 - Grid, 103
- InitializeDiagnostics
 - Config, 14
- InitializeFirstGrid
 - SquareSubdivisionMesh, 168
 - SquareSubdivisionMeshV2, 176
- InitializeScreenOutput
 - Config, 14
- InitializeTerminations
 - Config, 15
- InputCertainPixelsMesh, 108
 - EndCurrentLoop, 110
 - GeodesicFinished, 110
 - getCurNrGeodesics, 110
 - getFullDescriptionStr, 110
 - getNewInitConds, 111
 - InputCertainPixelsMesh, 109
 - IsFinished, 111
 - m_Finished, 111
 - m_PixelsToIntegrate, 111
 - m_RowColumnSize, 112
 - m_TotalPixels, 112
- InputOutput.cpp
 - GetLoopMessageFrequency, 238
 - ScreenOutput, 238
 - SetLoopMessageFrequency, 238
 - SetOutputLevel, 238
- InputOutput.h
 - GetLoopMessageFrequency, 240
 - Level_0_WARNING, 240
 - Level_1_PROC, 240
 - Level_2_SUBPROC, 240
 - Level_3_ALLDDETAIL, 240
 - Level_4_DEBUG, 240
 - MaxLevel, 240
 - OutputLevel, 240
 - ScreenOutput, 240
 - SetLoopMessageFrequency, 241
 - SetOutputLevel, 241
- IntegrateGeodesicStep_RK4
 - Integrators, 17
- IntegrateGeodesicStep_Verlet
 - Integrators, 17
- IntegratorDescription
 - Integrators, 18
- Integrators, 16
 - delta_nodiv0, 18
 - Derivative_hval, 18
 - epsilon, 18
 - GetAdaptiveStep, 16
 - GetFullIntegratorDescription, 17
 - IntegrateGeodesicStep_RK4, 17
 - IntegrateGeodesicStep_Verlet, 17
 - IntegratorDescription, 18
 - SmallestPossibleStepsize, 18
 - VerletVelocityTolerance, 18
- Integrators.h
 - GeodesicIntegratorFunc, 245
- interpolate
 - BicubicSplineInterpolator, 25
- IsCollection
 - ConfigReader::ConfigCollection, 42
- IsFinished
 - InputCertainPixelsMesh, 111
 - Mesh, 129
 - SimpleSquareMesh, 152
 - SquareSubdivisionMesh, 168
 - SquareSubdivisionMeshV2, 176
 - ViewScreen, 199
- JohannsenMetric, 112
 - getFullDescriptionStr, 114
 - getMetric_dd, 114
 - getMetric_uu, 116
 - JohannsenMetric, 114
 - m_alpha13Param, 116
 - m_alpha22Param, 116
 - m_alpha52Param, 116
 - m_aParam, 116
 - m_eps3Param, 117
- KerrMetric, 117
 - getFullDescriptionStr, 119
 - getMetric_dd, 119
 - getMetric_uu, 119

- KerrMetric, 119
- m_aParam, 120
- m_mParam, 120
- KerrSchildMetric, 120
 - getFullDescriptionStr, 122
 - getMetric_dd, 122
 - getMetric_uu, 123
 - KerrSchildMetric, 122
 - m_aParam, 123
- l_solve
 - tk::internal::band_matrix, 22
- largecounter
 - Geometry.h, 228
- LARGECOUNTER_MAX
 - Geometry.h, 228
- LeftNbr
 - SquareSubdivisionMeshV2::PixelInfo, 141
- Level_0_WARNING
 - InputOutput.h, 240
- Level_1_PROC
 - InputOutput.h, 240
- Level_2_SUBPROC
 - InputOutput.h, 240
- Level_3_ALLDETAIL
 - InputOutput.h, 240
- Level_4_DEBUG
 - InputOutput.h, 240
- linear
 - tk::spline, 161
- LookupValue
 - ConfigReader::ConfigCollection, 42, 43
- LookupValueInteger
 - ConfigReader::ConfigCollection, 43–45
- LowerNbrIndex
 - SquareSubdivisionMesh::PixelInfo, 139
- lu_decompose
 - tk::internal::band_matrix, 22
- lu_solve
 - tk::internal::band_matrix, 22
- m_ActivePixels
 - SquareSubdivisionMeshV2, 177
- m_AllCachedData
 - GeodesicOutputHandler, 93
- m_AllDiagnostics
 - Geodesic, 88
- m_AllPixels
 - SquareSubdivisionMesh, 169
 - SquareSubdivisionMeshV2, 177
- m_AllSavedPoints
 - GeodesicPositionDiagnostic, 98
- m_AllSingularities
 - SingularityMetric, 155
- m_AllTerminations
 - Geodesic, 88
- m_alpha13Param
 - JohannsenMetric, 116
- m_alpha22Param
 - JohannsenMetric, 116
- m_alpha3Param
 - MankoNovikovMetric, 126
- m_alpha52Param
 - JohannsenMetric, 116
- m_alphaParam
 - MankoNovikovMetric, 126
- m_aParam
 - JohannsenMetric, 116
 - KerrMetric, 120
 - KerrSchildMetric, 123
 - MankoNovikovMetric, 126
 - RasheedLarsenMetric, 145
- m_b
 - tk::spline, 162
- m_beg
 - Utilities::Timer, 196
- m_betaPhi
 - GeneralCircularRadialFluid, 78
- m_betaR
 - GeneralCircularRadialFluid, 78
- m_c
 - tk::spline, 162
- m_c0
 - tk::spline, 163
- m_ClosestRadius
 - ClosestRadiusDiagnostic, 37
- m_coeffs
 - BicubicSplineInterpolator, 25
- m_curLambda
 - Geodesic, 88
- m_CurNrSteps
 - TimeOutTermination, 193
- m_CurrentFullFiles
 - GeodesicOutputHandler, 93
- m_CurrentGeodesicsInFile
 - GeodesicOutputHandler, 93
- m_CurrentPixelQueue
 - SquareSubdivisionMesh, 169
 - SquareSubdivisionMeshV2, 177
- m_CurrentPixelQueueDone
 - SquareSubdivisionMesh, 169
 - SquareSubdivisionMeshV2, 177
- m_CurrentPixelUpdating
 - SquareSubdivisionMeshV2, 177
- m_CurrentPos
 - Geodesic, 88
- m_CurrentVel
 - Geodesic, 88
- m_d
 - tk::spline, 163
- m_DiagNames
 - GeodesicOutputHandler, 93
- m_Direction
 - ViewScreen, 200
- m_DistanceDiagnostic
 - Mesh, 129
- m_eps3Param

- JohannsenMetric, [117](#)
- m_EquatPasses
 - EquatorialPassesDiagnostic, [65](#)
- m_FileExtension
 - GeodesicOutputHandler, [93](#)
- m_FilePrefix
 - GeodesicOutputHandler, [93](#)
- m_Finished
 - InputCertainPixelsMesh, [111](#)
 - SimpleSquareMesh, [153](#)
- m_fInterpolator
 - RotatingBosonStarMetric, [149](#)
- m_FirstLineInfoString
 - GeodesicOutputHandler, [94](#)
- m_gamma
 - GLMJohnsonSUEmission, [102](#)
- m_GeodType
 - ViewScreen, [200](#)
- m_gInterpolator
 - RotatingBosonStarMetric, [149](#)
- m_grid_f
 - RotatingBosonStarMetric, [149](#)
- m_grid_g
 - RotatingBosonStarMetric, [149](#)
- m_grid_l
 - RotatingBosonStarMetric, [149](#)
- m_grid_Omega
 - RotatingBosonStarMetric, [149](#)
- m_HorizonRadius
 - SphericalHorizonMetric, [159](#)
- m_InfinitePixels
 - SquareSubdivisionMesh, [169](#)
 - SquareSubdivisionMeshV2, [178](#)
- m_InitialPixels
 - SquareSubdivisionMesh, [169](#)
 - SquareSubdivisionMeshV2, [178](#)
- m_InitialSubDivideToFinal
 - SquareSubdivisionMesh, [169](#)
 - SquareSubdivisionMeshV2, [178](#)
- m_Intensity
 - EquatorialEmissionDiagnostic, [59](#)
- m_ISCOexists
 - GeneralCircularRadialFluid, [79](#)
- m_ISCOlowerbound
 - GeneralCircularRadialFluid, [79](#)
- m_ISCOpphi
 - GeneralCircularRadialFluid, [79](#)
- m_ISCOpt
 - GeneralCircularRadialFluid, [79](#)
- m_ISCO_r
 - GeneralCircularRadialFluid, [79](#)
- m_ISCOupperbound
 - GeneralCircularRadialFluid, [79](#)
- m_IterationPixels
 - SquareSubdivisionMesh, [170](#)
 - SquareSubdivisionMeshV2, [178](#)
- m_kParam
 - MankoNovikovMetric, [126](#)
- m_lambda
 - ST3CrMetric, [183](#)
- m_left
 - tk::spline, [163](#)
- m_left_value
 - tk::spline, [163](#)
- m_lInterpolator
 - RotatingBosonStarMetric, [149](#)
- m_lower
 - tk::internal::band_matrix, [23](#)
- m_m_filename
 - BosonStarMetric, [29](#)
- m_made_monotonic
 - tk::spline, [163](#)
- m_MaxPixels
 - SquareSubdivisionMesh, [170](#)
 - SquareSubdivisionMeshV2, [178](#)
- m_MaxSubdivide
 - SquareSubdivisionMesh, [170](#)
 - SquareSubdivisionMeshV2, [178](#)
- m_Metric_dd
 - ViewScreen, [200](#)
- m_mParam
 - KerrMetric, [120](#)
 - RasheedLarsenMetric, [145](#)
- m_mSpline
 - BosonStarMetric, [29](#)
- m_mu
 - GLMJohnsonSUEmission, [102](#)
- m_nrGeodesicsPerFile
 - GeodesicOutputHandler, [94](#)
- m_nrOutputsToCache
 - GeodesicOutputHandler, [94](#)
- m_num_lines
 - BosonStarMetric, [29](#)
- m_OmegaInterpolator
 - RotatingBosonStarMetric, [149](#)
- m_OmegaSign
 - RotatingBosonStarMetric, [149](#)
- m_OwnerGeodesic
 - Diagnostic, [53](#)
 - Termination, [186](#)
- m_P
 - ST3CrMetric, [183](#)
- m_Phi_filename
 - BosonStarMetric, [29](#)
- m_Phi_infinity
 - BosonStarMetric, [29](#)
- m_PhiSpline
 - BosonStarMetric, [30](#)
- m_PixelsIntegrated
 - SquareSubdivisionMeshV2, [178](#)
- m_PixelsLeft
 - SquareSubdivisionMesh, [170](#)
 - SquareSubdivisionMeshV2, [178](#)
- m_PixelsToIntegrate
 - InputCertainPixelsMesh, [111](#)
- m_Pos

- ViewScreen, 200
- m_pParam
 - RasheedLarsenMetric, 145
- m_PrevCached
 - GeodesicOutputHandler, 94
- m_PrevTheta
 - EquatorialPassesDiagnostic, 65
- m_PrintFirstLineInfo
 - GeodesicOutputHandler, 94
- m_q0
 - ST3CrMetric, 183
- m_qParam
 - RasheedLarsenMetric, 145
- m_quadrant
 - FourColorScreenDiagnostic, 74
- m_right
 - tk::spline, 163
- m_right_value
 - tk::spline, 163
- m_rLogScale
 - Metric, 132
 - ViewScreen, 201
- m_RowColumnSize
 - InputCertainPixelsMesh, 112
 - SimpleSquareMesh, 153
 - SquareSubdivisionMesh, 170
 - SquareSubdivisionMeshV2, 179
- m_ScreenCenter
 - ViewScreen, 201
- m_ScreenIndex
 - Geodesic, 89
- m_ScreenSize
 - ViewScreen, 201
- m_Settings
 - ConfigReader::ConfigCollection, 48
- m_settingtrace
 - ConfigReader::ConfigReaderException, 49
- m_sigma
 - GLMJohnsonSUEmission, 102
- m_StepsSinceUpdated
 - Diagnostic, 53
 - Termination, 186
- m_subKeplerParam
 - GeneralCircularRadialFluid, 79
- m_Symmetries
 - Metric, 132
- m_TermCond
 - Geodesic, 89
- m_theIntegrator
 - Geodesic, 89
- m_theMesh
 - ViewScreen, 201
- m_theMetric
 - FluidVelocityModel, 71
 - Geodesic, 89
 - Source, 157
 - ViewScreen, 201
- m_theSource
 - Geodesic, 89
- m_theta
 - BicubicSplineInterpolator, 25
- m_TimeStamp
 - GeodesicOutputHandler, 94
- m_TotalPixels
 - InputCertainPixelsMesh, 112
 - SimpleSquareMesh, 153
- m_type
 - tk::spline, 163
- m_upper
 - tk::internal::band_matrix, 23
- m_Vielbein
 - ViewScreen, 201
- m_WriteToConsole
 - GeodesicOutputHandler, 94
- m_x
 - BicubicSplineInterpolator, 26
 - tk::spline, 163
- m_y
 - tk::spline, 163
- main
 - Main.cpp, 248
- Main.cpp
 - main, 248
- make_monotonic
 - tk::spline, 162
- MankoNovikovMetric, 123
 - getFullDescriptionStr, 125
 - getMetric_dd, 125
 - getMetric_uu, 126
 - m_alpha3Param, 126
 - m_alphaParam, 126
 - m_aParam, 126
 - m_kParam, 126
 - MankoNovikovMetric, 125
- MaxLevel
 - InputOutput.h, 240
- MaxSteps
 - TimeOutTermOptions, 194
- MaxStreamSize
 - ConfigReader, 16
- Maxterms
 - Terminations.h, 267
- Mesh, 127
 - ~Mesh, 128
 - EndCurrentLoop, 128
 - GeodesicFinished, 128
 - getCurNrGeodesics, 128
 - getFullDescriptionStr, 128
 - getNewInitConds, 128
 - IsFinished, 129
 - m_DistanceDiagnostic, 129
 - Mesh, 128
- Metric, 129
 - ~Metric, 130
 - getChristoffel_udd, 130
 - getFullDescriptionStr, 131

- getKretschmann, 131
- getMetric_dd, 131
- getMetric_uu, 131
- getRiemann_uddd, 132
- getrLogScale, 132
- m_rLogScale, 132
- m_Symmetries, 132
- Metric, 130
- N_col
 - Grid, 104
- N_row
 - Grid, 104
- NaN
 - Terminations.h, 267
- NaNTermination, 133
 - CheckTermination, 134
 - getFullDescriptionStr, 134
 - NaNTermination, 134
 - TermOptions, 135
- NaNTermOptions, 135
 - NaNTermOptions, 136
 - OutputToConsole, 136
- NewGeodesicOutput
 - GeodesicOutputHandler, 92
- NoSource, 136
 - getFullDescriptionStr, 137
 - getSource, 137
 - NoSource, 137
- NrSettings
 - ConfigReader::ConfigCollection, 46
- Null
 - ViewScreen.h, 277
- num_lower
 - tk::internal::band_matrix, 22
- num_upper
 - tk::internal::band_matrix, 22
- OneIndex
 - Geometry.h, 229
- OpenForFirstTime
 - GeodesicOutputHandler, 92
- operator()
 - Grid, 103
 - tk::internal::band_matrix, 22
 - tk::spline, 162
- operator+
 - Geometry.h, 231
- operator-
 - Geometry.h, 231
- operator/
 - Geometry.h, 232
- operator=
 - Geodesic, 87
- operator[]
 - ConfigReader::ConfigCollection, 46
- operator*
 - Geometry.h, 230
- Output_Important_Default
 - Config, 15
- Output_Other_Default
 - Config, 15
- OutputFinished
 - GeodesicOutputHandler, 92
- OutputLevel
 - InputOutput.h, 240
- OutputNrSteps
 - GeodesicPositionOptions, 100
- OutputToConsole
 - GeneralSingularityTermOptions, 83
 - NaNTermOptions, 136
- pi
 - Geometry.h, 234
- PIXEL_MAX
 - Geometry.h, 228
- pixelcoord
 - Geometry.h, 229
- PixelInfo
 - SquareSubdivisionMesh::PixelInfo, 139
 - SquareSubdivisionMeshV2::PixelInfo, 141
- Point
 - Geometry.h, 229
- PrepareForOutput
 - GeodesicOutputHandler, 92
- r_solve
 - tk::internal::band_matrix, 23
- RasheedLarsenMetric, 142
 - getFullDescriptionStr, 144
 - getMetric_dd, 144
 - getMetric_uu, 145
 - m_aParam, 145
 - m_mParam, 145
 - m_pParam, 145
 - m_qParam, 145
 - RasheedLarsenMetric, 144
- read_data
 - BosonStarMetric, 29
- ReadCollection
 - ConfigReader::ConfigCollection, 47
- ReadFile
 - ConfigReader::ConfigCollection, 47
- ReadSettingName
 - ConfigReader::ConfigCollection, 47
- ReadSettingSpecificChar
 - ConfigReader::ConfigCollection, 48
- ReadSettingValue
 - ConfigReader::ConfigCollection, 48
- real
 - Geometry.h, 229
- RedShiftPower
 - EquatorialEmissionOptions, 61
- Reset
 - ClosestRadiusDiagnostic, 36
 - Diagnostic, 52
 - EquatorialEmissionDiagnostic, 59
 - EquatorialPassesDiagnostic, 65

- FourColorScreenDiagnostic, 73
- Geodesic, 87
- GeodesicPositionDiagnostic, 98
- Termination, 185
- TimeOutTermination, 192
- reset
 - Utilities::Timer, 195
- resize
 - tk::internal::band_matrix, 23
- RightNbr
 - SquareSubdivisionMeshV2::PixelInfo, 141
- RightNbrIndex
 - SquareSubdivisionMesh::PixelInfo, 139
- RLogScale
 - ClosestRadiusOptions, 38
 - EquatorialEmissionOptions, 61
- rLogScale
 - BoundarySphereTermOptions, 33
 - GeneralSingularityTermOptions, 84
 - HorizonTermOptions, 108
- RotatingBosonStarMetric, 146
 - ~RotatingBosonStarMetric, 147
 - getFullDescriptionStr, 148
 - getMetric_dd, 148
 - getMetric_uu, 148
 - m_fInterpolator, 149
 - m_gInterpolator, 149
 - m_grid_f, 149
 - m_grid_g, 149
 - m_grid_l, 149
 - m_grid_Omega, 149
 - m_lInterpolator, 149
 - m_OmegaInterpolator, 149
 - m_OmegaSign, 149
 - RotatingBosonStarMetric, 147
- saved_diag
 - tk::internal::band_matrix, 23
- ScreenIndex
 - Geometry.h, 229
- ScreenOutput
 - InputOutput.cpp, 238
 - InputOutput.h, 240
- ScreenPoint
 - Geometry.h, 229
- Second
 - Utilities::Timer, 195
- second_deriv
 - tk::spline, 160
- SEdiagNbr
 - SquareSubdivisionMeshV2::PixelInfo, 141
- set_boundary
 - tk::spline, 162
- set_coeffs_from_b
 - tk::spline, 162
- set_grid
 - BicubicSplineInterpolator, 25
- set_points
 - tk::spline, 162
- SetLoopMessageFrequency
 - InputOutput.cpp, 238
 - InputOutput.h, 241
- SetNewInitialConditions
 - ViewScreen, 200
- SetOutputLevel
 - InputOutput.cpp, 238
 - InputOutput.h, 241
- SettingError
 - Config, 12
- SettingName
 - ConfigReader::ConfigCollection::ConfigSetting, 50
- SettingValue
 - ConfigReader::ConfigCollection::ConfigSetting, 50
- SimpleSquareMesh, 150
 - EndCurrentLoop, 151
 - GeodesicFinished, 151
 - getCurNrGeodesics, 152
 - getFullDescriptionStr, 152
 - getNewInitConds, 152
 - IsFinished, 152
 - m_Finished, 153
 - m_RowColumnSize, 153
 - m_TotalPixels, 153
 - SimpleSquareMesh, 151
- Singularities
 - GeneralSingularityTermOptions, 84
- Singularity
 - Geometry.h, 229
- SingularityCoord
 - Geometry.h, 229
- SingularityMetric, 153
 - getSingularities, 155
 - m_AllSingularities, 155
 - SingularityMetric, 154
- SingularityToString
 - GeneralSingularityTermination, 82
- size
 - Grid, 104
- SmallestPossibleStepsize
 - Integrators, 18
- Source, 155
 - ~Source, 156
 - getFullDescriptionStr, 156
 - getSource, 156
 - m_theMetric, 157
 - Source, 156
- Spacelike
 - ViewScreen.h, 277
- SphereRadius
 - BoundarySphereTermOptions, 33
- SphericalHorizonMetric, 157
 - getHorizonRadius, 159
 - m_HorizonRadius, 159
 - SphericalHorizonMetric, 158
- spline
 - tk::spline, 161
- spline_type

- tk::spline, 160
- SquareSubdivisionMesh, 164
 - EndCurrentLoop, 166
 - Explnt, 166
 - GeodesicFinished, 167
 - getCurNrGeodesics, 167
 - getFullDescriptionStr, 167
 - getNewInitConds, 167
 - InitializeFirstGrid, 168
 - IsFinished, 168
 - m_AllPixels, 169
 - m_CurrentPixelQueue, 169
 - m_CurrentPixelQueueDone, 169
 - m_InfinitePixels, 169
 - m_InitialPixels, 169
 - m_InitialSubDivideToFinal, 169
 - m_IterationPixels, 170
 - m_MaxPixels, 170
 - m_MaxSubdivide, 170
 - m_PixelsLeft, 170
 - m_RowColumnSize, 170
 - SquareSubdivisionMesh, 166
 - SubdivideAndQueue, 168
 - UpdateAllNeighbors, 168
 - UpdateAllWeights, 168
- SquareSubdivisionMesh::PixelInfo, 138
 - DiagValue, 139
 - Index, 139
 - LowerNbrIndex, 139
 - PixelInfo, 139
 - RightNbrIndex, 139
 - SubdivideLevel, 139
 - Weight, 139
- SquareSubdivisionMeshV2, 171
 - EndCurrentLoop, 173
 - Explnt, 173
 - GeodesicFinished, 174
 - getCurNrGeodesics, 174
 - GetDown, 174
 - getFullDescriptionStr, 174
 - GetLeft, 175
 - getNewInitConds, 175
 - GetRight, 175
 - GetUp, 176
 - InitializeFirstGrid, 176
 - IsFinished, 176
 - m_ActivePixels, 177
 - m_AllPixels, 177
 - m_CurrentPixelQueue, 177
 - m_CurrentPixelQueueDone, 177
 - m_CurrentPixelUpdating, 177
 - m_InfinitePixels, 178
 - m_InitialPixels, 178
 - m_InitialSubDivideToFinal, 178
 - m_IterationPixels, 178
 - m_MaxPixels, 178
 - m_MaxSubdivide, 178
 - m_PixelsIntegrated, 178
 - m_PixelsLeft, 178
 - m_RowColumnSize, 179
 - SquareSubdivisionMeshV2, 173
 - SubdivideAndQueue, 176
 - UpdateAllWeights, 177
- SquareSubdivisionMeshV2::PixelInfo, 140
 - DiagValue, 141
 - DownNbr, 141
 - Index, 141
 - LeftNbr, 141
 - PixelInfo, 141
 - RightNbr, 141
 - SEdiagNbr, 141
 - SubdivideLevel, 141
 - UpNbr, 142
 - Weight, 142
- ST3CrMetric, 179
 - f_om_phi, 181
 - f_phi, 181
 - get_omega, 182
 - getFullDescriptionStr, 182
 - getMetric_dd, 182
 - getMetric_uu, 183
 - m_lambda, 183
 - m_P, 183
 - m_q0, 183
 - ST3CrMetric, 181
- SubdivideAndQueue
 - SquareSubdivisionMesh, 168
 - SquareSubdivisionMeshV2, 176
- SubdivideLevel
 - SquareSubdivisionMesh::PixelInfo, 139
 - SquareSubdivisionMeshV2::PixelInfo, 141
- Term
 - Terminations.h, 267
- Term_BoundarySphere
 - Terminations.h, 268
- Term_GeneralSingularity
 - Terminations.h, 268
- Term_Horizon
 - Terminations.h, 268
- Term_NaN
 - Terminations.h, 268
- Term_None
 - Terminations.h, 268
- Term_ThetaSingularity
 - Terminations.h, 268
- Term_TimeOut
 - Terminations.h, 268
- TermBitflag
 - Terminations.h, 266
- Termination, 184
 - ~Termination, 185
 - CheckTermination, 185
 - DecideUpdate, 185
 - getFullDescriptionStr, 185
 - m_OwnerGeodesic, 186
 - m_StepsSinceUpdated, 186

- Reset, 185
- Termination, 184
- TerminationOptions, 186
 - ~TerminationOptions, 187
 - TerminationOptions, 187
 - UpdateEveryNSteps, 187
- Terminations.cpp
 - CreateTerminationVector, 264
- Terminations.h
 - BoundarySphere, 267
 - Continue, 267
 - CreateTerminationVector, 267
 - GeneralSingularity, 267
 - Horizon, 267
 - Maxterms, 267
 - NaN, 267
 - Term, 267
 - Term_BoundarySphere, 268
 - Term_GeneralSingularity, 268
 - Term_Horizon, 268
 - Term_NaN, 268
 - Term_None, 268
 - Term_ThetaSingularity, 268
 - Term_TimeOut, 268
 - TermBitflag, 266
 - TerminationUniqueVector, 266
 - ThetaSingularity, 267
 - TimeOut, 267
 - Uninitialized, 267
- TerminationUniqueVector
 - Terminations.h, 266
- TermOptions
 - BoundarySphereTermination, 32
 - GeneralSingularityTermination, 82
 - HorizonTermination, 106
 - NaNTermination, 135
 - ThetaSingularityTermination, 189
 - TimeOutTermination, 193
- TheEmissionModel
 - EquatorialEmissionOptions, 61
- TheFluidVelocityModel
 - EquatorialEmissionOptions, 62
- ThetaSingEpsilon
 - ThetaSingularityTermOptions, 190
- ThetaSingularity
 - Terminations.h, 267
- ThetaSingularityTermination, 187
 - CheckTermination, 189
 - getFullDescriptionStr, 189
 - TermOptions, 189
 - ThetaSingularityTermination, 188
- ThetaSingularityTermOptions, 189
 - ThetaSingEpsilon, 190
 - ThetaSingularityTermOptions, 190
- theUpdateFrequency
 - DiagnosticOptions, 54
- ThreeIndex
 - Geometry.h, 230
- Threshold
 - EquatorialPassesOptions, 67
- Timelike
 - ViewScreen.h, 277
- TimeOut
 - Terminations.h, 267
- TimeOutTermination, 191
 - CheckTermination, 192
 - getFullDescriptionStr, 192
 - m_CurNrSteps, 193
 - Reset, 192
 - TermOptions, 193
 - TimeOutTermination, 192
- TimeOutTermOptions, 193
 - MaxSteps, 194
 - TimeOutTermOptions, 194
- tk, 19
- tk::internal, 19
- tk::internal::band_matrix, 21
 - ~band_matrix, 22
 - band_matrix, 21
 - dim, 22
 - l_solve, 22
 - lu_decompose, 22
 - lu_solve, 22
 - m_lower, 23
 - m_upper, 23
 - num_lower, 22
 - num_upper, 22
 - operator(), 22
 - r_solve, 23
 - resize, 23
 - saved_diag, 23
- tk::spline, 159
 - bd_type, 160
 - cspline, 161
 - cspline_hermite, 161
 - deriv, 161
 - find_closest, 161
 - first_deriv, 160
 - get_x, 161
 - get_x_max, 161
 - get_x_min, 161
 - get_y, 162
 - linear, 161
 - m_b, 162
 - m_c, 162
 - m_c0, 163
 - m_d, 163
 - m_left, 163
 - m_left_value, 163
 - m_made_monotonic, 163
 - m_right, 163
 - m_right_value, 163
 - m_type, 163
 - m_x, 163
 - m_y, 163
 - make_monotonic, 162

- operator(), 162
- second_deriv, 160
- set_boundary, 162
- set_coeffs_from_b, 162
- set_points, 162
- spline, 161
- spline_type, 160
- toString
 - Geometry.h, 232, 233
- trace
 - ConfigReader::ConfigReaderException, 49
- TwoIndex
 - Geometry.h, 230
- Uninitialized
 - Terminations.h, 267
- Update
 - Geodesic, 88
- UpdateAllNeighbors
 - SquareSubdivisionMesh, 168
- UpdateAllWeights
 - SquareSubdivisionMesh, 168
 - SquareSubdivisionMeshV2, 177
- UpdateData
 - ClosestRadiusDiagnostic, 37
 - Diagnostic, 53
 - EquatorialEmissionDiagnostic, 59
 - EquatorialPassesDiagnostic, 65
 - FourColorScreenDiagnostic, 74
 - GeodesicPositionDiagnostic, 98
- UpdateEveryNSteps
 - TerminationOptions, 187
- UpdateFinish
 - UpdateFrequency, 196
- UpdateFrequency, 196
 - UpdateFinish, 196
 - UpdateNSteps, 196
 - UpdateStart, 197
- UpdateNSteps
 - UpdateFrequency, 196
- UpdateStart
 - UpdateFrequency, 197
- UpNbr
 - SquareSubdivisionMeshV2::PixelInfo, 142
- Utilities, 19
 - GetDiagNameStrings, 19
 - GetFirstLineInfoString, 20
 - GetTimeStampString, 20
- Utilities::Timer, 194
 - Clock, 195
 - elapsed, 195
 - m_beg, 196
 - reset, 195
 - Second, 195
- VerletVelocityTolerance
 - Integrators, 18
- ViewScreen, 197
 - ConstructVielbein, 199
 - EndCurrentLoop, 199
 - GeodesicFinished, 199
 - getCurNrGeodesics, 199
 - getFullDescriptionStr, 199
 - IsFinished, 199
 - m_Direction, 200
 - m_GeodType, 200
 - m_Metric_dd, 200
 - m_Pos, 200
 - m_rLogScale, 201
 - m_ScreenCenter, 201
 - m_ScreenSize, 201
 - m_theMesh, 201
 - m_theMetric, 201
 - m_Vielbein, 201
 - SetNewInitialConditions, 200
 - ViewScreen, 198
- ViewScreen.h
 - GeodesicType, 277
 - Null, 277
 - Spacelike, 277
 - Timelike, 277
- Weight
 - SquareSubdivisionMesh::PixelInfo, 139
 - SquareSubdivisionMeshV2::PixelInfo, 142
- WriteCachedOutputToFile
 - GeodesicOutputHandler, 93